

+ 데이터 모델링 정의

▣ 데이터 모델링 탄생 배경

현재의 기업 정보시스템은 데이터베이스 관리 시스템(DBMS, DataBase Management System)을 논외로 하고는 설명할 수 없을 것이다. 하지만 초창기에는 데이터의 저장 매체가 존재하지 않았으며, 기업의 정보시스템은 저장 매체가 없는 단지 배치(Batch) 프로그램 위주의 정보시스템이었다. 이후 파일이나 데이터베이스 관리 시스템과 같은 데이터 저장 매체의 발전과 더불어 온라인(Online) 데이터 처리 정보시스템이 태동하게 되었다. 현재의 관계형 데이터베이스 관리 시스템이 아닌 이러한 시기의 파일이나 데이터베이스 관리 시스템(대표적인 예로 IBM의 계층형 데이터베이스 - HDB, Hierarchical Database)의 데이터 중심 관리 기법이 아니라 배치 프로세스에서 태동한 프로세스 중심의 데이터 관리 기법(구조적 방법론)에 의하여 정보의 고립화라는 현상을 초래하게 되었으며, 많은 기업들은 정보시스템을 유지 관리하는 데 막대한 비용을 투자해야만 했던 것이다. 이에 많은 학자들은 프로세스 중심의 정보시스템 분석, 설계 기법에 문제점이 있다고 생각하게 되었고, 진정 기업 정보시스템의 핵심은 데이터(정보)를 어떻게 하면 중복없이 정확하게 유지 관리할 수 있을까에 대한 보다 근본적인 안을 제시하게 되었다. 이와 함께 기업의 경영 정보시스템에 근본적인 문제가 설계나 개발의 문제보다는 정확한 업무의 파악(데이터에 대한 정확한 분석)이 선행되어야 한다는 결론에 이르렀으며, 이러한 환경에서 보다 현실적(실세계를 좀 더 잘 표현할 수 있는)인 관계형 데이터베이스나 개체 관계 모델링 기법(ERD, Entity Relationship Diagram)을 발전시켜 왔던 것이다.

▣ 모델 정의

일상생활에서 많이 접할 수 있는 단어인 모델(Model)은 다양한 현장에서 사용된다. 예술 분야에서 그림, 사진 등과 같은 작품의 대상이 되는 인물이나 대상을 보통 모델이라고 한다. 또한 아파트 분양을 할때 시공 회사에서 제공하는 모델 하우스는 분양을 원하는 사람들에게 아직 아파트 자체가 현존하지는 않지만 어떤 모습과 환경으로 시공자가 아파트를 건설할 것인가에 대한 정보를 제공한다. 이와 같이 모델이란 어떤 대상을 의미하는 포괄적 의미를 가지고 있다고 할 수 있으며, 특히 데이터 모델은 현실 세계에 대해 우리가 관심있어 하는 대상을 데이터베이스화 하기 위한 개념적 도구라고 정의할 수 있다.

아래는 모델의 일반적 의미를 실례로 설명한 내용이다.

1) 물리적 모델

현실에 실재하는 것이 아닌 복잡한 자동차의 모형, 대형 선박의 스케치 또는 설계도, 자동차의 모의 실험용으로 사용되는 바람 터널에서의 자동차 축소 모형 등이 물리적 모델의 실례이다.

2) 개념적 모델

특정 시점에 맞게 기상을 예측하기 위해서 사용되는 수리적 공식이나 원형을 파괴하지 않고 조작·수정·변경시키기 위한 경제 모형 등을 들 수 있다.

▣ 모델링 정의

모델링이라는 단어는 실체를 나타내는 일과 모형화라는 의미로 해석된다. '실체를 나타낸다'의 의미는 '대상을 나타낸다'라는 말로 해석될 수도 있다. 모형화라는 의미는 '형태를 만드는 일' 혹은 '대상을 만드는 일'이라고 해석할 수 있다. 따라서 데이터 모델링이란 사용자의 요구사항으로부터 데이터의 실체를 나타내는 일이라고 해석할 수 있을 것이다.

다음은 데이터 모델링의 다양한 관점에 대한 일반적 정의 사례이다.

- Webster 사전
 - 가설적 또는 일정 양식에 맞춘 표현(a hypothetical or stylized representation).

- 어떤 것에 대한 예비 표현으로, 그로부터 최종 대상이 구축되도록 되어있는 계획으로 기여하는 것
- 복잡한 '현실 세계'를 단순화시켜 표현하는 것이다.
- 모델이란 사물 또는 사건에 관한 양상(Aspect)이나 관점(Perspective)을 연관된 사람이나 그룹을 위하여 명확하게 하는 것이다.
- 모델이란 현실 세계의 추상화된 반영이다.

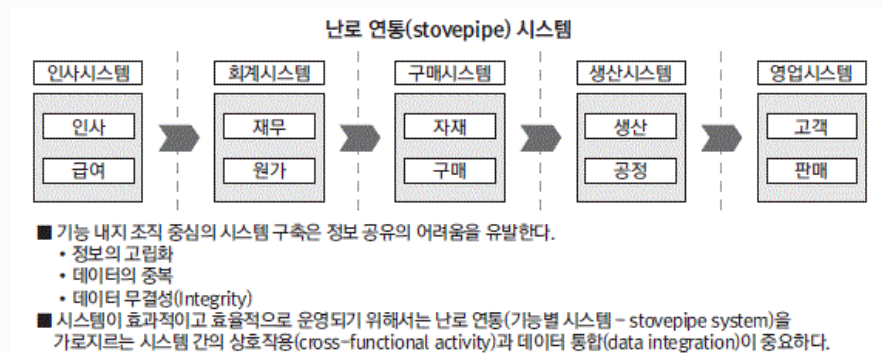
데이터 모델링을 다시 정의하면 기업 업무에 대한 종합적인 이해를 바탕으로 데이터에 존재하는 업무 규칙(Business Rule)에 대하여 참(True) 또는 거짓(False)을 판별할 수 있는 사실(사실명제)을 어떻게(How), 누가(Who) 접근하는지 또한 이에 대한 전산화와는 별개의 관점에서 이를 명확하게 표현하는 추상화 기법이라 할 수 있다. 즉, 현재 업무를 파악하여 문제점을 인식하고 개선 사항을 도출하며 미래에 적합한 설계를 이끌어 내기 위해 인간이 해야 할 대부분의 결정들을 내리는 단계까지를 모두 포함하는 것이 데이터 모델링이다.

▣ 데이터 모델이 제공하는 것

- 시스템을 현재 또는 원하는 모습으로 가시화하도록 도와준다.
- 시스템의 구조와 행동을 명세화 할 수 있게 한다.
- 시스템을 구축하는 틀을 제공한다.
- 우리가 결정한 것을 문서화한다.
- 다양한 영역에 집중하기 위해 다른 영역의 세부 사항은 숨기는 다양한 관점을 제공한다.
- 특정 목표에 따라 다양한 상세 수준을 제공한다.

⊕ 데이터 모델링 필요성

데이터 모델링은 프로세스 모델링과 함께 시스템 개발에 있어서 중요한 두 개의 축을 이룬다. 프로세스 중심의 분석/설계 방법을 통해 설계한 데이터 모델은 업무 프로세스의 변화에 따라 영향을 많이 받기 때문에 상대적으로 업무 변화에 대한 영향을 적게 받으면서 유연한 시스템을 만들기 위해 데이터 중심의 설계에 많은 관심이 모아지고 있다. 개발 방법론에 따라 다소간 차이는 있으나 데이터 모델링과 프로세스 모델링은 상호 보완적인 관점에서 이해되어야 하며 특히 데이터 모델은 시스템의 뼈대가 되기 때문에 데이터 모델링의 결과에 따라 시스템의 안정성은 많은 영향을 받게 된다고 할 수 있다. 고품질의 데이터 모델은 시스템의 안정성과 유연성, 성능 등에 미치는 영향이 크기 때문에 고품질의 데이터 모델을 확보하기 위한 데이터 모델링은 시스템 개발에 있어서 가장 핵심적인 과정이라 할 수 있다. 과거의 시스템 구축 방법의 주를 이루는 프로세스 중심적인 시스템 구축 방법에서는 위와 같이 데이터와 관련한 정보 공유의 문제점이 다수 발생하게 된다. 이러한 요인들은 결국 데이터의 무결성에 좋지 않은 영향을 주게 된다. 이러한 문제점들은 데이터 품질에도 악영향을 미치게 된다. 이러한 문제점들을 해소하기 위한 방안으로 기업들은 데이터 품질 툴을 고려하고 있다. 많은 기업이 데이터 품질 툴을 이미 도입하였거나 또는 도입을 검토하고 있다. 하지만 데이터 품질 툴 도입과 같은 일회성의 데이터 품질 향상 방안으로는 문제점을 근본적으로 치유하기에는 무리가 있다.



[그림 4-1-1] 프로세스 중심 개발 방법의 문제점

근본적인 조치로는 난로 연통형의 개별 업무 영역을 초월하여의 설계가 근본적인 해결 방안이다. 이러한 통합 데이터 구조의 가장 큰 특징은 데이터의 중복이 최소화되어 있다는 점이다.

데이터 모델링이 중요하다고 한다. 시스템 구축 과정 중에는 여러 설계 과정이 존재한다. 그 중에서도 특히 데이터 설계가 중요한 이유를 다음과 같이 정리할 수 있다.

■ 파급 효과(Leverage)

시스템 구축이 완성되어 가는 시점에서는 많은 애플리케이션이 테스트를 수행하고, 대규모의 데이터 이행을 성공적으로 수행하기 위한 많은 단위 테스트가 수행되고, 이러한 과정이 반복된다. 각 단위 테스트가 성공적으로 수행되고 완료되면 전체를 묶어 병행 테스트, 통합 테스트를 수행하게 된다. 만약 이러한 시점에 데이터 모델을 불가피하게 약간 변경해야 하는 상황이 발생한다고 가정해 보자. 이를 위해서 많은 영향 분석 과정이 일어나야 한다. 데이터 구조의 변경에 따른 표준 영향 분석, 응용 변경 영향 분석 등 많은 영향 분석이 일어난다. 그 이후에 해당 분야의 실제적인 변경 작업이 발생하게 된다. 변경을 해야 하는 데이터 모델의 형태에 따라 그 영향 정도는 차이가 있겠지만 이 시기의 데이터 구조의 변경으로 인한 일련의 변경 작업은 전체 시스템 구축 프로젝트에서 큰 위험 요소가 아닐 수 없다. 이러한 이유로 인해 시스템 구축 작업 중에서 다른 어떤 설계 과정보다 데이터 설계가 더 중요하다고 볼 수 있다.

■ 복잡한 정보 요구 사항의 간결한 표현(Conciseness)

데이터 모델은 구축할 시스템의 정보 요구 사항과 한계를 가장 명확하고 간결하게 표현할 수 있는 도구이다. 정보 요구

사항을 파악하는 가장 좋은 방법은 수많은 페이지의 기능적인 요구 사항을 파악하는 것보다 간결하게 그려져 있는 데이터 모델을 리뷰하면서 파악하는 것이 훨씬 빠른 방법이다. 데이터 모델은 건축물로 비유하면 설계 도면에 해당한다. 이것은 건축물의 설계 도면이 건축물을 짓는 많은 사람이 공유하면서 설계자의 생각대로 일사불란하게 움직여서 아름다운 건축물을 만들어 내는 것에 비유할 수 있다. 데이터 모델은 시스템을 구축하는 많은 관련자가 설계자의 생각대로 정보 요구 사항을 이해하고 이를 운용할 수 있는 애플리케이션을 개발하고 데이터 적합성을 유지할 수 있도록 하는 것이다. 이렇게 이상적으로 역할을 할 수 있는 모델이 갖추어야 할 가장 중요한 것은 정보 요구 사항이 정확하고 간결하게 표현되어야 한다는 점이다. 활용하고 있는 데이터 모델이 이와 같은 요소들을 충족한 모델인지를 확인해 볼 필요가 있다.

■ 데이터 품질(Data Quality)

데이터베이스에 담겨 있는 데이터는 기업의 중요한 자산이다. 이 데이터는 기간이 오래되면 될수록 활용가치는 훨씬 높아진다. 그런데 오랫동안 저장된 데이터가 그저 그런 데이터, 정확성이 떨어지는 데이터라면 어떨까? 이것은 일부 시스템의 기능이 잘못되어 수정하는 성격의 일이 아니다. 이것은 해당 데이터로 얻을 수 있었던 소중한 비즈니스의 기회를 상실할 수도 있는 문제이다. 데이터 품질의 문제가 중요한 이유가 여기에 있다. 데이터 품질의 문제는 데이터 구조가 설계되고 초기에 데이터가 조금 쌓일 때에는 인지하지 못하는 경우가 대부분이다. 이러한 데이터의 문제는 오랜 기간의 숙성된 데이터를 전략적으로 활용하려고 하는 시점에 대두되기 때문이다. 데이터 품질의 문제를 야기시키는 중대한 이유 중 하나가 바로 데이터 구조의 문제로 인해 야기된다. 중복 데이터의 미정의, 데이터 구조의 비즈니스 정의의 불충분, 동일한 성격의 데이터를 통합하지 않고 분리함으로써 데이터 불일치 등등 데이터 구조의 문제로 인한 데이터 품질의 문제는 치유하기에 불가능한 경우가 대부분이다.

데이터 모델링의 필요성은 다음과 같은 관점에서 설명될 수 있다.

▣ 애플리케이션과 데이터의 통합

현재 기업에서는 진정한 정보 인프라 구축을 위해 데이터와 애플리케이션의 통합에 많은 노력을 기울이고 있다. 데이터 기반의 통합이 아닌 애플리케이션 코딩 차원의 통합 시도에는 너무나 많은 비용과 시간이 소요된다. 데이터를 기반으로 한 통합은 효과적이면서 동시에 저비용으로 통합 프로젝트를 안정적으로 수행하면서 성공적으로 완수하기 위한 필요조건이 되고 있다. 이로 인해 데이터 기반 통합의 핵심 요소인 데이터 모델링의 중요성은 날로 부각되고 있다.

▣ 개발자들의 시스템 이해

개발자들은 하나의 애플리케이션에서 사용되는 데이터가 어떤 것인지 파악하고 이해하는 것뿐만 아니라 전체 시스템에서 데이터가 어떻게 상호 연관성을 가지고 유기적으로 움직이고 있는가에 대해 명확하게 이해하는 데 많은 어려움을 겪고 있다. 개발자들은 그들이 개발할 시스템과 데이터를 좀 더 확실하게 이해하기 위해 데이터의 모형화를 필요로 한다.

1) 사용자 관점 데이터

프로세스 모델링과 같이 데이터 모델링은 사용자가 원하는 것의 논리적 개념과 시스템이 어떻게 그것을 제공하는지의 물리적 개념을 명확하게 나타낸다.

2) 물리적 표현 또는 사용에 관계없는 데이터 그 자체의 본질

물리적인 것과 논리적인 것을 구별함으로써 저장 기법, 데이터와 파일 접근 방법 그리고 데이터를 사용하는 사람과 사용법에 대한 내역 등 변화되는 물리적인 것으로부터 독립되어 조직과 사용자가 필요로 하는 필수적이고 기본적인 데이터를 정의할 수 있다.

3) 애플리케이션간 데이터 사용

데이터 모델링은 데이터 정의, 생명주기 정보(CRUD) 그리고 언제 어떻게 데이터가 사용되었는지를 추적할 수 있는 방법(매트릭스 분석 기법을 이용한 상호작용 분석) 등을 제공함으로써 애플리케이션을 통해 데이터가 어떻게 사용되는지를 개발자들이 쉽게 이해할 수 있도록 한다.

▣ 데이터 모델링시 주의점

1) 중복(Duplication)

데이터 모델은 같은 데이터를 사용하는 사람, 시간, 그리고 장소를 파악하는데 도움을 준다. 이러한 지식 응용은 데이터 베이스의 여러 장소에 같은 정보를 저장하는 잘못을 하지 않도록 한다.

2) 비유연성(Inflexibility)

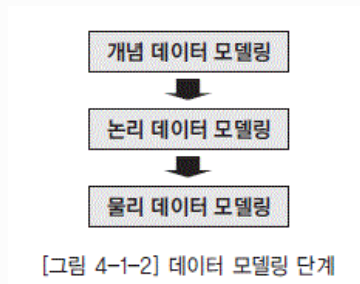
데이터 모델을 어떻게 설계했느냐에 따라 사소한 업무 변화에도 데이터 모델이 수시로 변경됨으로써 유지보수의 어려움을 가중시킬 수 있다. 데이터의 정의를 데이터의 사용 프로세스와 분리함으로써 데이터 모델링은 데이터 혹은 프로세스의 작은 변화가 애플리케이션과 데이터베이스에 중대한 변화를 일으킬 수 있는 가능성을 줄인다.

3) 비일관성(Inconsistency)

데이터의 중복이 없더라도 비일관성은 발생한다. 예를 들어 신용 상태에 대한 갱신 없이 고객의 납부 이력 정보를 갱신하는 것이다. 개발자가 다른 데이터와 모순된다는 고려 없이 일련의 데이터를 수정할 수 있기 때문이다. 데이터 모델링 시 데이터와 데이터간 상호연관 관계에 대한 명확한 정의는 이러한 위험을 사전에 예방할 수 있도록 해준다.

➤ 데이터 모델링 단계

데이터 모델링은 현실 세계의 기업 업무에서 발생하는 데이터에 대하여 물리적으로 데이터베이스화하기 위해 이루어지는 과정 중의 한 단계이다. 이 데이터 모델링은 개념 데이터 모델링, 논리 데이터 모델링, 물리 데이터 모델링 등 3단계로 나눌 수 있으며, 엄밀한 의미에서범주로 보기도 한다.



개념 데이터 모델링 단계에서는 주제별로 분류 가능한 업무를 분석한 후 핵심 엔터티(Entity)를 추출 하고 그들 간의 관계를 정의하여 전체 데이터 모델의 골격을 생성한다. 이렇게 도출된 엔터티(업무) 간 의 관계를 표현하기 위해 개체-관계 다이어그램(ERD, Entity-Relationship Diagram)을 작성한다.

[Tip] -----

데이터 모델링 과정은 다음과 같은 문제를 해결하려고 한다.

1. 어떠한 데이터가 중요한가?
2. 어떻게 데이터가 표시될 것인가?
3. 어디에 데이터가 저장될 것인가?

우선 시스템 분석 및 설계와 데이터베이스 설계의 차이를 알아야 한다. 데이터 모델링에서는 시스템 분석보다 조직의 업무를 지원하기 위한 포괄적 데이터베이스를 개발하는 데 초점이 맞춰진다. 반대로 시스템 분석에서는 데이터베이스 요구 사항을 고려하기는 하지만 데이터의 흐름, 데이터 변환, 입력, 출력 설계, 데이터 처리를 위한 과정에 초점이 맞춰진다. 보통 데이터 모델링 과정은 개념적 데이터 모델링(개념 데이터 모델링), 논리적 데이터 모델링(논리 데이터 모델링), 물리적 데이터 모델링(물리 데이터 모델링) 과정을 포괄한다.

논리 데이터 모델링 단계에서는 개념 데이터 모델링 단계에서 정의한 핵심 엔터티와 관계를 바탕으로 상세 속성을 정의하고 식별자를 확정하며 정규화와 같은 상세화 과정을 수행한다. 마지막으로 물리 데이터 모델링 단계에서는 논리 데이터 모델을 기반으로 목표하는 DBMS의 특성 및 구현 환경 등을 감안한 스키마(데이터 구조)를 일정한 기준과 규칙에 의해 도출하고 칼럼(Column)의 데이터 타입과 크기를 정의한다. 또한 데이터 사용량을 분석 예측하는 과정을 통해 효율적인 데이터베이스가 될 수 있도록 인덱스의 정의 및 역정규화 작업을 수행한다.

이러한 단계별 내용은 프로젝트 진행과도 일치한다. 물리 데이터 모델링 단계 후에 얻어진 스키마를 실제 데이터베이스로 생성하면 본격적인 애플리케이션 개발 단계로 넘어가게 된다. 각 단계에 대해 상세히 알아보면 다음과 같다.

▣ 개념 데이터 모델링(Conceptual Data Modeling)

개념 데이터 모델링은 조직, 사용자의 데이터 요구 사항을 찾고 분석하는 데서 시작한다. 이 과정은 어떠한 자료가 중요하며 또 어떠한 자료가 유지되어야 하는지를 결정하는 것도 포함한다. 이와 같이 데이터 요구 사항은 분석 과정의 초기에 본질적으로 기술과 무관한 사항들의 집합인 개념 데이터 모델로 형상화되고, 비즈니스 이해 관계자들과 초기 요구 사항을 논의하기 위해 사용된다. 이 단계에 있어서의 주요한 활동은 핵심 엔터티와 그들 간의 관계를 발견하고, 그것을 표현하기 위해서 개체-관계 다이어그램을 생성하는 것이다. 개체-관계 다이어그램은 조직과 다양한 데이터베이스 사용자에게 어떠한 데이터가 중요한지를 나타내기 위해서 사용된다. 데이터 모델링에서 다루는 범위와 수행 과정이 전 조직에 걸쳐 이루어진다면, 그것은 전사적 데이터 모델(Enterprise Data Model)이라고 불린다.

개념 데이터 모델을 통해 조직의 데이터 요구를 공식화하는 것은 두 가지의 중요한 기능을 지원한다. 첫째, 개념 데이터 모델은 사용자와 시스템 개발자의 데이터 요구 사항 발견을 지원한다. 개념데이터 모델은 추상적이다. 그렇기 때문에 그 모델은 상위의 문제에 대한 구조화를 쉽게 하며, 사용자와 개발자가 시스템 기능에 대해서 논의할 수 있는 기반을 형성한다. 둘째, 개념 데이터 모델은 현 시스템이 어떻게 변형되어야 하는가를 이해하는 데 유용하다. 일반적으로 매우 고립된(StandAlone) 시스템도 간단하게 추상적 모델링을 통해서 좀 더 쉽게 표현되고 설명된다.

▣ 논리 데이터 모델링(Logical Data Modeling)

논리 데이터 모델링은 데이터 모델링 프로세스의 Input으로써 비즈니스 정보의 논리적인 구조와 규칙을 명확하게 표현하는 기법 또는 과정이라 할 수 있다. 논리 데이터 모델링의 결과로 얻어지는 논리 데이터 모델은 데이터 모델링이 최종적으로 완료된 상태라고 정의할 수 있다. 즉, 물리적인 스키마 설계를 하기 전 단계의 '데이터 모델'상태를 일컫는 말이다. 논리 데이터 모델링의 핵심은 어떻게 데이터에 액세스하고 누가 데이터에 액세스하며, 그러한 액세스는 전산화와는 독립적으로, 다시 말해서 누가(Who), 어떻게(How), 그리고 전산화는 별개로 비즈니스 데이터에 존재하는 사실들을 인식하여 기록하는 것이며, 이것은 기법으로서의 의미를 넘어 하나의 철학이라고도 할 수 있다.

특히 데이터 모델링 과정에서 가장 핵심이 되는 부분이 논리 데이터 모델링이라고 할 수 있다. 데이터 모델링이란 모델링 과정을 통해서 조사하고 결정한 사실을 단지 개체-관계 다이어그램이라는 그림으로 그려내는 과정을 말하는 것이 아

니다. 시스템 구축을 위해서 가장 먼저 시작할 기초적인 업무 조사를 하는 초기 단계부터 인간이 결정해야 할 대부분의 사항을 모두 정의하는 시스템 설계의 전 과정을 지원하는 ‘과정의 도구’라고 해야 할 것이다.

이 단계에서 수행하는 또 한 가지 중요한 활동은 정규화이다. 정규화는 논리 데이터 모델 상세화 과정의 대표적인 활동으로 논리 데이터 모델의 일관성을 확보하고 중복을 제거하여 속성들이 가장 적절한 엔터티에 배치되도록 함으로써 좀 더 신뢰성있는 데이터 구조를 얻는 데 목적이 있다. 논리데이터 모델의 상세화는 식별자 확정, 정규화, M:M 관계 해소, 참조 무결성 규칙 정의 등을 들 수 있으며, 추가적으로 이력 관리에 대한 전략을 정의하여 이를 논리 데이터 모델에 반영함으로써 데이터 모델링을 완료하게 된다. 이에 대한 상세한 내용은 제3장 논리 데이터 모델링에서 다룬다.

▣ 물리 데이터 모델링(Physical Data Modeling)

데이터 모델링 과정의 세 번째 단계인 물리 데이터 모델링은 논리 데이터 모델이 데이터 저장소로서 어떻게 컴퓨터 하드웨어에 표현될 것인가를 다룬다. 데이터가 물리적으로 컴퓨터에 어떻게 저장될 것인가에 대한 정의를 물리적 스키마라고 한다. 이 단계에서 결정되는 것은 테이블, 칼럼 등으로 표현되는 물리적인 저장 구조와 사용될 저장 장치, 자료를 추출하기 위해 사용될 접근 방법 등이 있다.

계층적 데이터베이스 관리 시스템 환경에서는 데이터베이스 관리자가 물리적 스키마를 설계하고 구현하기 위해서 보다 많은 시간을 투자하여야 한다. 계층적 데이터베이스 관리 시스템 환경에서의 물리적 스키마 설계의 예는 데이터의 디스크상의 위치, 색인화할 레코드, 다양한 최적화 문제 등이다.

이에 비해, 논리 데이터 모델은 매체에 물리적으로 구현되는 것과는 독립적으로 여겨진다. 현실적으로 물리 데이터 모델을 계층적 데이터베이스 관리 시스템에 구현하는 것은 그것이 구현될 하드웨어와 밀접한 관계를 맺고 있다. 그러나 관계형 데이터베이스 관리 시스템의 출현으로 인해 그 초점이 개념 및 논리 데이터베이스 설계로 이동하고 있다. 그래서 관계형 데이터베이스 관리 시스템을 사용하는 조직에서는 데이터베이스 관리자가 잘 유지해야 하는 데이터 항목의 발견과 데이터 항목 간에 존재하는 논리 관계를 이해하는데 더 많은 시간을 할애한다. 관계형 데이터베이스 관리 시스템의 출현으로 인하여 물리적 데이터베이스 설계와 관련된 문제들은 많은 부분 관계형 데이터베이스 관리 시스템 소프트웨어에서 처리되고 있다.

➤ 모델링 기본원칙

일반적으로 모델링의 방법론, 기법, 도구와 같은 개발 보조 수단의 사용 없이는 시스템이 잘 설계될 수 없을 것이라 생각하게 된다. 하지만 과거의 수많은 시스템들은 이러한 개발 보조기구의 사용없이 성공적으로 개발됐다. 물론 과거의 시스템들은 단순해서 그러할 수 있었다고 주장한다면 그 말 역시 맞는 말이다. 그러나 이러한 사실들은 여기에서 말하려는 모델링의 핵심 사항은 아니다. 프로젝트의 어려움 정도 또는 도구와 기술에 상관없이 좋은 설계를 포함한 훌륭한 시스템 개발은 다음의 단순한 단계로부터 시작한다.

- 해결해야 하는 문제점들을 선별, 결정한다.
- 해결책을 위한 문제점들을 구체화한다.
- 시스템을 구축한다.
- 실제로 구현한다.

시스템 개발 실패의 주요 요인을 무엇이라 생각하는가? 그것은 처음의 요구 사항을 충족시키지 못하는 설계자의 무능력을 들 수 있다. 즉 적절한 시스템이 개발되기 위해서는 요구 사항을 완전하고 정확하게 식별해야 한다. 논리 데이터 모델링은 기능 또는 조직이 사용하는 물리적 설계 구조에 관계없이 기업의 비즈니스에 존재하는 데이터의 명확한 구조 및 정의를 제시한다. 이는 최종 사용자 관점에서 바라보는 정보구조를 개념화하고 추상화시킨 데이터의 구조이다. 다음에 설명할 세 가지 최우선 원칙들은 논리 데이터 모델링의 접근 방식을 성공적으로 이끄는 토대이다.

▣ 커뮤니케이션 원칙(Communication Principle)

요구 사항은 모든 사람들이 이해할 수 있도록 명확하게 공표됨은 물론 최종 사용자 지향적으로 분명하게 파악되는 수준으로 작성되어야 한다. 논리 데이터 모델링의 주목적은 최종 사용자 데이터에 대한 뷰(View)를 개념화하고 추상화하여 시스템 설계자들에게 전달하는 것이다. 물론 커뮤니케이션 원칙이 최종 사용자들에게만 적용되는 것은 아니다. 다른 배경과 기술을 가진 많은 사람들도 논리 데이터 모형의 이해를 필요로 한다.

[표 5-1-1] 데이터 모델의 이해를 필요로 하는 그룹

그룹명	필요성
최종 사용자	개념화, 추상화, 정규화 기법을 통하여 중복없고, 데이터의 정확성을 보장하는 데이터 구조의 이해 및 사용
시스템 분석가	시스템에서 사용되어질 정확한 데이터의 구조 및 데이터가 갖는 업무 규칙의 이해
데이터베이스 관리자	논리 데이터 모델의 구조와 물리 스키마(데이터의 구조)의 차이점을 이해하고 최종 사용자에게는 데이터의 제공, 시스템 분석가에게는 물리 스키마의 제공을 위한 데이터 구조의 이해
타 프로젝트에서 작업하는 분석가	관련 프로젝트가 데이터를 어떻게 정의하고 있는지를 알아냄. 인터페이스(Interface)가 개발되어지고 데이터가 애플리케이션 또는 시스템 간에 공유되기 위한 데이터 구조 및 업무 규칙의 이해

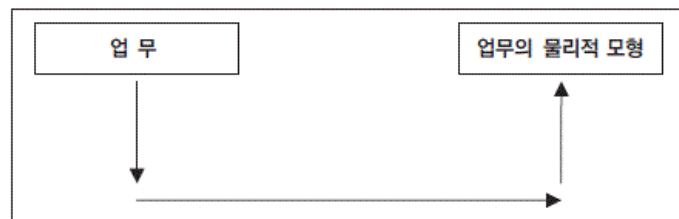
[표 5-1-1]의 그룹 구성원들과의 커뮤니케이션을 용이하게 하기 위해서는 두 가지의 논리 데이터 모델을 생각해 볼 수 있다. 그것은 비즈니스 지향적인 최종 사용자 데이터 모델과 기술적인 상세 데이터 모델이다. 논리 데이터 모델의 문서

와 그림을 작성할 때 그 목적을 명확히 해야 한다. 왜냐하면 모델을 보는 사람들에게 기술적으로 알아보기 힘든 혼란을 주는 것은 곤란하기 때문이다. 이와 같은 명확한 정의를 통해 시스템 개발 관련자와 최종 사용자들간에 비즈니스 요구 사항이 정확히 전달되어야 한다.

▣ 모델링 상세화 원칙(Granularity Principle)

데이터의 상세화 정도를 제시하고 조직이 사용하는 정보 구조의 '최소 공통 분모'를 제시해야 한다. 또한 복잡한 구조는 요소적인 부분들로 쪼개야 하며 불필요한 구조와 중복은 제거되어야 한다. 일반적으로 논리 데이터 모델은 너무 상세화 될 필요가 없다든지 애플리케이션에 대한 상세화는 물리 데이터 모델에서 이를 수 있다는 생각은 위험한 태도이다. 모델링의 상세화 원칙은 "데이터는 데이터의 본질과 잠재적 사용을 이해할 수 있을 만큼 상세화되어야 한다"는 것을 의미한다. 사실 좋은 설계는 분석 단계(논리 데이터 모델)를 위한 상세 수준이 다른 단계(물리 데이터 모델)에서 선택된 수준만큼 상세화되어야 한다는 것을 의미한다.

많은 모델러(Modeler)가 겪는 문제는 논리 데이터 모델에서 물리 데이터 모델로의 이동이 분해 과 정이라고 오해하는 것이다. 하지만 논리 데이터 모델에서 물리 데이터 모델로의 이동은 분해가 아니 라 보통 수준의 상세화에서 발생하는 변환(Transformation)이다. 즉, 물리 데이터 모델링은 논리 데이터 모델에 대한 변환 과정이다. [그림 4-1-3]을 참조한다



[그림 4-1-3] 논리 데이터 모델에서 물리 데이터 모델로의 이동

▣ 논리적 표현 원칙(Logical Representation Principle)

조직의 데이터에 대한 논리적 측면을 최대한 표현해야 한다. 모델은 물리적 제약 조건 없이 비즈니스를 그대로 반영해야 한다. 즉, 논리 데이터 모델은 특정 아키텍처, 기술 또는 제 사용자들이 항상 시스템 개발자들에게 가지는 주요 불만 사항은 바로 '이해의 부족'이다. 사용자가 원하는 것에 근거를 두지 않은 프로젝트는 실패할 확률이 높다. 또한 기존 애플리케이션에 대한 빈약한 문서화의 주요 원인 중 하나는 성급하게 물리적 설계 단계로 들어가려 하는 분석가들의 성향 때문이다. 논리적 설계와 물리적 설계를 구별하지 못하면 프로젝트가 추 구하고자 하는 물리적 선택 사항을 제한하거나 잘못된 방향으로 진행하게 된다. 이러한 문제들을 예를 통해 이해해 보자.

1) 웬만한 분석의 결과는 잘 알고 있다(지름길을 좋아하는 것)

어떤 이들은 여러 가지 이유(예를 들어, 자명하게 보이는 분석을 수행하는 데 시간 낭비를 줄이기 위해 또는 결과가 이미 정해져 있어서)때문에 분석을 생략하는 경우가 많다. 대부분의 경우에서 모델러들은 애플리케이션 개발 과정에서 발생할 수 있는 모든 것들을 알 수 없다. 또한 모델러들은 몇 십년내에 시스템에서 발생할 수 있는 것에 대해서도 확신하지는 못한다. 결론적으로 최적의 행동은 솔루션에 독립적인 방식으로 규칙을 따르고 문제를 문서화하는 것이다.

2) 조금하게 솔루션을 구체화하는 것

문제를 조사할 때 데이터베이스가 어떻게 구현되어야 하는가에 관한 기발한 아이디어가 나오기란 어렵다. 이러한 점에서 선불리 솔루션을 구체화하는 것은 잘못된 것이다. 그 이유는 문제의 일부분만을 파악했을 때 이를 해결할 수 있는 모든 데이터가 확보된 것은 아니며, 나중에 인식한 문제로부터 다른 솔루션을 발견할 수도 있기 때문이다. 또한 문제가 정확히 진술된 것이 아니기 때문에 작업물을 검토하는 다른 사람들은 솔루션이 문제를 해결하는 이유를 이해할 수 없을 것이다. 이것은 분석가들이 솔루션을 그들의 머리에서 없애야 한다는 것이 아니라 솔루션에 대한 좋은 생각이 표면화되었을 때 그것을 기록하여 논리적 설계 다음에 물리 데이터 모델 설계자에게 전달하여야 한다는 것을 의미한다.

논리적 설계의 컴포넌트가 부정확하거나 빠먹었을 때 물리 데이터 모델에서 논리 데이터 모델로의 회귀는 피할 수 없다. 물리 데이터 모델 제약 조건이 명시된 경우조차도 논리 데이터 모델은 제약 조건이 없는 것처럼 진행해야 한다. 그럼에도 불구하고 이러한 경우들이 논리적 표현 원칙을 변화시키지는 않는다. 논리 데이터 모델링이 2년짜리 프로젝트의 첫 단계이든지 수많은 2주 짜리의 반복 과정이든지에 상관 없이 원칙은 솔루션을 구체화하기 전에 문제를 이해해야 한다는 것이다.

+ 좋은 데이터 모델의 요소

일반적으로 시스템 구축 과정에서 생성되는 데이터 모델은 품질을 평가하는 것이 매우 어렵다. 사실은 특정 데이터 모델이 업무 환경에서 요구하는 사항을 얼마나 잘 시스템적으로 구현할 수 있는가를 객관적으로 평가할 수 있다면 가장 좋은 평가 방법일 것이다. 하지만 이것을 객관적으로 평가할 수 있는 기준이 존재하지는 않는 것이 현실이다. 다음에서는 이러한 상황에서 대체적으로 좋은 데이터 모델이라고 말할 수 있는 몇 가지의 요소를 설명한다.

▣ 완전성(Completeness)

업무에서 필요로 하는 모든 데이터가 데이터 모델에 정의되어 있어야 한다. 완전성은 데이터 모델을 검증하기 위해서 가장 먼저 확인해야 할 부분이다. 이 기준이 충족되지 못하면 다른 어떤 평가 기준도 의미가 없어진다. 만약 보험사의 데이

터 모델에 고객의 작업을 관리하기 위한 속성이 존재하지 않는다면 어떻겠는가? 이것은 심각한 데이터 모델의 문제점이다.

▣ 중복 배제(Non-Redundancy)

하나의 데이터베이스 내에 동일한 사실은 반드시 한 번만 기록하여야 한다. 예를 들면, 하나의 테이블에서 '나이' 칼럼과 '생년월일' 칼럼이 동시에 존재한다면 이것은 데이터 중복이라고 볼 수 있다. 이러한 형태의 데이터 중복 관리로 인해 여러 가지의 바람직하지 않는 형태의 데이터 관리 비용을 지불하고 있다. 예를 들면, 저장 공간의 낭비, 중복 관리되고 있는 데이터의 일관성을 유지하기 위한 추가적인 데이터 조작 등이 대표적으로 낭비되고 있는 비용이라고 볼 수 있다.

▣ 비즈니스 룰(Business Rules)

데이터 모델에서 매우 중요한 요소 중에 하나가 데이터 모델링 과정에서 도출되어 지고 규명되어지는 수많은 업무 규칙(Business Rules)을 데이터 모델에 표현하고 이를 해당 데이터 모델을 활용 하는 모든 사용자가 그 규칙을 공유할 수 있게 제공하는 것이다. 특히 데이터아키텍처에서 언급되는 논리 데이터 모델(Logical Data Model)에서 이러한 요소들이 포함되어야 함은 매우 중요한 요소라고 할 수 있다. 예를 들면, 보험사의 직원들은 매월 여러 가지 항목에 대해 급여를 지급받고, 이를 데이터로 관리하고 있다. 각 직원들은 월별로 하나 이상의 급여 항목(기본급, 상여금, 수당, 수수료, 등등)에 대해 급여를 지급받는다. 여기에 더 나아가 각 직원은 사원 구분별(내근, 설계사, 계약직, 대리점 등)로 위의 급여 항목을 차등적으로 지급받다는 업무 규칙이 존재한다. 이러한 내용을 데이터 모델에 나타내어야 한다. 이렇게 함으로써 해당 데이터 모델을 사용하는 모든 사용자(개발자, 관리자등)가 해당 규칙에 대해 동일한 판단을 하고 데이터를 조작할 수 있게 되는 것이다.

▣ 데이터 재사용(Data Reusability)

데이터의 재사용성을 향상시키고자 한다면 데이터의 통합성과 독립성에 대해서 충분히 고려해야 한다. 무슨 이야기인가? 과거에 정보시스템이 생성되고 운영되던 형태를 되짚어보면 철저하게 부서 단위의 정보시스템으로 설계되고 운용되어 왔다. 현재 대부분의 회사에서 진행하고 있는 신규 정보 시스템의 구축 작업은 어떻게 이루어지고 있는가? 회사 전체 관점에서 공통 데이터를 도출하고 이를 전 영역에서 사용하기에 적절한 형태로 설계하여 시스템을 구축하게 된다. 이러한 형태의 데이터 설계에서 가장 중요하게 대두되는 것이 통합 모델이다. 통합 모델이어야만 데이터 재사용성을 향상시킬 수 있다. 또 다른 측면에서 보면 과거 정보시스템의 데이터 구조의 가장 큰 특징은 데이터 모델이 별도로 존재하지 않고 애플리케이션의 부속품 정도로 인식되어져 왔던 것이 사실이다. 이러한 환경에서의 데이터는 프로세스의 흐름에 따라 관리되게 마련이다. 이러면 데이터 중복이 많이 발생하고 데이터의 일관성 문제가 심각하게 초래된다. 데이터가 애플리케이션에 대해 독립적으로 설계되어야만 데이터 재사용성을 향상시킬 수 있다.

▣ 안정성 및 확장성(Stability and Flexibility)

정보 시스템은 비즈니스의 변화에 대해 최적으로 적응하도록 끊임없이 요구 받고 있다. 하지만 정보시스템의 데이터 모델은 이러한 변화에 대해 어떻게 대응할 수 있는가? 현재의 데이터 구조를 거의 변화하지 않고도 변화에 대응할 수 있는 데이터 구조도 있을 것이고, 아주 적은 확장을 통해 이러한 변화에 대응하는 것도 있을 것이다. 하지만 이러한 변화에 대응하기 위해 데이터 구조적으로 아주 많은 변화를 주어야만 한다면 변화의 대상이 되는 부분뿐만 아니라 정보시스템의 나머지 부분들도 많은 영향을 받게 될 것이다. 그래서 많은 기업이 정보시스템을 구축하는 과정에서 데이터 구조의 확장성, 유연성에 많은 노력을 기울이고 있다. 그렇다면 어떤 데이터 모델이 이러한 확장성을 가진 데이터 모델인가?

현대의 기업들이 동종의 타 기업으로부터 경쟁 우위에 자리매김하려고 한다면 구축하는 데이터 모하게 대응할 수 있어야 한다. 특히 근래의 많은 패키지 시스템이 가지고 있는 데이터 모델들은 확장성을 강조하기 위해서 많은 부분을 통합한 데이터 모델 형태를 가지고 있는 것을 볼 수 있다. 여기에서도 잘 나타나듯이 확장성을 담보하기 위해서는 데이터 관점의 통합이 불가피하다. 특히 정보시스템에서의 '행위의 주체'가 되는 집합의 통합, '행위의 대상'이 되는 집합의 통합, '행위 자체'에 대한 통합 등은 전체 정보시스템의 안정성, 확장성을 좌우하는 가장 중요한 요소라 할 수 있다.

▣ 간결성(Elegance)

데이터 모델이 갖추어야 하는 중요한 요소 중 하나는 기업이 관리하고자 하는 데이터를 합리적으로 균형이 있으면서도 단순하게 분류하는 것이다. 아무리 효율적으로 데이터를 잘 관리할 수 있더라도 그것의 사용, 관리 측면에서 복잡하다면 잘 만들어진 데이터 모델이라고 할 수 없다. 동종의 비즈니스를 영위하는 기업이라고 하더라도 각 회사의 데이터 모델을 비교해 보면 복잡도에는 많은 차이가 있는 것을 볼 수 있다. A 보험사는 계약 업무를 수행하기 위해 10개의 테이블을 정의해 업무를 수행하는 반면에 B 회사는 100개의 테이블을 정의해 동일한 업무를 수행하고 있다. 두 회사의 데이터 모델의 차이점은 무엇인가? 10개의 테이블을 가지고 업무를 수행하고 있는 A 회사의 데이터 모델은 간결하지만 새로운 업무 환경의 변화에 대해 확장성을 가지고 있다. B 회사는 겉으로는 새로운 업무 환경의 변화(신규 상품의 출현 등)에 능동적으로 대처하고 있는 것처럼 보이지만 사실은 보유하고 있는 데이터 모델의 한계로 인해 테이블의 수가 지속적으로 증가해 왔다. 이렇게 됨으로써 데이터 모델은 간결하지 못하고 동일한 형태로 관리되어야 하는 데이터가 복잡한 형태로 관리되어지고, 그들과의 관계를 갖고 있는 다른 여러 가지의 데이터나 복잡한 형태의 관계들이 불가피해 복잡성을 증가시켜왔다. 결국 간결한 모델의 기본적인 전제는 통합이다. 합리적으로 잘 정돈된 방법으로 데이터를 통합하여 데이터의 집합을 정의하고, 이를 데이터 모델로 잘 표현하여 활용한다면 웬만한 업무 변화에도 데이터 모델이 영향을 받지 않고 운용될 수 있게 된다.

▣ 의사소통(Communication)

데이터 모델의 역할이 많다. 그 중에서도 중요한 것이 데이터 모델의 의사소통 역할이다. 데이터 모델은 대상으로 하는

업무를 데이터 관점에서 분석하고 이를 설계하여 나오는 최종 산출물이다. 데이터 분석 과정에서는 자연스럽게 많은 업무 규칙이 도출된다. 이 과정에서 도출되는 많은 업무 규칙은 데이터 모델에 엔터티, 서브타입, 속성, 관계 등의 형태로 최대한 자세하게 표현되어야 한다. 예를 들면, '사원' 테이블에는 어떠한 '사원구분'을 가지는 사원들이 존재하는가? '정규직', '임시직' 사원들이 같이 존재하는가? 아니면 또 다른 형태의 사원들이 존재하는지를 표현해야 한다. 더 나아가서 '호봉'이라는 속성은 '정규직'일 때에만 존재하는 속성인데, 이러한 업무 규칙이 데이터 모델에 표현되어야 한다. 또한 관리하는 사원 중에서 '정규직' 사원만이 '급여' 테이블과 관계를 가진다. 이러한 부분은 개별 관계로 데이터 모델에 표현되어야 한다. 이렇게 표현된 많은 업무 규칙은 해당 정보시스템을 운용, 관리하는 많은 관련자들이 설계자가 정의한 업무 규칙들을 동일한 의미로 받아들이고 정보시스템을 활용할 수 있게 하는 역할을 한다. 즉, 데이터 모델이 진정한 의사소통 (Communication)의 도구로서의 역할을 하게 된다.

▣ 통합성(Integration)

기업들이 과거로부터 정보시스템을 구축해왔던 방법은 개별 업무별로의 단위 정보 시스템을 구축하여 현재에까지 유지보수를 해오고 있는 것이 보통이다. 점진적인 확장과 보완의 방법으로 정보시스템을 사용해 왔기 때문에 동일한 성격의 데이터임에도 불구하고 전체 조직 관점에서 보면 여러 곳에서 동일한 데이터가 존재하기 마련이다. 특히 이러한 데이터 중에서도 고객, 상품 등과 같이 마스터 성격의 데이터들이 분할되어 관리됨으로 인해 전체 조직 관점의 데이터 품질, 관리, 활용 관점에서 많은 문제점이 나타나고 있는 것이 현실이다. 가장 바람직한 데이터 구조의 형태는 동일한 데이터는 조직의 전체에서 한 번만 정의되고 이를 여러 다른 영역에서 참조, 활용하는 것이다. 물론 이때에 성능 등의 부가적인 목적에 따라 의도적으로 데이터를 중복시키는 경우는 존재할 수 있다. 동일한 성격의 데이터를 한 번만 정의하려면 공유 데이터에 대한 구조를 여러 업무 영역에서 공동으로 사용하기 용이하게 정의할 수 있어야 한다. 이러한 이유로 데이터 아키텍처의 중요성이 한층 더 부각되고 있다.



데이터 모델링 이해

Home > DB 구축 가이드 > DA 가이드 > 데이터 모델링 > 데이터 모델링 이해

데이터 모델링 개요

데이터 모델링 기법 이해

데이터 모델링 표기법 이해

+ 데이터 모델 목적

데이터 모델은 데이터베이스 설계에 대한 계획 또는 청사진이다. 설계자와 개발자, 사용자 등 모든 관련자들은 데이터 모델을 통해 구축될 시스템의 데이터 구조에 대한 형상을 이해하고, 요구 사항의 구현과 변경 등에 대해 원활한 의사소통을 도모하게 된다. 예를 들어, 건물을 지을 때 건축설계 회사는 건물 공사 이전에 그 건물을 위한 계획이나 청사진을 작성하게 되는데, 만일 계획 단계에서 어떤 방이 너무 좁거나 불편한 곳이 있다면 단순히 설계 도면의 선을 다시 그램으로써 청사진이 변경될 수 있지만 건물이 완공된 후에 변경이 필요하게 되면 벽, 상하수도, 전기 등 모든 것을 다시 시공해야만 하기 때문에 비용이나 시간 측면에서 많은 손해를 보게 될 것이다. 이와 같이 완공된 건물을 변경하는 것보다는 계획 단계에서의 변경이 훨씬 쉽고 간결하며 비용 면에서도 저렴하기 때문에 설계자는 시공에 앞서 조감도, 평면도, 입체도 등 다양한 논리 도면을 통해 건물주(사용자)가 건물의 논리적인 형태를 쉽게 이해하고 의견을 제시하거나 확정을 할 수 있도록 유도한다. 이때 사용된 도면들은 설계자와 시공자, 건물주 등이 짓고자 하는 건물에 대해 동일한 논리적 모습을 공유하게 함으로써 원활한 의사소통의 수단이 되었다고 할 수 있다.

이러한 논리는 건축 공사뿐만 아니라 시스템 개발도 동일하게 적용될 수 있다. 데이터 모델링 단계에서 업무를 잘못 이해했거나 관계를 잘못 정의한 것이 발견되었다면 해당하는 다이어그램과 일부 관련된 문서만 변경하면 된다. 그러나 데이터베이스와 응용 프로그램 개발이 완료된 후 이러한 오류를 발견하여 수정하려면 이와 관련된 많은 프로그램과 SQL 문이 변경되어야 할 뿐만 아니라 데이터가 새로운 구조로 옮겨져야 하는 등 이러한 변경을 반영하는 데 많은 비용과 시간이 필요하게 되기 때문에 가능하면 설계 단계에서 조기에 오류들이 발견되고 수정되도록 원활한 의사소통이 필요하게 되고, 이러한 목적을 달성하는 데 데이터 모델은 최적의 수단으로 활용될 수 있다.

+ 개체-관계 모델 기법(Entity-Relationship Modeling)

개체-관계 모델은 1976년에 피터 첸(Peter Chen)에 의해서 최초로 제안되었으며 그의 논문을 통해 이 모델의 기본적인 구성 요소가 정립되었다. 그 후 데이터 모델을 만들어 주는 많은 도구와 기법 이 소개되었는데 계층적 데이터 모델, 네트워크 데이터 모델, ANSI/SPARC 데이터 모델, 개체-관계 데이터 모델, 의미 객체 모델 등을 예로 들 수 있다. 이 가운데에서 개체-관계 모델은 표준적인 데이터 모델로 부상했는데 이 모델이 지니고 있는 단순성 때문에 현재 개념/논리 데이터 모델링에서 가장 일반적으로 사용되고 있다. 이 모델에 서브타입이 추가되면서 확장된 개체-관계 모델 (Extended Entity-Relationship model)이 만들어졌으며 이제 일반적으로 개체-관계 모델이라고 하면 이 확장된 개체-관계 모델을 의미한다.

개체-관계 모델은 다음과 같은 목적으로 사용되고 있다.

- 데이터에 대해 관리자, 사용자, 개발자들이 서로 다르게 인식하고 있는 뷰들을 하나로 통합할 수 있는 단일화된 설계안을 만들 수 있다.
- 서로 다른 뷰를 충족시킬 수 있는 데이터 처리와 제약 조건 등의 요구 사항을 정의하기 위해서 이다.
개체-관계 모델은 개체-관계 다이어그램(ERD, Entity Relationship Diagram)으로 표현된다.
개체-관계 다이어그램은 최종 사용자의 관점에서 데이터 구조를 그림 형태로 묘사하기 위해 개체, 관계, 속성이라는 세 개의 기본 요소를 사용하며, 엔터티와 이들간의 관계를 미리 약속된 도형을 사용하여 알기 쉽고 일목요연하게 그림으로 표시한 것이라 정의할 수 있다.

본 절에서는 이들 각 요소들을 구체적으로 살펴본다.

+ 개체-관계 모델 구성 요소

개체-관계 데이터 모델링은 일반적으로 개발 방법론에 의하여 논리 모델링(논리 데이터 모델링)과 물리 모델링(물리 데

이터 모델링)으로 나눌 수 있을 것이다. 논리 데이터 모델이란 각 기업에서 업무를 수행하기 위해 필요한 개체(엔터티)가 무엇이며, 개체의 유일성을 보장해주는 식별자(Unique identifier)가 무엇이며, 각 엔터티 간에는 어떤 상관관계가 있고 필요한 속성이 무엇인지를 설명해주는 그림이다. 즉, 업무 담당자와 IT 시스템 설계자 사이의 의사소통이라 할 수 있다. 여기에서 중요한 것은 논리 데이터 모델링은 데이터베이스나 하드웨어 등과 같은 시스템과 아무런 상관없이 업무에서 필요한 데이터를 그림으로 그려놓고 이를 검증하는 방법으로, 주로 개체-관계 다이어그램을 사용한다는 것이다. 따라서 외래키(FK, Foreign Key), 기본키(PK, Primary Key), 액세스 성능, 분산 시스템 등의 물리적 시스템 환경은 고려하지 않는다. 이 때문에 논리적 모델의 특징은 초기에 엔터티와 엔터티의 관계에서 M:M 관계, 순환 관계(Recursive), 슈퍼/서브 타입(Super/Sub Type), 배타적(Arc) 관계 등을 갖고 있는 개체(엔터티)가 많이 보인다는 점이다. 잘 설계된 논리 데이터 모델은 비록 업무 방식이 바뀌어도 업무 영역이 바뀌지 않는 한 설계 변경이 거의 발생하지 않는다. 이러한 이유 때문에 많은 시스템 설계자는 프로세스 중심의 설계보다 데이터 중심 방식의 설계를 주로 사용한다. 논리 데이터 모델에서 하나의 엔터티는 반드시 물리적으로 하나의 테이블이나 세그먼트가 되지는 않으며, 하나 이상 또는 테이블 한 개의 일부가 될 수도 있다. 물리 데이터 모델링은 논리 데이터 모델링을 기초로 현재의 시스템 환경(네트워크, 하드웨어, 운영체제, 미들웨어, 데이터베이스, 디스크 용량, 분산 등)을 고려하여 최고의 성능 향상을 목적으로 한다.

논리 데이터 모델링에서 물리 데이터 모델링 단계로 넘어오면서 고려해야 하는 작업의 사례는 다음과 같다.

- Super/Sub 관계의 엔터티를 몇 개의 테이블로 만들 것인가
- 배타적(Arc) 관계 엔터티의 외부키(Foreign Key)를 몇 개로 할 것인가
- 성능 향상을 위해 테이블을 추가해야 할 것인가 혹은 통합해야 할 것인가
- 통계 작업을 위해 합계(Summary) 테이블 같은 임시성 테이블을 몇 개로 할 것이며, 유일키를 무 엇으로 할 것인가
- 테이블의 칼럼을 다른 테이블에 중복할 것인가, 중복한다면 어떤 애플리케이션이 관련되어 있는 가, 인덱스의 설정, 스냅샷(Snapshot) 또는 뷰(View) 등의 객체가 필요한가
- 분산 환경에서 테이블을 중복할 것인가, 중앙에 필요한 테이블을 따로 가져갈 것인가
- 데이터가 분산 환경에서 이동 시 문제를 어떻게 해결할 것인가

물리 데이터 모델링 단계에서는 이러한 해결 과제가 산적해 있으며, 이를 종합적으로 해결할 때 전체적인 성능 향상을 기대할 수 있다. 위의 개념은 매우 중요하며 논리 데이터 모델과 물리 데이터 모델의 구분이 반드시 필요하다. 왜냐하면 물리적인 디자인은 시스템의 변경에 따라 필수적으로 변경이 되는 요소지만 논리 데이터 모델은 업무 영역이 바뀌지 않는 이상 변경이 거의 없는 특성이 있기 때문이다. 따라서 하나의 그림으로 모델을 관리하게 되어 변경이 일어날 때마다 유지 보수를 하다 보면 현재의 시스템과 상관없는 것으로 전락하게 되고, 결국은 막대한 비용을 들여 만들었던 개체-관계 다이어그램이 존재 가치를 상실하게 될 수도 있다.

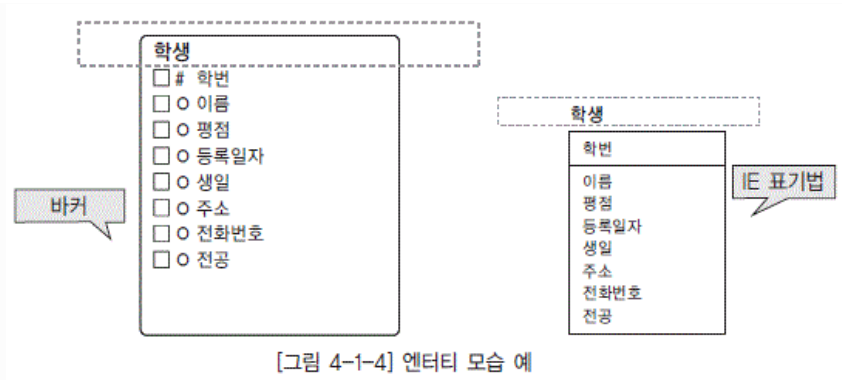
▣ 엔터티(Entity)

엔터티는 업무 활동상 지속적인 관심을 가지고 있어야 하는 대상으로서, 그 대상들 간에 동질성을 지닌 것으로 볼 수 있는 개체 집합이나 그들이 행하는 행위의 집합으로 정의할 수 있다. 동질성은 집합을 어떻게 정의하느냐에 따라 달라질 수 있는데, 집합에 들어갈 개체들의 동일한 성질을 어디까지로 한정할 것인지를 결정하는 것으로 동질성 유무를 판단할 수 있다. 예를 들면, ‘고객’이라는 집합을 ‘우리 상품을 구매한 사람이나 법인’으로 정의했다면 아직 구매를 한 적이 없는 잠재 고객이나 구매 상담자, 또 법인번호가 없는 단체나 개인 사업자 등은 이들과 동질성을 갖지 못한다. 그러나 이들이 현재 어떠한 방법으로도 관리가 되고 있거나 앞으로 관심을 갖고자 하는 범주에 해당한다면 집합의 정의를 더 확장해야 만 이들을 고객 집합 내에 끌어들일 수 있다. 집합에 대한 정의 문제는 엔터티의 구성 속성과 관계 정의에도 영향을 미치고, 이것은 결과적으로 데이터 모델 전체의 구성에 영향을 미치게 되므로 엔터티, 즉 집합에 대한 명확한 정의는 데이터 모델링에서 가장 핵심적인 사안이라 할 수 있다. 그러므로 엔터티를 정의할 때는 어떤 대상이 그 엔터티에 속하는지 여부를 명확하게 구분할 수 있도록 정의해야 한다.

엄밀한 의미에서 엔터티는 동질성을 갖는 개별 개체나 행위를 지칭하며, 달리 엔터티 인스턴스(Entity Instance) 라고도 한다. 그리고 이들의 집합을 엔터티 타입(Entity Type)이라고 부른다. 동질성을 갖는다는 의미는 어떤 한 개체(엔터티 인스턴스)를 설명하거나 묘사할 수 있는 특성들(예컨대, 학번, 이름, 학과 등과 같은 항목들)에 대해 다른 여러 개체들이 동일한 특성들로 설명 또는 묘사될 수 있음을 말하며, 이와 같은 개체들의 집합을 엔터티 타입이라고 한다. 예를 들면, 어떤 특정한 학번, 이름, 학과로 특징지어질 수 있는 ‘홍길동’ 학생은 엔터티 인스턴스이고, 학번, 이름, 학과 등과 같은 공통적인 특성 항목들을 공유하는 학생들의 집합인 ‘학생’은 엔터티 타입이 된다. 엔터티와 엔터티 타입은 각각 ‘실체’와 ‘실체형’으로 번역되어 사용되기도 하나, 일반적으로 엔터티와 엔터티 타입으로 사용하기로 하며, 통상적으로 많은 책들과 실무상에서 엔터티와 엔터티 타입은 엄격하게 구분하지 않고 엔터티로 줄여서 사용하고 있기 때문에, 본 과목에서는 개체나 행위의 집합을 엔터티로 통칭할 것이다.

엔터티는 그 집합에 속하는 개체들의 특성을 설명할 수 있는 속성(Attribute)을 갖는데, 예를 들어 ‘학생’이라는 개체 집합은 학번, 이름, 이수학점, 등록일자, 생일, 주소, 전화번호, 전공 등의 속성으로 특징지어질 수 있다. 이러한 속성 가운데에는 개체 집합 전체가 공유할 수 있는 공통 속성도 있고, 개체 집합 중 일부에만 해당하는 개별 속성도 있을 수 있다.

개체-관계 다이어그램을 표기하는 방법은 여러 가지가 있는데, 그것들에 대해서는 뒤에서 설명하기로 하고, 여기서는 그 중 한가지만 리처드 바커(Richard Barker)가 제안한 바커 표기법(Barker Notation)과 정보공학표기법(IE Notation)에 따른 표현을 예로 설명한다. 엔터티를 개체-관계 다이어그램으로 나타낼 때는 [그림 4-1-4]와 같은 모양으로 나타내고 이름을 부여할 수 있다. [그림 4-1-4]에서 좌측은 바커 표기법을, 우측은 정보공학표기법(이하 IE 표기법이라 지칭)을 적용한 엔터티 모습의 예이다.

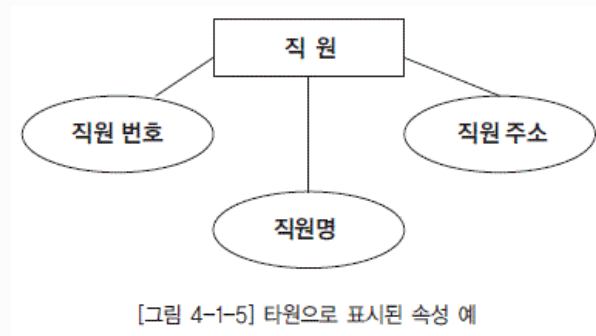


[그림 4-1-4] 엔티티 모습 예

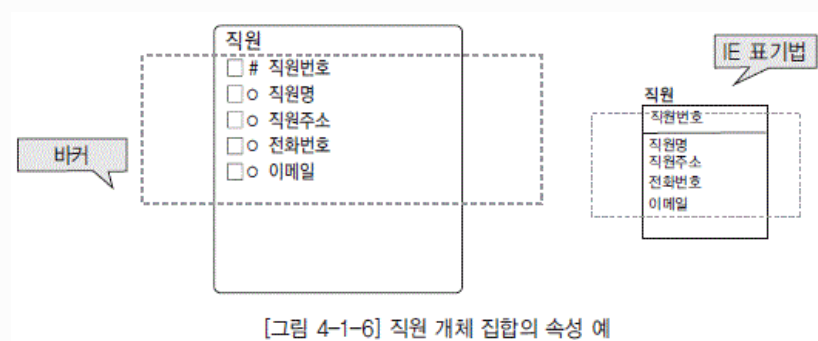
□ 속성(Attribute)

속성은 엔티티에 저장되는 개체 집합의 특성을 설명하는 항목이라고 할 수 있다. 엔티티를 명확하고 구체적으로 정의했다 하더라도 이것만으로는 개체 집합의 특성을 설명하기에는 부족함이 있다. 예를 들면, '직원' 엔티티의 개체인 '김철수'와 '홍길동'은 독립적인 사람 개체임에는 분명하지만 이들의 특성을 설명할 수 있는 보다 구체적인 항목(속성)이 없으면 이 집합을 명쾌하게 객관화할 수 없다. '직원' 집합의 특성을 설명하기 위해 직원번호, 직원명, 주민등록번호, 연락처번호, 거주지, 주소 등과 같은 속성을 정의했다면 이제 이러한 속성 구성을 통해 어떻게 '홍길동'이라는 개체를 특징짓고 변별할 것인지를 분명하게 이해할 수 있고, 집합의 의미 또한 좀 더 명확해진다고 할 수 있다. [그림 4-1-5]와 [그림 4-1-6]에서 엔티티의 속성을 표현하는 서로 다른 방법을 보여 주고 있다. [그림 4-1-5]는 엔티티에 연결되는 속성을 타원으로 보여 주고 있다. 이 스타일은 데이터 모델링 소프트웨어 제품이 출현하기 이전에 원래의 개체-관계 모델에서 사용되었다. [그림 4-1-6]은 현재의 데이터 모델링 소프트웨어 제품에서 보편적으로 사용되고 있는 바커 표기법과 IE 표기법의 스타일을 보여 주고 있다.

[그림 4-1-5]의 개체-관계 다이어그램에서 속성은 개체 집합을 나타내는 직사각형에 실선으로 연결된 타원형으로 표현되며, 각 타원형은 고유의 속성 이름을 갖는다. 또한 각 속성은 가질 수 있는 값의 범위가 있는데, 이를 그 속성의 도메인(Domain)이라 한다. 예를 들면 학생이라는 엔티티가 있을 때 학점이라는 속성의 도메인은 0.0에서 4.0 사이의 실수 값이며, 주소라는 속성은 길이가 20자리 이내의 문자열로 정의할 수 있다. 여기서 물론 각 속성은 도메인 이외의 값을 갖지 못한다.



[그림 4-1-5] 타원으로 표시된 속성 예



[그림 4-1-6] 직원 개체 집합의 속성 예

일반적으로 서로 다른 개체 집합에 정의된 속성은 같은 도메인을 공유할 수 있다. 예를 들면, 학생 개체 집합에서 주소 속성에 해당하는 도메인과 교수 개체 집합에서의 주소 속성 도메인은 같은 값들의 범위를 가질 수 있다. 개체 집합 내에서 각각의 개체를 식별할 수 있도록 하나 또는 그 이상의 속성들로 이루어진 속성 집합을 식별자(Unique Identifier)라고 하는데, 이 속성들이 개체 집합 내의 각 개체를 유일하게 지정한다. 예를 들면 학번, 성명, 주소, 생년월일, 학과 등의 속성들로 이루어진 학생 엔티티에서 학번 속성은 하나의 식별자가 될 수 있다.

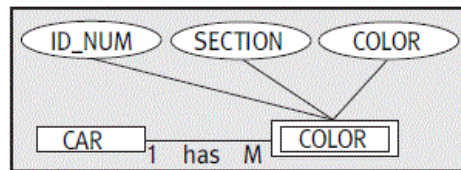
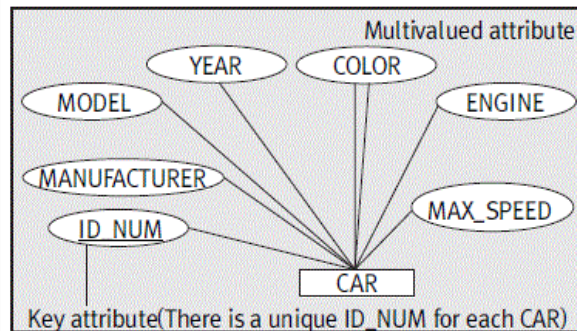
[그림 4-1-7]은 자동차 개체 집합의 속성 구성을 나타낸 것이며, 'ID_NUM'이라는 속성에 밑줄을 친 것은 식별자를 의미한다. [그림 4-1-6]과 같은 표현에서는 '#' 표시가 붙은 속성이 식별자이다. 속성은 단순형 혹은 복합형으로 분류할 수 있다. 예를 들면, 주소 속성은 시, 구, 동, 번지 등과 같은 여러 세부 속성들로 구성될 수 있는데, 이를 복합 속성(Composite Attribute)이라 한다. 또한 나이, 성별 등의 속성은 더 이상 다른 속성들로 구성될 수 없는 단순한 속성이므로 단순 속성(Simple Attribute)이라 한다. 속성은 관리 목적이나 상세화 정도에 따라서 단일 값(Single Value) 또는 다중 값(M

ulti Value)을 가질 수 있다. 주민등록번호와 같은 속성은 반드시 하나의 값만 존재하므로 이 속성은 단일치 속성(Single-Valued Attribute)이라 하고, 어떤 사람 전화번호와 같이 여러 개의 값을 가질 수 있다. 자동차의 색상 속성도 차 지붕, 차 체, 외부의 색이 다를 수 있다. 이런 속성을 다중치 속성(Multi-Valued Attribute)이라 한다.

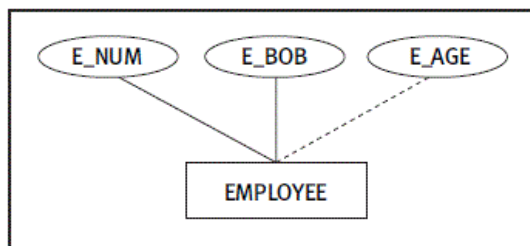
[그림 4-1-7]과 같은 표현에서 다중치 속성은 두 개의 실선으로 표시한다. 개체-관계 모델에서는 복합 속성, 다중치 속성을 정의할 수 있지만, 직접 구현은 불가능할 수도 있다. 이를 위해서는 M:M 관계나 다수의 속성 또는 1:M 관계의 추가 엔터티를 사용하여 수정해 주어야 한다. 만약 다중치 속성이 존재하면 설계자는 아래의 방법 중 하나를 선택하여 해결할 수 있다.

1) 엔터티 내에서 다중치 속성을 여러 개의 새로운 속성으로 나눈다. [그림 4-1-7]에서의 'COLOR' 속성에 대해 [그림 4-1-8]과 같은 새로운 속성을 만들어 [그림 4-1-7]에서 'CAR' 개체형의 속성들로 부착시킨다.

2) 다중치 속성을 구성하는 속성들로 구성된 새로운 엔터티를 만든다. 이때 새로운 엔터티는 원래 엔터티와 M:1 관계를 맺게 한다. [그림 4-1-8]의 'COLOR' 속성을 떼어내어 새로운 엔터티로 만들면 새로운 엔터티 'COLOR'는 'CAR' 개체형과 M:1 관계가 될 것이다. 속성 중에는 유도 속성 (Derived Attribute)으로 분류될 수 있는 것도 있는데, 이 속성은 다른 속성의 값으로부터 어떤 계산을 통해 새로운 값을 얻게 되므로 집합의 본질이나 특성을 규명하기 위한 속성을 고려할 때 제외할 수도 있다. 예를 들면, 어떤 사람의 나이 속성은 현재 날짜로부터 생년월일 속성의 값을 뺄으로써 나이의 값을 유도할 수 있다. [그림 4-1-9]에서와 같은 표현에서 유도 속성은 점선으로 표시된다.



[그림 4-1-8] 다중치 속성의 구성 속성들로 만들어진 새로운 개체형



[그림 4-1-9] 유도 속성

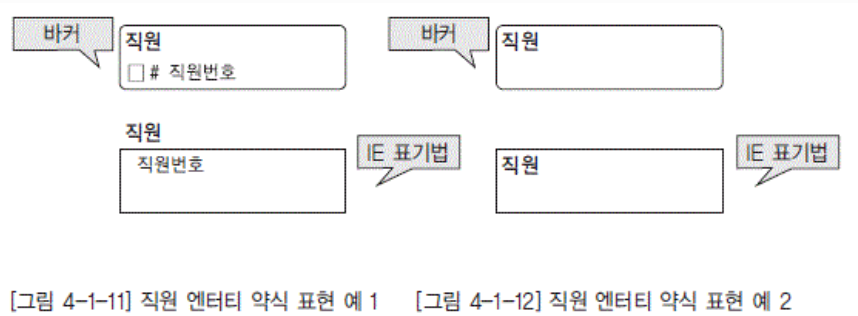
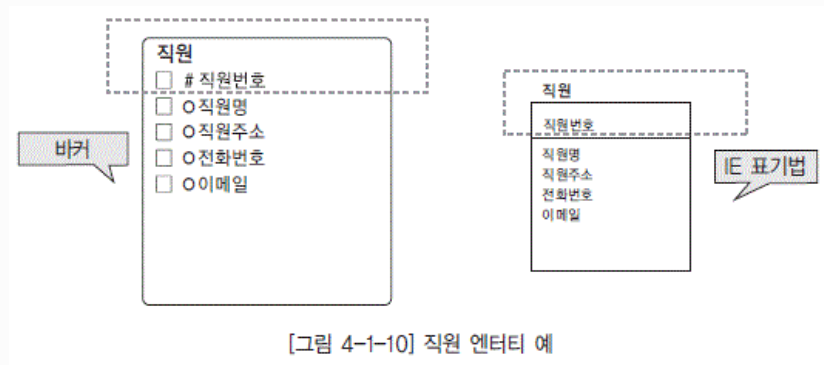
▣ 식별자

엔터티의 각 개체들은 인스턴스라고 하는데, 인스턴스는 그들을 지칭하거나 식별해 주는 속성인 식별자(Unique Identifier)를 가지고 있다. 예를 들어, 직원 인스턴스는 직원번호, 주민번호, 직원명으로 식별될 수 있다. 이들 중 직원명은 실제 계에서는 직원 각각을 식별하는 데 이용될 수 있으나 데이터 측면에서는 동명이인의 문제 때문에 식별성이 미약한 편이다. 봉급이나 입사일 같은 속성으로는 각각의 인스턴스를 유일하게 식별할 수 없다. 이와 마찬가지로 고객은 고객번호나 고객이름으로 식별될 수 있고, SALES-ORDER는 OrderNumber(주문번호)에 의해서 구별될 수 있다.

식별자는 하나 또는 그 이상의 속성으로 구성된다. 특히 두 개나 그 이상의 속성으로 이루어진 식별자를 복합 식별자(Composite Identifier)라 부른다. 그 예로는 (Area Code, Local Number), (Project Name, Task Name), (First Name, Last Name, Date Of Hire) 등이 있다. 식별자와 키는 구별하여 사용되어야 하며, 이 둘은 서로 일치할 수도 있고 그렇지 않을 수도 있다. 즉 식별자는 논리적인 관점에서 사용되고 키(Key)는 물리적인 관점에서 사용된다. 따라서 엔터티는 식별자를 가지며, 테이블은 키를 가진다고 할 수 있다. 이러한 이유로서 식별자가 엔터티에 대해 하는 역할은 키가 테이블에 대해 하는 역할과 동일하다.

엔터티는 데이터 모델에서 3단계의 상세 수준으로 표현된다. 때로는 엔터티와 그것이 가지는 속성 모두가 표시되기도 한다. 이 경우 [그림 4-1-10]에 나와 있는 것과 같이 식별자 속성을 표현하기 위해 바커 표기법에서는 #이 식별자 속성 앞에 그려지고, IE 표기법에서는 엔터티 박스 내부의 상단에 식별자 속성을 표시한다. 데이터 모델에서는 모든 속성을

자세히 표현하면 개체-관계 다이어그램을 다루기가 힘들어진다. 이 경우에는 [그림 4-1-11]에서처럼 식별자만 보이거나 [그림 4-1-12]에서처럼 엔터티 박스 안에 엔터티 이름만 간결하게 보일 수도 있다.



데이터 모델링의 초기 단계에서부터 모델을 지나치게 상세히 표현하면 개체-관계 다이어그램을 이해하고 다루기가 힘들어질 수도 있기 때문에 [그림 4-1-11]에서처럼 간단하게 식별자만 보이거나 [그림 4-1-12]에서처럼 직사각형 안에 개체 이름만 간결하게 보임으로써 많은 모델들 중에서 사용자가 필요한 것들을 손쉽게 식별하게 하기도 한다. [그림 4-1-11]과 [그림 4-1-12]에서 위는 바커 표기법을 사용한 엔터티의 약식 표현이며, 아래는 IE 표기법을 사용한 엔터티의 약식 표현이다. 세가지 기법 모두가 실제로 사용되며, [그림 4-1-12]의 더 간결한 형식은 상위 수준의 개요와 전반적인 개체-관계를 보여주는 데 사용된다.

▣ 관계(Relationship)

관계는 엔터티와 엔터티 간 연관성을 표현하는 것으로, 엔터티의 정의에 따라 영향을 받기도 하고, 속성 정의 및 관계 정의에 따라서도 다양하게 변할 수 있다. 엔터티 간에 논리적으로 존재할 수 있는 수많은 관계 중에서 정말로 의미가 있고 관리할 가치가 있는 관계를 식별해 낸다는 것이 쉬운 일은 아니다. 최초의 개체-관계 다이어그램에서 관계는 속성을 가질 수 있었으나 현재는 그렇지 않다. 관계의 표현에는 이항 관계(Binary Relationship), 삼항 관계(Ternary Relationship), n항 관계가 존재할 수 있는데, 실제 삼항 관계 이상은 잘 사용되지 않는다. [그림 4-1-13] 참조 매핑 카디널리티(Mapping Cardinality)는 개체-관계 다이어그램에서 개체와 연결될 때 나타나는 대응(Mapping)되는 수로 대응수라고도 한다. 대응수는 최대 대응수(Maximum Cardinality)와 최소 대응수(Minimum Cardinality)로 구분된다. 매핑 카디널리티가 포함된 개체-관계 다이어그램에 대하여 예제를 통해 설명하고자 한다. [그림 4-1-13] 참조

다음과 같은 조건하에서 교수 개체와 학생 개체 간에 성립하는 지도 관계의 매핑 카디널리티를 결정하고, 개체-관계 다이어그램을 작성하면 [그림 4-1-13]과 같은 이항 관계 구조의 다이어그램을 완성할 수 있다.

■ 조건 1 : 교수는 꼭 학생에 대한 지도를 해야 한다.

■ 조건 2 : 교수는 여러 명의 학생을 지도할 수 있다.

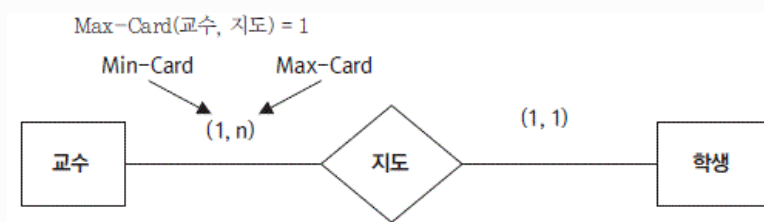
Max-Card(교수, 지도) = n

■ 조건 3 : 학생은 꼭 교수에게 지도를 받아야 한다.

Min-bCard(교수, 지도) = 1

■ 조건 4 : 학생은 여러 명의 교수에게 지도를 받을 수 없다.

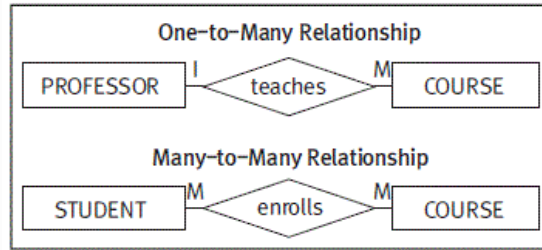
Max-Card(교수, 지도) = 1



[그림 4-1-13] 이항 관계 예

▣ 카디널리티(Cardinality)

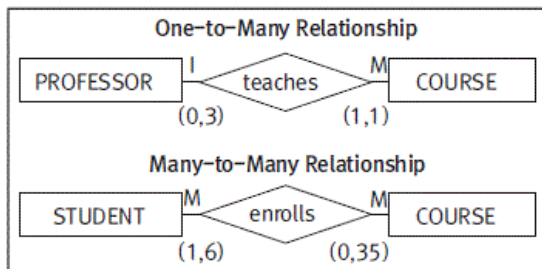
개체-관계 다이어그램은 데이터베이스가 지켜야 할 제약 조건들을 명시할 수 있다. 중요한 제약 조건 중의 하나로 연결(Connectivity)이라는 것이 있는데, 이는 한 개체가 관계를 통하여 다른 개체와 관련된 개체들의 수를 나타낸다. 관계의 연결은 1:1, 1:M, M:M으로 분류된다. 개체-관계 다이어그램에서는 관계의 연결을 1, M을 사용하여 표현하며 [그림 4-1-14]의 예와 같다. 표기법에 따라서는 M:M을 M:N으로 표현하기도 하지만 여기서는 M:M으로 나타내기로 한다.



[그림 4-1-14] 관계 연결

- 일대일(One To One, 1:1) : X에 속하는 한 개체는 Y에 속하는 한 개체에만 연결되며, Y에 속하는 한 개체도 X에 속하는 한 개체에만 연결된다.
- 일대다(One To Many, 1:M) : X에 속하는 한 개체는 Y에 속하는 한 개체에만 연결되며, Y에 속하는 한 개체는 X에 속하는 여러 개체와 연결된다
- 다대다(Many To Many, M:M) : X에 속하는 한 개체는 Y에 속하는 여러 개체와 연결될 수 있으며, Y에 속하는 한 개체도 X에 속하는 여러 개체와 연결될 수 있다.

또한 카디날리티(Cardinality)란 관계에 참여하는 하나의 개체에 대해 다른 엔터티에서 몇 개의 개체가 참여하는지를 나타낸다. 예를 들면, 한 명의 학생이 1개 이상 6개 이하의 과목에 등록할 수 있다면 카디날리티는 (1, 6)이다. 한 명의 교수가 최대 3개의 과목을 가르칠 수 있다면 카디날리티는 (0, 3)이다. 카디날리티는 (Min, Max)의 값 한 쌍으로 표현하는데, 여기서 Min은 관계에 참여하는 개체의 최소 개수, Max는 관계에 참여하는 최대 개수를 각각 의미한다. 여기서 Max의 값이 M으로 표시되면 최대 개수에 제한이 없음을 의미한다. [그림 4-1-15]에서 PROFESSOR-COURSE의 관계를 살펴보면 다음과 같이 해석할 수 있다.



[그림 4-1-15] 개체-관계 다이어그램에서의 카디날리티 예

- 1) PROFESSOR-COURSE는 1:M 관계이다.
- 2) PROFESSOR의 카디날리티는 (0, 3) 이다. 즉 각 교수는 3개 이하의 과목을 가르칠 수 있다.
- 3) COURSE의 카디날리티는 (1, 1)이다. 즉 각 과목을 가르치는 교수는 반드시 1명이어야 한다.

□ 존재 종속

만약 한 엔터티의 존재가 다른 엔터티(들)의 존재에 영향을 받는다면 이를 존재 종속(Existence-Dependent)이라 한다. 예를 들어, 어느 보험회사가 J. H. Callifante라는 사람과 그의 부양 가족들에게 보험 혜택을 준다고 할 때 EMPLOYEE, DEPENDENT라는 엔터티들을 정의한다고 하자. J. H.Callifante에게 Jill, Annelise, Jorge라는 3명의 부양 가족이 있다면 부양 가족 3명은 J. H.Callifante 없이는 보험 혜택을 받을 수 없다. 다시 말해 부양 가족 3명의 정보가 DEPENDENT라는 테이블에 존재하지만 EMPLOYEE와 연관지어 있을 때만 존재하게 된다. 이러한 것을 존재 종속이라 한다. 만약 J. H. Callifante가 직장을 그만 두어 테이블에서 없어지게 되면 부양 가족 3명도 함께 없어지게 된다. 여기서 DEPENDENT의 식별자는 EMPLOYEE의 키인 E_NUM과 DEP_NUM으로 구성된다. 두 개의 테이블들을 연결시켜 보면 J. H. Callifante는 3명의 부양 가족이 있고, W.K. Smithson은 부양 가족이 없음을 알 수 있다.

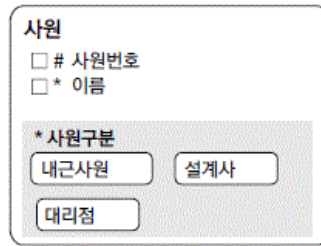
TABLE : EMPLOYEE		TABLE : DEPENDENT		
E_NUM	E_NAME	E_NUM	DEP_NUM	E_NUM
21234	J.H.Callifante	21234	1	Jill
21458	W.K.Smithson	21234	2	Annelise
18791	H.H.Washington	21234	3	Jorge
		18791	1	Robert

[그림 4-1-16] 존재 종속 예

□ 서브타입

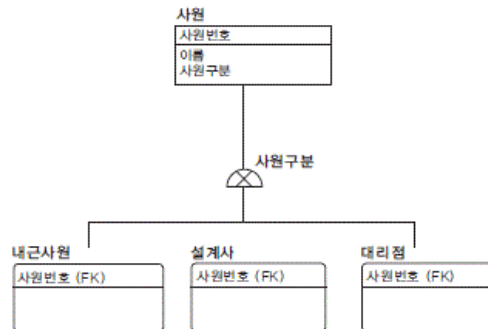
확장된 개체-관계 다이어그램은 서브타입의 개념을 도입했다. 서브타입은 정확한 의미로는 서브 타입 엔터티이며 그것

의 전체집합인 슈퍼타입(Supertype)의 부분집합이다. 예를 들어 학생들은 학 부학생이나 대학원생으로 분류될 수 있다. 이 경우 학생이 슈퍼타입이고 학부학생과 대학원생은 서브타입이다.



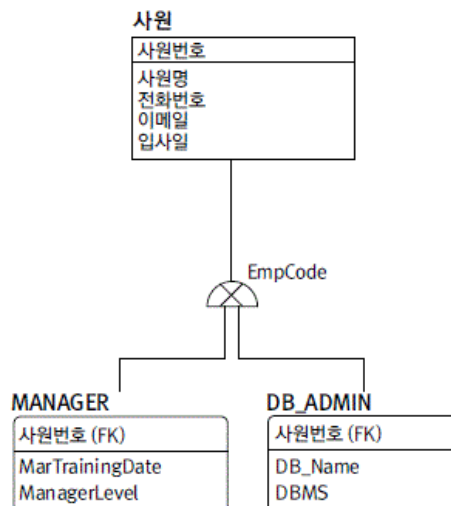
[그림 4-1-17] 서브타입과 서브타입 구분자 예(바커 표기법)

[그림 4-1-17]은 바커 표기법에 따른 서브타입 표현으로, 슈퍼타입인 사원은 서브타입인 내근사원, 설계사, 대리점을 가지고 있다. 슈퍼타입은 세 서브타입에 공통적인 모든 속성을 포함하고 있고, 서브타입은 각 서브타입에 적절한 속성만을 포함하고 있다. [그림 4-1-17]에서 속성 '사원구분'은 내근사원, 설계사, 대리점인지를 가리킨다. 어떤 서브타입이 적합한지를 결정해 주는 속성을 구분자(Discriminator)라고 부른다. IE 표기법을 사용하는 툴을 사용하면 [그림 4-1-18]과 같이 구분자는 서브타입 부호 바로 옆에 나타난다.

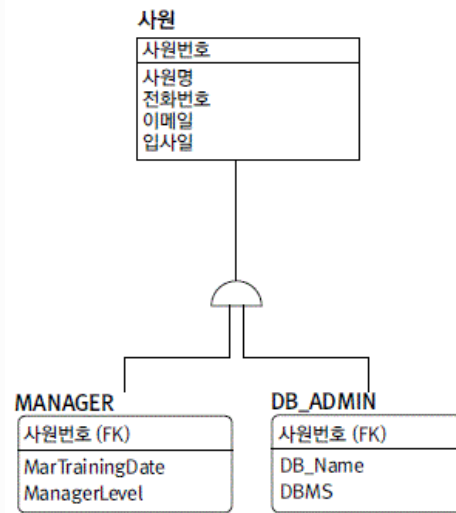


[그림 4-1-18] 서브타입과 서브타입 구분자 예(IE 표기법)

모든 슈퍼타입이 구분자를 가지고 있는 것은 아니다. 만일 구분자가 없다면 적합한 서브타입을 생성하기 위해 응용 코드가 작성되어야 한다. 서브타입은 배타적(Exclusive) 또는 포괄적(Inclusive)일 수 있다. 만약 배타적이라면 슈퍼타입은 많아야 1개의 서브타입과 관련될 수 있다. 만약 포괄적이라면 슈퍼타입은 1개 또는 그 이상의 서브타입과 관련될 수 있다. [그림 4-1-19]는 IE 표기법에 따라 MANAGER와 DB_ADMIN을 서브타입으로 가진 사원 슈퍼타입의 형태를 표현한 사례이다. 둠 안의 X는 서브타입이 배타적인 것을 의미한다. 따라서 사원은 MANAGER나 DB_ADMIN 중 하나가 될 수는 있으나 둘 다 될 수는 없다. 이에 대한 구분은 서브타입을 의미하는 둠 옆에 서브타입 구분자를 표시하여 이 서브타입 구분자 속성을 통해 사원이 어느 서브타입에 속하는지 알 수 있음을 표현하고 있다. [그림 4-1-20]은 이러한 동일한 서브타입을 포괄적으로 보여주고 있다. 여기서 사원 슈퍼타입은 MANAGER, DB_ADMIN, 또는 양쪽 모두 될 수도 있는 포괄적 서브타입의 형태로 표현되어 있으며, X가 없는 둠 형태를 사용하여 두 개 이상의 서브타입과 관련된 포괄적 서브타입을 표현한다. 여기서 둠 옆에 서브타입 구분자를 나타내지 않음은 특정한 서브타입 구분자 속성을 가지고 있지 않음을 의미한다. 이와 같이 배타적 서브타입이나 포괄적 서브타입을 표현할 때 서브타입 구분자를 지정하지 않고 서브타입을 표현하는 것은 가능하나, 서브타입 구분자를 사용하지 않게 되면 인스턴스가 어느 유형에 해당하는지 알기 위해 주변 속성들을 확인해 보아야 하기 때문에 불가피한 경우가 아니면 효율성 측면에서 바람직하지 않다.



[그림 4-1-19] 배타적 서브타입 예



[그림 4-1-20] 포괄적 서브타입 예

▣ 객체지향 모델링(Object-Oriented Modeling)

과거, 소프트웨어 생산성 향상은 상당히 힘든 일로 여겨졌다. 구조적 기법, 방법론, 4세대 언어, 관계형 DBMS, CASE 도구는 시스템 개발상의 문제점을 없애고 비용을 절감하고 소프트웨어 개발의 품질을 향상시키기 위해 시도되었다. 그러나 대부분의 시도는 실패로 끝났다. 최근의 접근 방식 중의 하나가 재사용 코드(Reusable Code)의 개념에 초점을 둔 것이다. 이 개념은 한번 소프트웨어 루틴 (Routine)을 저장해 놓고 그것을 모든 프로그래머들과 공유하는 것을 말하는 것이다. 많은 노력 끝에 방법론과 CASE와 같은 많은 접근 방식들의 결합(Combination)은 재사용 코드에 대한 실제적인 성과로 생산성의 이익을 가져왔다. 많은 개발자들은 이 새로운 접근 방식이 향후 시스템 개발에 있어 지배적인 대세가 될 수 있다고 확신하고 있다. 만약 이것이 성공한다면 객체지향 접근 방식은 애플리케이션이 어떻게 개발되어야 하는지를 과감하게 변화시킬 수 있을 것이다. 객체지향 개발은 전체 시스템 개발 영역을 다루고 있다. 그것의 종류는 아래와 같다.

- 객체지향 방법론
- 객체지향 분석
- 객체지향 설계
- 객체지향 모델링
- 객체지향 프로그래밍
- 객체지향 DBMS

객체지향 개발이 시작된다면 그것은 시스템 개발 생명 주기의 많은 부분(논리적 데이터 모델링을 포함해서)에 영향을 미칠 것이다. 객체지향 모델링의 간단한 개요를 제시하고 그것이 어떻게 논리적 데이터 모델링의 스키마에 적합하게 되는지를 살펴보자.

▣ 객체지향 개념

객체지향 모델링은 객체지향 개발의 기본이다. 여기에서 객체지향에 대한 특성과 개념을 발견할 수 있을 것이다. 우선 객체지향의 용어를 검토해 보자. 조직이나 시스템은 객체(Object)라는 것에 관련되어 있고 객체에 대한 정보를 저장한다. 고객, 직원, 교실, 책, 그리고 전표 등이 객체의 예이다. 객체는 대개 객체를 기술하는 데이터와 그 기술 데이터를 운영하는 메소드(Method)로 구성된다. 속성 유형과 메소드를 공유하는 객체가 그룹화되어서 객체 클래스(또는 클래스)로 된다. 객체 인스턴스는 객체 클래스의 어커런스이다. 예를 들어, 직원이 객체 클래스이면 '홍길동'은 클래스 직원의 객체 인스턴스이다.

객체는 속성과 메소드로 구성된다. 속성은 객체클래스의 성질이다. 이름, 주문번호, Color, 그리고 책제목 등은 속성의 예이다. 속성값이 바로 객체 인스턴스의 성질인 것이다. 메소드는 하나 이상의 속성을 접근, 조작, 수정, 삭제, 생성하는 프로세스이다. 예를 들어 '신규조직을 추가하라' 또는 '주문서를 작성하라' 등이 그 예이다.

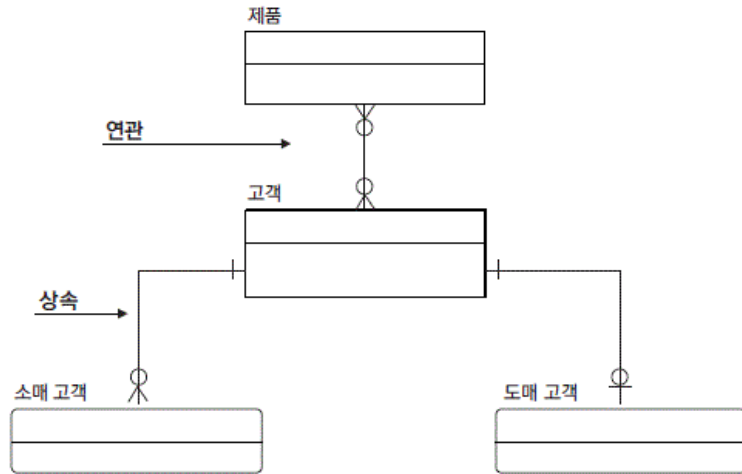
객체는 연관(Association) 또는 상속(Inheritance)을 통해서 다른 객체들에 연결된다. 연관은 객체 간의 자연적 관계이다. 비??다. 예를 들어, 고객은 제품을 산다. 그래서 고객 객체와 제품 객체 간의 자연적 관계인 '산다'가 있다. 연관은 카디날리티와 모달리티(선택적 그리고 의무적)를 가질 수 있고 연관은 하나, 둘, 셋 이상 객체들 간에 존재할 수 있다.(즉 일항, 이항 그리고 다항)

반적인 객체(상위클래스)를 가지고 제일 아래 (Hierarchy) 조직을 생각해 보면 하위 클래스는 상위 클래스로부터 속성과 메소드를 상속한다. 예를 들어 도매 고객 객체와 소매 고객 객체는 상위 클래스 '고객'의 하위 클래스이다. 이 하위 클래스는 고객 클래스로부터 모든 속성과 메소드를 상속한다. 게다가 하위 클래스는 하위 클래스 고유의 속성과 메소드를 가질 수 있으며 다른 객체들에게 상위클래스로 착는 것은 그 객체 외부에서 일어나는 것과는 관계가 없다는 것과 같은 구체적 경계 (Boundary)를 가지고 있다. 이것을 일명 캡슐화(Encapsulation)라고 한다. 예를 들어 고객 객체는 '신규 고객을 추가하라'라는 메소드를 가지고 있다. 고객 객체는 그 메소드가 호출될(Invoke) 때 무엇을 해야 하는지를 정확히 알고 있다. 그러나 제품 객체에 적용될 때에 그 메소드는 의미가 없다.

고객 객체의 속성과 메소드는 다른 객체들로부터 감추어져 있다. 본질적으로 그 객체는 적절한 메 시지가 전달될 때 작업을 수행하는 ‘블랙박스’이다. 그 작업이 무엇이고 그 작업이 어떻게 수행되는 지는 다른 객체에게 알려지지 않는다. 객체는 메시지를 주고 받음으로써 다른 객체와 통신한다. 예를 들어 구매 객체는 고객 객체에게 고객의 신용 상태를 변화시키라는 메시지를 보낼 수 있다. 캡슐화 때문에 구매 객체는 고객 객체가 신용 상태를 어떻게 변화시키는지 알 필요가 없다. 즉, 메시지 ‘고객 신용 상태를 변화시켜라’를 받은 결과로서 고객 객체가 사용하거나 수정하려는 메소드와 속성은 구매 객체에게 전달된다. 이러한 정보는 객체 모형에 표시될 수 있다. [그림 4-1-21]과 같이 객체 모형에서 객체는 굵은 선으로 된 상자(Bold-lined Box)로 표시된다. 연관과 상속의 연결은 두꺼운 선으로 나타난다. [그림 4-1-22] 참조

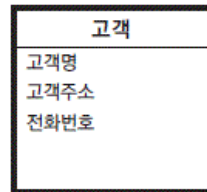


[그림 4-1-21] 객체 다이어그램 규약

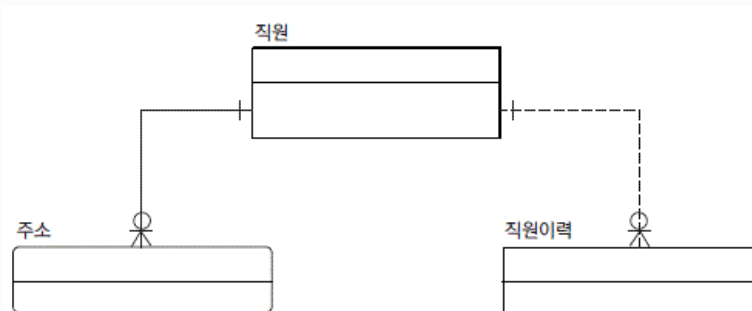


[그림 4-1-22] 연관과 상속의 다이어그램 규약 예

속성이 적거나 별도 문서에 있다면 속성은 [그림 4-1-23]과 같이 객체 상자 안에 기입될 수도 있다. 객체 안의 속성이 속성 자신의 구조를 가질 수 있다면 그 객체는 자기 자신의 논리적 데이터 모델을 가질 수 있다. [그림 4-1-24] 참조

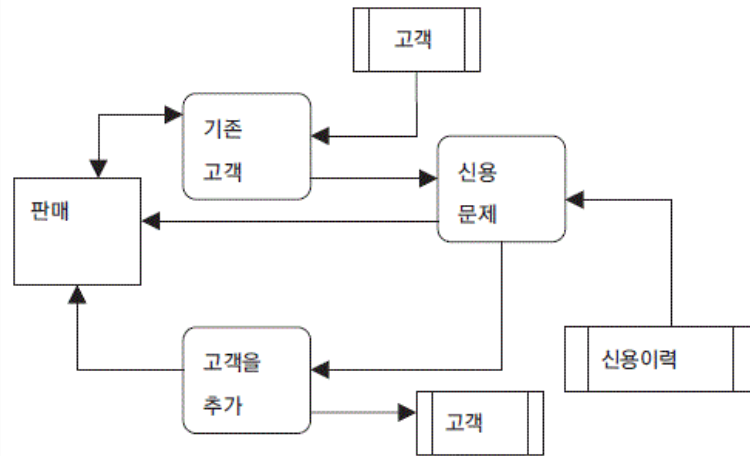


[그림 4-1-23] 속성 다이어그램의 규약



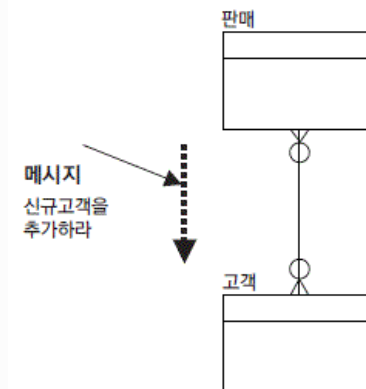
[그림 4-1-24] 하나의 객체를 위한 논리적 데이터 모형 예

대부분의 객체 메소드는 메소드를 기술하는 서술(Narrative)에 의해서 표현될 수 있다. 그러나 메소드가 복잡하다면, 데이터 흐름 다이어그램(DFD, Data Flow Diagram)과 같은 다양한 프로세스 모델링 기법 중의 하나가 사용될 수 있다. [그림 4-1-25] 참조



[그림 4-1-25] 하나의 객체를 위한 데이터 흐름 다이어그램 예

메시지는 메시지 이동의 방향을 나타내는 화살표와 점선에 의해 객체 모형에 표현된다. [그림 4-1-26] 참조



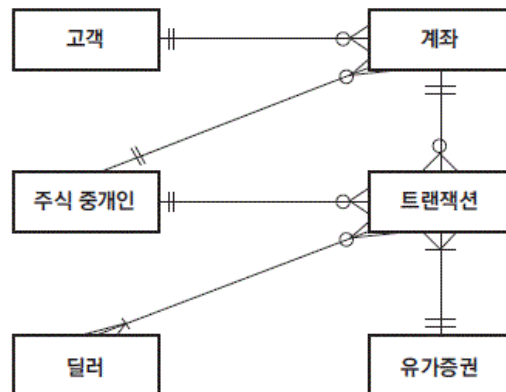
[그림 4-1-26] 메시지 다이어그램의 규약

▣ 객체 모형

객체 모형을 생성시키기 위해서 아래와 같은 방법을 취한다.

- 1) 주제에 연관된 기본(Basic) 객체를 식별한다. 그 결과는 엔터티 목록과 같은 것이다.
- 2) 객체 간의 연관(Association)을 식별한다.
- 3) 많이 알려져 있는 객체 모델링의 규약을 이용해서 객체의 다이어그램을 그린다.
- 4) 객체 간의 메시지는 다이어그램에 추가될 수 있다.

객체 모형이 완료되었을 때, 각 객체 내부에 대해서 표시하면 된다. 즉, 객체의 속성과 메소드가 문서화될 필요가 있다.



[그림 4-1-27] 객체 모형의 개체-관계 관점 예

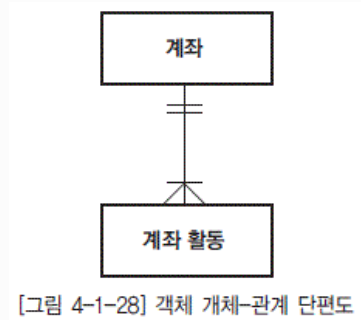
▣ 객체 모형 데이터

객체 내의 데이터를 문서화하는 것은 상대적으로 간단하다. 그 이유는 객체 내에서 발견된 속성들은 그 객체의 성질이거나 메시지 안에 전달된 다른 객체의 성질이기 때문이다. 예를 들어 [그림 5- 1-28]에서 메시지 부분으로서의 트랜잭션 객체는 트랜잭션 번호를 주식 중개인 객체에 전달한다. 두 객체의 메시지가 트랜잭션 번호를 사용한다고 할지라도 트

랜잭션 번호는 트랜잭션의 성질이지 주시 중개인의 성질은 아니다. 트랜잭션 번호는 트랜잭션 객체 안에 정의되어 있다. 그래서 주시 중개인 객체 안에서 재정의될 필요는 없다. 오직 데이터만이 한번 문서화할 필요가 있다. 그래서 다른 객체들에 속하는 데이터는 소유 객체 안에만 문서화 된다.

대부분의 경우에서 한 객체 안의 데이터는 구조를 가지지 않는다. 구조상 데이터가 개체-관계 다 이어그림에 있다면, 다양한 엔터티 내에 있을 수 있다는 것을 뜻한다. 즉 데이터 구조가 없는 객체는 모든 객체의 속성들이 개체-관계 다이어그램에서 하나의 엔터티 내에 있다는 것을 의미한다. 데이터 구조가 없을 때 객체의 데이터를 문서화하는 것은 객체의 정의와 도메인을 구체화시키는 것으로 제 한된다. 그러나 객체의 데이터가 구조를 가진다면 그 구조는 개체-관계 다이어그램의 단편도 (Fragment Diagram) 안에 모형화되어야 한다.

예를 들어 계좌 객체의 데이터가 정적인(Static) 계좌 정보뿐만 아니라 다양한 활동 항목(Activity Items)을 포함하고 있다면 분석가는 개체-관계 다이어그램 단편도에 계좌와 계좌 활동 간의 일대다 관계를 표현할 것이다. [그림 5-1-28] 참조



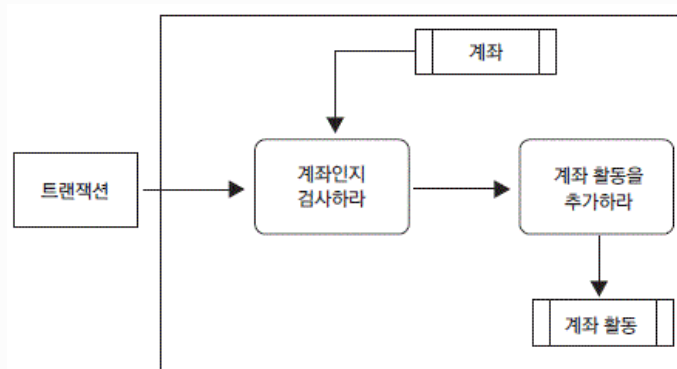
[그림 4-1-28] 객체 개체-관계 단편도

▣ 객체 모형의 메소드

객체 메소드가 복잡하지 않다면 메소드는 간단한 설명부(Narrative) 또는 구조화된 기법으로 가장 잘 문서화된다. 복잡한 메소드는 좀 더 정교한 문서화 접근 방식이 필요하다. 이벤트 지향적인 프로세스는 상태 전이 모델링(State Transition Modeling)과 같은 동적(Dynamic) 프로세스 모델링 기법이 필요하다. 반면에 정적인 프로세스는 데이터 흐름 다이어그램이나 기능 분해(Functional Decomposition)와 같은 기법을 사용한다. 계좌 객체의 계좌 갱신 메소드는 데이터 흐름 도표에 의해서 문서화된다. [그림 4-1-29] 참조

다이어그램에서 어떻게 트랜잭션 객체가 외부 엔터티로 표현되었는지, 또한 계좌 갱신 메시지가 계좌 객체에 대한 데이터 흐름이라는 점을 유의해야 한다.

다이어그램에서 어떻게 트랜잭션 객체가 외부 엔터티로 표현되었는지, 또한 계좌 갱신 메시지가 계좌한다.



[그림 4-1-29] 객체 데이터 흐름 다이어그램 예

▣ 불명료한 객체 접근 방식

객체 접근 방식에는 두 가지 주의 사항이 있다. 첫번째는 객체지향 개발의 가장 단순화된 표현이다. 이것은 객체지향 개발을 분명하게 이해하는데 필요한 많은 객체지향 특징을 무시하는 것이다. 불행히도 그러한 특징을 탐색하는 것은 설명하기가 매우 어렵다. 두번째는 객체지향 모델링의 미성숙이다. 객체가 어떻게 표현되고 기호가 어떻게 사용되어야 하는가 등의 정의는 아직까지 초기단계이다. 현재 접근 방식은 크게 변화하고 있고 많은 개발자들이 표준안을 원한다고 할지라도 이러한 상황은 당분간 계속될 것이다.

▣ 객체지향 모델링과 논리 데이터 모델링간의 관계

객체지향 모델링과 논리 데이터 모델링은 공통점이 많다. 객체지향 모델링 개념은 논리 데이터 모델링의 개념과 매우 유사하다. 그러나 많은 유사점에도 불구하고 차이점 또한 존재한다. 객체지향 모델링에서만 유일한 것은 데이터와 프로세스를 같은 엔터티로 결합시킨 것이다.

[표 4-1-2] 객체지향 모델링과 논리 데이터 모델링 간의 비교

객체지향 모델링	논리 데이터 모델링	차이점

객체	엔터티	객체는 프로세스를 포함한다.
속성	속성	-
연결 (Link)	관계 (Relationships)	유사하다. - 연관은 동일하다. - 상속은 데이터 모델링이 메소드를 포함하지 않는다는 것만 제외한다면 서브타입/슈퍼타입과 동일하다.
캡슐화	-	대응하는 논리적 데이터 모델링의 개념은 없다.
객체 클래스	엔터티 유형	없음
객체 인스턴스	엔터티 인스턴스	없음
메시지	-	메시지는 프로세스와 연관되어 있기 때문에 대응하는 개념은 없다.

논리적 기초가 잘 갖춰진 데이터 모델러는 일반 프로세스 모델러보다 객체지향 모델링 개념을 더 쉽게 소화한다. 많은 객체지향 접근 방식은 논리적 데이터 모델링 접근 방식과 유사하다. 그래서 객체지향 모델링을 논리적 데이터 모델링의 연장으로 간주하기도 한다. 다른 주요 유사성은 두 가지 접근 방식이 다음의 어려움을 공유한다는 것이다. 많은 분석가는 데이터가 프로세스에 종속하지 않는 방식으로 데이터와 프로세스의 개념을 결합한다. 이것은 객체지향 모델링에 기반한 애플리케이션을 개발하는 사람들이 논리적 데이터 모델링의 개념도 잘 이해하고 있기 때문이다. 또한 좋은 접근 방식들은 서로 결합하기 때문이다.

▣ 객체지향 모델링 장점

객체지향 모델링의 장점은 재사용 코드와 같은 개념이 실제로 가능한 환경을 제공한다는 것이다. 이것은 또 다른 문제를 해결해 준다. 데이터나 프로세스 중 어느 것도 조직의 모든 규칙을 표현할 수는 없다. 일부 규칙은 프로세스 모형에서 표현될 수 있고 일부는 데이터 모형에서 표현된다. 그러나 객체지향 모델링은 모든 비즈니스 규칙이 표현될 수 있는 유일한 환경을 제공한다. 객체지향 모델링은 또한 프로세스와 데이터 모델링을 함께 운영한다. 데이터와 프로세스가 객체의 성질인 반면 데이터와 프로세스 각각은 그들의 풍부함(Richness)을 달성하는 다른 접근 방식을 필요로 한다는 사실이 위의 사항들을 확신하게 한다.

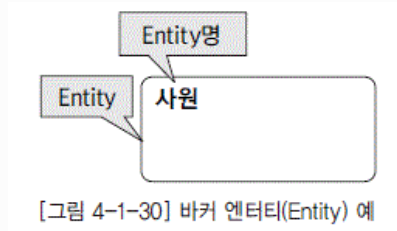
+ 바커 표기법 (Baker Notation)

바커 표기법은 영국 컨설팅 회사 CACI에 의해 처음 개발되었고 리처드 바커(Richard Barker)에 의해 지속적으로 업그레이드되었다. 오라클에서 Case Method(Custom Development Method)로 채택하여 사용하고 있다.

▣ 엔터티(Entity)

엔터티는 기업에서 지속적으로 저장하고 관리해야 할 대상이다. 하나의 관리 대상이 엔터티가 되기 위해서는 반드시 두 개 이상의 속성을 가져야 한다. 속성이 없는 실체는 존재할 수 없다. 엔터티란 실제 세상에 있는 객체(Object)이다.

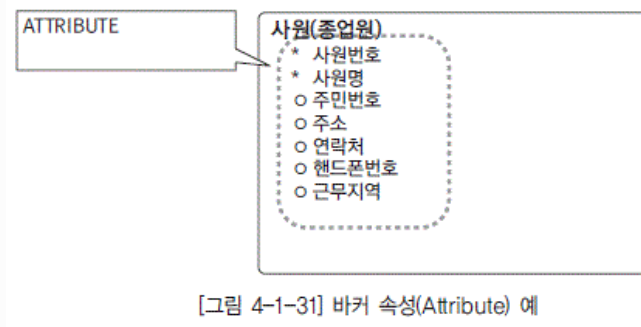
- 엔터티는 네 부분의 모서리가 둥근 형태인 소프트-박스(Soft-box)로 표현한다.
- 엔터티는 하나 이상의 속성으로 구성된다.



[그림 4-1-30] 바커 엔터티(Entity) 예

▣ 속성(Attribute)

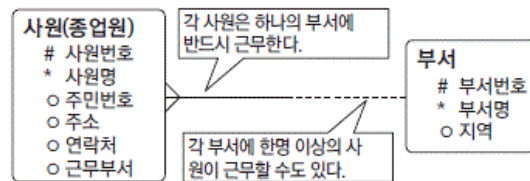
속성은 하나의 엔터티에 종속되는 명사적 단어들을 말한다. 일반적으로 명사적 단어들 중에 구성 요소를 포함하고 있는 명사들은 엔터티가 되고 그렇지 못한 명사들은 속성이 된다. 속성의 상태에는 2가지 종류가 있다. 해당 속성에 어떤 값을 반드시 저장해야 하는 경우에는 * (Mandatory)를 표시하며 해당 속성에 어떤 값이 존재할 수도 있고 존재하지 않을 수도 있는 경우에는 o (Optional)를 표시하게 된다.



[그림 4-1-31] 바커 속성(Attribute) 예

▣ 관계(Relationship)

두 개의 엔터티간에 CONDITIONAL을 표기한 후 해당 엔터티의 가까운 위치에 관계 명칭을 표기하고 관계(Relationship)는 실세계의 해당 엔터티에서 발생하는 동사적 단어들을 표기한다.



[그림 4-1-32] Barker Conditional 작성 단계 예

■ 1) 엔터티와 엔터티간의 관계

- 1:1 관계
A 엔터티에 존재하는 데이터 1개와 관계되는 B 엔터티에 존재하는 데이터의 개수도 1개인 엔터티간의 관계를 1:1 관계라고 한다.
- 1:M 관계
A 엔터티에 존재하는 데이터 1개와 관계되는 B 엔터티에 존재하는 데이터의 개수가 여러 개인 엔터티 간의 관계를 1:M의 관계라고 한다.
- M:M 관계
A 엔터티에 존재하는 데이터 1개와 관계되는 B 엔터티에 존재하는 데이터의 개수가 여러 개이며, B 엔터티에 존재하는 데이터 1개와 관계되는 A 엔터티에 존재하는 데이터의 개수도 여러 개인 엔터티 간의 관계를 M:M 관계라고

말한다.

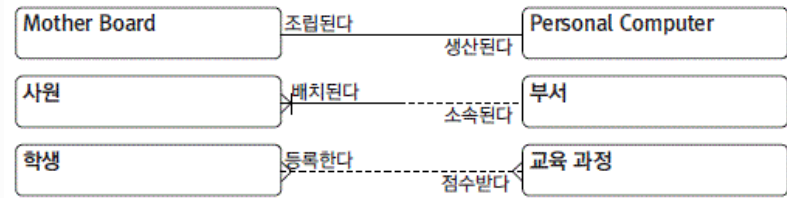
■ 2) 엔터티와 엔터티간 상관 관계의 조건

▪ 필수 조건

필수 사항은 실선으로 표시하고 상대 엔터티에 대해 해당 엔터티에 조건을 만족하는 엔터티가 반드시 존재할 경우에 표시한다.

▪ 선택 조건

선택 사항은 점선으로 표시하고 상대 엔터티에 대해 해당 엔터티에 조건을 만족하는 엔터티가 존재할 수도, 존재하지 않을 수도, 있을 경우 표시한다.



[그림 4-1-33] 바커 관계 예

□ 식별자(Unique Identifier)

식별자란 하나의 엔터티에 구성되어 있는 여러 개의 속성 중에 엔터티를 대표할 수 있는 속성을 의미하며 하나의 엔터티는 반드시 하나의 식별자가 존재해야 한다. 보통 식별자와 키(Key)를 동일시 생각하고 있는 경우가 있는데 식별자는 논리 데이터 모델링 단계에서 사용하고 키는 물리 데이터 모델링 단계에서 사용한다.

■ 1) 식별자의 유형

▪ 본질 식별자

속성들 중에서 집합의 본질을 명확하게 설명할 수 있는 의미상의 주어를 본질 식별자라한다. 의미상의 주어에는 사원번호, 상품번호처럼 집합을 식별하기 위한 임의의 유일값을 사용하는 인조 식별자(Artificial Unique Identifier)도 있고, 내가 태어나기 위해서 절대적으로 존재했어야만 하는 본질 속성들에 해당하는 것으로 자신의 고유 속성과 부모로부터 물려받은 속성(릴레이션십)들로 이루어진 것도 있을 수 있다.

▪ 후보 식별자

각 인스턴스를 유일하게 식별할 수 있는 속성 또는 속성들의 조합이며, 후보 식별자로 속성 집합을 선택하는 경우에는 개념적으로 유일해야 한다.

▪ 대체(보조) 식별자

보조 식별자란 원래의 식별자를 대신할 수 있는 또 다른 속성들이나 릴레이션십을 말한다. 가령 사원 엔터티에 공식적으로 부여된 식별자(실질식별자)는 사원번호이지만, 만약 주민등록번호 속성이 유일한 값을 가지면서 필수적(mandatory)으로 정의되었다면 비록 공식적인 식별자는 아니지만 식별자로서의 역할을 할 자격은 충분히 갖추고 있다. 특히 보조 식별자는 여러 참조 엔터티 중에서 원래의 식별자보다 보조 식별자로 연결을 맺는 것이 자신에게는 훨씬 유리한 경우에 의미가 있게 된다.

▪ 인조 식별자

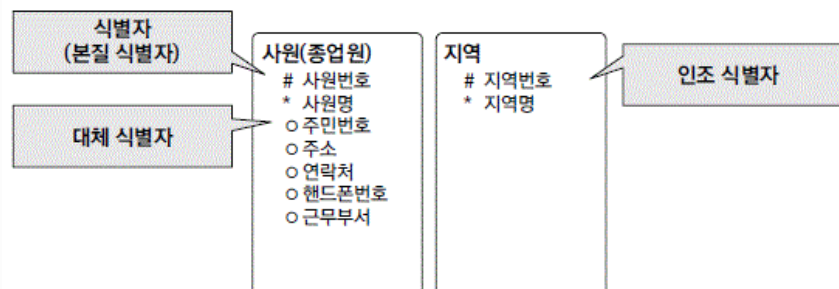
인조 식별자란 식별자 확정시 기존의 본질 식별자를 그대로 실질 식별자로 인정할 수 없는 여러 가지 상황이 발생했을 때, 전부 혹은 일부를 임의의 값을 가진 속성들로 대체하여 새롭게 구성한 식별자를 말한다. 가령, 사원 엔터티에 이미 존재하고 있는 속성 중에서 원래의 본질 식별자를 찾으라고 한다면 주민등록번호가 될 것이다. 그러나 이 속성은 너무 길고 관리상 여러 가지 문제점이 발생하기 때문에 새롭게 사원번호라는 임의의 값을 가진 인조 속성을 영입하여 공식적인 식별자 자리까지 부여받은 것이다.

▪ 실질 식별자

인스턴스를 식별하기 위해 공식적으로 부여된 식별자를 말한다. 본질 식별자나 인조 식별자 모두가 실질 식별자가 될 수 있다.

■ 2) 작성 방법

식별자 앞에는 # 기호를 표시하고 여러 개# 기호를 반복적으로 표시한다.



[그림 4-1-34] 바커 식별자(Unique identifier) 예

▣ 서브타입(Sub-type)

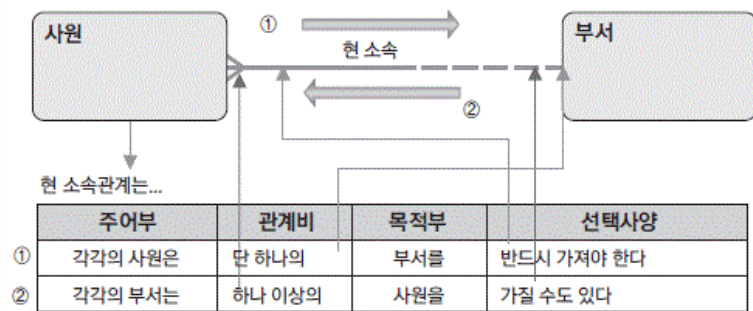
바커 표기법에서는 슈퍼타입(super-type) 안에 서브타입(sub-type)을 상자로 나타낸다. 이것은 다이어그램에서 공간을 적게 사용하는 장점을 가지고 있다. 서브타입은 서브타입의 중복을 허락하지 않는 상호 배타적 관계이다.



[그림 4-1-35] 바커 서브타입 예

▣ 관계의 표현 비교

다음은 바커 표기법과 I/E 표기법의 가장 큰 차이점인 관계의 표현 비교를 설명하겠다. [그림 4-1-36]은 사원과 부서 엔터티 간의 관계를 표현한 모습이다. 릴레이션이란 양방향 관계를 합성한 것이라 할 수 있다. 각 관계를 설명하자면 사원을 주어로 부서를 보는 관계에서 의미는 “각각의 사원은 단 하나의 부서를 반드시 가져야 한다”라고 정의할 수 있다. 또한 부서를 주어로 사원을 보는 관계의 정의는 “각각의 부서는 하나 이상의 사원을 한 주소속 부서로서 배치 받을 수도 있다”라는 의미를 표현한 것이 바로 관계선이다. 이때 관계선을 살펴보면 실선과 점선으로 구분할 수 있다. 실선은 필수를 뜻하고 점선은 필수가 아님을 뜻한다. 현재 [그림 4-1-36]의 사원 엔터티가 주어일 경우 사원 엔터티 쪽에 가까이 있는 실선이 바로 사원이 주어일 때 부서를 바라보는 관계를 설명한 것이다. 때문에 까마귀 발 모양은 이미 설명한 것처럼 하나 이상의 개체를 허용한다는 뜻이다. 이런 표시를 근거로 “각각의 사원은 단 하나의 부서를 반드시 가져야 한다”와 같이 정의할 수 있다.



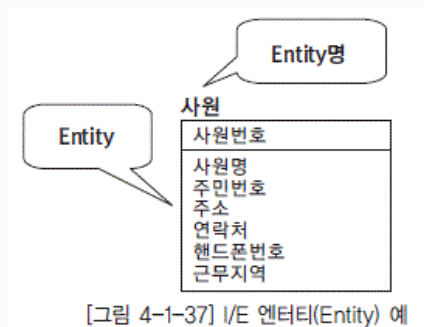
[그림 4-1-36] 바커 표기법의 관계의 표현 예

✚ I/E 표기법 (Information Engineering Notation)

Information Engineering(I/E)은 1981년에 Clive Finkelstein과 James Martin이 공동 저술로 발표하였으며, 80년대 중반에 James Martin에 의해 그 체계가 정리되면서 본격적으로 활용이 되었고, 정보시스템을 구축하는데 있어서 데이터 분석(Data Analysis)과 데이터베이스 설계(Database Design)를 위한 매우 유용한 기법으로 자리 잡게 되었다. 이 모델은 관계의 다(Many) 쪽을 나타내기 위해 까마귀 발을 사용하기 때문에 때때로 까마귀 발모델(Crow's Foot Model)이라 부른다.

▣ 엔터티(Entity)

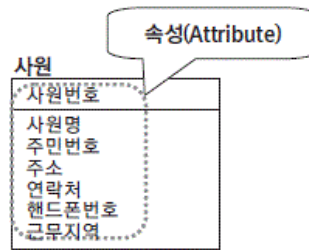
엔터티란 사용자가 추적하고자 하는 어떤 사물이다.



[그림 4-1-37] I/E 엔터티(Entity) 예

▣ 속성(Attribute)

엔터티는 엔터티의 특징을 기술해 주는 여러 개의 속성을 가진다. 속성은 [그림 4-1-38]과 같이 엔터티 안에 위치한다.



[그림 4-1-38] I/E 속성(Attribute)의 예

▣ 관계(Relationship)

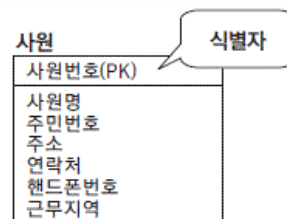
까마귀 발 부호는 관계의 다(Many) 쪽을 보여주는 데 사용되고, 타원(Oval), 해쉬 마크 및 까마귀발의 다양한 조합들은 [표 4-1-3]과 같이 사용된다.

[표 4-1-3] I/E 관계(Relationship) 예

ER win의 부호 사용	Identifying	Non-identifying	의미
타원, 해쉬 마크 및 까마귀 발	+-----<	+-----<	0,1, 또는 그 이상의 개체 허용
까마귀 발이 있는 해쉬 마크	+-----<	+-----<	1 또는 그 이상의 개체 허용
해쉬 마크가 있는 타원	+-----<	+-----<	0 또는 1 개체 허용
해쉬 마크만 있음	+-----<	+-----<	정확히 1 개체 허용

▣ 식별자(Unique Identifier)

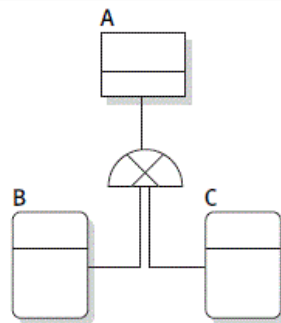
엔터티는 그들을 지칭하거나 식별해 주는 속성인 식별자를 가지고 있다. 속성의 식별자는 엔터티의 상단에 나타나며, [그림 4-1-39]와 같이 수평선이 식별자 밑에 그려진다.



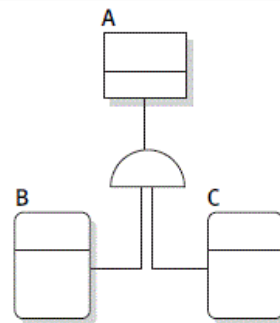
[그림 4-1-39] I/E 식별자(Unique Identifier) 예

▣ 서브타입(Sub-type)

서브타입은 배타적 또는 포괄적일 수 있다. 만일 배타적이라면 슈퍼타입은 많아야 1개의 서브타입과 관련될 수 있다. 만일 포괄적이라면 슈퍼타입은 1개 또는 그 이상의 서브타입과 관련될 수 있다. [그림 4-1-40]에서 A는 슈퍼타입, B와 C는 배타적 서브타입이다. 구분자는 보이지 않고 있다. 관계는 실선으로 그려져 있다. [그림 4-1-41]에서 A는 슈퍼타입, B와 C는 포괄적 서브타입이다. 관계는 실선으로 그려져 있다.



[그림 4-1-40] I/E 배타적 서브타입 예



[그림 4-1-41] I/E 포괄적 서브타입 예



개념 데이터 모델링

Home > DB 구축 가이드 > DA 가이드 > 데이터 모델링 > 개념 데이터 모델링

개념 데이터 모델링 이해

주제 영역 정의

후보 엔터티 선정

핵심 엔터티 정의

관계(Relationship) 정의

개념 데이터 모델 작성

+ 개념 데이터 모델 정의

개념적 데이터 모델이란 건물로 말하면 철제빔으로 건물의 골격을 세워 놓은 형태와 유사하다. 건물의 골격이 주요 골조 자재로 구성되어 있듯이 개념 데이터 모델도 주요 핵심 엔터티들로 구성된다. 핵심 엔터티란 행위의 주체나 목적물이 되는 개체 집합에 해당하는 엔터티를 의미한다. 이들은 부모가 존재하지 않는 창조된 집합이어서 다른 집합의 존재 유무에 상관없이 독립적으로 탄생할 수 있다. 핵심 엔터티는 대체적으로 여러 가지 하위의 행위 엔터티를 탄생시킨다.

+ 개념 데이터 모델 의의

개념 데이터 모델은 단지 대상을 주요 핵심 엔터티로 한정한다는 것일 뿐이지 모델링 기법은 논리적 모델링과 특별히 다를 것이 없다. 만약 우리가 하향식으로 데이터아키텍처를 수립해 간다면 개념적 모델은 개괄적 모델을 좀더 상세화된 형태로 진화시키거나 중요 엔터티만 선정하여 모델링을 함으로써 탄생된다. 물론 이후에도 모델링의 상세화는 계속되어 갈 것이며, 필연적으로 개념적 모델도 따라서 약간씩 변화가 일어날 수 밖에 없다. 하지만, 개념 데이터 모델은 앞으로 아무리 상세한 모델링이 진행되더라도 전체적인 골격은 개념적 모델을 벗어나지 않는다는 것이다.

+ 데이터아키텍처 프레임워크 상에서 개념 데이터 모델

전사아키텍처에서 개념 레이어는 최상위의 개괄 레이어와 하위의 논리 레이어 중간에 존재하는 레이어이다. 데이터아키텍처에서는 개괄 데이터 모델과 논리 데이터 모델 사이에 존재하는 모델이라고 할 수 있다. 여기에서 개괄 데이터 모델은 흔히 사용되어지는 주제 영역 관계도와도 대응되는 부분이라고 할 수 있다. 개념 데이터 모델은 상위의 주제 영역별로 핵심 엔터티와 핵심 속성, 또한 핵심 엔터티들 사이의 관계들로 이루어진 데이터 모델이라고 할 수 있다. 아키텍처의 개념을 종종 건축물에 비유하곤 한다. 개념 데이터 모델을 건축물에 비유해 보면 건물의 기둥들과 중요 벽체들로만 만들어진 건물 설계 도면에 비유할 수 있다. 이 단계에서 정의된 핵심 엔터티들과 관계들을 기반으로 논리 데이터 모델에서는 업무 요구 사항에 기반을 둔 상세 데이터 모델이 생성된다.



개념 데이터 모델링

Home > DB 구축 가이드 > DA 가이드 > 데이터 모델링 > 개념 데이터 모델링

개념 데이터 모델링 이해 주제 영역 정의 후보 엔터티 선정 핵심 엔터티 정의 관계(Relationship) 정의

개념 데이터 모델 작성

+ 주제 영역 개념

주제 영역(Subject Area)은 기업이 사용하는 데이터의 최상위 집합이다. 예를 들어, 제조 업체의 경우 인사, 생산, 자재, 판매 등의 주제 영역이 있을 수 있다. 하나의 주제 영역으로 정의되는 데이터간의 관계는 밀접하고, 다른 주제 영역에 포함되는 데이터 간의 상호작용은 최소화할 수 있도록 정의한다. 데이터는 기본적으로 관계 구조로 표현된다. 관계 구조는 데이터 간의 관계가 복수 개로 표현되면서 서로 연결되어 있기 때문에 하향식(Top-down) 분석이 용이하지 않다. 계획 수립 단계는 하향식 분석을 원칙으로 하고, 검증을 위해서 부분적으로 상향식 분석을 사용한다. 데이터를 하향식으로 분석하기 위한 개념으로 유용한 것이 주제 영역이다. 주제 영역은 계층적으로 표현될 수 있다. 주제 영역을 분해하면 하위 수준의 주제 영역이나 엔터티가 나타난다.

특히 현행 시스템을 개선하거나 현행 시스템에 대한 데이터 관리 체제를 생성하고 체계화하고자 할 경우에는 상향식 분석을 수행한다. 이 경우에는 현재 운용하고 있는 시스템에서 사용하고 있는 데이터에 대해 상위 수준에서 파악하기 위해 전사적인 관점에서의 데이터 분류를 수행하고, 이의 결과를 주제 영역(Subject Area)이라는 이름으로 관리하게 된다. 이렇게 분류된 결과는 보통 업무에서 관리하고자 하는 데이터 집합들의 그룹으로, 친밀도가 높아 동질성이 있는 데이터들로 이루어지는 것이 보통이다. 각 데이터 분류들은 일반적으로 관련된 업무 기능이 존재한다.

+ 주제 영역 분류 원칙 및 기준

▣ 주제 영역 분류 원칙

1) 데이터 중복 최소화

동일한 기능을 하는 자원(지역 및 정보)이 중복 정의되어 낭비되지 않도록 체계적인 분류 작업이 필요하다.

2) 데이터 확장성 보장

가까운 미래의 추가되어지는 정보에 대해 최대한의 확장성을 고려하여 분류 체계가 설계되어야 한다.

3) 데이터 관련성 및 편의성 확보

- 타 자원(정보 및 지역)과의 인접성을 고려해 설계한다.
- 고객 편의(정보 요건)를 고려한 자원 내의 핵심적인 데이터 집합에 대한 것을 명시한다.
- 핵심 관계에 대한 것도 명시한다.
- 필요에 따라서는 표준화된 타 영역의 설계(참조 모델)도 참조하여 데이터 분류를 생성한다.

▣ 주제 영역 분류 기준

1) 데이터 관점의 분류

- 기존의 분류 방침이 데이터와 업무 영역의 개념이 혼재되어 있었으나 본 주제 영역 분류에서는 업무를 발생시키는 주제, 대상 및 행위 등 데이터 관점에서 데이터를 생성시키고 사용하는 유형에 근거하여 분류를 수행한다.
- 데이터와 시스템/애플리케이션 간의 독립성이 계속 증가되는 추세하에서는 더욱더 장기적이고 전사적인 관점에서 데이터 유사성을 고려하여야 한다.
- 시스템이나 애플리케이션이 다르더라도 동일한 유형의 데이터를 유사한 방식으로 활용한다면 이를 동일한 영역으로 분류하여 통합된 관점에서 데이터를 관리하여야 한다.

2) 업무 요건 추가에 대한 유연성 보장

- 업무 요건의 변경이나 추가 시 유연성을 보장할 수 있도록 전사 분류 체계를 설정한다.
- 업무 요건 변경에 대해서 데이터 구조의 변경을 최소화하려면 동일한 유형의 데이터를 본질이 희석되지 않는 한도 내에서 최대한

집합으로 통합해야 한다. 이렇게 함으로써 모델의 유연성과 확장성, 융통성이 보장되어 신규, 추가 요건에 대해 기존의 분류 구조에 적절하게 수용될 수 있다.

- 만일 새로운 영역이 필요한 상황이 발생한다면 전사적인 협의를 통하여 적절한 계층의 분류 구조를 조정한다.

3) 주제 영역 간 균형 유지

- 데이터 분류는 일부분에 국한된 것이 아니고 전체적인 균형을 유지하는 것이 중요하다.
- 특정 부분을 너무 상세하게 분류하거나, 분류 방식이 타 영역과 다른 방식으로 되어 있으면 전체적인 혼란을 야기할 수 있다.
- 데이터 분류에 대한 추가나 변경이 발생할 경우 해당 부문만을 고려하여 수행하기보다는 타 영역의 분류 체계와의 형평성 및 균형을 고려하여 분류 구조를 관리한다.

□ 주제 영역 명명

1) 실 업무에서 보편적으로 사용하는 업무 용어를 부여하는 것이 바람직하다.

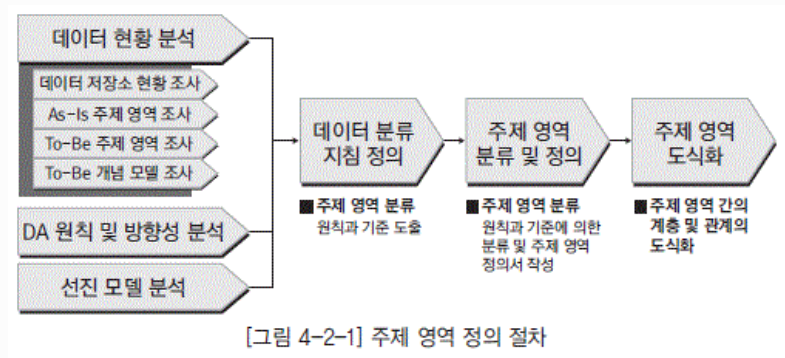
예) 인사, 생산, 판매, 구매, 재무 등

2) 유일한 단수형 명사를 사용한다.

3) 데이터의 그룹을 의미하는 이름을 부여한다.

- 업무 활동(Activity)을 의미하는 이름을 배제하고 데이터 그룹을 의미하는 이름을 사용하도록 한다.

□ 주제 영역 정의 절차



□ 주제영역 분류 방법

■ 1차 분류 : 주요 데이터 집합의 유형 정의

- 기존의 시스템별로 제공되는 데이터의 성격 및 특성을 고려하여 영역을 분류한다.
- 업무의 변화에 민감하지 않도록 정의한다.
- 분류 예시(아래 예시들을 복합적으로 적용하는 것도 가능)
 - 데이터를 발생시키는 주체로서의 분류 → 관계자, 상품/서비스, 자산, 채널 등
 - 데이터를 발생시키는 주체 간의 상호 작용으로 발생하는 대상으로서의 분류 → 계약, 리스크, 상품, 조건
 - 공통 및 관리 성격의 상위 개념으로서의 분류 → 경영 관리(정보, 방침, 지원 등)

■ 2차 분류 : Biz 활동에 필요한 데이터 분류

- Biz 활동에 필요한 분석 주제와 현황 등의 영역으로 분류한다.
- 기본(정보), 상세(정보), 관계(정보) 등과 같은 데이터의 기능적 구성 관점에서 접근하여 1차 분류를 세분화한다.
- 업무 변화 수용의 융통성을 반영하여 정의한다.
- 분류 예시
 - 관계자 : 관계자 기본, 관계자 상세, 관계자 관계, 관계자 분류 등

■ 3차 분류 : 2차 영역의 세부 주제 영역 분류

- Biz 활동에 필요한 분석 주제와 현황 등의 영역으로 분류한다.
- 사용자에게 제공되는 실제 데이터로서의 관점에 근거하여 정의한다.
- 업무적인 관점에서 분류한다.
- 분류 예시
 - 관계자(기본) : 고객, 법인, 조직, 직원 등
 - 계약(기본) : 수신계약, 예금계약, 신탁계약 등

■ 4) 데이터의 그룹을 의미하는 이름을 부여한다.

업무 활동(Activity)을 의미하는 이름을 배제하고 데이터 그룹을 의미하는 이름을 사용하도록 한다.

+ 주제 영역 활용

□ 목적

- 데이터의 계층적 구조를 파악하는 데 도움이 된다.
- 업무 기능(Function)과 병행하여 분석하는 경우에 분석의 최상위 단위 역할을 하여 품질 확보에 기여한다.
- 주제 영역 계층과 업무 기능 계층 간의 대응 관계를 확인한다.
- 주제 영역은 기업의 전사 업무를 위한 전체 데이터 구성에 대한 청사진을 제공한다.

- 데이터 구성 및 통합에 대한 방향 제시(선언적 성격)
- 효율적 데이터 관리를 위한 기준을 제공한다.

▣ 장점

- 데이터 및 업무 활동 모델의 품질 보증(Quality Assurance)
- 프로젝트 관리(Project Management) 용이
- 모델 개발 조정(Development Coordination) 용이
- 리포지터리(Repository) 관리 용이
- 상세 사항의 전개 혹은 축약 가능

+ 주제 영역 도출

▣ 업무에서 사용하는 데이터의 명사형 도출

- 정보 수집 소스로부터 명사형 찾기

▣ 업무 기능의 이름으로부터 도출

- 데이터와 업무 활동의 상호 보완 관계

▣ 하향식(Top-down) 접근 방법

- 주제 영역에서 출발하여 엔터티 타입으로 전개

▣ 상향식(Bottom-up) 접근 방법

- 엔터티 타입을 그룹핑하여 주제 영역 도출

▣ 분석 단계에서의 도출

- 아키텍처 모델(Architecture Model)을 정련하는 과정에서 도출
- 데이터 모델을 상세화하는 과정에서 도출

+ 주제 영역 정의 내용

▣ 주제 영역 목록

- 레벨 : 주제 영역의 계층 수준(1차, 2차, ... 단위)
- 주제 영역명
- 설명(단위 주제 영역의 경우)
- 대표 엔터티 : 해당 주제 영역 내에서 대표적인 엔터티를 기술한다.

▣ 주제 영역 정의서 샘플 양식

레벨	주제 영역명	설명	대표 엔터티

[그림 4-2-2] 주제 영역 정의서 예

개념 데이터 모델링 이해 주제 영역 정의 **후보 엔터티 선정** 핵심 엔터티 정의 관계(Relationship) 정의

개념 데이터 모델 작성

+ 개념

엔터티를 선정하기 위해서 우리가 가장 먼저 해야 할 일은 엔터티 후보를 수집하는 것이다. 이 단계는 말 그대로 후보를 찾아내는 것이지 엔터티를 확정하는 것이 아니므로 후보의 자격 여부만 가려내는 선에서 수사를 멈출 수 있어야 한다. 엔터티 후보를 수집할 때는 다양한 경로를 통해 수집하는 것이 바람직하며, 후보인지를 검증하는 객관적인 기준을 적용하여 후보라는 것을 판명하는 엔터티 후보 판정 단계를 거치게 된다. 이들은 엔터티 후보 분류 단계에서 3가지 형태로 분류된다.

+ 엔터티 후보 수집

엔터티 후보를 수집할 수 있는 방법은, 기존 시스템이 있었다면 시스템 도큐먼트(Document)가 있고, 현업에서 사용하는 각종 장표들도 있다. 또한 현업 사용자와의 인터뷰를 통해 후보를 찾을 수도 있고, 관련 서적을 통해서도 가능하다. 만약 프로세스 모델링을 먼저 수행하여 자료 흐름도가 나와 있다면 그 속에 있는 데이터 스토어 또한 훌륭한 엔터티 후보가 될 수 있다. 뿐만 아니라 타사 혹은 유사 시스템의 자료를 확보하고 있다면 거기에서도 많은 후보가 나타날 것이다. 현업 담당자들이 기안한 각종 보고 자료, 또는 현장에 가서 직접 업무를 조사해 보는 방법도 있다.

■ 기존 시스템 도큐먼트

시스템 도큐먼트에는 데이터 구조 및 프로세스 명세들이 나타나 있는 설계 자료에서부터 사용자를 위한 지침서에 이르기까지 다양한 도큐먼트가 있다. 이 자료를 가지고 있다면 엔터티 후보를 도출하는데 가장 유용하게 사용할 수 있다.

■ 현업 장표/보고서

실제 업무를 하는 조직에서는 대체적으로 정보시스템과 관련되지 않았더라도 오래 전부터 그들이 수행하는 업무의 효과적인 처리를 위하여 많은 장표를 가지고 있다. 또한 처리된 업무의 집계·분석·관리를 위해서 많은 종류의 보고서를 가지고 있다. 이러한 장표나 보고서에서 진짜 엔터티를 찾으려면 이 자료를 만들기 위해서 어떤 본질적인 집합이 필요한지를 분석해 보아야 한다. 좀 더 자세히 말하면 자료에 기술된 항목들을 속성이라 가정하고 이 속성들의 주인이 될 본질적인 집합이 무엇이 될지를 검토해 보라는 것이다. 여기서 말하는 본질 집합은 가공된 결과가 아닌 원재료(Source)가 되는 정보를 말한다.

■ 현업 인터뷰(Interview)

엔터티 후보를 도출할 때부터 현업 담당자들과 같이 시작하는 것이 최상의 모델링 방법이다. 사전에 충분한 분석을 통해 짧은 시간에 많은 것을 확인할 수 있도록 질문을 준비해야 한다.

■ 관련 전문 서적

현재의 업무를 있는 그대로의 모습으로 형상화하는 것이 아니라 좀 더 개선되고 발전된 모습으로 재창조하는 것이다. 이를 위해서 얻을 수 있는 정보는 현업만으로는 분명히 한계가 있다. 관련 전문 서적을 통하여 현실에 적용하기 위해 고민하고 있는 문제를 해결하는 데 필요한 아이디어나 힌트를 얻을 수만 있어도 충분히 목적을 달성한 것이다. 이러한 시각에서 전문 서적을 참고하면 생각보다 얻을 것이 많다.

■ 데이터 흐름도 (DFD, Data Flow Diagram)

업무 파악 및 시스템 분석을 위해 기능 설계를 데이터 흐름도로 작성했다면, 여기에 있는 데이터 저장소(Data Store)와 데이터 사전(Data Dictionary)에 있는 정보를 이용하여 엔터티 후보를 도출할 수 있다. 데이터 흐름도를 작성할 때 생성되는 데이터 저장소는 비록 테이블처럼 표현되지만 사실은 특정 데이터를 저장해야 한다는 데이터의 추상화된 집합이지 엔터티가 되는 것은 아니다. 예를 들어, 자재 구매 기능에서 자재 정보 자료 저장소로 정보가 저장되고 여기에는 자재

코드+자재명+규격+구매 수량+... 등의 자료 사전이 생성되지만 실제 모델링을 해보면 자재, 자재 유형, 단가정보 등의 많은 엔터티가 나타나게 된다.

■ 타 시스템 자료

타 시스템이 가지고 있는 엔터티를 면밀히 조사하고 이것들이 우리가 가져가야 할 집합과 어떻게 다른지를 밝히고 앞으로는 어떻게 정의해야 할 것인지를 조사한다. 엔터티 후보를 손쉽게 찾을 수 있는 또 하나의 방법은 관계사나 유사 업종의 시스템 도큐먼트를 입수해서 참조하는 것이다.

■ 현장 조사

본격적인 모델링을 하기 전에 현장을 답사하는 것이 좋다. 그렇게 해야 모델러와 개발 종사자들 간에 공통적인 의사소통 수단(Communication Protocol)이 좀 더 쉽게 만들어진다. 그렇지 못하면 설계 시에는 서로를 충분히 이해한 것 같은데, 나중에 가서 보면 무엇인가 앞뒤가 맞지 않는 일이 자주 발생한다. 이것은 결국 동일한 사실에 대해서 서로가 동상이몽을 하고 있었다는 것을 의미한다.

+ 엔터티 후보 식별

다양한 경로를 통해 엔터티 후보를 수집하는 과정에서 가장 중요한 것은 엔터티 후보를 어떻게 식별할 것인가에 대한 문제이다. 이러한 판단을 위해서는 무엇보다도 누구나 인정할 수 있는 구체적이고 객관적인 잣대가 있어야 한다. 엔터티 후보의 식별은 다음의 세 가지 단계에 대한 검증을 통해 객관화할 수 있다.

- 후보 엔터티의 개념 정립을 명확히 한다.
- 우리가 관리하고자 하는 것인지를 따져 본다.
- 가로와 세로를 가진 면적(집합)인지를 확인한다.

▣ 엔터티 후보의 개념 정립

엔터티 후보로 검토해 볼 대상의 최초 상태는 한낱 단어에 불과하다. 단어란 단지 사전적 의미에 지나지 않으므로 우리는 이 단어가 의미하는 진정한 집합이 무엇인지를 정의해야 한다. 예를 들어 사원이란 단어의 사전적 의미는 회사에 근무하는 사람 혹은 사단법인을 구성하는 사람이라고 되어 있다. 그러나 우리가 검토할 사원은 우리 회사에 근무하거나 축탁(임시) 사원, 관계사, 협력사 등에서 지원받은 사원들을 포함한 집합이 될 수도 있다. 우리는 반드시 이와 같은 구체적 집합을 정의한 다음에야 비로소 다음 단계에 대한 검토를 진행할 수 있다. 그것은 애매모호한 대상을 놓고 구체적인 판단을 하고자 노력하는 것은 도로(徒勞)에 지나지 않기 때문이다.

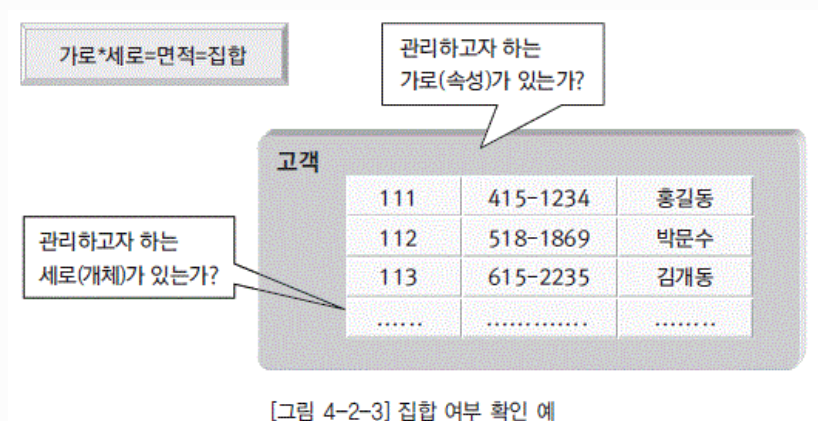
▣ 관리 대상 판정

엔터티 후보로 선정하기 위해 개념을 정립해 둔 단어에 대해 우리가 검토해야 할 첫 번째 작업은 과연 이것이 우리가 관리하고자 하는 대상이 맞는지를 확인하는 일이다. 이 말은 앞으로 우리가 모델링을 하면서 던지는 모든 질문에 반드시 기본적으로 포함되어야 하는 주어부(主語部)에 해당하는 구문이다. 이 문장이 있음으로 인해서 모델링을 할 대상 업무에 따라 모델링의 결과가 달라지는 원인이 된다.

- 사람이라는 논리적인 집합은 구멍가게 업무를 모델링을 하느냐 혹은 행정 전산망에 대한 모델링을 하느냐에 따라 관리하고자 하는 집합이 달라지므로 데이터 모델의 모습도 달라질 수 있다.
- 현재 관리하고 있느냐 뿐만 아니라 앞으로는 관리해야 하지 않느냐를 모두 포함하고 있다. 사실 현재 관리되고 있다는 것을 확인하는 것도 쉽지 않겠지만, 앞으로 관리할 것인지를 결정하는 것은 매우 전략적인 판단이 필요하다.

▣ 집합 여부 확인

엔터티는 집합이어야 하지만 모든 집합이 모두 엔터티가 되는 것은 아니다. 여기서는 엔터티를 결정하는 것이 아니라 엔터티 후보를 선정하려는 것이므로 검토하고자 하는 대상이 집합이 되는지 여부만 확인한다.



집합이란 다른 측면에서는 면적이라고 할 수도 있다. 면적이 되려면 가로 선분과 세로 선분이 있어야 한다. 선분이란 서로 다른 독립적인 두개 이상의 점(點)이 있어야 한다. 여기서 점이란 속성을 의미한다. 만약 우리가 검토할 대상이 하나의 점만 있다면 선분이 되지 못하므로 비록 세로가 선분이 되더라도 수직선 밖에 될 수 없다. 마찬가지로 세로가 하나의 점이라면 최대 수평선 밖에 될 수 없기 때문에 이런 경우는 면적(집합)이 되지 못하므로 엔터티가 될 수 없다.

➤ 엔터티 후보 선정시 유의사항

엔터티 후보의 선택은 데이터 모델링의 시작 부분이다. 엔터티 후보를 선정하는 동안에 아래와 같은 몇 가지의 유의사항을 염두에 두어야 한다.

■ 엔터티 가능성이 있다고 예상되면 일단 검토 대상에 올려라

엔터티 후보라고 해서 모두가 최종적으로 엔터티가 되는 것은 당연히 아니다. 그 대신 후보란 다른 후보들과 비교됨으로써 결정되는 것이므로 이런 후보들을 충분히 수집한 후 언젠가는 후보들을 하나씩 비교하면서 검토하게 된다. 버리게 되더라도 이 때 버리면 되므로 일단 모든 가능성 있는 대상을 최대한 수집한다는 생각을 가지고 접근하는 것이 좋다.

■ 너무 깊게 들어가지 마라

후보의 자격이 있다/없다 정도만 판단한다. 물론 필요에 따라 좀더 깊은 곳에 있는 내용을 통해 현재 결정하고자 하는 것에 대한 확신이 필요하다면 얼마간은 더 깊이 들어가 볼 수는 있다. 그러나 엔터티 후보를 선정하는 단계에서 절대로 너무 깊이 들어가서는 안 된다

■ 동의어처럼 보이더라도 함부로 버리지마라

특정 단어의 경우에는 의미를 일반적인 의미와는 다르게 사용하는 경우가 많다. 마치 새로운 언어가 생겨나는 것과 유사하게 누가 사용하기 시작했는지는 모르지만 언젠가부터 그렇게 알고 사용하기 시작하면서 의미가 그대로 굳어 버린다. 예를 들어 어떤 회사에는 상품과 제품이 있는데 일반적으로는 동일한 의미로 사용하지만 이 회사는 다른 의미로 사용하고 있었다. 이와 같이 동일한 의미로 여겨지지만 실제 정의된 집합은 같지 않을 수가 있고, 비슷하기는 하지만 정확하게 일치하지 않는 경우도 종종 발견된다. 그러므로 동의어처럼 보인다고 함부로 버릴 게 아니라 일단 후보로 도출하는 것이 중요하다. 데이터 모델링의 뒷부분에서 이러한 부분들을 가지고 다른 어떤 집합과 중복된 것인지, 완전히 포함되는 부분집합을 말하는 것인지, 아니면 일부가 서로 겹쳐져 있는 형태인지를 정확하게 규명하게 된다.

■ 개념이 모호한 대상은 일차로 그 개념을 상식화하여 이해하라

어려운 전문 용어나 특화된 업무에 대해서는 일반적이고 상식적인 예를 들어서 엔터티의 개념만을 파악한다. 이 단계에서 해당 업무 또는 데이터에 대해 정확한 개념을 파악하는 것은 무리가 따르기 마련이다.

■ 프로세스에 너무 연연해 하지마라

지금까지 우리는 모든 업무를 프로세스 중심으로 이해했기 때문에 프로세스 중심으로 설명을 듣지 않으면 이해하기가 매우 어렵다. 그러나 데이터 모델에는 프로세스가 없다. 흐름도 없고, 시간도 없다. 이것이 우리를 참으로 어렵게 만든다. 모델링을 하면서 자꾸 업무의 흐름에 집착하면 데이터흐름도처럼 데이터 모델이 업무를 따라 흘러가는 이상한 현상이 발생한다. 물론 데이터 모델링에서도 업무의 흐름을 완전히 배제하지는 않는다. 다만 그것은 현재 목적인 데이터 모델링의 어떤 결정을 좀더 완벽하게 하기 위해서 프로세스를 파악해 보는 것일 뿐이다.

■ 예외 경우에 너무 집착하지마라

예외 처리에 집착하지 말라는 뜻은 예외 처리가 중요하지 않다는 의미가 아니다. 지금 엔터티 후보를 수집하고 선정하는 단계에서는 가볍게 넘어가라는 것이다. 나중에 엔터티의 내부 구성 집합(Subtype)이나 릴레이션십을 정의하는 단계에서 이러한 예외 경우를 분석하여 설계에 반영하게된다.

■ 단어 하나하나에 집중해서 판단해라

누구나 알고 있는 것처럼 보이는 단어에 대해서도 좀더 깊이 구체적인 집합을 생각해야 한다. 예를들어 누구나 사원이 어떤 집합인지는 확실히 알고 있다고 생각한다. 그러나 좀더 깊이 들어가서 외부 용역사원이나 협력사 직원도 우리 사원 집합에 들어오는지를 물어보면 대답을 못한다. 왜냐하면 이렇게 구체적으로 사원이라는 집합에 대해서 정의한 적이 없기 때문이다. 매우 상식적이고 일반적인 용어로 정의된 엔터티 후보라도 사실은 매우 개략적이고 모호하게 정의되어 있는 경우가 많다.

➤ 수집된 엔터티 분류

개념 데이터 모델링에서는 일단 데이터 모델의 골격을 잡는 일부터 철저하게 수행해야 한다. 이렇게 하기 위해서는 엔터티 후보를 분류하는 작업부터 수행해야 한다. 분류 작업의 궁극적인 목적은 골격에 해당하는 엔터티만을 따로 분류해서 거기에 노력을 집중함으로써 나무만 보고 숲을 보지 못하는 오류를 범하지 않도록 하기 위한 것이다. 엔터티 후보를 분류하는 작업은 다음 두 단계에 걸쳐 진행된다. 첫 번째 단계는 우선적용 대상을 분류하는 것이고, 두 번째 단계는 첫 번째 단계에서 선별한 핵심 엔터티를 데이터 영역별로 분류하는 것이다.

▣ 우선적용 대상 분류

엔터티 후보를 우선적용 대상별로 분류하는 목적은 모델링의 골격에 해당하는 주요 엔터티를 먼저 도출하여 이를 명확히 정의함으로써 모델링의 기초를 단단하게 하는 것이다. 이를 바탕으로 흔들림없이 상세한 모델링을 진행해 갈 수 있게 된다. 모델링을 순차적으로 접근해 가야 할 형태별로 분류한다면 크게 세 가지로 나눌 수 있다.

첫 번째는 사람, 상품, 자재 등과 같이 행위를 발생시키는 주체나 목적어가 되는 엔터티라(여기서는 키 엔터티라 부르기로 한다)이다. 두 번째는 키 엔터티가 행위를 함으로써 발생되는 행위의 집합 중에서 좀더 하위의 행위를 발생시키는 주체나 목적어가 될 수 있는 엔터티(여기서는 메인 엔터티라 부르기로 한다)이다. 그리고 나머지 엔터티(여기서는 액션 엔터티라 부르기로 한다)를 세 번째로 분류한다.

1) 키 엔터티(Key Entity)

자신의 부모를 가지지 않는 엔터티를 말한다. 여기서 부모 엔터티란 자신을 태어나게 한 - 만약 이 엔터티가 없다면 논리적으로 자신이 태어날 수 없는 - 엔터티를 말한다. 태초부터 창조된 집합이란 다른 엔터티의 도움을 받지 않더라도 얼마든지 어떤 개체를 새롭게 정의할 수 있는 집합을 말한다. 예를 들어, 사원 엔터티에 있는 홍길동이란 개체는 아직 부서가 정해지지 않았더라도 사원으로서 정의하는 데 문제가 없다. 물론 부서 엔터티와 관계를 정의할 때 반드시 부서를 갖도록 하는 규칙을 부여할 수도 있겠지만 그렇다고 해서 부서가 직접적으로 사원을 태어나게 한 부모인 것은 결코 아니다. 다시 말해 사원은 부서 없이도 태어날 수 있지만 부서가 결정되지 않으면 결코 탄생시키지 않겠다는 규칙을 부여할 수는 있다. 키 엔터티를 구별할 때 이러한 규칙과 부모의 차이를 혼동하는 경우가 자주 나타나고 있으므로 여기서 정확히 개념을 정립해야 한다. 키 엔터티의 예로는 다음과 같은 엔터티가 있을 수 있다.

- 사원
- 부서
- 고객
- 상품
- 자재

2)메인 엔터티(Main Entity)

키 엔터티를 제외한 다른 모든 엔터티는 부모 엔터티를 가지고 있어야만 태어날 수 있다. 이러한 엔터티는 업무의 크기에 따라 엔터티 간 계층의 깊이가 달라진다. 여기서 키 엔터티를 제외한 엔터티 중에서 업무의 중심에 해당하는 엔터티를 메인 엔터티라 정의하고, 나머지를 액션 엔터티로 정의한다. 이렇게 정의하는 것은 전체를 단계적으로 파악하고자 하는 의도가 있다. 메인 엔터티의 예로는 다음과 같은 엔터티가 있다.

- 보험계약, 사고, 예금원장, 청구 ...
- 구매의뢰, 출하지시, 공사 ...
- 주문, 예산편성, 매출 ...

3)액션 엔터티(Action Entity)

매출, 출고, 입금과 같은 엔터티는 눈에 잘 보이지만 이들을 태어나게 한 고객, 공정, 창고 등은 엔터티로서는 눈에 잘 보이지 않는다. 키 엔터티에 해당하는 것들은 다만 우리 눈에는 코드 정도로 보일 뿐이고 실제 모델링에서도 이러한 부모들이 나중에 나타나는 경우가 많다. 액션 엔터티는 수행된 업무를 담고 있으므로 중요도를 가지고 따진다면 물론 가장 중요하다고 할 수 있다. 그러나 이들은 스스로는 결코 태어날 수 없는 - 반드시 부모를 가져야만 하는 - 자손들이다. 이들을 제대로 정의하기 위해서는 낱을 부모들부터 먼저 정확하게 정의하는 것이 일의 당연한 순서일 것이다.

액션 엔터티를 구별하는 방법은 매우 쉽다. 키 엔터티와 메인 엔터티가 아닌 것은 모두 액션 엔터티이기 때문이다. 앞으로 모델링이 좀더 구체적으로 진행되더라도 키 엔터티와 메인 엔터티는 집합의 본질이 크게 달라지지 않는다. 그러나 액션 엔터티는 상위 엔터티들이 어떻게 결정되느냐에 따라서 크게 영향을 받기 때문에 업무의 본질은 살아 있지만 최초에 예상했거나 과거에 정의했던 의미상의 주어가 크게 달라질 수도 있다. 액션 엔터티의 예로는 다음과 같은 엔터티가 있다

- 상태 이력, 차량 수리 내역, 상세 주문 내역,...

▣ 우선적용 대상 분류 사례

[그림 5-2-3]에서 예시된 엔터티 후보를 접근 형태별로 분류해 보기로 하자. 검토 대상은 가능한 무순으로 하고 간단한 설명만 하도록 하겠다.

고객	도난	법인	상품
계약	교육과정	직종	진단서
납입자	금융기관	통화	예금원장
우편번호	자동차체	수납기관	청구서
계좌	계정과목	사고유형	소송
납입	보험사고	대출	피보험자
불량거래자	거래자	계약대상물	행정구역

[그림 4-2-4] 우선 적용 대상 분류 예

위의 후보 대상 중에서 키 엔터티에 해당하는 몇 가지만 분류해 보면 다음과 같다.

■ 고객

고객은 상식선에서 생각하더라도 당연히 개체 집합이므로 키 엔터티로 분류해야 한다. 문제는 어떤 구체적인 집합을 고객으로 할 것이냐를 결정하는 것이 중요하다. 가령 개인과 법인을 모두 고객이라고 할 것인지, 현재 우리와 직접 관계를 가진 것만이 고객인지, 아니면 잠재고객까지 포함한 것을 고객이라 할 것인지에 대한 결정을 하는 것이 중요하다.

■ 법인

법인 또한 당연히 개체 집합이라는 것은 굳이 설명할 필요도 없다. 강의를 하다보면 가끔 법인을 메인 엔터티라고 생각하는 사람도 있다. 그 이유를 물어보면 고객의 자식이 아니냐고 반문한다. 이것은 착각이다. 법인은 고객의 부분집합이

지 자식은 아니다. 부분집합과 자식은 절대로 같은 개념이 아니다. 만약 사원 엔터티를 남자, 여자로 구분했다면, ‘남자는 사원의 아들인가’ 그렇다면 ‘여 자가 사원의 자식인가’ 나중에 엔터티 확정 단계에 가면 집합의 독립성과 동질성에 대한 평가를 하게 되는데 여러 가지 결론이 나올 수 있다. 즉, 고객 집합에 법인 집합이 포함된다면 이는 독립성에 저촉을 받는 것이므로 법인은 엔터티가 아니라 고객의 서브타입에 불과하다. 만약 개체의 일부만 고객에 포함된다면 고객 집합의 동질성을 확장하여 법인 집합 모두를 포함시킬 것인지, 아니면 어떤 특별한 목적이 있다면 중복이 있더라도 별도의 개체 집합으로 할 것인지 등의 결정에 따라 판단은 달라진다.

■ 금융기관

이것은 법인의 한 부분 집합일 수 있으므로 당연히 키 엔터티로 보아야 한다. 이 또한 고객과 법인과 관계에서 고려했던 문제처럼 법인과 금융기관 관계에서도 동일한 고민이 발생한다. 즉, 금융기관을 법인의 서브타입으로 보아야 할 것인지, 독립적인 집합으로 보아야 할 것인지를 검토하면서 유사해 보이는 두 집합의 개념을 좀 더 정확하게 정의를 해야 한다는 것이다. 얼핏 생각하면 금융기관은 당연히 법인의 일부분처럼 보인다. 그러나 집합을 좀 더 명확히 해보면 미묘한 차이가 있을 수도 있다. 가령, 법인 엔터티는 국가가 인정한 공식 법인이 있다면 그것이 하나의 단위 개체가 되는 집합이라고 정의했다고 가정하자. 그렇지만 금융기관 엔터티는 금융기관의 본·지점이 하나의 개체가 되는 집합이라고 했다면 이들은 성격이 다른 집합이 될 것이다.

■ 수납기관

단어의 구성을 보면 금융기관과 유사해 보이지만 결론은 정반대이다. 수납기관은 엔터티가 아니다. 물론 다른 엔터티와 개체의 중복이 발생하더라도 수납을 받을 수 있는 모든 개체들을 모아두고이를 수납기관이라고 정의한다면 안 될 것도 없지만 개체 본연의 본질이 많이 희석되는 일이 발생한다. 예를 들어 행정기관인 세무서에서 세금을 수납 받는다면 이것을 수납기관의 개체로 두어야 하겠는가? 만약 법이 바뀌어 수납을 받지 않는다면 이 집합에서 다시 빼내어야 하는가? 또 수납의 반대 개념인 지급도 발생할 것이므로 그렇다면 지급기관 엔터티도 만들어야 하고 여기에도 이 개체가 존재해야 하지 않겠는가? 수납기관이란 말에는 기관이란 개체 본질과 수납이라는 행위 본질이 같이 들어 있는-다시 말해서 두 개의 집합을 연결한-모습이므로 이것은 엔터티가 아니라 릴레이션십이 되는 것이다.

■ 거래자

거래자는 엔터티가 아니다. 엔터티는 순수한 개체 본질이 되거나 행위 본질이 되어야 하는데 거래자는 거래라는 행위 본질과 자라는 개체 본질이 혼합되어 있으므로 엔터티가 아니라 관계이다. 다시 말해서 개체 집합인 자(고객)와 거래내역이라는 행위 집합의 관계 명칭이 바로 거래자인 것이다. 이러한 집합에 대해 얼핏 생각하면 엔터티 정의에서 검토했던 우리가 관리하고자 하는 집합이며, 2개 이상의 속성과 2개 이상의 개체를 가져야 한다는 조건을 모두 만족하고 있으므로 엔터티로 오해하는 경우가 많은데 절대 그렇지 않다.

▣ 데이터 영역별 분류

이 골격에 해당하는 엔터티들을 명확하게 정의해야 한다. 상위 엔터티가 단단하게 고정되지 않고서는 사상누각이 될 수 밖에 없기 때문에 앞으로 좀 더 상세한 모델링이 진행되더라도 굳건하게 건널 수 있도록 견고하게 정의를 해야 한다. 앞서 도출했던 골격 엔터티들의 의미를 정밀하게 비교하여 구체화시키기 위해서는 유사한 의미를 가진 엔터티를 서로 모아 함께 비교하는 것이 필요하다. 뚜렷한 차이를 가지는 것들 간이라면 대충만 비교해도 그 차이가 분명하게 나타나지만 미묘한 차이를 가지고 있는 것들은 곁에 놓고 같이 비교해 보지 않으면 쉽게 그 차이를 알아낼 수 없다. 데이터 영역을 정의하는 방법은 접근 방법별로 볼 때 크게 하향식(Top-Down)과 상향식(Bottom-Up) 접근 방법으로 나누어 볼 수 있다. 또한 그 구성에 따라 세상의 모든 집합을 크게 나누어 행위의 주체나 목적물이 되는 개체들이 모인 집합과 그들의 활동으로 얻어지는 행위??, 지역, 시설, 재료 등은 행위의 주체로서 개체 집합의 예가 될 수 있고, 계약, 주문, 청구, 매출, 수납 등은 기업의 업무를 크게 분류한 것으로서 행위 형태의 데이터 영역으로 분류될 수 있다. 실제 업무에서 자주 발생하는 엔터티의 일부를 몇 가지 데이터 영역별로 분류해보면 [표 4-2-1]과 같다.

[표 4-2-1] 엔터티 영역별 분류 정의서 예

모델	대 상 엔 터 티 후 보
사람	사원, 가입자, 회원, 고객, 학생, 교사, 환자, 협력사 직원 ...
물건	부품, 원재료, 연료, 저장품, 상품, 건물, 운송센터 ...
사건 관리자	계약, 수주, 주문, 발주, 재해, 고장, 사건, 신고, 입고, 출고 ...
장소	창고, 생산라인, 행정구역, 하천, 선거구, 공항, 항만, 공정 ...
개념	판매목표, 생산계획, 평가기준, 할인기준, 배부기준, 공법 ...
금전	입금, 청구, 차입금, 예적금, 연간예산, 응자, 사내대출 ...
조직	부서, 판매망, 채널, 거래처, 법인조직, 교대조, 대리점 ...

개념 데이터 모델링 이해	주제 영역 정의	후보 엔터티 선정	핵심 엔터티 정의	관계(Relationship) 정의
개념 데이터 모델 작성				

+ 개념

▣ 엔터티란?

- 엔터티란 업무 활동상 지속적인 관심을 가지고 있어야 하는 대상으로서 그 대상에 대한 데이터를 저장할 수 있고 대상들 간의 동질성을 지닌 개체 또는 행위의 집합
- 엔터티를 정의할 때는 어떤 대상이 그 엔터티에 속하는지 혹은 속하지 않는지를 명확하게 정의할 수 있어야 한다.
- 예를 들어 영화배우라는 엔터티를 영화에서 극중 배역으로 분하여 연기하는 사람(?)으로 정의한 경우에 파트라슈는 주연이지만 배우인지 아닌지를 명확히 정의할 수 있어야 한다.

▣ 엔터티 정의의 요건

- 우리가 관리하고자 하는 것인지를 확인한다.
- 가로와 세로를 가진 면적(집합)인지를 확인한다.
- 대상 개체들 간의 동질성이 있는지를 확인한다.
- 다른 개체와 확연히 구분되는 독립성을 가지는지를 확인한다.
- 순수한 개체이거나 개체가 행위를 한 행위 집합인지를 확인한다.

+ 의미상 주어 정의

엔터티에는 인조 식별자(Artificial Unique Identifier)가 있을 수 있고, 이를 가주어(假主語)라 한다면 진주어(眞主語)에 해당하는 관계나 속성이 어딘가에 있을 수 있다. 예를 들어 신용카드의 가주어는 임의의 번호를 부여해서 만든 카드번호를 말하며, 진주어는 카드를 발급 받은 사람(고객 번호)과 발급 받은 상품(상품 코드), 그리고 발급일자가 될 것이다. 이 진주어를 의미상의 주어라고 할 수 있으며, 의미상의 주어를 모델링 입장에서 본다면 원래의 본질적인 식별자에 해당한다. 여기에서는 이것을 본질 식별자라고 칭한다.

▣ 본질 식별자 정의의 의의

모델링 진행 과정에서 본질 식별자를 특별히 중시하는 이유는 집합의 의미가 모호한 상태에서는 더 이상 객관적인 판단을 진행해 가는 것이 불가능하기 때문이다. 집합의 인스턴스가 생성되는 정확 한 단위를 모르고서는 집합이 명확해질 수 없다. 다시 말해서 집합의 의미가 명확하게 정의되지 않은 모호한 집합에 인조적인 유일한 이름만 가져다 붙인다고 해서 갑자기 집합의 정의가 명확해지지는 않는다.

▣ 본질 식별자 정의 예

신용카드 엔터티의 본질 식별자인 고객 번호와 상품 코드는 부모에게서 상속받은 관계 속성이며, 이 부모들은 바로 키 엔터티이다. 다시 말해 본질 식별자로 상속 관계를 규명해 올라갔을 때 최상위에 존재하는 것이 바로 키 엔터티라는 것이다. 여기서 주의할 것은 임의의 엔터티가 다른 엔터티와 1:M이나 1:1 관계일 때 1 쪽의 엔터티가 항상 본질 식별자가 되는 것은 아니다. 예를 들어, 사원 엔터티와 부서 엔터티는 M:1의 관계를 가지지만 사원 엔터티의 부모가 부서 엔터티는 아니다. 다시 말해 1쪽의 엔터티라고 해서 항상 본질 식별자가 되는 것은 아니다. 본질 식별자란 만약 그가 없다면 자신이 절대로 태어날 수 없을 때만 해당된다.

+ 코드성 키 엔터티 모델링

소위 코드성 엔터티라고 부르는 엔터티를 개념적 모델링 단계에서 도출할 것인지 아니면 나중에 상세 모델링 단계의 정

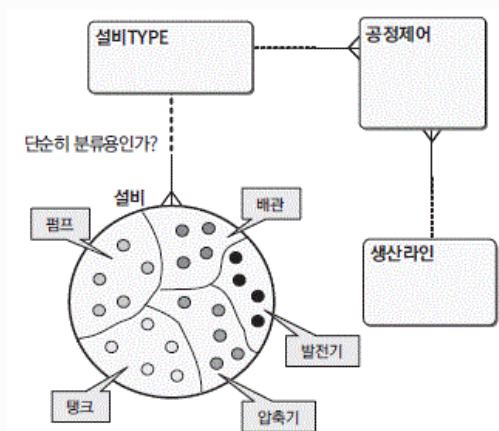
규화 단계에서 처리할 것인지에 대한 판단은 모델링의 효율적인 진행에 커다란 영향을 미친다. 이런 유형의 엔터티를 현 단계에서 너무 많이 도출시킨다면 모델링은 복잡성의 함정에 빠져 버릴 것이다. 반면, 꼭 필요한 것을 누락시켰다면 태어나야 할 중요한 자식 엔터티가 나타나지 않을 수도 있다. 이러한 고민을 해결하는 방법은 다음의 네 가지 기준에 따라 판단하는 것이다.

- 이 엔터티가 자식 엔터티를 가지는가?
- 이 엔터티가 자신만의 다양한 속성을 가질 것인가?
- 이 엔터티가 여러 엔터티에 관계를 가질 것인가?
- 이 엔터티가 다양한 종류의 관계를 가질 것인가?

▣ 자식 엔터티 유무 확인

이것은 해당 엔터티가 다른 엔터티의 본질 식별자가 되고 있는지를 찾아내는 것이다. 다시 말해서, 비록 겉모양은 볼품 없이 초라하지만 만약 이 엔터티가 도출되지 않았을 때 중요한 자식 엔터티도 같이 태어날 수 없다면 반드시 미리 도출해야 한다는 것이다.

[그림 4-2-5]에 있는 엔터티를 검토해 보자. 이 그림은 어떤 생산 공장에 있는 생산에 관련된 설비를 정의한 엔터티이다. 여기에 들어 있는 개체를 크게 나누어 펌프, 탱크, 배관, 발전기, 압축기 등으로 분류했다고 하자. 이러한 분류를 모델링에서는 서브타입(Sub-type)이라고 한다. 이 분류가 그 림에서처럼 단지 설비를 분류하는 데에만 사용한다면-그것이 비록 나중에 코드화를 함으로써 별도의 엔터티로 생성된다고 하더라도-지금 단계에서 굳이 엔터티로 도출할 필요가 없다. 그 이유는 지금 엔터티로 도출한 것이나 나중에 정규화 단계에서 도출한 것이나 별차이도 없으면서 초기 단계의 복잡성만 가중시키기 때문이다.



[그림 4-2-5] 자식 엔터티 유무 확인 예

그러나 만약 설비 타입과 생산 라인별로 우리가 관리하고자 하는 공정 제어값이 있다면 이 설비타입은 분명히 부모의 역할을 하게 되므로 지금 도출해 두지 않으면 자식 엔터티도 같이 누락된다.

▣ 속성 존재 여부 확인

해당 엔터티가 현재 혹은 미래에 단순한 코드명이나 코드의 의미에 대한 설명 외에 또 다른 속성을 가질 수 있는가를 확인해 보는 것이다. 엔터티가 이러한 속성을 가지고 있다는 것은 자기만의 확실한 사유재산이 있다는 것을 의미하므로 마땅히 독립적인 개체 집합으로서의 자격이 있다고 하겠다. 이러한 엔터티는 필연적으로 다른 엔터티와 다양한 관계를 가지게 될 것이며, 앞으로 자신을 참조하는-향후에 조인(join)이 발생하는-행위 엔터티가 분명히 발생하게 될 것이므로 미리 엔터티로 도출해 두는 것이 바람직하다.

▣ 관계 존재 여부 확인

엔터티가 향후 다른 엔터티와 많은 관계를 가질 가능성이 있는지를 검토해 보는 것이다. 엔터티가 다양한 관계를 맺고 있다는 것은 활동성이 왕성하다는 것을 의미한다. 여기서 말하는 관계는 반드시 부모(의미상의 주어)로써의 관계만 의미하는 것이 아니라 일반적인 관계까지 모두 포함한 포괄적인 관계를 말한다. 만약 이러한 엔터티를 누락시킨다면 앞으로 우리가 정의해야 할 많은 릴레이션십들 도 같이 누락될 것이다.

+ 집합 순수성 예

▣ 집합 순수성 의미

엔터티는 반드시 순수한 본질(本質) 집합이 되어야 한다는 것은 정의하고자 하는 집합이 사람, 상품 등과 같이 단위 사물을 정의한 개체(個體) 집합이 되든지, 아니면 이들이 어떤 활동을 함으로써 만들어진 임금, 계약 등과 같은 행위(行爲) 집합이 되든지 간에 반드시 둘 중의 어느 하나가 되어야 한다는 것이다. 그렇지 못하고 이들이 서로 결합된 형태라면 그것은 관계이다.

▣ 집합 순수성 예

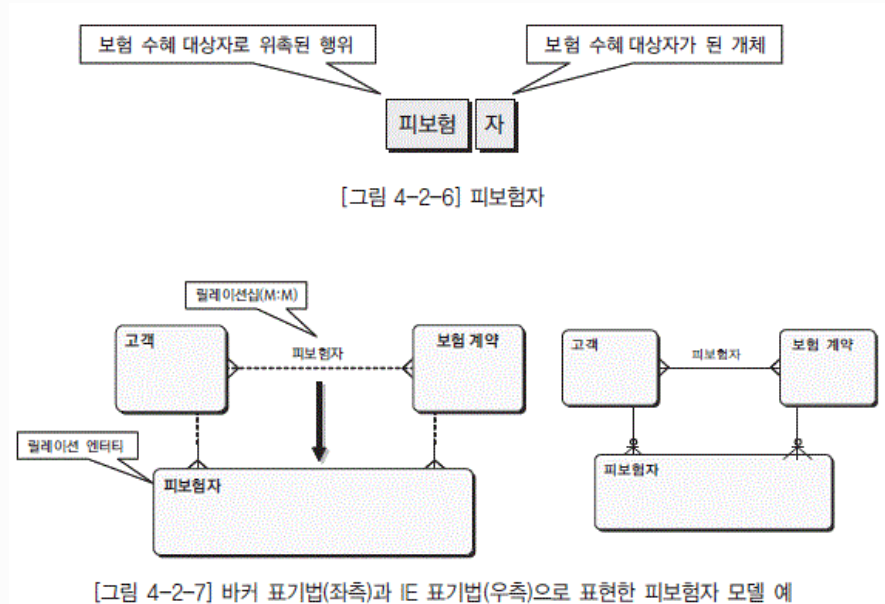
납입자와 같은 단어 개체 집합이 합성되어 있음을 알 수 있다. 물론 여기서 '자'란 사람이나 단체와 같은 개체 집합을 의

미하며, 이러한 개체가 포함될 수 있는 본질 집합은 고객과 같은 집합들이 이에 해당할 수 있다. 그렇다면 납입자란 순수 본질 집합인 고객과 납입이 결합되어 만들어진 것, 다시 말해 이것은 엔터티가 아니라 관계라는 것이다.

▣ 집합 순수성 적용 예외 사항

1) 관계의 엔터티화

관계가 엔터티로 변하는 경우이다. 관계가 M:M이 되면 더 이상 관계로만 존재할 수 없기 때문에 엔터티로 바뀌게 되며, 이를 릴레이션 엔터티라고 한다. 예를 들어 피보험자란 단어를 분석해 보면, [그림 4-2-6]에 있는 것처럼 보험 수혜자로 위촉된 행위, 보험 수혜 대상자가 된 개체



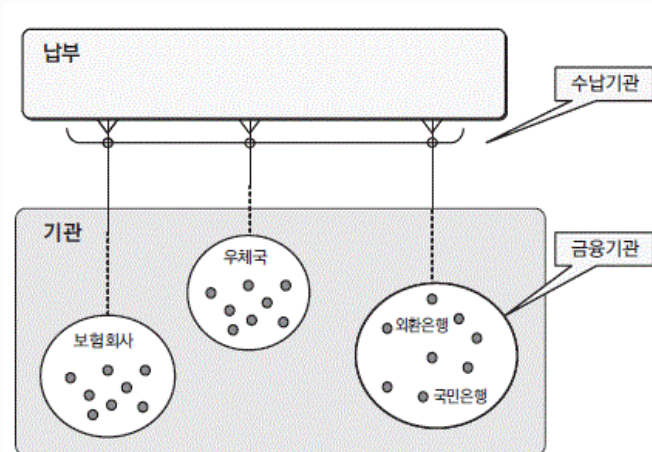
하지만, M:M 관계는 최종적으로는 관계로써 존재할 수 없기 때문에 1:M으로 연결된 교량 역할을 해주는 엔터티로 바뀌어야 한다. 이 엔터티는 관계가 변해서 생성되었기 때문에 릴레이션 엔터티(Relation Entity)라고 부른다. 방법론에 따라서 제휴 엔터티(Association Entity)나 교차 엔터티(Intersection Entity)라고 부르기도 한다.

2) 일부 집합 정의

전체 논리적인 집합 중에서 관리하고자 하는 일부의 집합만을 엔터티로 정의하고자 할 때, 구체적인 명칭을 부여하다 보면 이를 수식하고 있는 단어로 인해 마치 개체 집합과 행위 집합의 합성처럼 보일 수가 있다. 예를 들면 금융기관이란 엔터티 후보가 있을 때 언뜻 생각하면 금융과 기관의 합성어로 구성되어 있는 것처럼 보이므로 이것을 관계로 오해할 수도 있지만 이것은 엔터티이다.

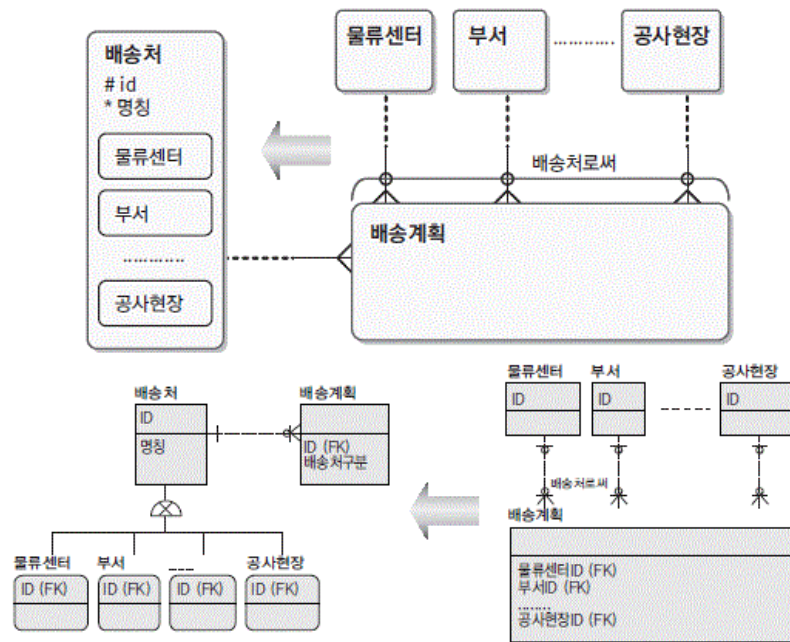
[그림 4-2-8]에 있는 금융기관과 수납기관은 언뜻 보기에는 마치 모두 행위처럼 보이는 금융, 수납의 수식을 하는 부분과 업체들을 말하는 개체 집합인 기관이 결합된 형태처럼 보인다. 그렇지만 이 두 가지 단어는 서로 결론이 다르다. 먼저 결론부터 말한다면 금융기관은 엔터티가 될 수 있지만 수납기관은 엔터티가 아니라 관계일 뿐이다.

단순하게 생각하면 금융이란 말이나 수납이란 말은 모두 동일한 역할을 하는 것처럼 보인다. 그러 나 조금만 더 깊이 생각해 보면 이들 간에는 아주 뚜렷한 차이가 있음을 알 수 있다. 금융은 기관이라 는 큰 집합의 특정 부분 집합을 나타내기 위해 사용된 말이다. 마치 전체 집합인 고객의 일부를 법인 고객으로 표현한 것과 같은 의미로 사용되었다. 반면, 수납 기관에 사용된 수납이란 뜻은 기관의 개체 가 행하는 여러 행위 중의 하나이다. 가령 은행에서는 수납만 하는 것이 아니라 출납 행위도 한다.



3)배타적 관계 대체

엔터티가 여러 엔터티와 동일한 내용의 관계를 갖는 배타적 관계를 가질 때 만약 배타적 관계의 변화 가능성이 높다면 이를 피하기 위해 배타적 관계에 있는 것들만 모아서 별도의 엔터티를 구성하는 경우이다. 예를 들어보자. 물건을 배달하기 위해서는 목적지가 있을 것이다.



[그림 4-2-9] 바커 표기법(위)과 IE 표기법(아래)에 따른 배송모델 예

일반적으로 이것을 배송처라고 부르는데, 이 단어를 풀어보면 '배송'이라는 행위와 '처'라는 개체 집합이 결합되어 만들어졌음을 발견할 수 있다. 이론적으로 보면 이것은 관계가 틀림없다. 여기서 먼저 '처'에 해당하는 원래의 집합을 찾아보면 [그림 4-2-9]와 같이 다양한 개체 집합들이 이에 해당할 것이다. 그런데 만약 업무의 변화가 심해서 배송처가 될 대상 집합이 앞으로 계속 늘어날 가능성이 높다면 필연적으로 배타적 관계 선분이 증가할 것이고, 이에 따라 애플리케이션의 처리 방법에는 빈번한 수정이 발생하게 된다. 이러한 경우라면 약간의 집합적 중복이 발생하더라도 별도의 엔터티를 생성할 필요가 있다.

+ 집합 동질성

▣ 집합 동질성 의미

엔터티에서 동질성(同質性)을 정의한다는 것은 말 그대로 집합에 들어갈 개체들의 동일한 성질을 어디까지로 한정할 것인가를 결정하는 것을 말한다. 가령, 사원이라는 집합을 우리 회사에 공식적인 적(籍)을 두고 있는 사람들로만 정의했다면 협력회사 사원이나 관계사 직원들은 이들과 동질성을 갖지 못한다. 그러나 개념을 확장하여 우리 회사에 공식적인 적을 두고 있거나 사내 업무에 관련된 준 사원에 해당하는 사람들이라고 정의했다면 협력회사나 관계사 직원은 정규 사원과 동질성을 갖게 된다.

▣ 집합 동질성 부여의 예

고객이라는 집합의 동질성을 부여하는 두 가지 경우를 가정해 보자. 첫 번째는 고객집합에 대해 우리 회사의 상품을 구매했거나, 구매할 사람들의 집합이라고 규정했다고 가정하자. 두 번째 경우는 우리 회사의 상품을 구매했거나, 구매할 가능성이 있는 모든 사람 혹은 법인·단체들의 개체집합이라고 규정했다고 가정하자.

1) 사람의 집합이라고 규정한 경우

이 경우에는 사람만의 집합이고, 우리 상품과 구체적인 관계를 맺은 사람들만 존재하는 집합으로 정의했다는 특수성이 분명히 나타나야만 한다. 이러한 구체적인 정의는 잠재 고객은 이 집합에 포함되지 않는다는 집합적 비교를 확실하게 해 주고 법인, 금융기관 등도 전혀 동질성을 갖고 있지 않다는 것을 분명하게 나타내고 있다

2) 사람 또는 법인이라고 규정한 경우

위의 이러한 집합들을 분명하게 포함시켰다는 것을 내포하고 있다.

+ 엔터티 명칭

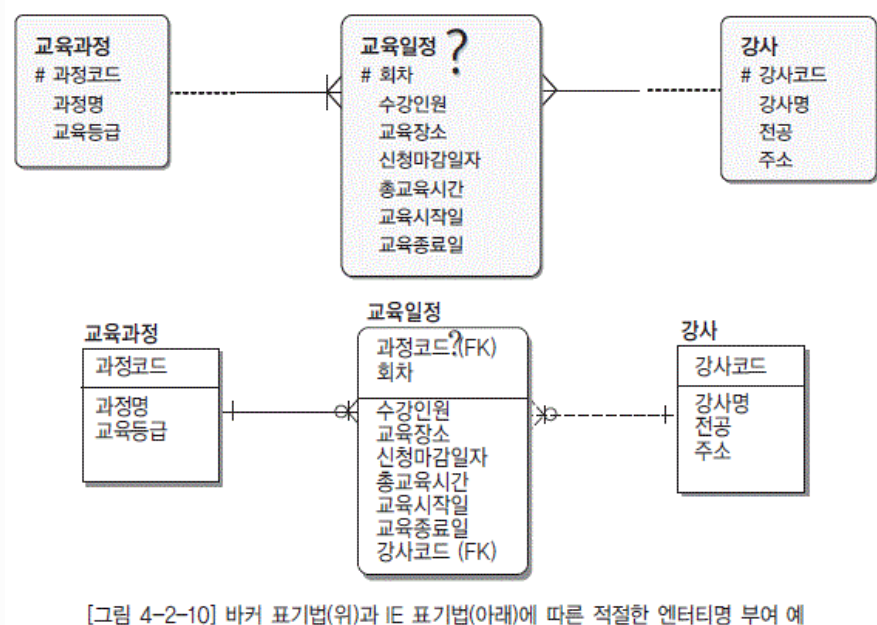
엔터티에 적절한 명칭을 붙여주는 작업은 생각보다 매우 중요한 일이다. 도출한 집합의 의미에 가장 어울리는 명칭을 부여해야만 앞으로의 커뮤니케이션에서 오류를 줄일 수 있고, 제3자에게 설명할 때도 의미의 전달이 명확해질 수 있다. 사실 지금까지 명확화를 했다고 생각하는 것은 겨우 엔터티 단위일 뿐이다.

▣ 적절한 엔터티 명칭

엔터티 명칭은 그 엔터티를 함축시킨 의미를 담고 있어 남에게 일일이 설명하지 않아도 오해를 최소화시킬 수 있어야 한다.

▣ 엔터티 명칭 부여하기 예제

엔터티의 의미를 명확하게 정의하는 작업은 매우 중요한 일이며, 그에 못지 않게 적절한 명칭을 부여하는 일에도 심혈을 기울여야 한다. 적절한 엔터티 명칭을 부여하기 위한 예를 들어보자.



[그림 4-2-10] 바커 표기법(위)과 IE 표기법(아래)에 따른 적절한 엔터티명 부여 예

[그림 4-2-10]에서 좌측에 있는 교육과정이란 엔터티는 의미상으로 본다면 교육과목에 가까운 엔터티이다. 다시 말해서 이 과정이 몇 차례 강의가 개설되더라도, 혹은 아직 언제 강의를 개설할지 결정이 나지 않았더라도 과정마다 단 하나씩의 개체를 가지게 되는 키 엔터티라고 하자. 이 엔터티는 그 진행 일정과 강사가 정해지면서 여러 번에 걸쳐 강의가 진행될 것이다. 이 엔터티는 교육과정을 부모로 해서 태어난 자식에 해당하는 행위 집합이며, 그 이름을 교육일정으로 하여 생성됐다. 이 엔터티의 의미는 명확해 보이며, 그 명칭에도 큰 문제가 없어 보인다. 그러나 분명히 엔터티의 명칭에 문제가 있다. 교육일정이라고 되어 있는 엔터티의 속성을 보면 일정만 있는 것이 아니다. 교육장소·강사 등의 일정과는 전혀 다른 속성도 있다. 그렇다면 이 집합의 개체는 교육일정이 아니라 개설된 강좌로 보는 것이 더 어울린다.

✚ 서브타입

▣ 서브타입 지정 의미

엔터티를 명확화하는 단계에서 해야 할 또 하나의 중요한 작업은 엔터티 내에 들어가는 구체적인 부분 집합(서브타입)의 종류를 명시하는 것이다. 개체-관계 도표(ERD, Entity Relationship Diagram)를 입체적이고 구체적으로 작성하기 위해서는 집합의 부분 집합을 표현해 주어야 한다. 예를 들어 어떤 보험회사의 직원정보 엔터티를 사원의 유형별로 바라보면 내근사원, 설계사, 대리점으로 구성되어 있지만, 직무별로 바라본다면 경영자, 관리자, 영업사원, 사무직 등으로 구분할 수 있다. 만약 성별로 바라본다면 남자, 여자로 구분할 수 있다.

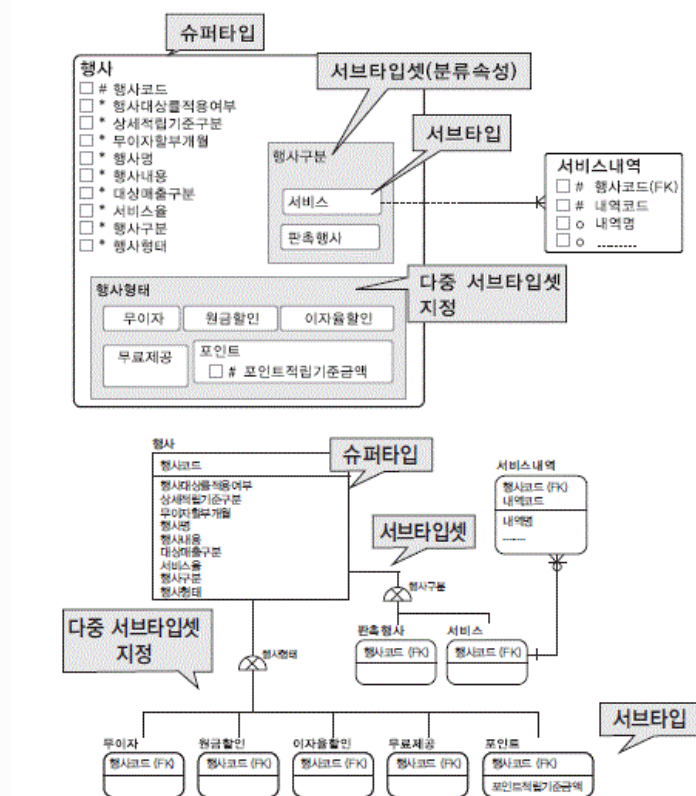
▣ 서브타입 지정시 고려사항

1) 교집합 허용 불가

서브타입 간에 교집합(交集合)을 허용하지 않는다. 다시 말해 남자와 여자 집합의 교집합, 즉 남자도 되고 여자도 되는 개체가 존재한다면 이는 서브타입이 될 수 없다.

2) 서브타입의 합이 전체집합

서브타입을 모두 결합하면 반드시 전체 집합이 되도록 해야 한다. 만약 전체 집합이 될 자신이 없다면 나머지 부분(여집합, 餘集合)을 기타로 정의해서라도 반드시 이 규칙을 준수해야 한다.

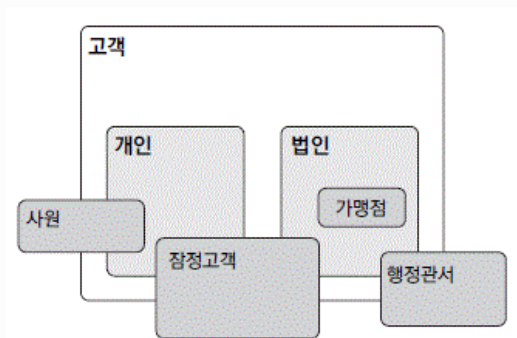


엔터티를 크게 하나로 할 것인지 아니면 여러 개의 개별 엔터티로 할 것인지를 결정은 향후에 일어날 데이터 모델링의 많은 과정에 커다란 영향을 미치게 된다. 예를 들면, 개발 단계의 생산성, 난이도, 수행 속도, 업무 변경에 대한 유연성, 유지·보수의 비용 등 많은 부분에 영향을 미친다. 좋은 형태의 통합은 변화에 대한 유연성을 증가시키고, 마치 분모의 개

수가 감소하듯이 복잡한 업무를 단순화시킬 수 있지만 나쁜 통합 형태는 본질의 희석을 가져와 모호한 집합을 만들게 된다. 엔터티 통합/분할의 결정 사례를 알아보자.

▣ 엔터티 독립성

엔터티를 결정하기 위해서 우리가 따져 보아야 할 독립성이란 우리가 새롭게 정의하고자 하는 엔터티가 앞서 정의해 둔 어떤 엔터티에도 포함되지 않는 독립적인 집합인지를 확인하는 것이다.



[그림 4-2-12] 집합 독립성 예

- 고객이라는 엔터티의 동질성을 우리 회사 제품의 구매 주체가 되었거나 될 가능성을 가진 사람이나 법인, 단체로 규정했다고 하자. 이때 새롭게 법인이라는 엔터티 후보를 검토한다면 이 집합에 속할 개체들은 이미 고객 엔터티에 속할 수 있기 때문에 독립적인 집합이 아니다.
- 고객 엔터티를 사람의 집합만으로 국한해서 동질성을 부여했다면 법인은 독립적인 집합이 된다.
- 우리가 독립성을 따질 때는 먼저 검토할 엔터티 후보의 의미를 명확히 해야 한다. 그것은 자신의 집합에 어떠한 개체들이 속해 있는지를 구체적으로 알아야만 다른 엔터티와의 독립성을 확인할 수 있다.

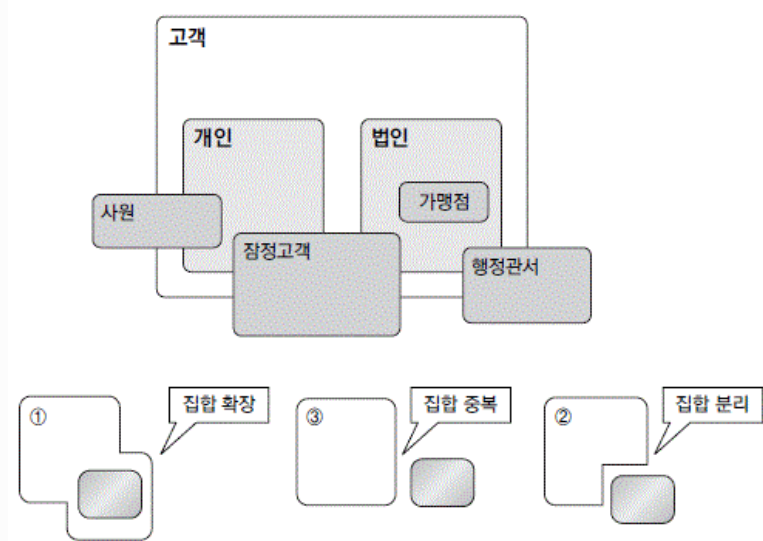
▣ 엔터티 분할/통합

집합 간에는 상품과 고객처럼 전혀 포함 관계를 가지지 않거나, 고객과 직원처럼 일부가 포함되거나, 아니면 직원과 임원처럼 전체가 포함되는 세 가지 경우가 존재한다. 그런 집합 간에 서로 포함되지 않는 경우라도 동질성을 크게 확장하면 포함될 수도 있고, 반대로 동질성을 구분하면 별도로 분리될 수도 있는 것이기 때문에 상황에 따라 종합적인 판단이 필요해지는 것이며, 그 판단의 합리성에 따라 모델의 품질은 크게 영향을 받게 된다.

이러한 결정에 따라 집합은 이합집산이 일어날 수 있으며 최종적으로 엔터티의 모습이 결정되는 것이다. 이와 같이 집합의 일부가 서로 겹쳐져 있을 때 이들 간의 모양을 결정하는 방법은 크게 세가지로 나눌 수 있다.

- 첫째, 어느 한 집합을 확장??차된 부분을 어느 한쪽 집합에서만 가지도록 해 둘 사이를 완전 분리하는 방법
- 셋째, 비록 교차되는 부분이 있더라도 두 집합을 별개의 집합으로 간주하고 필요시 이들 간에 관계를 맺어 주는 방법

어떤 상황에서 이것들을 선택해야 할 것인지는 매우 중요하고 어려운 판단이다. 엔터티를 결정하는 것은 결국 집합의 형태를 결정하는 것이므로 이러한 결정의 적절성이 바로 모델링의 품질을 좌우하는 가장 기본적인 요소이다.



[그림 4-2-13] 집합 이합집산 예

▣ 집합 동질성 확대를 통한 집합 통합 사례

1) 보험회사의 대리점 사례

보험회사에서 대리점은 상황에 따라 조직과 유사한 집합으로 볼 수도 있고, 직원과 유사한 집합으로 볼 수도 있다. 보험회사라면 보험모집, 고객관리, 실적 등의 주체가 되는 개체는 사람인 직원이나 설계사뿐만 아니라 대리점이 될 수도 있으므로 이러한 집합은 많은 행위를 공통적으로 한다는 입장에서 보면 매우 유사성이 강한 집합이다. 또한 개인도 자격조

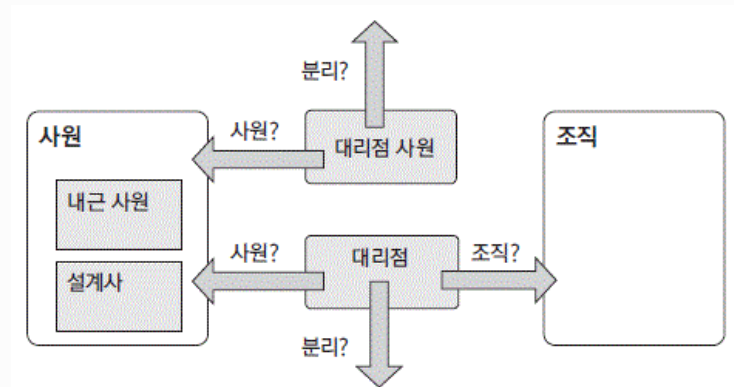
건에 따라서 대리점이 될 수 있어 조직의 의미보다는 설계사나 사원의 역할을 하는 경우가 많기 때문에 사원 집합을 확장하여 여기에 포함 시키고 있다.

2)통신회사의 대리점 사례

통신회사의 대리점은 회사의 조직처럼 실적 관리의 단위가 되거나 일반 부서와 유사한 업무 처리 를 하기 때문에 조직으로 보는 것이 일반적이라 할 수 있다.

3)대리점 사원

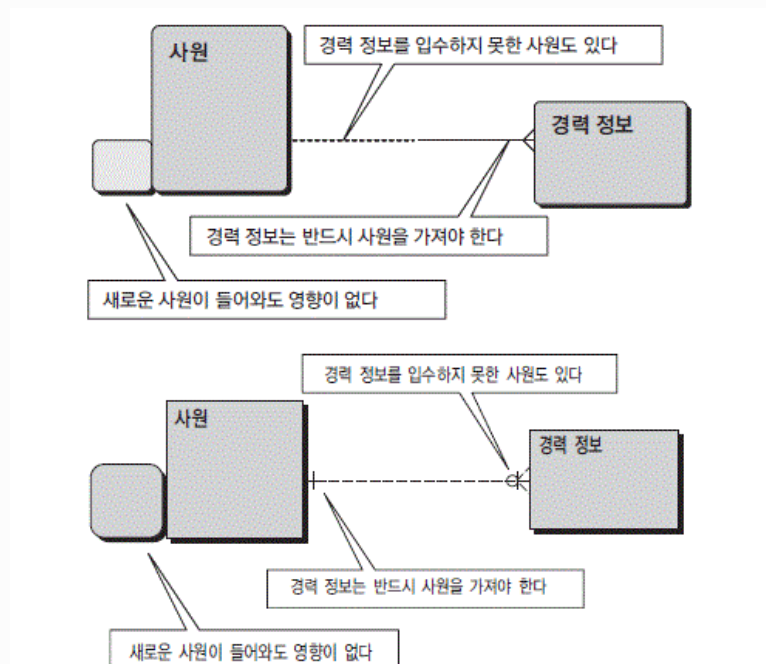
대리점 사원을 사원이라는 의미에서 동질성을 찾았다면 사원 엔터티에 포함시켜야 옳을 것이다. 가령 대리점에 소속된 사원이지만 어떤 행위의 주체로서의 의미는 일반 사원과 동일하다고 한다면, 사원 엔터티에 포함시키고 일반 사원과 희석되지 않도록 서브타입으로 구분하는 것이 좋다. 그러나 우리가 관리할 행위의 주체(모집자, 고객관리자 등)로서 일반 사원과 동격의 의미를 가지는 것은 대 리점이고, 대리점 사원은 단지 참조 정보로서 관리하고자 한다면 대리점은 사원 집합에 포함시키고 대리점 사원은 별도의 엔터티로 분리하여 불필요하게 사원의 의미를 희석시키지 않는 것이 좀더 좋은 방법이다.



[그림 4-2-14] 집합 동질성 확대 예

▣ 유연성 향상을 위한 통합

집합의 동질성을 확장시켜 중첩된 집합을 포함시키면 이들 간의 집합적 관계는 명확해진다. 이러 한 집합의 확장은 비록 유연성과 단순성을 증가시키기는 하지만 지나친 확장은 집합의 의미를 희석 시키므로 주의해야 한다.



[그림 4-2-15] 바커 표기법(위)과 IE 표기법(아래)에 따른 유연성 향상을 위한 통합 예

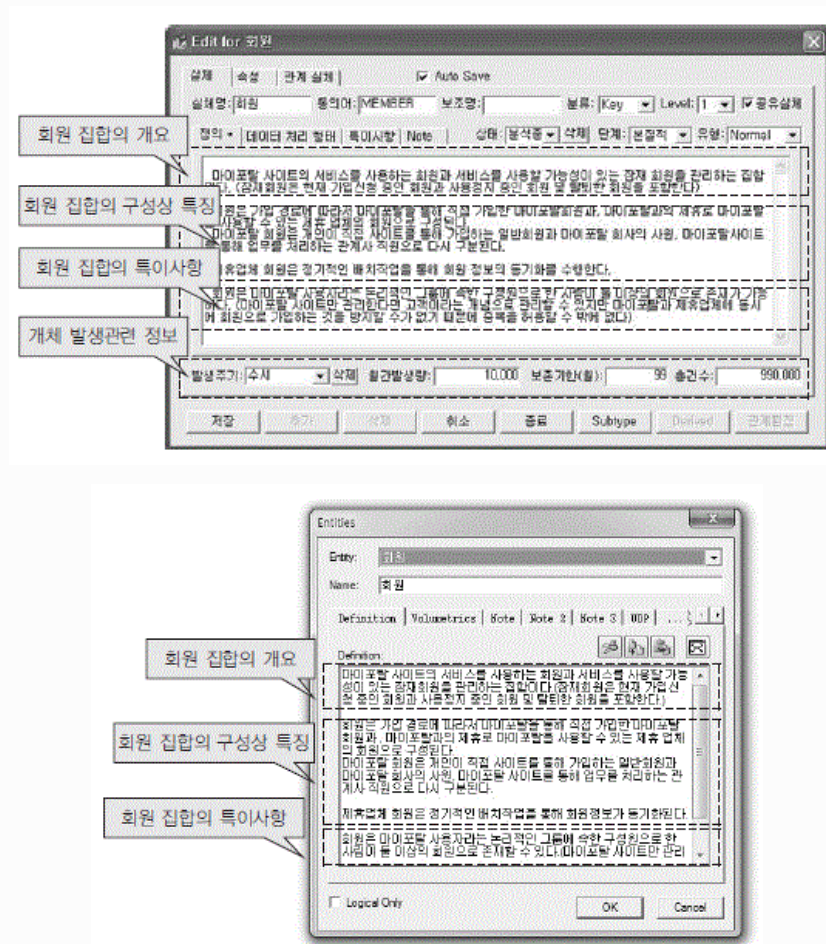
[그림 4-2-15]에서와 같이 부모 집합인 사원 엔터티에는 아직 경력 정보를 입수하지 못했거나 신입사원이기 때문에 경력 정보가 없는 사원도 있다는 것은 이상할 것이 없다. 어떤 사원의 경력 정보라고 하더라도 그 정보의 주인이 되는 사원이 없다면 정보로서 아무런 가치가 없다. 부모는 상황에 따라 자식을 가질 수도, 갖지 않을 수도 있지만 자식은 실사 부모가 누군지 모를지언정 결코 없을 수는 없다. 이것이 바로 보편 타당한 규칙이다. 이 말을 다른 의미로 해석을 해보면 그림 좌측과 같이 사원 엔터티에만 새로운 개체가 추가되더라도 경력 정보에는 영향을 미치지 않는다는 것을 뜻한다. 어차피 부모인 사원은 경력 정보를 가지지 않을 수 있도록 허용되었기 때문에 경력 정보를 가지지 않은 새로운 사원이 추가되었다고 해서 이 규칙에 영향을 줄 수 없는 것은 당연하다. 결과적으로 키 엔터티(사원 엔터티)는 최대한 통합되어 집합이 늘어나더라도 하위 엔터티에는 거의 영향을 미치지 않으므로 과감한 통합을 시도하는 것이 바람직하다.

+ 엔터티 정의의 기술과 엔터티 정의서 작성

엔터티 후보를 도출하고 충분한 검토와 검증을 거쳐 엔터티의 구성 단위와 형태 등을 결정하고 나면, 다시 서브타입 정의, 동질성 파악을 통한 엔터티 통합 및 분할 등의 과정을 거쳐서 최종적인 엔터티 형태가 확정된다. 엔터티 도출 및 정의 과정에서 엔터티의 개념을 정리하고 적절한 이름을 부여한 후 엔터티 분할-통합 과정을 거쳐 가장 적절한 명칭을 확정해야 한다. 이와 같은 과정을 거쳐 확정된 엔터티의 이름은 이미 이름만으로도 해당 엔터티가 어떤 집합인지 최대한 표현될 수 있어야 하며, 나머지 미진한 부분은 엔터티 정의 또는 엔터티 설명에서 구체적으로 기술하여 이해관계자 간의 명확한 의사소통과 개념 공유가 가능하도록 해야 한다. 이를 위해 엔터티 정의 기술 시 다음과 같은 내용으로 서술함으로써 해당 엔터티에 대한 상세하고 구체적인 내용이 온전하게 전달될 수 있도록 해야 한다.

- 데이터 집합의 개념 및 성격
- 집합 구성상의 특징
- 데이터 생성, 변경, 삭제 시의 특이사항 또는 데이터 오너쉽 등 기타 특이사항 등

[그림 4-2-16]은 이와 같은 기준에 따라 작성한 엔터티 정의 기술 사례이다.



[그림 4-2-16] 엔터티 정의 기술 사례

엔터티 정의의 기술이 완료되면 엔터티 정의에 관련된 추가적인 상세 정보 항목들을 작성하여 문서화 및 의사소통 수단으로 활용한다. 엔터티 정의서는 이러한 목적으로 작성하는 대표적인 산출물이다. [표 4-2-17]은 엔터티 정의서에 대한 작성 사례이다. 엔터티 정의서에 대한 항목 구성과 서식은 각 조직이나 프로젝트 여건에 맞춰 변형하여 사용할 수 있으며, 가능하면 최대한 상세한 내용으로 작성되어야 산출물의 작성 목적을 달성할 수 있다.

문서ID	FR-E-001	엔터티 정 의 서								
엔터티명 ①	기업계약 정보	서명시스템	고객서비스관리	대리인수②	16,000,000	작성일	2002-10-15			
대리인명 ③		작성자	홍길동	날	증가	500,000	배간	1.0		
엔터티 정의 ④	고객의 기업계약 행위를 관리하는 집합으로 가장 최근 상태, 계약기념일, 평가일자 등의 정보를 관리한다. 단편이라는 별도의 입력 엔터티에서 관리하며 이차적 상태의 정보를 가지고 있다. 유사 명의로명이 발생하며, 고객이 바뀌어도 새로운 기업계약을 생성하지 않는다. 평가일, 계약은 발행기간 이내에 복구할 수 있으며 그 기간을 초과하면 새로운 계약을 발행시킨다. 평가일인 경우는 이전 계약의 연계를 맺고 있어 필요한 정보를 참조한다. 계약의 핵심 사항은 고객, 전화번호, 단말기, 상품에 변경된다. 단말기자가 아닌 이상 계약은 유지된다. 000 서비스는 별도의 계약으로 취급되며, 발행일인이 사용하는 자원을만을 위해 기존의 기업계약이 존재한다.									
이	분	속성명	출산 엔터티	비	고	작	속성명	출산 엔터티	비	고
별	식	고객번호	고객	번호	누가?	4	계약번호		이것	식별자
자	자	계약번호	계약번호	정보	양식	문				
		계약일자			일자?	문				
대리인 ⑤	1) 신규 고객 가입 등록 시 2) 고객입력 정보관리 3) 연회 변경 등록 시 4) 가입 해지 시									
특기사항 ⑥	1) 해당 연도로 인한 고객의 변경 등록에, 기업계약은 장기적으로 보면 대리인의 고객에 연결된다. 이 엔터티의 하위 정보들을 고객 레벨로 승격시키는데 문제가 발생함 2) 이것을 해결하기 위해 고객에, 동속적인 기업계약번호와 필요성이 대두됨 3) 이를 만들기는 별도의 엔터티에서 관리하며 이 엔터티에서는 전혀 관여하지 않는다. → 별도의 기업계약으로 처리하는 것이 기존의 정책을 훼손할 수 있는 방법으로 권장됨 4) 자동 로딩은 고객을 확인할 수가 있는데 이런 경우와 기업계약을 어떻게 관리할 것인가?									
출산 엔터티	발행일인	출	속속사항	출	출	출	비	고		
고객	정보	사용자번호	*	고객번호						
대리인	정보	판매대리인번호	a	대리인번호						
출산 엔터티	출	출	출	출	출	출	출	출	출	출
기업계약상대	기업계약의 현재 계약유요상태	[1]사용 중, [2]폐지, [3]중지, [4]일시정지								
고객입력	고객의 입력을 구분하는 서명리입	[1]제입, [2]공적입력, [3]계약상대자, [4]입력입력								

[그림 4-2-17] 엔터티 정의서 작성 사례



개념 데이터 모델링

Home > DB 구축 가이드 > DA 가이드 > 데이터 모델링 > 개념 데이터 모델링

개념 데이터 모델링 이해

주제 영역 정의

후보 엔터티 선정

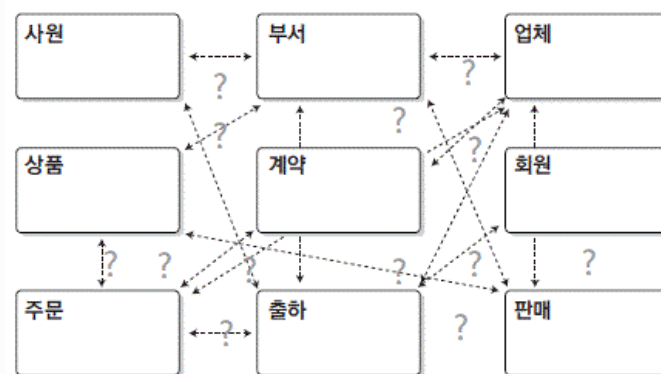
핵심 엔터티 정의

관계(Relationship) 정의

개념 데이터 모델 작성

+ 관계(Relationship)란 ?

관계(Relationship)란 엔터티와 엔터티 사이의 관계(關係)를 말한다. 즉, 관리하고자 하는 업무 영역 내의 특정한 두 개의 엔터티 사이에 존재하는 많은 관계 중 특별히 관리하고자 하는 직접적인 관계(업무적 연관성)를 의미한다.

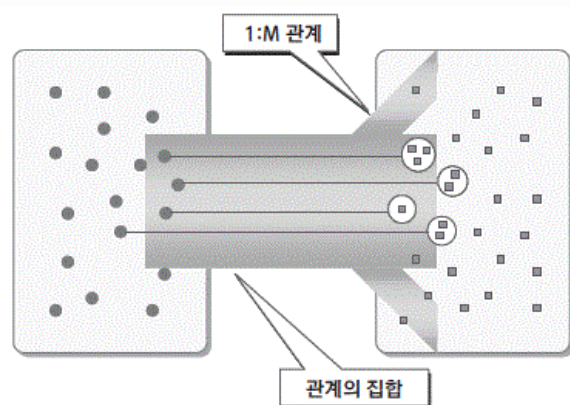


[그림 4-2-18] 집합 간의 다양한 관계 예

+ 관계 이해

▣ 관계도 집합이다

관계는 두 엔터티 사이에 그 목적과 내용이 다른 여러 개의 관계가 동시에 존재할 수 있다. 마치 교통량이 너무 많거나 좀더 나은 이동을 위해 특수 목적을 가진 더 세분화된 교량을 추가할 수도 있고, 작은 교량을 없애고 커다란 교량으로 통합할 수도 있듯이 관계 또한 크게 묶을 수도 있고, 구체적으로 분할시킬 수도 있다.



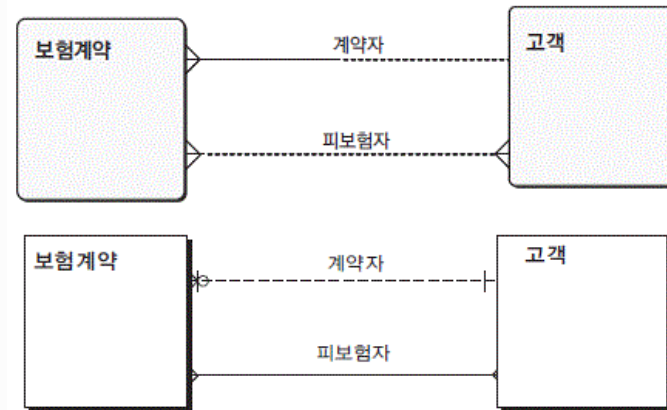
[그림 4-2-19] 관계 집합 예

▣ 직접 관계를 관계라고 한다

데이터 모델링에서의 데이터 집합(엔터티) 간에는 무수한 관계가 존재한다. 하지만 이러한 모든 관계를 표현(설계)하는 것은 아니다. 많은 관계 중에서 직접 종속인 것만을 관계로 보고 모델링하는 것이다.

▣ 두 엔터티 간에는 하나 이상의 관계가 존재할 수 있다

두 개의 엔터티 사이에는 서로 다른 업무 규칙을 가진 별개의 관계가 존재 할 수 있다.



[그림 4-2-20] 바커 표기법(위)과 IE 표기법(아래)에 따른 복수 관계 예

보험계약과 고객과의 관계는 [그림 4-2-20]에서 볼 수 있듯이 계약자 관계는 1:M이지만 피보험자 관계는 M:M이 될 수 있다. 물론 명의 변경을 할 수도 있고 그 이력까지 관리하겠다고 한다면, 계약자 관계도 다시 M:M이 된다. 보험 종류에 따라 하나의 보험계약에 여러 명의 고객이 피보험자가 될 수 있으며, 고객 또한 하나 이상의 보험계약에 피보험자가 될 수 있으므로 이 관계는 당연히 M:M 관계를 가진다.

▣ 외래키로 정의

관계는 외래키(FK, Foreign Key)로 구현되어 참조 무결성(RI, Referential Integrity)으로 데이터 정합성 유지의 역할을 하게 된다.

참조무결성 : 제3장-제3절-4항 참조 무결성 규칙 정의 참조

▣ 관계의 관점

- 항상 두 엔터티 간에 존재한다.
- 항상 두개의 관점을 가지고 있다.
- 데이터의 양방향 업무 규칙(Business Rule)을 표현
- 관계를 통하여 정보로서의 활용가치 상승
- 외래키로 구현되어 참조무결성으로 데이터의 정합성 유지
- 참조무결성

+ 관계표현

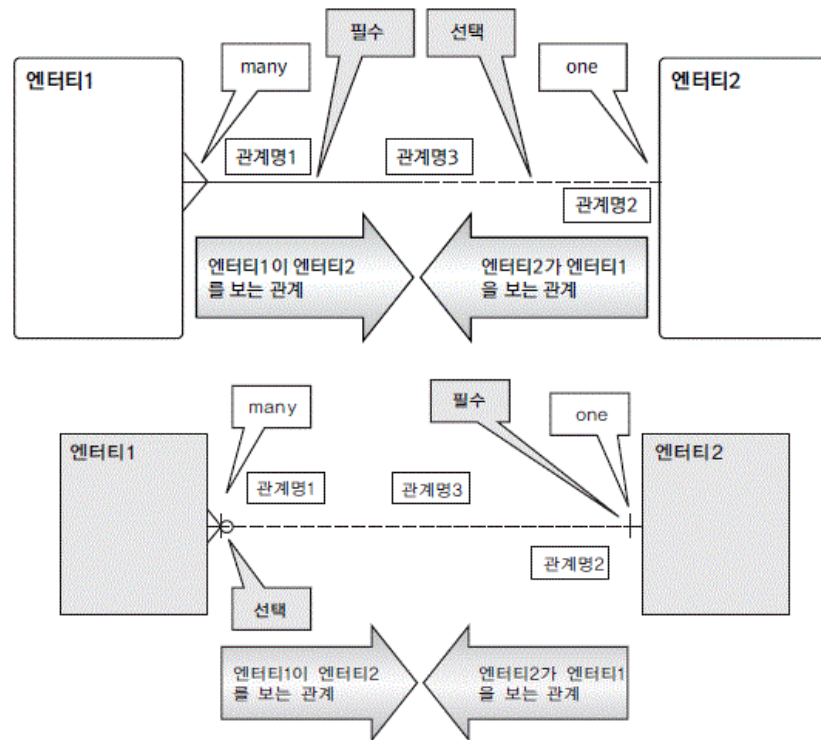
▣ 관계 형태(Degree/Cardinality)

1) 하나 이상(Many)

[그림 4-2-21]에 있는 까마귀 발가락처럼 표시된 모양은 하나 이상(Many)을 나타낸다. 하나 이상이란 반드시 하나 이상이 되어야 한다는 것이 아니라 하나 이상인 것이 적어도 한가지는 존재하고 있다는 것을 의미한다. 이 말은 1:M의 관계에는 1:1의 관계가 포함되어 있다는 것을 의미하고 있다. 나중에 우리가 관계 형태를 규명할 때도 하나 이상인 경우가 한가지라도 있는지에 대해 집중적으로 검토함으로써 이 관계를 밝혀내게 된다.

2)단 하나(Only One)

[그림 4-2-21] 우측에 있는 수평선은 단 하나임을 의미한다. 여기서 수평선이란 직선이 될 수도 있고 점선으로 표현될 수도 있으며 이것은 다음에 설명할 선택 사양에 따라 결정된다. 단 하나뿐임을 규명하는 것 또한 하나 이상인 경우가 하나라도 존재하는지를 검토함으로써 밝혀 낼 수 있다.



[그림 4-2-21] 바커 표기법(위)과 IE 표기법(아래)에 따른 관계 형태 예

□ 선택 사양(Optionality)

직선은 반드시 존재해야 함을 의미하고, 점선은 존재하지 않을 수도 있음을 의미한다.

1) 일반적인 형태

주로 참조하는 엔터티(M쪽)는 참조되는 엔터티(1쪽)가 반드시 존재해야 하는 경우가 많이 나타난다. 반대로 1쪽 엔터티는 M쪽 엔터티와 반드시 관계를 맺지 않아도 되는 경우가 가장 일반적인 형태라고 할 수 있다. 일반적이란 말은 이러한 경우가 가장 많이 나타난다는 것이지 항상 그러해야 한다는 것을 말하는 것은 아니다.

2) 바람직한 형태

우리는 모델링에서 가능한 직선 관계를 가지도록 - 특히 자식 쪽에서 - 노력해야 한다. 즉, M쪽 관계의 선택사양을 직선 관계로 만들려고 노력해야 한다. 부모 엔터티에 개체가 존재하지 않는 자식 엔터티의 개체가 많이 발생한다면 정보의 정합성에 많은 문제점이 나타나게 된다.

□ 관계명

1) 두 개의 관계 멤버십에 각각 부여

- 각자 상대방 입장에서의 관계명을 기술한다.

2) 현업에서 사용하는 간결한 동사형으로 표현

- 두 엔터티 타입 간의 업무적 연관성을 나타내는 이름 부여
- 현재 시제를 사용
- 다른 관계명과의 유일성은 확보되지 못함
- 능동/수동형의 사용은 가급적 배제



[그림 4-2-22] 관계명 예

3) 업무적 의미가 없거나 애매모호한 용어는 배제

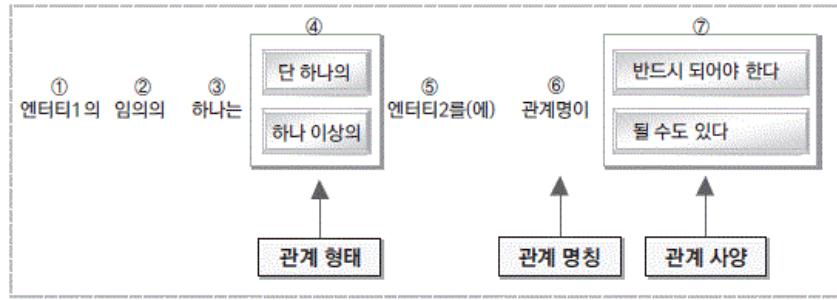
- 예: ~(관계가)있다, ~(관련이)있다, ~이다, ~한다 등

+ 관계 정의 방법

먼저 한쪽 엔터티를 기준으로 상대 엔터티와의 관계를 규명하고 다시 반대 방향으로 관계를 규명 한다. 관계를 규명할 때 가장 먼저 해야 할 일이다.

□ 관계 구문 이해

관계를 규명할 때 사용하는 관계 구문(Relationship Syntax)은 다음과 같은 전형적인 형태를 가진다.



[그림 4-2-23] 관계 구문 예

위의 관계 부분을 설명하면 다음과 같다.

■ 엔터티1

관계를 규명할 두 엔터티 중에서 어느 한쪽이 주어가 되어 상대방과의 관계를 규명하고자 할 때 주어가 되는 엔터티

■ 임의의

영어적 표현이라면 'ANY'와 같다고 이해하면 된다. 따라서 '임의의'란 결국 '모든'과 같은 뜻이라 하겠다. 즉, 집합 내의 개체 중에서 아무것이나 하나(임의의 개체)를 선택하여 만족하는지 증명하는 것이다.

■ 하나는

주어 집합에 있는 '임의의' 나 '하나'의 개체를 말한다. 이 하나가 어떤 형태로 상대방과의 관계를 가지는지를 검증한다.

■ 관계 형태

하나가 몇 개의 상대방을 가질 수 있는지를 집중적으로 밝히는 것이다. 여기에서 단 하나인 것을 증명하기 위해서 하나만 가지는 경우를 수없이 내세우는 것보다 하나 이상인 경우를 한 가지만 제시하는 것이 보편적인 방법이다.

■ 엔터티2

주어가 되는 엔터티가 바라 본 상대방 엔터티이다.

■ 관계명

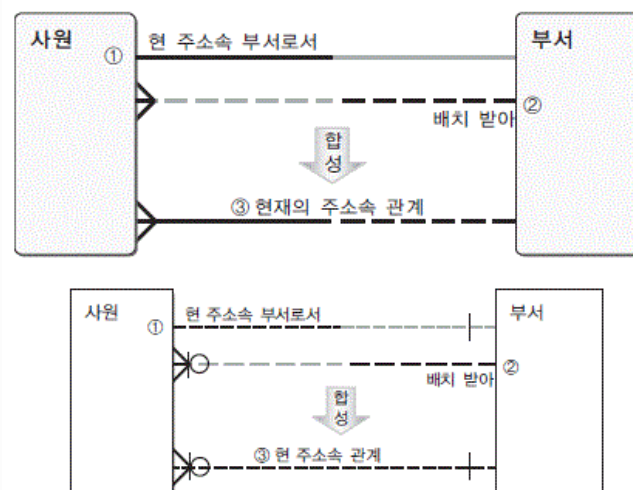
관계 규명을 통해서 정의된 관계 명칭이다.

■ 선택 사양

필수(실선), 선택(점선)의 모습을 결정하는 과정이다. 여기서 '반드시 관계명이어야 한다'와 '관계명일 수도 있다'를 결정하기 위해서는 관계명을 만족하는 경우를 아무리 많이 제시하더라도 없을 수도 있다는 한 가지 경우만 예를 들면 결정할 수 있다.

▣ 관계 정의 절차

[그림 4-2-24]는 어디에서나 접할 수 있는 사원과 부서 엔터티 간의 관계를 정의한 모습이다.



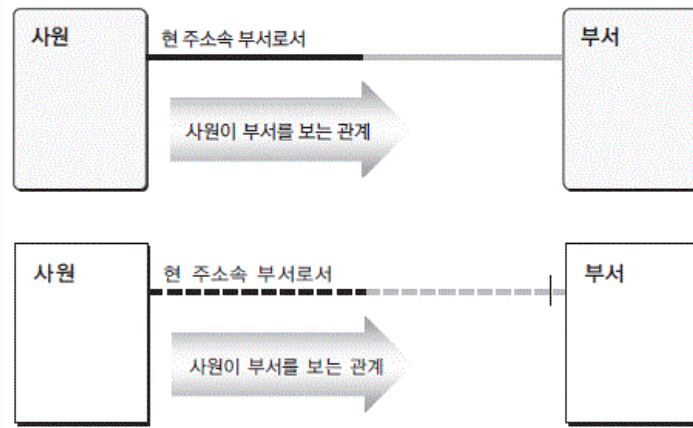
[그림 4-2-24] 바커 표기법(위)과 IE 표기법(아래)에 따른 관계 정의 예

[그림 4-2-24]와 같이 두 집합 간의 관계는 두 집합 각각의 관점에서 바라 본 관계가 궁극적으로는 합성된 형태가 완성된 관계가 된다. 이 관계에 대해서 각각의 관점에서 생각하고 판단해야 할 사항은 예를 들면 다음과 같다.

1) 사원을 주어로 부서를 보는 관계에서 판단해야 할 사항 예제

- 사원 엔터티의 임의의 한 개체(사원 한 사람)가 상대인 부서 엔터티에 단 하나의 개체(부서 하나)에만 소속되는지, 아니면 하나 이상에 소속될 수 있는지 판단한다.
- 지금까지 소속되었던 모든 부서를 찾고자 하는 것인지, 아니면 현재 소속되어 있는 부서만 찾고자 하는지를 판단한다.
- 겸임(兼任)을 하는 사원(주로 관리자)에 대한 처리 방안을 판단한다.

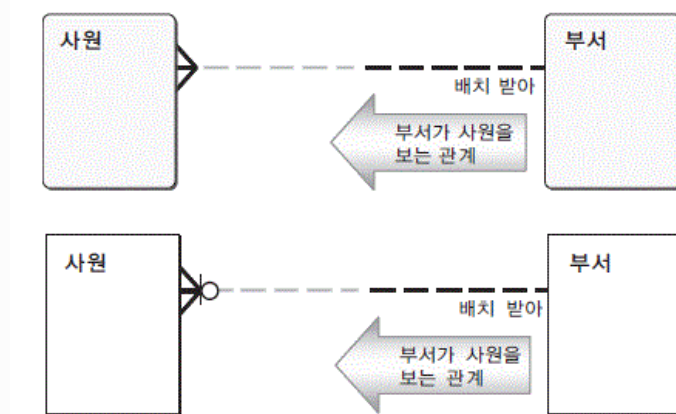
- 현재 소속을 가지지 않는 사원이 단 한 명이라도 있는지를 판단한다.
- 사원 엔터티가 우리 회사에 적을 두고 있는 사람만이 아니라 관계사나 협력사에서 파견되어 근무하는 사람들까지 포함된 집합인지를 판단한다.



[그림 4-2-25] 바커 표기법(위)과 IE 표기법(아래)에 따른 사원 주어 관계 예

2) 부서를 주어로 사원을 보는 관계에서 판단해야 할 사항 예제

- 하나 이상의 사원을 가지는 부서가 있을 수 있는지를 판단한다.
- 부서가 사원을 반드시 가져야 하는지, 그렇게 하지 않을 수도 있는지에 대해 판단한다.
- 최종적인 관계는 임의의 부서는 하나 이상의 사원을 현 주소속 부서로서 배치 받을 수도 있다고 결정한다.



[그림 4-2-26] 바커 표기법(위)과 IE 표기법(아래)에 따른 부서 주어 관계 예

+ 관계 형태

관계의 형태는 크게 1:1, M:1, M:M의 세 가지로 나눌 수 있다. 물론 선택 사양까지 결합하면 훨씬 많은 형태가 나타나지만 형태로만 나눈다면 이렇게 세 가지로 분류할 수 있다. 각각의 형태들은 고유한 특징을 가지고 있으며, 각자 가장 흔하게 발생하는 기본 형태(Default Style)를 가지고 있다.

▣ 1:1 관계(Relationship)

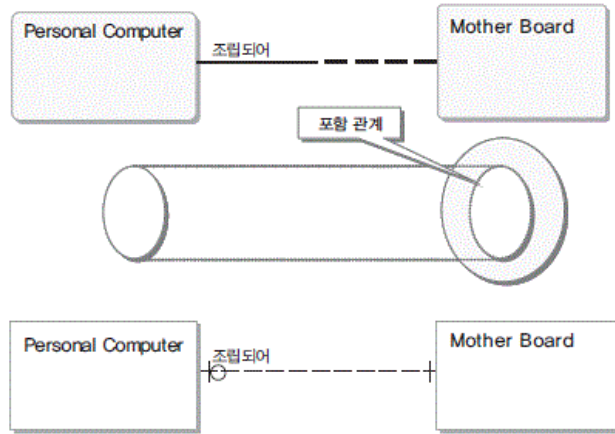
1:1 관계란 어느 쪽 당사자의 입장에서 상대를 보더라도 반드시 단 하나씩과 관계를 가지는 것을 말한다.

1) 특징

- 현실에서 매우 드물게 나타나는 형태
- 업무의 흐름에 따라 데이터가 설계된 형태에서 많이 나타남.
- 엔터티 수직 분할시에 많이 나타남.

2) 필수-선택 형태

- 한쪽은 실선이고 다른 쪽은 점선이 되는 모습이다. 좌측에 있는 엔터티의 개체는 대응되는 우측 엔터티의 개체가 반드시 존재해야 하지만 우측 엔터티의 개체와 대응되는 좌측 엔터티의 개체는 없을 수도 있다는 것이다.



[그림 4-2-27] 바커 표기법(위)과 IE 표기법(아래)에 따른 1:1 필수-선택 관계 형태 예

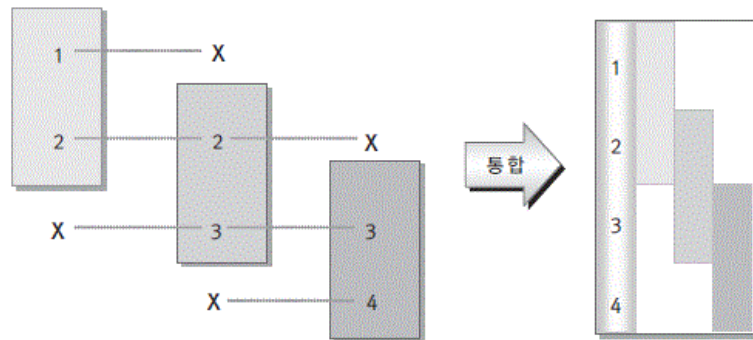
- 좌측의 Personal Computer가 조립되기 위해서는 반드시 단 하나씩의 Mother Board를 가져야 한다. 그러나 Mother Board는 아직 조립에 사용되지 않은 것이 존재할 수 있으므로 위와 같은 관계가 나타난다.

3) 필수-필수 형태

- 양쪽 모두 실선인 모습이다. 이 관계를 수학적으로 풀어보면 두 엔터티는 같은 엔터티라는 것이 증명된다. 수학에서 사용하는 필요충분조건은 $P \Rightarrow Q$ 이고, $Q \Rightarrow P$ 이면, $P \Leftrightarrow Q$ 라는 법칙이다. 이것은 서로 필요충분조건을 만족하면 두 개는 서로 동치(同値)라는 것을 의미한다.
- 대부분의 경우가 데이터 양을 감안하여 수직분할을 한 경우가 대부분이다. 하지만 이러한 데이터들은 논리적 관점에서 보면 하나의 데이터 집합이라고 볼 수 있다.

4) 선택-선택 형태

- 바커표기법에서는 양쪽 모두가 점선인 모습으로, IE표기법에서는 양쪽 끝에 모두 타원(oval)이 나타나는 모습의 관계로서 자주 발생되지 않는 형태이다.
- [그림 4-2-28]에서 좌측 3개의 엔터티는 서로 1:1 관계를 가지고 있지만 서로 존재할 수도, 존재하지 않을 수도 있는 형태이다. 이것을 만약 나중에 그대로 테이블로 설계한다면 전체 집합(1,2,3,4)을 찾기 위해서는 항상 합집합을 만들어야 한다.



[그림 4-2-28] 1:1 선택-선택 형태 예

▣ M:1 관계(Relationship)

M:1 관계는 가장 흔하게 나타나는 매우 일반적인 형태라 할 수 있다. 이는 마치 부모와 자식과의 관계처럼 계층적인 구조로 이해해도 좋다.

1) 특징

- 가장 흔하게 나타나는 관계의 형태이다.
- 내가 참조할(부모쪽) 정보는 나에게 대해서 반드시 하나만 존재해야만 참조(join)할 때 내 집합에 변화가 생기지 않으므로 내가 참조하는 엔터티는 반드시 1쪽이 될 수 밖에 없다.
- M:1 관계는 한쪽은 M(many)이고 다른 한쪽은 1(one)인 것을 말한다. 이 관계의 가장 일반적인 선택 사양의 모습은 [그림 4-2-29]와 같이 참조하는 쪽(M)은 실선(바커표기법)이거나 타원(oval)을 갖는 형태(IE표기법)이고 참조되는 쪽(I)은 점선(바커표기법)이거나 타원(oval)이 없는 형태로 나타난다.
- [그림 4-2-29]와 같이 참조하는 쪽(M)은 실선이 되도록(바커표기법)하거나 타원이 포함되도록 (IE표기법) 하는 것이 바람직하며, 반대로 참조되는 쪽(I)은 점선이 되도록(바커표기법) 하거나 타원이 없는 형태가 되도록(IE표기법) 하는 것이 좋은 모양이다.

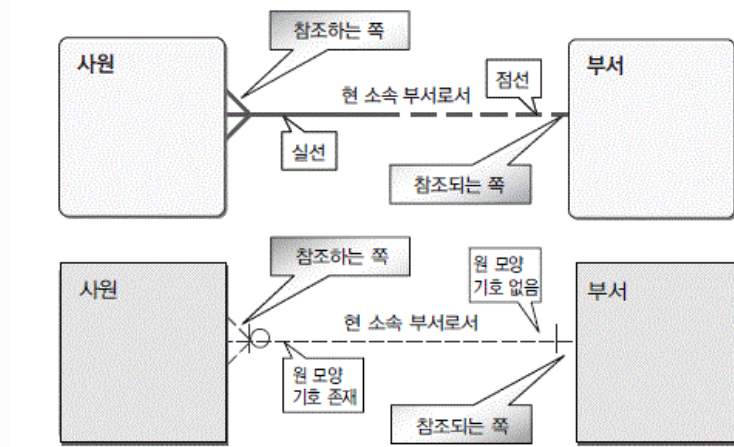
2) One to many relationship (Both Side Mandatory)

- 현실 세계에서 그리 많은 경우는 아니지만 가끔 발생하는 형태이다.
- 주문과 주문 아이템의 관계에서 하나의 주문이 여러 개의 아이템을 포함하고 있는 경우, 주문 아이템이 없는 주문은 업무적으로 의미가 없고 마찬가지로 주문이 없는 주문 아이템도 의미가 없다.

3) One to many relationship (One Optional)

- 현실 세계에서 가장 흔한 형태이다.
- 부서와 사원 간의 관계에서 모든 사원은 반드시 하나의 부서에 소속되어야 한다. 하지만 사원이 하나도 없는(신생) 부서도

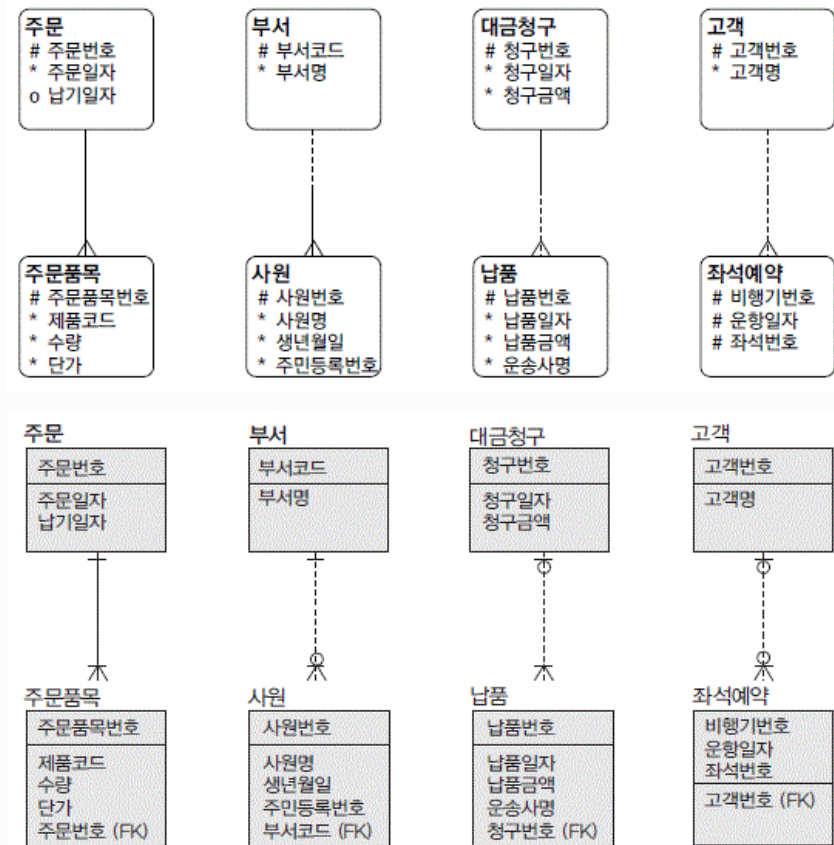
존재할 수 있다.



[그림 4-2-29] 바커 표기법(위)과 IE 표기법(아래)에 따른 M:1 관계의 일반적인 형태 예

4) One to many relationship (Many Optional)

- one to many 관계에서 현실적으로 드문 형태.
- [그림 4-2-30]의 예는 여러 번의 납품 건에 대해서 한번에 모아 대금 청구를 하는 경우이다. 즉 선 납품 후 대금 청구하는 계약 조건에서 아직 대금이 청구되지 않는 납품 건은 청구번호가 정해지지 않은 경우이다.



[그림 4-2-30] 바커 표기법(위)과 IE 표기법(아래)에 따른 M:1 관계의 종류 예

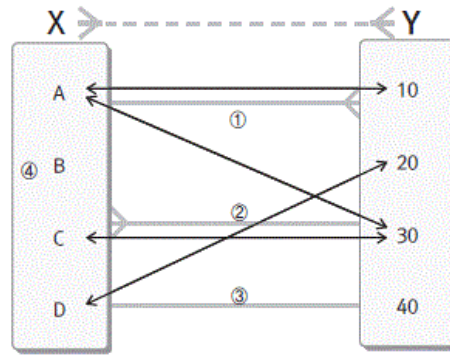
5) One to many relationship (Both Side Optional)

- 이 또한 현실 세계에서 흔한 형태 중의 하나.
- 관계의 선택성(Optional)이 증가할수록 모델의 모호성도 증가하므로 될 수 있으면 선택성을 해소해야 한다.

▣ M:M 관계(Relationship)

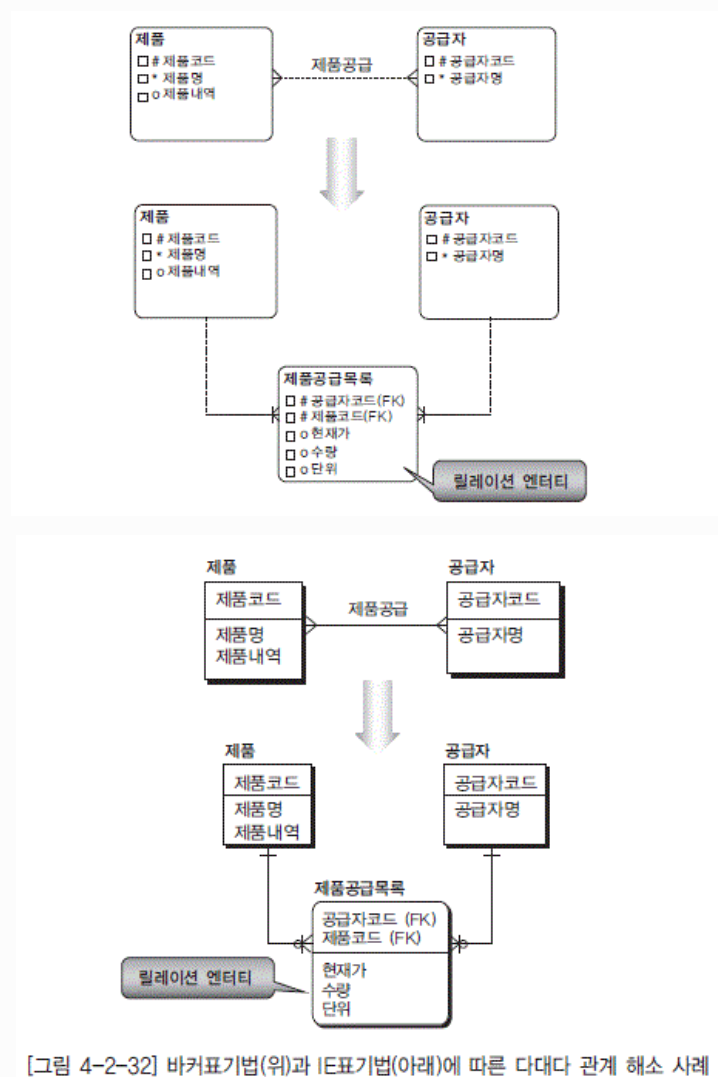
1) 특징

- M:M 관계는 관계를 가진 양쪽 당사자 모두에서 1:M 관계가 존재할 때 나타나는 모습이다.
- M:M관계의 선택 사양은 양쪽 모두가 선택적(바커표기법에서는 점선으로, IE표기법에서는 실선의 M:M형태)인 것이 기본형이다.

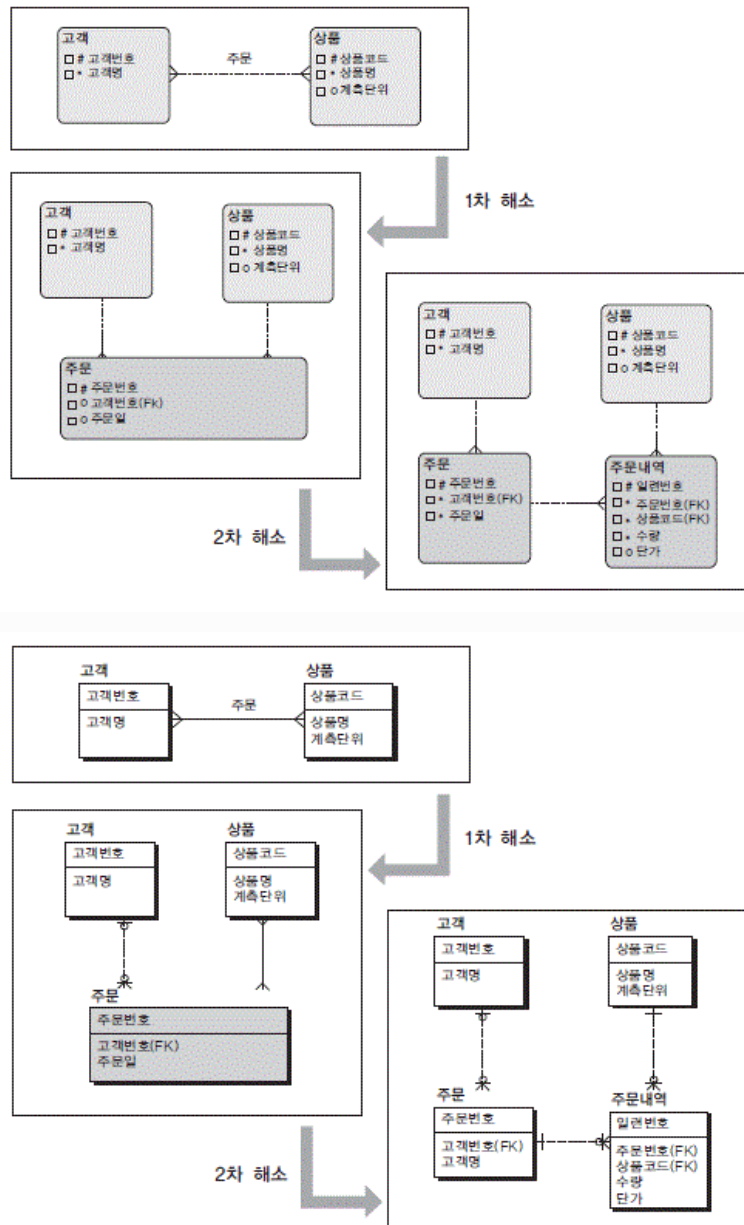


[그림 4-2-31] M:M 관계 예

- M:M 관계는 의외로 아주 빈번하게 발생하는 관계이다. 그러나 데이터 모델링이 완료된 후에 M:M은 존재하지 않는다. 이 말은 M:M 관계가 아직 덜 풀려진 형태라는 것을 의미한다.
- M:M 관계는 관계가 해소되면서 새로운 엔터티(Relation Entity/Intersection Entity)를 생성시킨다. [그림 4-2-32]와 같이 새로 생성된 릴레이션 엔터티와 기존 엔터티 사이에는 1:M 관계로 분해된 새로운 관계가 나타난다. M:M 관계를 분해했을 때 항상 1:M 관계로 분해되는 것은 아니다. [그림 4-2-33]의 상품과 주문 사이의 M:M 관계와 같이 때때로 덜 분해된 M:M 관계가 사이에 나타날 수도 있다. 이 경우 앞의 작업과 동일하게 분해 작업을 수행하여 M:M 관계가 완전히 해소될 때까지 관계를 분해해야 한다.



[그림 4-2-32] 바커표기법(위)과 IE표기법(아래)에 따른 다대다 관계 해소 사례

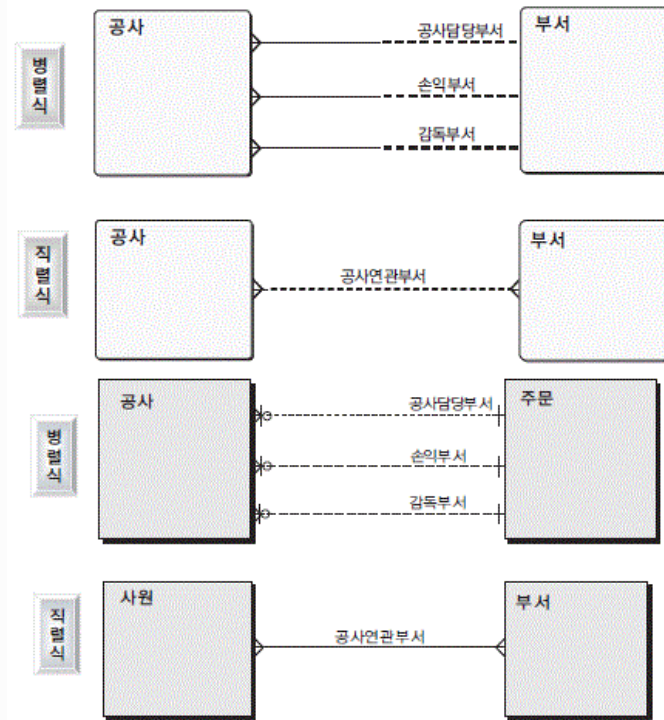


[그림 4-2-33] 바커표기법(위)과 I표기법(아래)에 따른 다대다 관계 해소 특이 사례

다중 관계 처리

1) 특징

다중 관계란 어떤 두 엔터티의 사이에 하나 이상의 관계를 가지고 있는 상태를 말한다. 서로 다른 엔터티 사이에도 경우에 따라 얼마든지 하나 이상의 다양한 관계가 존재할 수 있다. 관계도 집합이기 때문에 내용을 엄격하게 구분하여 여러 개의 관계로 정의할 수도 있고, 크게 묶어서 한두 개로 정의할 수도 있다. 묶거나 분리한다는 입장에서 보면 앞에서 설명한 관계의 통합, 분할과 비슷하게 보일 수도 있지만 엄연히 다른 개념이다.

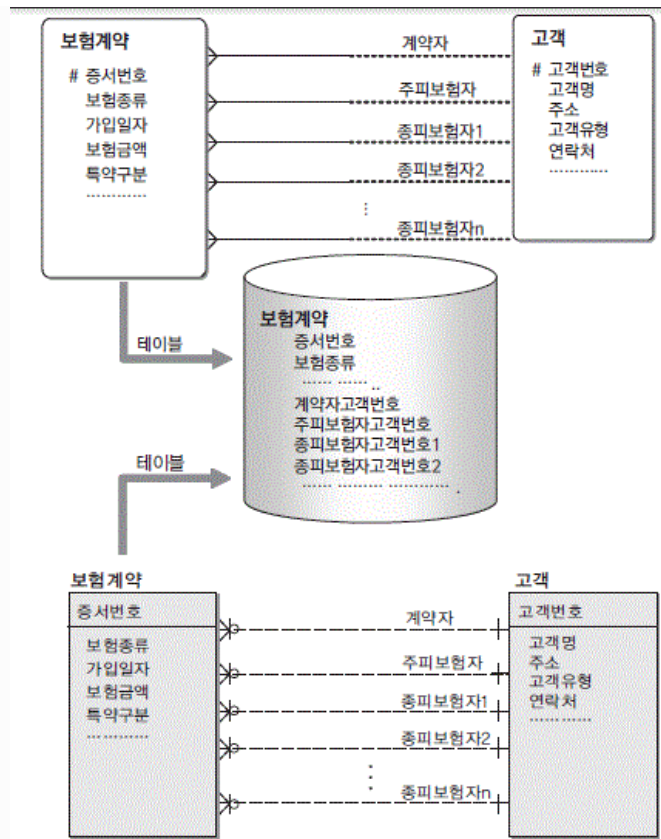


[그림 4-2-34] 바커 표기법(위)과 IE 표기법(아래)에 따른 다중 관계 예

[그림 4-2-34]에 있는 공사(工事) 엔터티는 말 그대로 회사 내에서 일어나는 각종 토목이나 건축물을 구축하거나 보수하는 작업을 말한다. 이러한 작업에는 실제 공사의 수행을 담당하는 부서(예를들면, 공무1과)와 투입된 비용에 대한 손익 처리(원가배정 부서)의 대상이 되어야 할 부서가 있을 것이다. 또 그 공사의 진행을 감독하는 부서도 관리할 수 있을 것이다. 가령, 생산1과가 관장하는 공정의 설비 개선 공사를 공무1과가 담당하고, 공무관리과에서 공사를 감독한다면 손익 부서는 생산1과, 공사담당부서는 공무1과, 감독 부서는 공무관리과가 될 것이다. 우리가 관리해야 할 이런 관계를 [그림 4-2-34]의 상단에 있는 것처럼 여러 개의 관계로 나누어 관리하는 것을 병렬식이라고 하고, 하단에서처럼 하나로 묶어서 관리하는 것을 직렬식 관계라고 부른다.

2) 병렬식 관계

병렬식이란 말 그대로 두 엔터티 사이에 존재하는 관계들을 별도의 관계로 간주함으로써 여러개의 관계 선분이 나란히 병렬로 관계가 맺어지게 되는 경우이다.



[그림 4-2-35] 바커 표기법(위)과 IE 표기법(아래)에 따른 병렬식 관계 예

병렬식으로 결정한다는 것은 [그림 4-2-35]와 같이 계약자, 피보험자, 수익자 등을 별도의 관계로 정의하겠다는 의미다. [그림 4-2-35]의 하단은 이렇게 정의했을 때 나중에 테이블의 구조 (관계형 데이터베이스로 설계 시)가 나타나는 모습을 보여주고 있다.

3) 병렬식 관계 특징

- 가) 테이블이 될 때 여러 개의 칼럼으로 나열된다.

물리 데이터 모델에서 테이블로 변환할 때 이러한 관계는 여러 개의 칼럼으로 생성되어진다.

- 나) 하나의 로우(row)에서 관리되므로 새로운 테이블을 추가할 필요가 없다.

나열된 모든 관계가 결국에는 모두 외래키 칼럼이 되어서 같은 로우에 펼쳐지게 되므로 이들을 위하여 새로운 테이블이 추가될 필요가 없다는 것이다. 이 말은 다시 말하면 이렇게 관리하는 방법은 기존 집합에 변화가 발생하지 않는다는 것이다. 이 특징이 바로 다음에 설명할 직렬식과 가장 큰 차이라고 할 수도 있겠다. 직렬식은 관계를 관리하기 위한 전용 엔터티(즉, 릴레이션 엔터티)를 가짐으로써 변화에 대한 유연성은 크게 증가하지만 새로운 엔터티를 추가해야 하고, 관계의 형태가 나빠진다 - M:M이 발생한다 - 는 단점이 있다.

- 다) 인덱스 수가 증가되고 SQL이 복잡해진다.

각각의 관계가 여러 개의 독립적인 칼럼으로 나타나기 때문에 이들이 검색 조건에 사용된다면 이들 모두에게 별도의 인덱스가 존재해야만 한다. 만약 [그림 4-2-33]에 있는 피보험자 관계처럼 동질성이 강함에도 불구하고 병렬식으로 정의했다면 피보험자의 고객 정보들을 참조하고자 할 때마다 나열된 각각에 대해 동일한 액세스를 해야 한다. 뿐만 아니라 만약 단 한 곳이라도 인덱스가 없다면 모두 없는 것이나 다름없는 매우 심각한 일이 발생한다. 이들이 혼자로서는 분포도가 너무 나빠 다른 칼럼과 결합 인덱스를 만들어야 한다면 각각에 대해서도 모두 동일하게 결합해 주어야 하기 때문에 상황에 따라서는 매우 심각한 일이 발생할 수도 있다.

- 라) 새로운 관계의 추가, 관계 형태의 변경 등에 매우 취약한 형태이다.

업무의 변화가 테이블이나 칼럼에 영향을 주지 않고, 데이터의 변화로 나타난다면 유연성은 크게 향상된다. 특히 관계형 데이터베이스에서 사용하는 SQL에서는 테이블이나 칼럼을 변수로 하는 것이 불가능하다. 물론 다이나믹 SQL을 사용하면 마치 변수로 처리할 수 있는 것처럼 보이지만 사실은 그렇지 않다. 비록 겉모양은 그렇게 하였더라도 DBMS로 날라가기 전에 바인딩(Binding)을 하여 최종적으로는 이들 변수를 없애기 때문에 결국 DBMS는 변수로 이들을 처리하지 않는다.

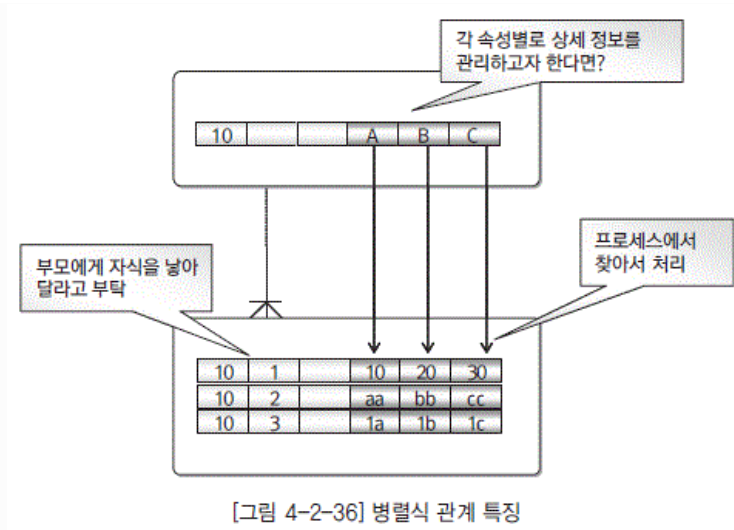
좀 더 정확하게 말한다면 DBMS는 이들을 변수로 해석하는 처리가 불가능하다는 것이 옳은 표현이다. DBMS는 데이터 사전을 이용하여 사용자 SQL을 해석하고 실행 계획을 수립해야 한다. 그러나 근거가 될 테이블과 칼럼이 변수로 지정되어 그 값을 알지 못한다면 아무것도 참조할 수 없게 된다. 이런 이유로 인해 테이블이 추가된다거나 새로운 칼럼이 추가되어 기존의 처리에 영향을 미친다면 반드시 SQL은 수정되어야만 한다. 즉, 업무의 변화로 인해 데이터 모델에 변화가 발생했을 때 많은 애플리케이션을 수정해야 하므로 유연성이 크게 나빠질 수밖에 없는 것이다.

이런 의미에서 병렬식은 직렬식에 비해 유연성이 훨씬 부족하다. 병렬식은 새로운 관계가 추가되면 결국 새로운 칼럼(외래키)이 추가되어야 하고, 관련된 애플리케이션이 수정되어야 하지만 직렬식에서는 거의 변화가 일어나지 않는다.

- 마) 관계 내용별로 상세 정보를 관리할 수 없다.

병렬식으로 나열된 관계는 기존의 집합을 유지하도록 하는 장점이 있지만, 이들이 칼럼으로 나열됨으로써 나중이라도 각자에 대한 상세한 정보를 관리할 지식 엔터티를 가질 수 있는 길이 원천적으로 봉쇄된다는 매우 중요한 제한 요소를 가지고 있음을 잊어서는 안된다.

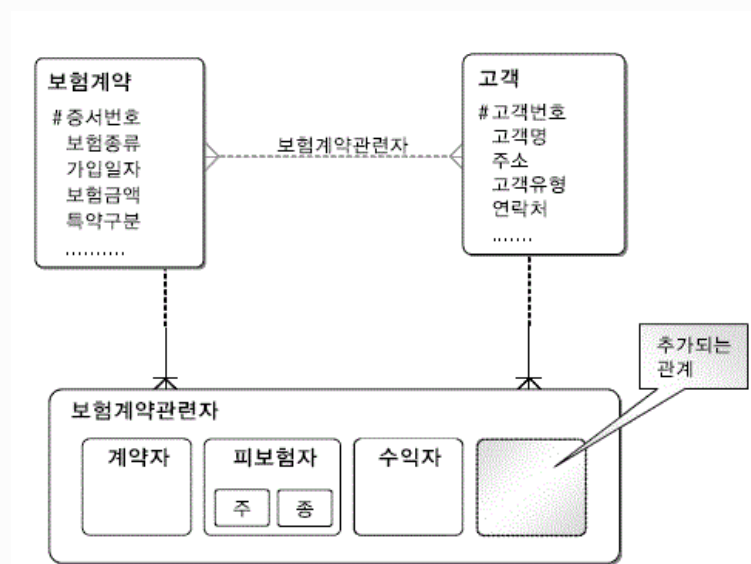
[그림 4-2-36] 상단의 A, B, C는 병렬식 관리로 인해 옆으로 나열된 속성이다. 이 속성에 대해 좀 더 상세한 정보를 관리해야 할 일이 생겨서 그림 하단에 있는 새로운 엔터티를 추가하게 되었다.

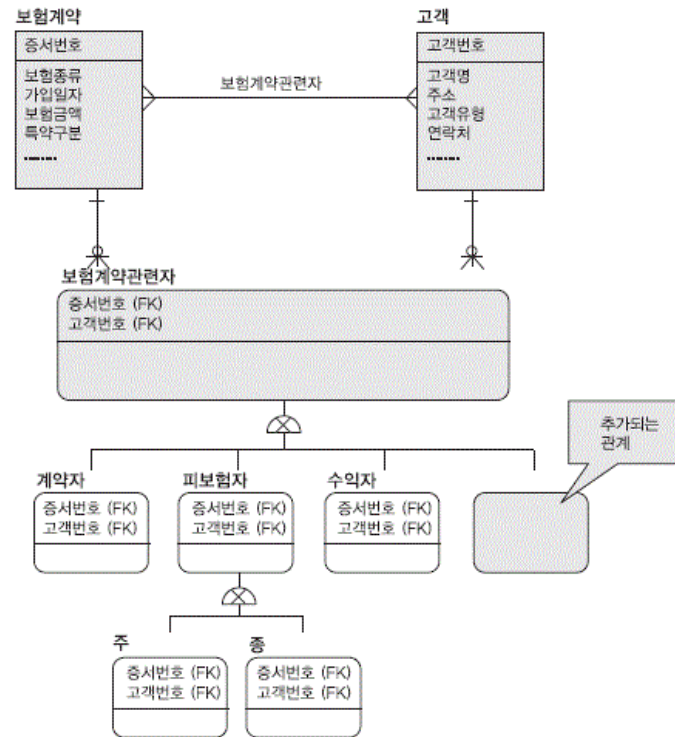


4) 직렬식 관계

두 엔터티 사이에 존재하는 몇 개의 관계를 모아 상위 개념으로 통합함으로써 하나의 관계로 관리하는 방법이다. 이 방법은 여러 가지의 관계를 하나로 통합했기 때문에 관계의 명칭 또한 당연히 상향으로 조정되어야 한다.

[그림 4-2-37]은 앞서 병렬로 정의된 여러 관계를 모두 묶어 '보험계약관련자'라는 명칭의 관계로 통합한 모양이다. 여러 개의 M:1 관계를 통합하면 당연히 M:M 관계가 된다. 이 관계를 해소하면 보험계약관련자란 관계 엔터티가 탄생한다. 이때 각각의 관계들이 서로 섞이지 않도록 하기위하여 서브타입을 사용하였다.





[그림 4-2-37] 바커 표기법(위)과 IE 표기법(아래)에 따른 직렬식 관계 예

5) 직렬식 관계의 특징

- 가) 관계들을 관리하는 새로운 엔터티가 추가되어야 한다.

M:M 관계를 해소시켜 관계 엔터티를 생성하는 것을 말하는 것이다. 이 방법은 앞으로 설명할 양호한 유연성을 얻는 대가로 새로운 엔터티가 추가되는 비용이 발생한다. 물론 특별하게 관리할 가치가 있다면 얼마간의 비용이 발생하더라도 그렇게 관리해야 한다는 것에는 이론의 여지가 없다.

- 나) 관계들이 로우(Rows) 형태로 나타난다.

칼럼의 나열로 나타나는 병렬식과 크게 대비되는 모습이다. 앞서 유연성을 얻고 싶다면 업무의 변화를 엔터티나 속성의 변화로 만들지 말고, 데이터를 증가하게 만들어야 한다고 했었다. 직렬식은 계약자, 피보험자 등등 아무리 많아지고, 변경이 발생하더라도 새로운 로우들이 만들어지거나 수정됨으로써 쉽게 해결되기 때문에 유연성이 높아지는 것은 당연하다.

- 다) 인덱스 수가 감소하고 SQL이 단순해진다.

향후 물리 데이터 모델링, 데이터베이스 디자인 과정에서 생성할 인덱스 전략에 영향을 미친다. 즉, 병렬식인 경우와 반대로 생각하면 쉽게 이해할 수 있다. 만약, 병렬식으로 설계되었다면 많은 관계들이 다수의 칼럼(계약고객번호, 주피보험자고객번호...)이 되었으므로 칼럼마다 인덱스가 필요해진다. 하지만 직렬식으로 되어 있는 보험계약관련자 엔터티에는 고객번호 칼럼이 하나밖에 존재하지 않으므로 당연히 고객번호에 대한 인덱스는 하나만 있어도 충분하다.

- 라) 새로운 관계의 추가, 관계 형태의 변경 등에 매우 유연한 형태이다.

직렬식 관계는 엔터티의 형태를 특별한 목적으로 보장하는 것 즉, 구조 변화에 대해서도 유연성을 가지고 있으므로 병렬식에 비해 유연성의 차이는 매우 크다는 것을 인정하지 않을 수 없다. 그렇기 때문에 대부분은 직렬식으로 하는 것이 바람직하다고 할 수 있다. 그렇다고 해서 앞서 소개한 공사와 부서간의 관계처럼 관리할 관계가 분명하고 내용이 독립적인 경우까지 무조건 그렇게 해야 한다는 뜻은 아니므로 오해하지 말기 바란다.

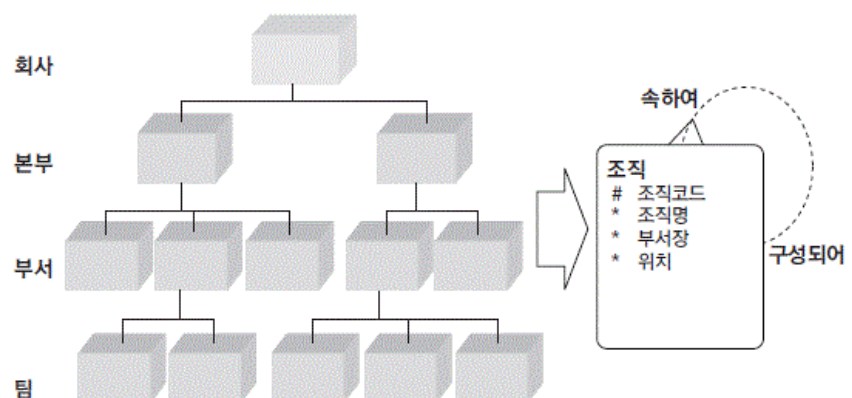
- 마) 관계 내용별로 상세 정보를 관리할 수 있다(자식 엔터티를 거느릴 수 있다).

모든 관계들이 관계 엔터티에서 별개의 개체로 관리되고 있으므로 이들이 자식 엔터티를 거느릴 수 있다

▣ 특수한 형태 관계

1) 순환 관계 (Recursive Relationship)

하나의 엔터티가 다른 엔터티가 아닌 자기 자신과 관계를 맺는 관계를 순환 관계(Recursive Relationship)라고 한다. 이와 같은 순환 관계를 표현한 모델을 순환 구조 모델 또는 순환 모델이라고 하며, [그림 4-2-38]의 좌측에 표현된 조직 구조와 같은 계층 구조를 표현하는 데 유용하다.



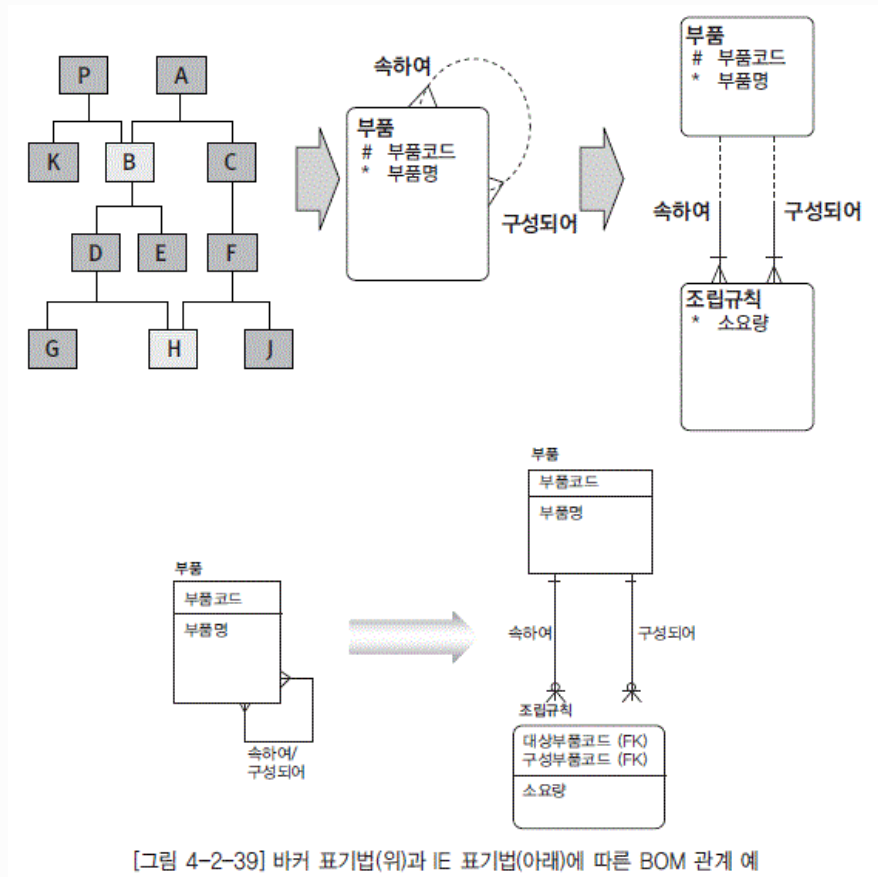
[그림 4-2-38] 순환 관계 예

순환 관계는 다음과 같은 특징을 가지고 있다.

- 하나의 순환 엔터티는 각 엔터티의 모든 속성을 포함해야 한다.
- 각 계층에 있는 속성은 동일하게 하는 것이 가장 좋다.
- 순환 모델은 필수(직선) 관계로 취급될 수 없고(무한 LOOP 발생), 반드시 선택사양 관계이다.
- 순환 모델의 특징은 조직의 변경(추가/삭제)에 쉽게 대응할 수 있다는 것이다.

2) BOM(Bill of Materials) 관계

BOM 관계의 모델을 네트워크 구조(전철 노선이나 하이퍼링크를 갖는 웹 문서 같은)라 한다. 이러한 네트워크 구조를 제조업에서 [그림 4-2-39]와 같이 부품의 소요량 파악 등의 모델에 유용하게 이용하게 되면서 M:M 순환 구조는 BO M(Bill of Material) 모델이라 알려지게 되었다.



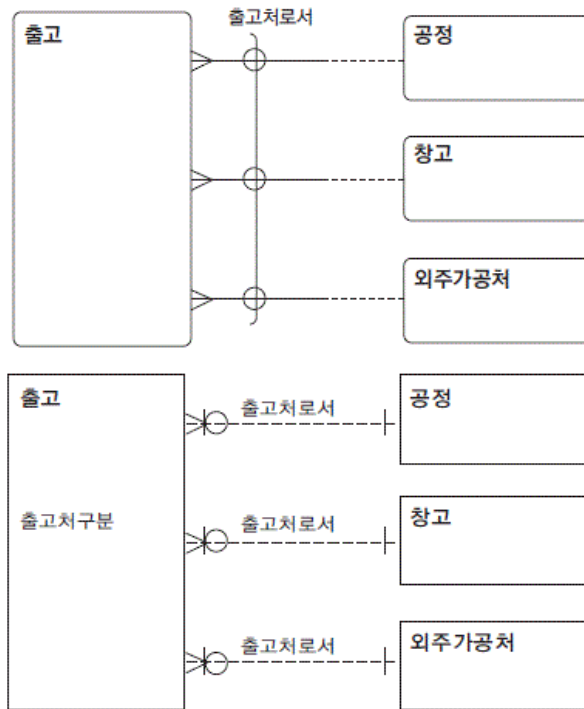
BOM 모델은 M:M 순환 관계라 하며, 상세 모델링 과정에서 새로운 관계 엔터티(Relation Entity, Intersection Entity)를 추가하여 두 개의 일대다(1:M) 관계로 구성된 모델로 구체화된다.

3) Arc (Mutually Exclusive) 관계

어떤 엔터티가 두 개 이상의 다른 엔터티의 합집합과 관계를 가지는 것을 배타적(Exclusive) 관계 혹은 아크(Arc) 관계라 한다. 이러한 아크 관계는 동일한 의미의 관계가 서로 다른 하나 이상의 엔터티와 배타적으로 관계를 갖고 있을 때 이를 하나로 통합함으로써 발생하게 된다.

이러한 관계의 특징은 다음과 같다.

- 아크 내에 있는 릴레이션십은 보통 동일하다.
- 아크 내에 있는 릴레이션십은 항상 필수이거나 선택사양이어야 한다.
- 아크는 반드시 하나의 엔터티에만 속해야 한다(하나의 아크가 여러 엔터티를 가질 수 없다).
- 어떤 엔터티는 다수의 아크를 가질 수 있다. 그러나 지정된 관계는 단 하나의 아크에만 사용되어야 한다.



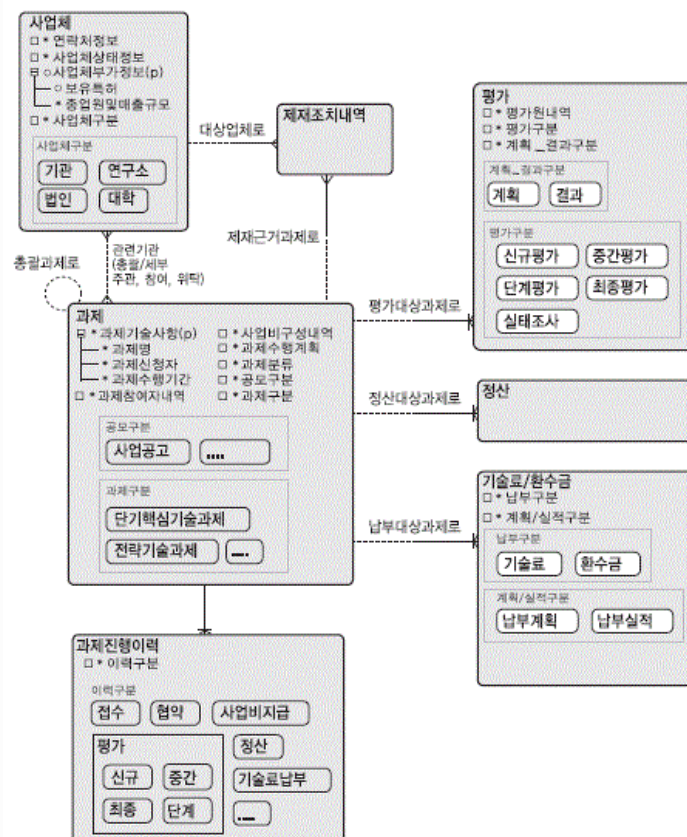
[그림 4-2-40] 바커 표기법(위)과 IE 표기법(아래)에 따른 Arc 관계 표현 예

개념 데이터 모델링 이해 주제 영역 정의 후보 엔터티 선정 핵심 엔터티 정의 관계(Relationship) 정의

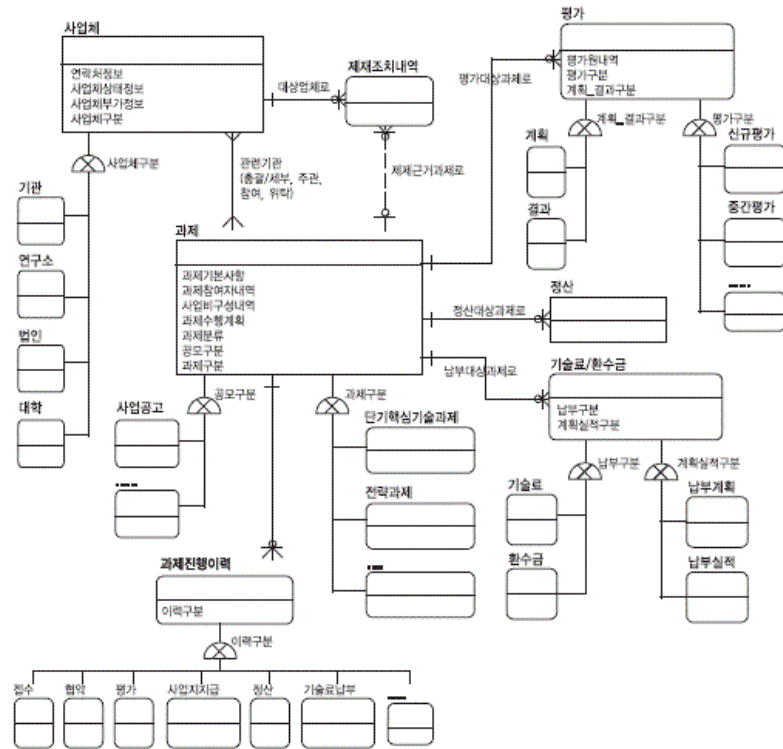
개념 데이터 모델 작성

+ 개념 데이터 모델의 구성 요소

개념 데이터 모델은 핵심 엔터티(키엔터티, 메인엔터티)와 핵심 엔터티 사이의 관계 도출을 통해 핵심 구조라 할 수 있는 데이터 모델의 골격을 정의한 것이다. 이것은 데이터 아키텍처 상에서 개괄 데이터 모델로부터 업무 요건을 충족하기 위해서 데이터 주제영역별로 상세화하여 핵심 엔터티들을 도출하고 관계를 정의함으로써 생성되기도 하고, 수집된 엔터티 후보들을 검토하고 분류하여 핵심 엔터티들을 도출하고 이들 간의 관계를 정의함으로써 생성되기도 한다. 주제영역별로 작성된 개념 데이터 모델을 전사 영역으로 확장하여 하나의 개념 데이터 모델로 작성한 것을 전사 개념 데이터 모델(Enterprise Conceptual Data Model) 또는 확장 개념 데이터 모델(Extended Conceptual Data Model)로 부르기도 한다. 전사 개념 데이터 모델에서는 전사 관점의 핵심 엔터티와 관계를 통해 전사적인 핵심 데이터 구조를 정의한다.



[그림 4-2-41] 바커 표기법에 따른 개념 데이터 모델 작성 사례



[그림 4-2-42] IE 표기법에 따른 개념 데이터 모델 작성 사례

개념 데이터 모델은 시스템에 대한 사용자의 요구사항을 정형화된 모델로 표현함으로써 사용자가 요구하는 데이터의 범위 및 구조를 용이하게 확인할 수 있고, 사용자의 요구 사항을 사용자와 함께 검토하여 신규 시스템에 해당 요구사항을 반영할지 여부를 결정하여 개발범위를 정하는데도 도움을 준다. 또한 개괄 데이터 모델 및 전사 개념 데이터 모델과의 불일치가 발생하지 않도록 데이터의 골격을 유지하고 향후의 논리 및 물리 데이터 모델과의 구조적 정렬(얼라인먼트)을 지원한다.

개념 데이터 모델은 주요 핵심 엔터티와 이들 간의 관계 정의 위주로 구성되어 데이터의 전체적인 골격을 파악하는 것이 목적이기 때문에, 작성된 개념 데이터 모델 상에서는 속성 표현이 불필요할 수 있다. 그러나 개념 데이터 모델을 구성하는 핵심 엔터티들이 데이터의 발생 기원 측면에서 상위 수준에 해당하기 때문에, 전체 데이터 구성에 대한 이해를 돕기 위해 향후에 이들이 세분화되거나 자식으로 탄생될 수 있는 하위 엔터티나 부가적인 엔터티들을 속성이나 서브타입 형태로 표현하는 경우도 있다. 드물지만 때로는 엔터티를 이해하는데 도움을 주거나 해당 엔터티의 속성으로 명백한 일부 핵심 속성이 함께 도출되기도 한다. [그림 4-2-41]과 [그림 4-2-42]는 각각 바커 표기법과 IE 표기법에 따라 작성한 개념 데이터 모델의 사례이다.

▣ 개념 데이터 모델 미작성시 영향

- 개괄 데이터 모델 및 전사 개념 데이터 모델과의 불일치 사항이 발생할 수 있다.
- 논리 및 물리 데이터 모델 작성 시 사용자 요구사항 반영이 누락되거나 잘 못 반영될 수 있다.
- 주제영역 간 혹은 업무 간 데이터 연관에 있어 범위가 불명확해져 오류가 발생할 수 있다.

▣ 개념 데이터 모델의 작성이 불필요한 경우

개념 데이터 모델은 반드시 작성하는 것을 원칙으로 하는 것이 바람직하지만, 시스템의 특성이 대체적으로 연산처리 중심이거나 업무에 연관된 데이터를 저장하지 않는 경우는 생략할 수 있다.

+ 개념 데이터 모델의 작성 방법

▣ 개괄 데이터 모델로부터 상세화하는 경우

- 개괄 데이터 모델 또는 전사 개념 데이터 모델로부터 개념 데이터 모델을 작성할 단위 주제영역을 결정한다.
- 도출된 핵심 엔터티를 주제영역과 매핑한다.
- 주제영역별로 해당 엔터티들 간의 관계를 정의하여 개념 데이터 모델을 작성한다.
- 개념 데이터 모델을 상위 수준의 어플리케이션 모델이나 프로세스 모델과 비교하여 검증한다.

▣ 수집된 엔터티 후보로부터 작성하는 경우

- 수집된 엔터티 후보들을 검토하고 분류하여 핵심 엔터티를 도출한다.
- 수집된 엔터티 후보들과 핵심 엔터티들을 분류하여 데이터 주제영역을 정의한다.
- 데이터 주제영역별로 해당하는 핵심 엔터티들을 배치하고 이들 간의 관계를 정의하여 개념 데이터 모델을 작성한다.
- 개념 데이터 모델을 상위 수준의 어플리케이션 모델이나 프로세스 모델과 비교하여 검증한다.

▣ 현행 데이터 리버스를 통해 작성하는 경우

-

현행 물리 데이터 모델을 생성하고 상세화 및 논리화를 거쳐 현행 논리 데이터 모델을 작성한다.

- 현행 논리 데이터 모델의 엔터티들을 분류하여 핵심 엔터티를 도출하고 현행 데이터 주제영역에 매핑한다.
- 현행 논리 데이터 모델로부터 현행 핵심 엔터티 간의 관계를 정의하여 현행 개념 데이터 모델을 작성한다.
- 사용자 요구사항이나 현행 시스템 분석 결과 또는 선진 사례 등을 검토하여 개선 사항을 반영함으로써 현행 개념 데이터 모델로부터 목표 개념 데이터 모델을 생성한다.
- 목표 개념 데이터 모델을 상위 수준의 어플리케이션 모델이나 프로세스 모델과 비교하여 검증한다.



논리 데이터 모델링

Home > DB 구축 가이드 > DA 가이드 > 데이터 모델링 > 논리 데이터 모델링

논리 데이터 모델링 이해

속성 정의

엔터티 상세화

이력 관리 정의

논리 데이터 모델 품질 검토

+ 논리 데이터 모델링 정의

논리적 데이터 모델링이란 데이터베이스 설계 프로세스의 Input으로써 비즈니스 정보의 구조와 규칙을 명확하게 표현하는 기법이다. 논리적 모델은 데이터 모델링이 최종적으로 완료된 상태를 말한다. 즉, 물리적인 스키마 설계를 하기 전 단계의 데이터 모델 상태를 일컫는 말이다.

논리적 데이터 모델링의 핵심은 어떻게 데이터에 액세스하며, 누가 데이터에 액세스하며, 그러한 액세스의 전산화와는 독립적으로 비즈니스 데이터에 존재하는 사실을 인식·기록하는 기법일 뿐만 아니라 철학이다.

특히 데이터 모델링 과정에서 가장 핵심이 되는 부분이 논리 데이터 모델링이라고 할 수 있다. 데이터 모델링이란 모델링 과정이 아닌 별도의 과정을 통해서 조사하고 결정한 사실을 단지 개체-관계 다이어그램(ERD, Entity Relationship Diagram)이라는 그림으로 그려내는 과정을 말하는 것이 아니다. 시스템 구축을 위해서 가장 먼저 시작할 기초적인 업무조사를 하는 초기 단계부터 인간이 결정해야 할 대부분의 사항을 모두 정의하는 시스템 설계의 전 과정을 지원하는 과정의 도구라고 해야 할 것이다.

+ 논리 데이터 모델링 목적 및 효과

■ 해당 비즈니스에 대한 데이터 관점에서의 명확한 이해

데이터 모델링을 한마디로 하면 업무의 데이터 관점에서의 표현 또는 설계라고 할 수 있다.

■ 전사적인 통합 데이터 체계 확립

논리 데이터 모델링을 통하여 전사의 데이터에 대한 구조를 체계화하고 이를 통한 통합 데이터 체계를 확립한다.

■ 데이터의 일관성 및 정확성 유지를 위한 규칙 도출

기업이 관리하는 데이터의 일관성, 정확성을 유지하기 위해서는 데이터에 대한 정확한 업무 규칙과 데이터 처리 규칙들을 생성 관리해야 한다. 이것을 표현하고 관리하는 것은 논리 데이터 모델을 통해서 하게 된다.

■ 안정적인 데이터베이스 설계의 토대 마련

논리 데이터 모델을 통하여 물리 데이터 모델이 생성된다. 특히 데이터 모델의 구체적인 실체(객체)인 데이터베이스를 안정적이고 체계적으로 생성하는 기초가 된다.

■ 사용자와의 명확한 의사소통을 위한 수단으로 활용

시스템 설계자와 업무 담당자 간의 의사소통의 수단으로 논리 데이터 모델이 사용된다. 반면 설계자와 개발자 간의 의사소통의 수단은 물리 데이터 모델이라고 볼 수 있다. 하지만 이 과정에서 개발자라고 하더라도 논리 데이터 모델에 표현되어 있는 비즈니스 규칙에 대한 이해가 선행되어야 한다.

+ 논리 데이터 모델링 필수 성공요소

■ 업무에 능통한 현업 사용자와 함께 데이터 모델링을 진행하라

데이터 모델링 작업은 결국 업무를 데이터 관점에서 체계화하는 작업이다. 즉, 비즈니스에서 사용되는 데이터를 집합(엔터티)으로 생성하고 이들 간의 관계(Relationship)를 지정하는 작업이다. 이러한 작업을 하는 과정에서 업무를 알고 있는 현업 사용자의 참여는 필수적이다.

■ 절차(Procedure)보다는 데이터에 초점을 두고 모델링을 진행하라

데이터 모델에는 순서와 시간, 흐름의 개념이 들어가지 않는다. 이러한 개념을 넣어서 마치 데이터 흐름도(DFD, Data F

low Diagram)처럼 업무의 흐름에 따라 엔터티가 발생하는 경우를 실전에서는 많이 보게 된다. 이런 식의 데이터 모델에서는 많은 데이터의 중복과 데이터 정합성을 훼손할 개연성을 내포한 모델이 만들어지게 마련이다. 그렇기 때문에 가능한 절차를 배제한 채로 데이터 모델을 생성해야 한다.

- 데이터의 구조(Structure)와 무결성(Integrity)을 함께 고려하라
- 개념화(Conceptualization)와 정규화(Normalization) 기법을 적용하라
- 가능하면 다이어그램(Diagram)을 이용하여 업무를 표현하라
- 데이터 모델링을 지원하는 데이터 사전을 구축하라

논리 데이터 모델링 이해 **속성 정의** 엔터티 상세화 이력 관리 정의 논리 데이터 모델 품질 검토

+ 속성 개념

▣ 속성 정의

속성은 엔터티에서 관리되는 구체적인 정보 항목으로 더 이상 분리될 수 없는 최소의 데이터 보관 단위이다. 예를 들어, 엔터티 사원에 속하는 모든 엔터티는 이름을 갖고 있다. 또한 모든 사원에는 입사일자, 사원번호, 생년월일 등의 특성을 가지고 있다. 엔터티 사원에 속하는 모든 인스턴스들이 공통으로 가지는 이러한 특성을 속성(Attribute)이라고 한다. 각 엔터티들은 일련의 속성들에 의해 상세화될 수 있다.

▣ 속성 특징

1) 속성의 어원적 의미

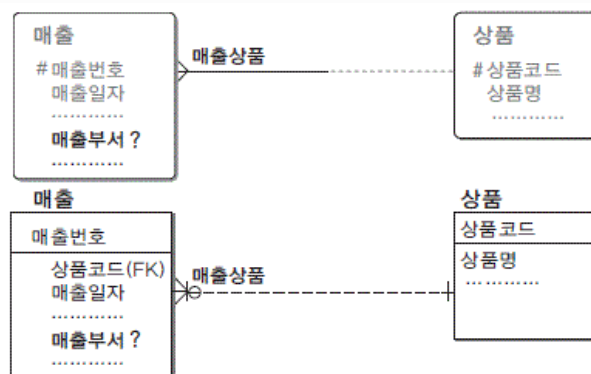
속성이라는 의미에는 가공되지 않은 것이라는 의미도 포함되어 있다. 이 말은 곧 원천적인 것을 의미한다. 또한 고유한 성질이란 의미도 가지고 있는데 이 말은 남의 도움을 받지 않더라도 자기만이 가지고 있는 독자적인 성질이 반드시 있어야 함을 뜻한다. 혼자서도 독자적인 성질을 가지고 있느냐에 대한 문제는 절대적이 아니라 상대적이라는 것 때문에 판단이 쉽지 않다. 즉 어떤 시스템의, 어떤 엔터티에서, 어떤 목적으로 사용하려고 하느냐에 따라서 달라지기 때문에 사람의 종합적인 판단이 필요하다는 것이다. 이러한 속성의 어원적인 특징들은 속성 검증의 원칙으로도 사용된다.

2) 속성도 일종의 집합이다.

물리 데이터 모델링 단계에서 엔터티는 테이블이 되고, 속성은 칼럼이 된다. 결국 속성에는 데이터 값이 들어가게 되며, 그 값들은 여러 종류를 가지게 된다. 가령, 사원 정보 엔터티의 직책 속성에는 사원, 대리, 과장, 부장 ... 등 여러 종류의 값들이 존재한다. 만약 이 값들의 종류 하나 하나를 개체 라고 한다면, 직책이란 속성은 결국 이들을 구성요소로 하는 집합이란 뜻이 된다.

3) 릴레이션십도 속성이다.

물리 데이터 모델링 단계에 가서 보면 릴레이션십 또한 결국은 일종의 속성이 될 수 밖에 없다.



[그림 4-3-1] 바커 표기법(위)과 IE 표기법(아래)에 따른 매출 상품 모델 예

■ 가) 매출 부서

[그림 4-3-1]에서 매출 부서는 엄격히 말해서 매출 엔터티의 속성이 아니라 부서 엔터티와의 릴레이션십이라고 하는 것이 정확한 표현이다. 사실 이론적으로 따진다면 속성은 전체 엔터티를 통틀어 반드시 유일하게 존재해야만 한다. 나 자신(속성)은 우리집(엔터티)에 속하고, 거기서 유일하지만 다른 모든 것(엔터티)들에 대해서도 반드시 유일한 존재이다.

매출 엔터티에 있는 매출 부서 속성에는 나중에 분명히 부서 코드가 들어온다. 부서 코드 속성이 오직 한 곳에만 위치해야 한다면 반드시 부서 엔터티에 있어야만 한다. 이 말은 다른 엔터티에 있는 모든 부서 코드는 사실 모두가 릴레이션십이라는 것을 의미한다.

다. 이 릴레이션십은 나중에 데이터베이스 설계 단계에서 관계형 데이터베이스로 설계하고자 한다면 매출 테이블에 매출 부서라는 외래키 컬럼으로 존재하게 된다.

■ 나) 매출 상품 릴레이션십

[그림 4-3-1]의 좌측에 있는 매출 상품 릴레이션십은 속성이 아닌 릴레이션십으로 제대로 표현 되어 있다. 설계 단계에서 결과적으로 동일해지는데 굳이 이것을 릴레이션십으로 표현하여 그림을 복잡하게 할 필요가 있겠는가?

물론, 원론적으로 본다면 이를 무조건 릴레이션십으로 표현하는 것이 옳다는 것에는 이견이 있을 수 없다. 왜냐하면 우리가 작성하는 논리적 데이터 모델이란 업무를 체계적으로 형상화하는 것이지 데이터베이스의 구조를 정의하는 것이 아니기 때문이다. 엄격히 말한다면 논리적 데이터 모델에는 데이터베이스나 컴퓨터와 같은 물리적 요소들은 전혀 감안하지 않은 상태에서 정의되어야 한다.

4) 속성들 간은 서로 독립적이다

속성들은 반드시 식별자에 직접 종속되어야 한다. 이 말은 정규화의 제2정규형에 해당하는 말이 다. 만약 속성들 간에 종속성이 존재한다면 이들은 별도의 엔터티로 분리되어야 한다. 이것이 바로 제3정규형이다.

+ 속성 후보 도출

속성을 결정할 때에도 다양한 후보를 준비하고 이들 중에서 속성의 검증 규칙에 부합하는 것을 속성으로 최종 결정하게 된다. 특히 이러한 후보 도출 작업은 기존의 현장조사에 국한하지 않고 좀더 적극적이고 개선적인 사고를 가지고 사용자에게 많은 질문을 하고 확인해 속성 후보를 도출한다.

▣ 속성 후보 수집처

속성은 결국 속성 후보 중에서 선택되기 때문에 우리는 일단 다양한 경로를 통해서 좋은 후보를 가능하다면 최대한 많이 확보해야 한다.

1) 구 시스템의 문서자료

가동 중인 기존 시스템의 데이터 구조 및 프로세스 명세들이 나타나 있는 설계 자료에서부터 사용자를 위한 지침서에 이르기까지 다양한 도큐먼트가 있을 것이다. 이 자료는 엔터티 후보 및 속성 후보를 도출하는데 가장 유용하게 사용할 수 있다.

2) 현업 장표/보고서

현업에서 사용하고 있는 각종 장표나 보고 자료들을 수집해서 조사해 보는 것이다. 현업 업무 중에는 업무를 효과적으로 처리하기 위하여 많은 장표와 각종 보고 자료를 만들고 있다. 물론 이들을 그대로 속성 후보로 선정할 수 있는 것은 결코 아니다. 그 중의 상당 부분은 다른 속성에 의해 만들어질 수 있는 가공된 결과(추출 속성, Derived Value)인 경우가 많기 때문이다. 더구나 이것들은 대부분 정규화가 되어 있지 않기 때문에 정확히 파악해야만 진정한 의미의 속성 후보를 찾아낼 수 있다.

3) 사용자와 협의

데이터 모델링 과정에서부터 시종일관 현업 담당자들과 같이 진행하는 것이 데이터를 모델링하는 최상의 방법이다.

4) DFD의 DD(Data Dictionary)

업무 파악 및 시스템 분석을 위한 기능 설계로 자료 흐름도가 존재한다면 여기에 있는 데이터 저장소(Data Store)와 데이터 사전(Data Dictionary)에 있는 정보를 이용하여 속성의 후보를 도출할 수 있다. 자료 흐름도를 작성할 때 생성되는 자료 저장소는 비록 테이블처럼 표현되지만 사실은 이러한 내용의 데이터가 저장되어야 한다는 데이터의 추상화된 집합이라 할 수 있으며, 그 속에 들어가야할 구체적인 속성이 데이터 사전에 기술된다.

5) 전문 서적 및 자료

동일한 업무에 대한 전문 서적 또는 자료를 통해서도 속성의 후보를 도출할 수 있다.

6) 다른 시스템 자료

우리가 개발할 시스템의 주변을 살펴보면 사내외에 이와 관련된 시스템이나 유사한 시스템을 찾을 수 있을 것이다. 만약 여러분이 그들이 가지고 있던 속성 또한 후보로서 참조하는 것도 속성 후보를 찾는 데 좋은 방법이다.

▣ 속성 후보 선정 원칙

속성의 후보를 선택할 때는 최대한 충분히 수집해야 한다. 그것은 검토한 결과 자신의 속성이 아닌것으로 판명될지라도 검토할 기회를 제공해 주는 단서라는 중요한 역할을 하기 때문이다.

1) 원시(Source) 속성으로 보이는 후보는 버리지 않는다

다른 속성에 의해 다시 재현할 수 있는 가공(추출, Derived) 속성이 아닌, 다시 말해서 만약 이 속성이 없다면 다시는 재현할 수가 없을 때 이 속성을 원시 속성이라고 부른다. 재현이 불가능하다면 이것을 버리는 순간, 이미 정보는 소실되어 버리므로 절대 버려서는 안된다.

2) 소그룹별로 후보군(Pool)을 만들고 가장 근접한 엔터티에 할당한다

모든 엔터티가 정의되어 있는 상태가 아니라 단지 핵심 엔터티들을 대상으로 모델링을 실시해왔을 뿐이므로 아직은 모든 엔터티가 드러나 있지는 않다. 그렇기 때문에 각 속성 후보들을 적절한 데이터 그룹으로 생성하여 두는 것이 필요하다.

▣ 속성의 기본 구성요소

1) 속성명

속성의 내용이나 목적이 무엇인지 알려주는 명사 또는 명사구이다. 기업에서 널리 사용하는 용어 를 쓴다.

- 속성의 의미를 명확히 표현하는 함축성 있는 명사 혹은 명사구를 사용한다.
- 해당 업무에서 일반적으로 사용하는 용어를 사용한다.
- 실제명은 속성명으로 사용하지 말아야 한다.
- 필요시 표준 약어를 제정하여 속성명을 생성하고 그 속성명을 단 하나의 실체에만 속하도록 하는 것이 바람직하다.

2) 도메인

속성이 지닐 수 있는 값에 대한 업무적인 제약 조건으로 파악된 일련의 특성이다. 모든 영역에서 같은 도메인을 사용하는 것이 좋다. 속성이 기존 도메인 집합에 속해 있지 않는 경우에는 새로운 도메인을 추가한다. 도메인은 다음과 같은 속성들을 가진다.

- 데이터 타입
- 길이
- 허용 값(Permitted Value): 속성에 지정할 수 있는 모든 값들의 집합
- 디폴트 값 및 디폴트 알고리즘

3) 선택성

모든 건의 해당 속성이 반드시 값을 가져야 하는지 여부를 나타낸다.

- 선택성 조건 => 선택성이 다른 속성 값에 의해 영향을 받는 경우
- 필요조건 / 금지조건 / 무관계조건

+ 속성 검증 및 확정

다양한 경로를 통해 수집된 속성 후보를 엔터티에 배정시킨 후 이제 우리가 해야 할 일은 이들을 검증하여 제자리를 찾게 해 주는 일이다. 속성을 검증하는 작업은 총 4단계로 나누어 실시한다.

▣ 최소 단위(Atomic Value)까지 분할하라

1) 검증 방법

- 집합 개념의 속성은 단순 개념으로 분할한다.
- 가능한 최소 단위까지 분할한 후 관리 필요에 따라 통합한다.
- 일자, 시간, 성명, 주민등록번호, 우편번호 등은 일반적으로 분할하지 않는 것이 좋다.
- 분할 및 통합의 기준은 업무의 요구 사항에 따른다.

2) 분할 속성의 대표적 유형

- 가) 일자(日字) 형태의 속성 : 매출일자

분리된 것들이 속성의 자격을 가지고 있는지를 검토하기 위해서는 이들이 자기 혼자서도 독립적 인 의미를 가지고 있는지 확인해 보아야 한다. 매출월(mm)이 매출년도(yyyy)나 매출일(dd)을 무 시하고 혼자서도 어떠한 의미를 가지고 있겠는가? 아마 대부분의 경우는 이들을 하나로 결합했을 때만 매출이 발생한 일자라는 의미를 가질 수 있을 것이므로 이런 경우라면 통합된 것이 속성이라고 보아야 한다.

	통합된 속성	분리된 속성
①	매출일자	매출년도 매출월 매출일
②	처리일시	처리일자 처리시간
③	우편번호	시군구 읍면동
④	전화번호	지역번호 국번 개별번호
⑤	전표번호	처리일자 일련번호
⑥	주소	지역주소 상세주소
⑦	발생일자	입고일자 출고일자 선적일자 하역일자 입문일자 출문일자

[그림 4-3-2] 분할 속성 예

- 나) 외부에서 공인된 속성 : 우편번호 유형

국가나 공공기관에 의해서 이미 공인되어 있는 각종 번호들(예; 우편번호, 주민등록번호, 사업자 등록번호, 법인번호, 여권번호 등)에 대한 속성 관리 형태는 매우 유사하다. 이들은 단지 길이에서 차이가 있지만 대부분 000-000 형식으로 나타난다. 우편번호의 앞의 세 자리는 마치 성명의 성과같이 나름대로 확실한 의미가 있다. 이 세자리 숫자는 반드시 단 하나의 시군구만 지칭하므로 분명한 의미가 있다는 것이다. 그러나 뒤의 세 자리는 앞의 시군구가 없으면 독자적으로 의미를 갖지는 못한다.

- 다) 전화번호 유형 : 전화, 팩스번호

전화번호는 지역번호(DDD)+국번+개별번호로 나누어진다. 일반적인 의미로 본다면 이들 각각 에는 분명히 독립적인 의미가 있다. 그러나 지금 우리가 검토하고자 하는 엔터티 내에서 과연 독립 적인 의미를 가지느냐는 또 다른 문제일 수 있다. 예를 들어, 고객 엔터티에 있는 고객의 연락전화 번호를 검토한다고 가정해보자. 국번만 독립적인 의미로 부여할 경우가 있는가? 뒷자리 4자리의 개별번호만 액세스하여 어떤 처리나 참조를 할 가치가 있는가? 지역번호만 독립적인 의미로 인정 하여 처리할 경우가 있겠는가? 물론 이러한 질문에 대한 대답은 그 엔터티의 내용이나 앞으로의 관리 수준에 많은 영향을 받는다.

- 라) 주소 유형 : 고객 주소

주소 형태로 관리되어야 하는 일단의 속성들은 어떤 시스템에서도 자주 등장하는 매우 중요한 속성의 한 유형이다. 주소를 세부적으로 나누는 방법은 여러 가지를 생각해 볼 수가 있다. 가령 도(광역시) + 시군구 + 읍면동 + 단지(아파트, 건물) + 동호수(번지)로 아주 세분화를 시킬 수도 있다. 크게 나누어 우편번호와 연결되어 사전에 이미 생성되어 있는 주소 부분(여기서는 지역주소라고 표현함)과 나머지 부분(상세주소로 표현)으로 분리하는 방법이다.

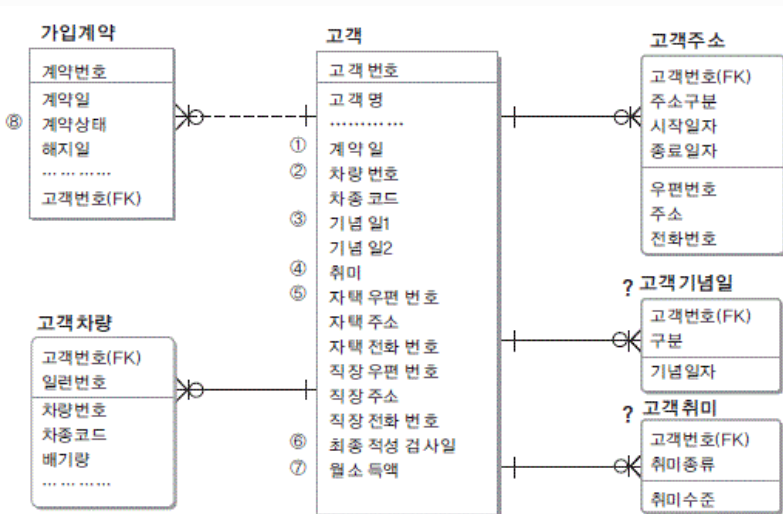
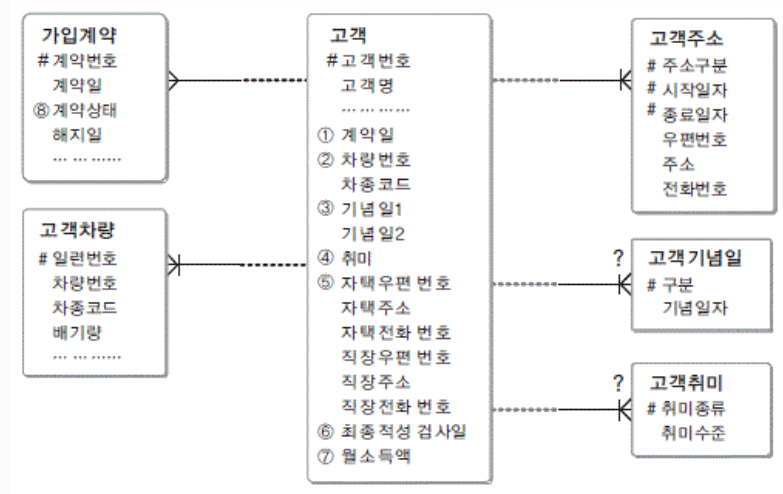
▣ 하나의 값(Single Value)만을 가지는지 검증한다

속성에서 관리되어야 할 값이 반드시 단 하나만 존재해야 한다는 것이다. 그러나 이 말의 진정한 의미는 그 속성에 들어올 수 있는 값의 종류가 반드시 하나만 존재하고 있다는 의미는 절대 아니다. 쉽게 설명한다면 그 엔터티에 들어가는 개체마다 반드시 하나의 값만 보유하고 있어야 한다는 것이다.

1) 검증 방법

- 여러 값을 가지거나 반복되는 속성은 잘못된 속성이다.
- 반복되는 속성은 새로운 엔터티로 분할해야 할 1차 정규화의 대상이 된다.

2) 대표적 유형



[그림 4-3-3] 바커 표기법(위)과 IE 표기법(아래)에 따른 Single Value 예

가) 계약일

계약일은 고객 엔터티의 속성이 아니라 가입계약의 속성이다. 즉, 고객은 여러 개의 계약일을 가질 수 있기 때문에 고객의 속성이 될 수 없다. 상황에 따라서 아직 가입계약 엔터티가 도출되지 않았을 수도 있고, 이미 도출되어 있을 수도 있다. 만약 아직 도출이 되지 않은 상태라면 이 속성에 대한 유일 값 검증에 의해서 가입계약 엔터티는 자연스럽게 도출될 것이며, 이미 존재한다면 이 속성을 거기에 적절히 반영하면 된다.

나) 차량번호

어떤 고객은 하나 이상의 차량을 가질 수도 있다. 물론 지금까지 보유한 차량의 이력을 관리하고자 한다면 당연히 유일하지 않을 것이고, 현재 입장에서만 보더라도 하나 이상의 차량을 보유한 사람들은 얼마든지 존재한다. 그러나 이처럼 실제로는 하나 이상이 존재하지만 업무적으로 판단했을 때 굳이 하나 이상의 차량을 관리할 필요성이 없거나 과거의 이력을 관리할 의사가 없다면 모델링 입장에서는 유일값이 되므로 이들은 이 엔터티의 속성이다.

다) 취미

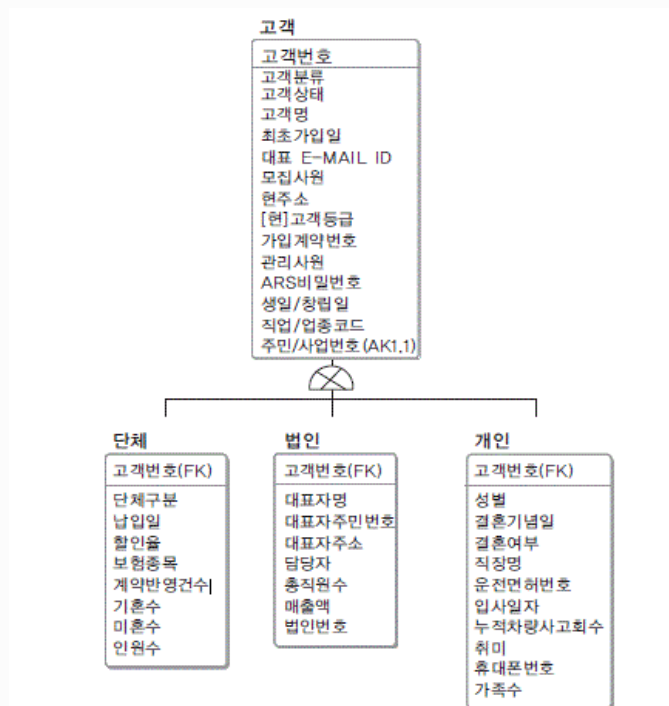
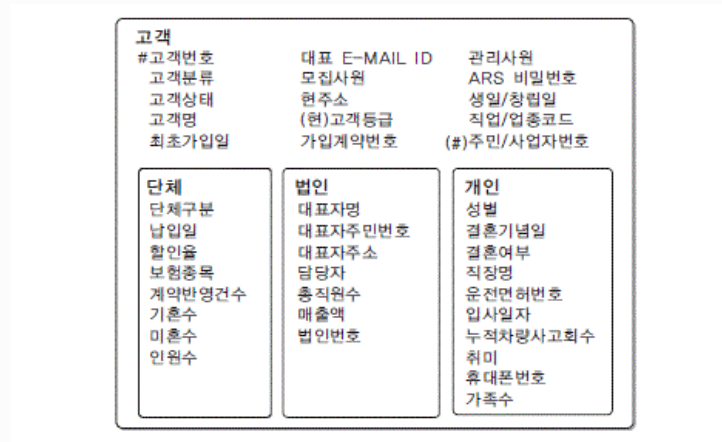
취미는 사람에 따라 당연히 여러 가지를 가질 수 있다. 취미와 같은 고객의 성향 정보는 마케팅의 중요한 자료가 된다. 엄청나게 많은 고객 모두를 대상으로 마케팅을 한다는 것은 너무 많은 비용이 들어가기 때문에 가능성이 높은 고객들을 대상으로 표적 마케팅을 하지 않을 수 없다. 이때 고객의 각종 성향 정보는 매우 중요한 가치를 가진 정보이므로 이를

최대한 확보하려는 노력은 반드시 필요하다.

▣ 추출 속성(Derived Attribute)인지 검증한다

속성 검증의 제3단계는 그 속성이 원천적인 값인지, 다른 속성에 의해 가공되어서 만들어진 값인지를 검증하자는 것이다. 원천적인 값이란 말 그대로 다른 것에 의해 만들어진 것이 아닌 태초부터 창조된 것을 말하며, 추출 값이란 이들을 가지고 언제라도 쉽게 재현할 수 있는 속성을 말한다. 어찌면 이것은 매우 분명하고 단순한 개념이지만 실전적인 시각에서 바라보면 그리 간단하지만은 않다.

1) 대표적 유형



[그림 4-3-4] 바커 표기법(위)과 IE 표기법(아래)에 따른 추출 속성 사례

가) 현재 정보만 관리하는 형태 : 현주소, 고객 등급 등

자식 엔터티에 있는 맨 마지막 정보를 현재의 정보라는 의미로 중복화하는 경우라고할 수 있다. 엄밀하게 말하면 이력(선분) 개념이 들어가 있는 자식 정보 중에서 현재 시점을 통과하고 있는 선 분에 해당하는 데이터를 미리 가져다 두는 형태이다.

나) 최초 정보를 보관하는 형태 : 최초 가입일, 모집 사원 등

자식 엔터티의 여러 정보 중에서 하나만 선택하는 또 하나의 방법은 바로 최초로 발생한 정보만 가져다 두는 것이다. 실전에서 발생하는 대부분의 경우는 현재 정보나 집계 형태로 나타나지만, 가끔은 최초의 정보를 보관하고자 하는 경우도 나타난다.

다) 집계 정보를 관리하는 형태 : 인원 수, 가족 수, 총직원 수 등

추출 값을 관리하는 속성 중에서 아마 가장 많이 나타나는 형태는 수행 속도를 위해 하위 엔터티의 정보를 집계해 가져다 둔 경우일 것이다. 물론 집계의 형태에는 금액 관련 속성들에서 많이 발생하는 합계(sum)와 발생회수(count)를 관리하기 위해 사용하는 건수, 회수, 차수 등의 단어가 들어가는 속성들이 대부분이다.

라) 추출 릴레이션만 관리하는 형태 : 가입 계약번호, 관리 사원 등

추출 속성 중에는 자식 엔터티의 식별자를 전부 또는 일부만 옮겨둔 것을 자주 발견할 수 있다. 논리적으로는 M쪽의 식별자가 1쪽으로 올 수는 없지만 대부분의 경우 마지막 건의 식별자를 가져 다 두는 경우가 많다.

마) 대표 정보만 관리하는 형태 : 대표 전자메일 ID, 취미, 법인의 대표자 정보

자식 엔터티에 있는 여러 개의 정보를 하나로 만들어 올리는 방법 중에는 가장 대표적인 것 하나만을 선정하는 경우도 있다. 예를 들어, 한 사람이 여러 개의 전자메일 ID를 가질 수 있다. 심지어 사람에 따라서는 십여 개를 가지고 있기도 한다.

바) 다른 속성의 일부 정보만 분리한 형태 : 성별, 결혼 여부 등

추출 속성 중에는 특별한 목적을 위해서 다른 속성의 일부를 떼어서 새로운 속성을 만드는 형태도 자주 등장하고 있다. 사실 우리가 확실한 속성이라고 할 수 있는 성별도 이러한 형태에 속한다. 만약 개인 서브타입에 속하는 모든 고객이 주민등록번호를 반드시 가져야 하고, 그 값이 확실하다고 가정한다면 일곱째 자리에 따라 남·여를 구별할 수 있다.

사) 일부분만 추출 값인 형태 : 인원 수, 법인의 대표자 정보 등

일부분이란 이 속성에 값이 들어가는 수많은 개체들 중에서 일부는 다른 엔터티에서 가공하여 만들 수 있지만 다른 일부는 원시 값인 경우를 말하는 것이다.

▣ 보다 상세하게 관리할 것이냐?

‘보다 근원적인 정보를 관리해야 하지 않겠느냐?’고 질문하는 것이다. 다시 말해 현재까지는 이대로도 충분했지만 수 많은 변화가 예상되는 미래를 대비하기 위해 좀더 근원적인 정보를 관리할 필요성에 대해 검토해 보자는 것이다. 이 단계가 바로 데이터 모델링이 현재뿐만 아니라 미래의 시스템 구조를 찾아 나가는 과정이라는 것을 의미한다

+ 추출 속성 규칙

다른 엔터티나 속성으로부터 유도되거나 계산된 속성들은 유도 또는 계산에 사용된 기준 속성(Base Attribute)에 대한 종속성과 함께 그 방법이 데이터 모델에 표현되어야 데이터 가공에 따른 연관관계를 명확하게 전달하고 기록으로서 의미를 가질 수 있다.

- 초기 데이터 모델링 이론을 보면 도출된 속성이나 가공된 속성은 중복으로 인식하고 이를 데이터 모델에 표현하지 말 것을 권고했다. 그러나 현재는 이러한 속성들이야말로 기업의 관리자들이 관심을 가지고 보는 속성으로 데이터 모델 내 반드시 기술할 것을 권장하고 있다.
- 추출(Derived) 속성이란 하나 이상의 다른 데이터 속성에 축적된 여러 사례의 값을 토대로 어떤 추가 계산 작업을 수행함으로써 선택적으로 주제를 도출하는 속성에 대하여 새로운 값을 창출하는 속성이다.
- 계산(Calculated) 속성은 엔터티의 단일 사례에 대한 어떤 특성을 기술하며, 일반적으로 관련 속성의 또다른 단일 사례로부터 계산된다.
- 추출 속성은 경영층이 진실로 원하고 필요로 하는 정보를 대표한다.
- 추출 속성은 사용자들의 데이터 요구 사항을 나타낸다.
- 데이터 모델에 추출 속성을 포함시키는 것은 그들의 물리적 구현에 대한 어떤 것도 내포하지 않는다.
- 추출 속성은 향후 과정에서 결코 기본키(Primary Key)로서의 역할을 맡지 않아야 한다.

+ 속성 정의시 유의사항

데이터 모델링을 진행하는 과정에서 속성의 정의 시 흔히 범하는 오류는 필요할지도 모른다고 생각 되는 속들을 필요할 때마다 이것저것 나열해 두는 방법으로 속성을 정의하는 것이다. 이런 방식의 속성 정의에는 진정한 엔터티 자신의 속성 이외의 다른 엔터티에서 빌려 온 것들이거나 여기저기에 있는 정보를 이용하여 가공을 해놓은 속성들이 많이 존재한다. 하지만 이러한 방법으로 정의된 속성들은 항상 데이터의 적합성을 해칠 수 있는 개연성을 가지고 있기 마련이다. 데이터 모델링에서 데이터가 들어가 살게 될 방이라 할 수 있는 속성을 명확하게 정의하는 일은 중요하다. 속성을 정의할 때에는 다음과 같은 유의 사항을 지켜야 한다.

▣ 의미가 명확한 속성 명칭을 부여한다.

속성의 명칭을 명확히 하는 것은 매우 중요하다. 이 말의 진정한 의미는 그 속성의 개념을 정확히 부여한다는 것을 뜻한다.

1) 직업이라는 속성 정의

직업의 사전적인 의미는 생계를 위하여 일상적으로 하는 일이라는 뜻이다. 하지만 이것이 ‘회사원, 전문직, 자영업, 공무원 ...’처럼 수십 가지 정도로 분류한 것을 말하는 것인지, 아니면 이 보다 훨씬 상세하게 ‘전산설계, 프로그래머 ...’ 등과 같이 수백 가지로 분류한 것을 말하는지, 아니면 수 천 가지 이상으로 상세하게 분류한 것을 말하는지를 분명하게 정의해야 한다.

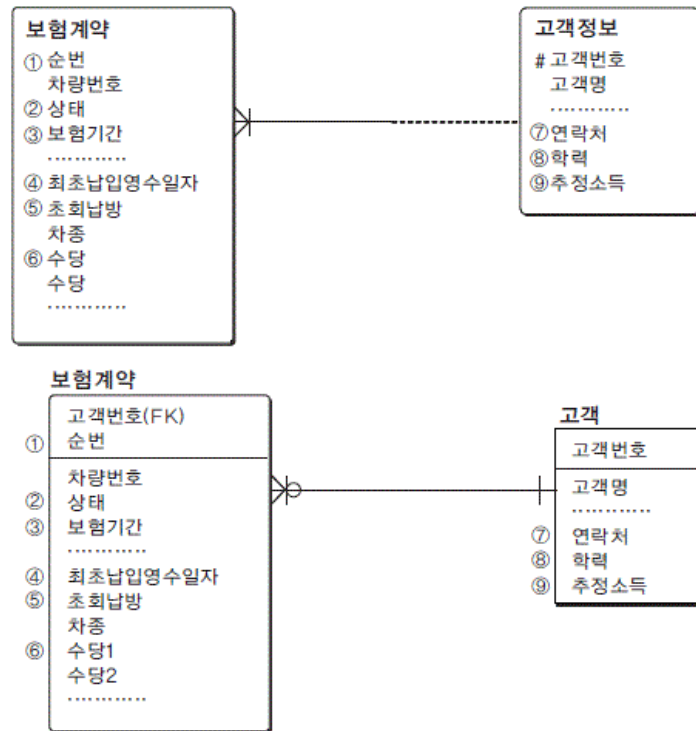
2) 학점이라는 속성

학생이 수강한 강좌에서 취득한 ‘A학점, B학점 ...’을 말하는 것인지, 그 강좌를 이수했을 때 받는 ‘2학점, 3학점 ...’을 말하는지 분명히 해야 한다는 것이다. 이런 상황에서는 설사 최초 설계 단계에서는 매우 명확한 정의가 되었다더라도 의미의 와전이나 단절은 필연적으로 발생한다.

▣ 유일한 복합명사 사용

속성의 개념을 구체적이고 명확하게 정의했다면 그 명칭을 제대로 부여하는 것도 매우 중요한 일이다. 남들에게 구구절절 보충설명을 하지 않더라도 거의 유사한 생각을 가질 수 있도록 보편적이고 함축성 있는 단어를 찾아내어야 한다. 속성이란 자신만이 가지는 분명한 독립적인 의미를 가지고 있 기 때문에 명칭 또한 단순히 일반 용어만으로 부여해서는 결코 구체적인 의미를 나타낼 수 없다. 결론적으로 말하면, 보편적인 용어를 적절히 결합한 복합명사를 만듦으로써 구체적인 표현을 할수 있다는 것이다.

[그림 5-3-5]와 같은 논리 데이터 모델을 예로 보면 모호하게 이름지어진 속성이 많이 존재한다.



[그림 4-3-5] 바커 표기법(위)과 IE 표기법(아래)에 따른 속성 명칭 예

1) 순번

많은 데이터 모델에서 순번 혹은 일련번호라는 명칭의 속성을 자주 접하고 있다. 이 속성은 인조식별자를 생성하기 위해서 사용한 것이 분명하며, 누구나 그렇게 받아들이고 있기도 하다. 그러나 분명히 이 속성의 명칭에는 문제가 있다. 여기서 만약 정확하게 표현하고자 한다면 보험계약순번 혹은 계약순번으로 지정하는 것이 옳다. 만약 순번이라고만 표현한다면 나중에 테이블로 설계가 되었을 때 이 테이블을 참조하는 하위 테이블에서 보면 어디서 온 칼럼인지 분명치 않게 된다. 만약 또 다른 엔티티에서도 순번이라는 속성을 가지고 있고 그것도 참조한다면 실제로는 서로 다른 속성이었지만 참조하는 쪽에서 보면 동일한 이름으로 나타난다는 문제가 있다.

2) 상태

이 명칭만으로는 속성의 의미가 참으로 애매모호하기 짝이 없다. 물론 심증적으로는 보험계약상태를 말하는 것으로 보이지만 확실하지 않다. 실전에서는 이처럼 오해의 여지가 남아있는 애매한 명칭들이 많이 나타난다. 이 상태가 자기 엔티티에서 지정되는 값을 관리하고자 하는 것인지, 하위 엔티티의 최종 상태를 가져다 둔 것인지, 그것도 아니면 하위 엔티티의 상태를 종합해서 새로 의미를 부여한 상태를 의미하는지 구분할 수 없다. 어떤 상태를 의미하는지 누가 보아도 알 수 있도록 명칭을 분명하게 지정해야 한다.

3) 보험기간

보험계약이 적용되는 기간일 것으로 예상되기는 하지만 가령 12개월, 20개월 ...로 표현되는 것인지 1년, 2년, ...으로 표현되는 것인지를 알 수가 없다.

4) 최초납입영수일자

간결한 명사를 최초 + 납입 + 영수 + 일자로 잘 결합하여 복합명사를 만듦으로서 누가 보더라도 오해가 없을 만큼 상세하게 표현되어 있다.

5) 초회납방

최초로 납입한 회차의 납입 방법을 말하는 것인데 설사 그 업무에 관련 있는 대부분의 사람들이 이해한다고 하더라도 최초납입회차 납입방법으로 표현하는 것이 바람직하다.

이렇듯 우리가 데이터 모델링에서 사용하는 속성명을 분명하게 표현하기 위해서 가능한 한 복합명사를 사용해야 한다.

▣ 단수형으로 속성명을 사용한다

속성은 단일 값을 저장하기 때문에 단수형으로 적는 것이 바람직하다. 예를 들어 고객 엔티티의 전자메일 속성 이름을 [전자메일들]과 같이 지었다면, 이 속성은 여러개의 값들을 저장하는 것인가? 그렇다면 속성이 될 수 없으므로 여러 속

성으로 분리하든지 아니면 별도의 고객 전자메일 엔터티로 분리해야 한다.

▣ 표준 단어 제정

표준 용어 사전을 데이터 모델링을 하기 이전에 생성하여 두면 데이터 모델의 각 객체들(엔터티, 속성, 테이블, 칼럼 등)이 최대한 그 기준을 준수하도록 유도하기가 용이하며 뚜렷한 일관성이 생길것이다. 또한 표준화 준수 여부를 관리하거나, 표준으로 변경할 대상을 선정하는 일, 어떤 속성의 파생 근원을 분석하는 데도 굉장한 힘을 발휘한다. 뿐만 아니라 속성을 구성하는 단어들을 관리하는 용어 사전에 데이터베이스 설계 단계에서 속성을 칼럼(Column)으로 전환시킬 때 사용할 영문 약어를 같이 제정해 두는 것도 바람직하다. 최근에는 모델링 도구뿐만 아니라 데이터 표준화를 전문적으로 수행하는 톨도 등장하고 있는 추세이다.

▣ 작의적인 전용 금지

전용(轉用)이란 말을 사전에서 찾아보면 예정되어 있는 곳에 쓰지 않고, 다른 데로 돌려서 쓰는것이라고 되어 있다. 이말과 통합이란 말에는 미묘한 관련이 있다. 가령 A를 위해??부터 A와 B를 위해서 만들었다면 통합이다. 그러나 결과만 놓고 본다면 어쨌거나 A와 B를 위해 사용되었다는 것은 동일하다.

전용이 발생하는 경우는 크게 세 가지로 나눌 수 있다.

- 모델러가 속성의 의미를 매우 추상적이고, 모호하게 정의했기 때문에 그것을 적용하는 개발자들이 각기 다르게 이해함으로써 나타나는 경우이다.
- 개발이 막바지에 도달했거나 이미 운용 중인 시스템에서 새로운 사용자 요구 사항으로 인해 새로운 속성의 추가가 필요할 때 나타나는 경우이다.
- ERP 패키지에서 자주 나타나는 형태로써 미리 전용의 용도로 속성을 정의해 두는 경우를 말한다.



논리 데이터 모델링

Home > DB 구축 가이드 > DA 가이드 > 데이터 모델링 > 논리 데이터 모델링

논리 데이터 모델링 이해 속성 정의 **엔터티 상세화** 이력 관리 정의 논리 데이터 모델 품질 검토

+ 식별자(UID, Unique Identifier) 확정

엔터티 내의 모든 인스턴스는 유일하게 구분되어야 한다. 이러한 유일성을 보장하기 위해서 필요한 것이 식별자이다. 식별자에는 크게 본질 식별자(Primary UID 또는 Intrinsic UID), 주(실질) 식별자(Actual UID), 대체(보조) 식별자(Secondary UID)로 구분할 수 있다. 대체 식별자는 방법론에 따라서 AK 혹은 Alternate Key로 부르기도 하지만 현재 단계에서는 Key 보다 식별자가 더 적절한 표현이다.

▣ 본질 식별자

엔터티에는 인조 식별자(Artificial Unique Identifier)가 있고, 진주어(眞主語)에 해당하는 관계나 속성이 어딘가에 있다. 개념 데이터 모델링 과정에서 키 엔터티와 일부 핵심 엔터티에 대해서 본질 식별자를 정의하였다. 여기에서는 엔터티 상세화 단계에서 정의되어질 핵심,액션(행위) 엔터티에 대한 본질 식별자를 정의한다.

본질 식별자를 정의하는 방법은 키 엔터티와 행위 엔터티가 서로 다르다. 행위 엔터티를 정의하는 방법에는 상황에 따라 하향식과 상향식으로 접근하는 방식을 적용할 수 있다. 키 엔터티는 부모가 없이 창조된 집합이므로 식별자 또한 창조시켜 주어야 한다. 그러나 행위 엔터티는 항상 부모가 누구인지를 확인하는 방식으로 진행된다.

1) 키 엔터티의 본질 식별자

키 엔터티는 우리가 잘 알고 있듯이 사원, 고객, 상품과 같이 부모 엔터티 없이도 혼자서 정의될 수 있는 엔터티이다. 예를 들면 사원 엔터티에는 이미 이전부터 정의해서 사용하고 있던 사원번호가 있다는 것은 굳이 언급할 필요도 없다. 그러나 좀더 깊이 생각해 본다면 사원 집합의 개체는 사람이므로 사실 본질 식별자는 주민등록번호라고 볼 수 있다. 물론 외국인 사원이 있다면 여권번호, 외국인 등록번호 등이나 임시로 부여한 임의의 값을 광의의 주민등록번호로 정의한다면 이것을 개념적으로는 원래의 본질 식별자로 볼 수 있다. 뒤에서 확정 식별자를 설명할 때 전략적인 부여 방법에 대해 자세하게 언급하겠지만 이 주민등록번호는 몇 가지 전략적인 이유 때문에 사원번호라는 인조 식별자에게 대표 권한을 물려준 것이다.

2) 절대 종속 / 상대 종속 의미

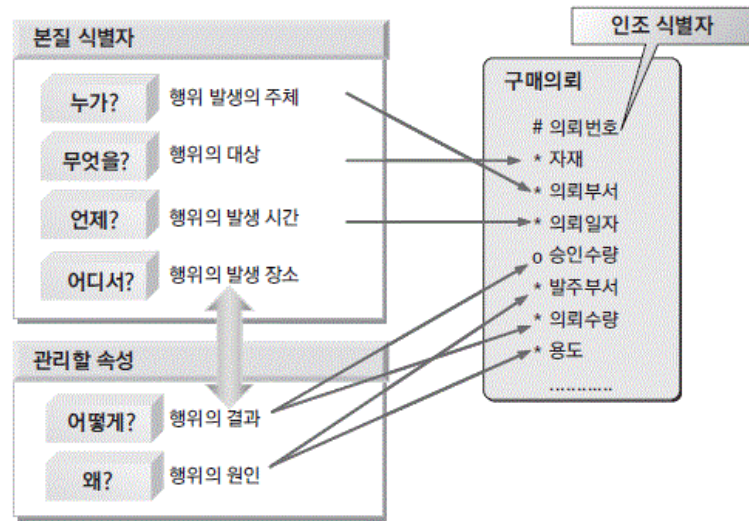
절대 종속과 상대 종속은 나를 태어나게 하는데 절대적인 영향을 주었는지, 그렇지 않는지를 따지는 것이다. 다시 말해서 내가 태어나기 위해서 절대적으로 존재했어야 하는지, 아니면 그것이 없어도 내가 태어날 수가 있는지를 확인하는 것이다. 절대 종속을 확인하는 방법은 매우 간단하다. 영화 백 투더 퓨처(Back To the Future)처럼 과거로 돌아가서 누구를 없앴더니 내가 태어나지 않게 된다면 그는 나를 태어나게 하기 위해 절대적으로 필요한 존재이다. 반면에 누군가가 우리 집안과 분명히 어떤 관계를 맺고 살았지만 설혹 그가 없다고 하더라도 내 탄생에는 아무런 영향을 미치지 않는다면 그는 나의 탄생에 대해서는 상대 종속이 된다.

3) 직접 종속 / 간접 종속 의미

‘1촌이냐? 1촌 이상이냐?’를 구분한다. 즉, 부모 엔터티와의 관계가 1촌이면 직접 종속이고 1촌 이상이면 간접 종속이라고 볼 수 있다.

4) 행위 엔터티의 본질 식별자

행위 엔터티의 본질 식별자란 이와 같이 절대종속이면서도 직접종속인 것을 찾고자 하는 것이며, 결국은 자신을 태어나게 한 근본을 찾는 것이다.



[그림 4-3-6] 본질 식별자 예

본질 식별자를 찾는 가장 확실한 방법은 사건의 전모를 가장 체계적으로 표현할 수 있다는 기사 작성의 여섯 가지 원칙인 육하원칙(六何原則)을 이용하는 것이다. ‘누가, 무엇을, 언제, 어디서, 어떻게, 왜’라는 이 여섯 가지의 질문을 통해 발생한 행위를 구체적으로 규명하면 상황에 대한 정확한 사실을 규명할 수 있다.

▣ 후보 식별자 도출

이전 단계에서 정의된 본질 식별자를 기본으로 식별자의 기본 목적인 자기를 식별할 수 있어야 한다는 유일성 유지의 목적과 다른 엔터티에서 정보로 참조해야 하는 목적을 적절히 판단하여 최종 식별자를 확정해야 한다. 하나의 엔터티 내에는 식별자로 사용할 수 있는 하나 이상의 식별자가 있다. 이 중에서 하나가 식별자로 선택되게 된다. 나머지 식별자들을 후보 식별자(Candidate UID, 방법 론에 따라서는 CK 혹은 Candidate Key라고도 하지만 현재 단계에서 Key라는 표현보다는 식별자라는 표현이 더 적절하다)라고 한다. 이러한 후보 식별자들은 다음과 같은 조건을 만족해야 한다.

1) 각 인스턴스를 유일하게 식별할 수 있어야 한다

가장 기본적인 전제 조건이며, 후보 키들은 유일한 값을 가지고 이를 통해 나머지 인스턴스와 자신을 식별하는 능력을 가져야 한다. 후보 식별자는 단일 속성뿐만 아니라 하나 이상의 속성이 모인 집합으로서도 후보 식별자가 될 수 있다. 그러므로 속성 혹은 여러 속성들이 조합된 속성 집합은 전체 인스턴스에서 유일값을 가져야 한다. 그래야만 특정한 인스턴스를 선택하기 위해서 전체 인스턴스에서 유일한 값을 가지는 후보 식별자를 선택할 수 있다.

2) 나머지 속성들을 직접 식별할 수 있어야 한다

인스턴스 간에서 뿐만이 아니라 후보 식별자는 나머지 속성을 식별할 수 있는 능력을 가지고 있어야만 한다. 이는 후보 식별자가 유일 값을 가지고 있는 상황에서 특정 인스턴스를 추출하기 위해서 유일값을 가진 후보 식별자를 찾아내면, 후보 식별자에 관련된 나머지 속성을 다시 찾아낼 수 있다는 의미이다.

3) NULL이 될 수 없다

후보 식별자들은 NULL이 될 수 없다. NULL이란 해당 속성에 값이 지정되지 않았다는 적극적인 표시라고 볼 수 있다. 따라서 NULL이 허용되는 속성이 있다면 값을 넣지 않은 경우에는 NULL이 할당된다. NULL이 할당되었다는 것은 값이 없다는 것이므로 NULL이 있는 속성은 식별할 수가 없다.

4) 후보 식별자로 속성 집합을 선택하는 경우에는 개념적으로 유일해야 한다

집합으로 후보 식별자를 선택하는 경우에는 개념적으로도 유일할 것이라는 판단을 하고서 후보 식별자로 선정해야만 한다. 만일, 회원 테이블에 대한 건본 데이터를 보고서 부서+성명 집합이 유일한 값을 가진다고 판단했다고 하더라도 개념적으로 부서에 동일한 이름을 가지는 사원이 없다고 확인할 수 있겠는가? 하지만 만일에 부서를 관리하는 엔터티에서 부서명+팀명을 후보 식별자로 선택하는 것은 개념적으로 유일할 수 있다. 한 부서에 동일한 이름을 가지는 팀은 없기 때문이다.

5) 후보 식별자의 데이터는 자주 변경되지 않는 것이어야 한다

데이터가 자주 변경된다고 해서 후보 식별자가 될 수 없는 것은 아니지만 일반적으로 후보 식별자의 값은 자주 변경되지 않는다. 데이터베이스 설계 단계에서 이러한 식별자들은 대부분 인덱스를 통해 구현된다. 인덱스는 물리적으로 트리 구조를 가지고 수많은 노드 및 포인터를 관리하게 된다. 어떤 노드의 데이터가 변경되면 트리 구조를 재수정하는 데에 너무나 많은 시간이 필요하게 되어 데이터베이스의 성능을 떨어뜨리게 되므로, 인덱스에 선택되는 칼럼의 데이터가 자주 변경되는 것은 좋지 않다.

▣ 대체(보조) 식별자

대체(보조) 식별자란 원래의 식별자를 대신할 수 있는 또 다른 속성들이나 관계(관계속성)를 말한다. 가령 사원 엔터티에 공식적으로 부여된 식별자는 사원번호이지만, 만약 주민등록번호 속성이 유일한 값을 가지면서 필수적(Mandatory)으로 정의되었다면 비록 공식적인 식별자는 아니지만 식별자로서의 역할을 할 자격은 충분히 갖추고 있다. 특히 대체(보조) 식별자는 여러 참조 엔터티 중에서 원래의 식별자 보다 대체(보조) 식별자로 연결을 맺는 것이 자신에게는 훨씬 유리

한 경우에 의미가 있게 된다.

▣ 인조 식별자 지정

인조 식별자란 식별자 확정시 기존의 본질 식별자를 그대로 실질 식별자로 인정할 수 없는 여러가지 상황이 발생했을 때, 전부 혹은 일부를 임의의 값을 가진 속성들로 대체하여 새롭게 구성한 식별자를 말한다. 가령, 사원 엔터티에 이미 존재하고 있는 속성 중에서 원래의 본질 식별자를 찾으라고 한다면 주민등록번호가 될 것이다. 그러나 이 속성은 너무 길고 관리상 여러 가지 문제점이 발생하기때문에 새롭게 사원번호라는 임의의 값을 가진 인조 속성을 영입하여 공식적인 식별자 자리까지 부여받은 것이다. 인조 식별자는 다음과 같은 기준을 가지고 지정하는 것이 바람직하다.

1) 최대한 범용적인 값을 사용한다

인조 식별자의 속성은 남들이 알지 못하는 임의의 값일 수 있기 때문에 특별한 결격 사유가 없다면 가능한 한 기존에 범용적으로 사용하던 것을 그대로 사용하는 것이 좋다. 예를 들어, 이미 회사 내에 공인되어 사용하고 있는 사원번호, 상품코드, 국가나 공공 기관에서 부여한 은행코드, 국제적으로 사용하고 있는 무역상품 분류체계인 HS코드, 범용적으로 사용하고 있는 국가나 통화(通貨)코드 등은 가능하다면 그대로 사용하는 것이 여러 가지로 유리한 점이 많다.

2)유일한 값을 만들기 위한 인조 식별자를 사용한다

지금까지 정의해 왔던 본질 식별자를 그대로 사용하면 심각한 문제가 발생하는 경우가 몇 가지 있다. 여기에서는 그 중에서 한 가지인 유일값에 대한 확신이 없을 때 이를 해결하기 위해 인조 속성을 영입하는 경우에 대해 설명하고자 한다. 어떤 경우에는 본질 식별자는 논리적으로야 문제가 없지만 실제로는 유일성에 대한 현실적인 문제가 발생하므로 인조 속성의 도입을 검토해야만 한다. 물론 그 인조 속성을 아예 전체 본질 식별자를 대체하도록 하든, 그 중의 일부를 대체하게 하든 종합적 인 판단에 따라 달라지겠지만 인조 속성이 필요하다는 것은 분명하다. 사실 실전에서 인조 속성을 도입하는 상당 부분은 바로 이런 경우라고 할 수 있다.

3)하나의 인조 식별자 속성으로 대체할 수 없는 형태를 주의한다

인조 속성을 만들 때 이 속성이 구체적으로 본질 식별자의 어느 부분을 대체하고 있는지를 분명하게 정의해야 한다. 그러나 하나의 인조 식별자 속성에 같이 대체될 수 있는 것과 절대로 그렇게 해서는 안 되는 경우가 있다는 것에 유의해야 한다. 만약 이런 원칙을 어긴다면 나중에 집합 내의 개체의 정의가 모호해져 엔터티 전체를 애매한 집합으로 변질시키게 되므로 주의해야 한다. 예를 들면, 이력을 대체하는 일련번호 인조식별자를 갖고 있는 엔터티가 자식을 갖게 되면 일련번호 인조식별자가 상속되게 되고, 이때 부모에 변경이력이 발생하게 되면 자식에 변경이 없더라도 새로운 이력일련번호를 상속하여 자식을 다시 만들어야 하는 상황이 발생할 수 있으며, 이로 인해 자식의 정보를 활용하는데 있어서 혼란이나 왜곡이 생길 수 있다.

4)편의성·단순성 확보를 위한 인조 식별자를 사용할 수 있다

속성의 길이가 너무 길거나 기억하기가 어려워서 좀더 쉽고 간편한 이름으로 변경할 목적으로도 인조 속성을 추가시킬 수도 있다. 마치 사람의 이름을 간편하고 부르기 쉽도록 애칭이나 별명을 만드는 것과 유사한 목적이라 할 수 있다. 성을 포함해서 대부분 세 글자에 지나지 않는 우리나라 사람들에게는 그리 흔치 않는 일이지만 훨씬 긴 이름을 가진 외국 사람들에게 흔하게 사용된다. 이처럼 본질 식별자를 그대로 사용해도 불편이 없다면 굳이 인조 속성을 만들 필요가 없겠지만 그렇지 않다면 고려해 볼 만한 충분한 가치가 있다.

5)의미의 체계화를 위한 인조 식별자를 사용할 수 있다

의미를 체계화한다는 말을 다른 말로 쉽게 표현하면 코드화를 한다는 것과 유사하다. 과거에는 전산 은 곧 코드라는 말이 있을 정도로 모든 것을 코드화하려고 애를 썼다. 코드화한다는 말에는 곧 속성의 자릿수마다 나름의 의미를 부여하겠다는 의미를 포함한다. 논리적으로 임의의 값이란 말에는 이미 정해진 값의 형태가 포함되어 있다. 인조 속성이란 임의의 값을 정의하는 것이라고 했다. 그렇다고 해서 그야말로 아무 값이나 괜찮다는 것은 아니다. 특별한 의미를 부여할 필요가 없다면 순차적인 번호, 곧 일련번호를 정의하는 것으로도 충분하겠지만 특정한 의미를 부여함으로써 변별력이 향상된다거나 처리의 규칙이 생겨난다면 너무 지나치지 않는한 그것을 무조건 나쁘다고만 할 수는 없다.

6)내부적으로만 사용하는 인조 식별자

인조 식별자를 생성하는 또 한 가지의 경우는 현업 사용자들에게는 전혀 알려주지 않으면서 시스템 내부적으로만 사용하는 형태이다. 물론 데이터 모델링 입장에서 본다면 현업 사용자뿐만 아니라 시스템 개발이나 관리를 하는 사용자 또한 염연한 사용자임에 틀림이 없다. 여기서 설명하고자 하는 것은 새롭게 정의한 인조 식별자가 비록 업무적으로는 아무런 의미도 없지만 시스템적인 필요성에 의해서 도입하는 경우를 소개하려는 것이다.

▣ 식별자 확정

식별자는 자기 엔터티를 위해 생성하는 것처럼 보인다. 하지만 보다 중요한 결정 요소는 나를 참조 할 다른 엔터티가 원하는 형태로 결정되어야 하는 것이기 때문에 주변 엔터티의 상황을 종합적으로 살피고, 주변 엔터티의 상황도 최대한 수렴할 수 있도록 하는 것이 매우 중요하다. 그래서 식별자를 확정할 때는 자기 자신에 대한 존재 가치뿐만 아니라 남들에 대한 배려를 어떻게 조화시키느냐가 중요한 관건이다.

1) UID BAR의 두 가지 의미

UID BAR가 가지고 있는 본질을 자세히 살펴보면 크게 두 가지의 의미를 가지고 있다.

가) 식별자로서의 역할

엔터티 자신의 입장에서 보았을 때 자신의 개체들을 다른 것들과 구별될 수 있도록 유일한 값을 만드는데 일조를 한다는 의미이다.

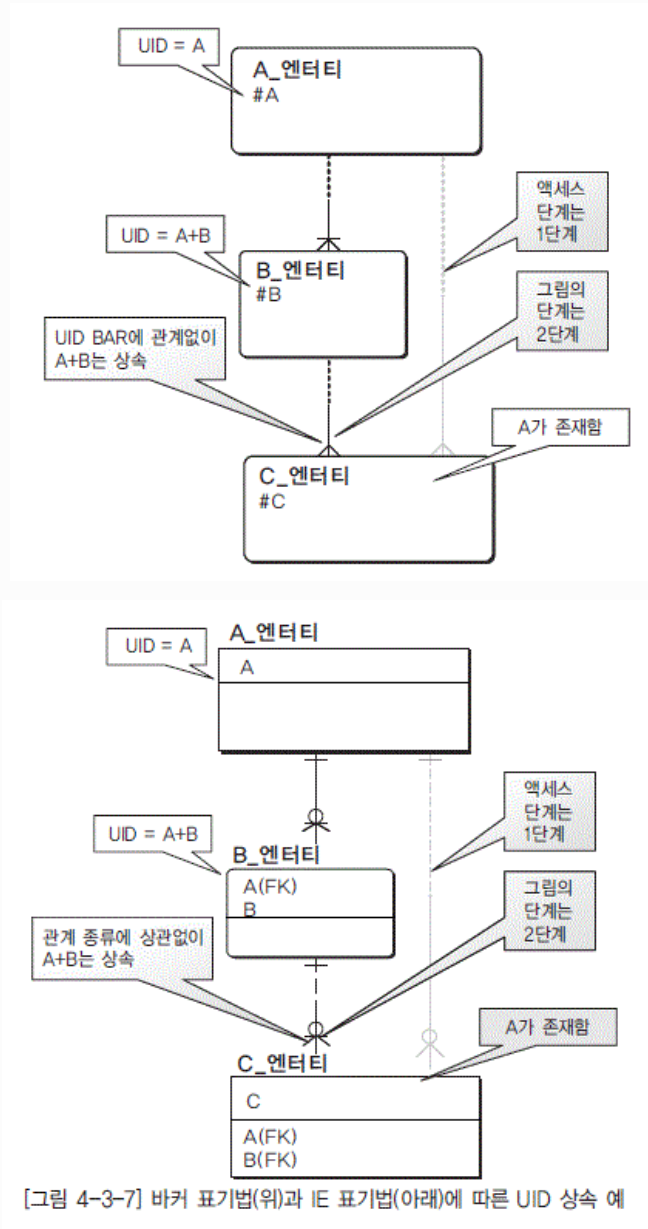
나) 정보로서의 역할

참조하는 엔터티의 입장에서 보았을 때 상대방의 식별자를 상속 받았기 때문에 자신이 보유한 정보가 증가했다는 의미도 가지고 있다.

조상의 식별자를 내가 상속 받고, 다시 그것을 내 자손들에게 물려준다면 자손들은 굳이 나를 경유하지 않고서도 조상이 누구인지 알 수 있는 것과 유사하다. 인조 식별자를 설명하면서 언급했듯이 단지 유일한 값을 만들기 위한 목적 뿐이었다면 굳이 부모에게 받은 식별자를 자??을 만드는 것이 가능하다. 이 말에는 부모의 식별자를 내 이름에 넣는 것에 상관없이 내가 태어난 이상 부모는 존재한다는 의미가 내포되어 있다. 비록 부모가 분명하더라도 부모의 식별자가 나의 식별자를 만들기 위해서 반드시 필요한 것은 아니라는 것이다. 그러나 내 식별자에 부모의 식별자를 포함시키지 않는 순간에 내 자손들에게는 더 이상 조상의 식별자를 상속시키지 못한다.

2) UID 상속과 단절의 원리

식별자 확정이란 단지 자기 식별자의 형태를 결정한다는 단순한 의미로만 생각해서는 안된다. 데이터 모델이 복잡한 업무로 인해 어쩔 수 없이 복잡해지더라도 식별자 상속이 전략적으로 이루어진다면 나중에 데이터를 처리할 때의 실제 액세스 단계에서는 훨씬 간편하게 만들 수 있다. 즉, UID의 적절한 상속과 단절 전략은 실질적인 처리의 단순화를 가져다 줄 수 있으므로 그 전략적 가치를 가지고 있다.



- [그림 4-3-7]에서 보면 A_엔터티와 B_엔터티 사이의 릴레이션에 UID BAR가 있다는 것은 C_엔터티와 B_엔터티 사이의 릴레이션에 UID BAR의 존재 여부와 상관 없이 무조건 조부모인 A_엔터티의 식별자 A를 상속 받게 된다는 것을 의미한다. 이는 곧 C_엔터티에는 A가 있으므로 이미 아버지인 B_엔터티를 경유하지 않고서도 직접 A_엔터티와 연결될 수 있음을 의미한다. 이 릴레이션십에 의해 부모나 그 이상의 조상에게 있던 식별자 속성이 설계 단계에서 나에게 상속된다는 의미일 뿐이지 참조하는 경로가 그렇게 되어야 함을 뜻하는 것은 결코 아니다. 이렇게 평범하고 당연해 보이는 사실이 갖는 진정한 가치는 상속 여부를 결정할 때 매우 중요한 판단의 기준이 된다.

3) 식별자 확정 절차

하향식(Top-down) 방식, 즉 상위 엔터티부터 시작해서 하위 엔터티로 순차적으로 결정해 가는 것이 좋다. 식별자 상속이란 상위에서 하위로 이루어지기 때문이다.

가) 키 엔터티 식별자 확정

부모를 가지지 않는 최상위 엔터티이므로 서로 독립적으로 식별자를 확정할 수 있다. 사실 엄밀히 말하면 최상위 엔터티인 키 엔터티는 본질 식별자와 굳이 다르게 할 필요가 없으므로 본질 식별자를 결정하는 단계에서 미리 식별자를 확정하는 것이 좀더 현실적이다. 물론 식별자 확정 단계에서 주변의 상황과 여건을 감안하여 일부를 조정하여 최종적으로 확정한다.

나) 메인 엔터티 식별자 확정

메인 엔터티는 해당 업무의 근본이 되는 엔터티라고 할 수 있으므로 자신이 하위에 거느리고 있는 수많은 엔터티의 상황을 종합적으로 감안한 전략적인 결정을 해야 할 것이다. 이러한 엔터티는 자신의 하위 엔터티에게는 최상위 조상이기도 하므로 될 수 있는 대로 식별자 속성의 개수를 적게 하는 것도 중요하다. 이런 이유 때문에 경우에 따라서는 전체를 대신하는 인조 식별자를 생성하기도 한다.

다) 하위 엔터티 식별자 확정

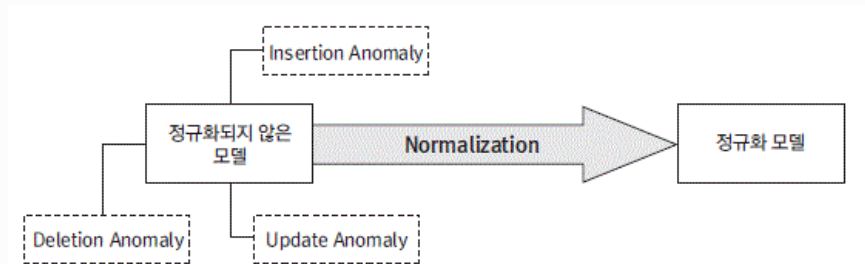
이런 부류의 엔터티는 가능하다면 인조 속성을 많이 사용하지 않는 것이 바람직하다. 그 이유는 인조 속성이란 임의의 값을 의미하므로 유일성에는 도움을 줄지 모르지만 정보로서의 가치는 현저하게 감소하기 때문이다.

+ 정규화(Normalization)

정규화는 논리적 데이터 모델을 일관성이 있고 중복을 제거하여 보다 안정성을 갖는 바람직한 자료구조로 만들기 위해 여러 단계를 거친다. 그 단계는 제 1차 정규형에서부터 제 5차 정규형과 BCNF(Boyce-Codd Normal Form)까지로 구성되어 있다. 대체로 적절하고 일관성을 유지하면서 중복이 없는 논리적 데이터 모델을 구축하는 데에는 흔히 3차 정규형이 사용된다.

▣ 정규화의 의미

- 잘 만들어진 데이터 모델이라고 해도 엔터티에 데이터를 삽입, 수정, 삭제할 때 오류가 발생할 개연성을 가지고 있다. 이러한 것들을 통칭해서 변경 이상(Modification Anomaly)이라고 한다. 여기에는 구체적으로 삽입 이상(Insertion Anomaly), 수정 이상(Update Anomaly), 삭제 이상(Deletion Anomaly) 등이 있다.
- 변경 이상이 발생하는 엔터티를 그대로 운용하게 되면 데이터가 신뢰할 수 없는 값들로 채워질 가능성이 있다. 즉, 데이터의 일관성, 무결성을 해칠 가능성이 있는 것이다.
- 정규화 과정을 통해서 변경 이상의 엔터티를 정규화된 엔터티로 변환하게 된다.



[그림 4-3-8] 정규화 과정

1) 입력 이상

별도의 사실이 발생하기 전까지 원하는 데이터를 삽입할 수 없음. 어떤 데이터를 삽입하려고 할 때 불필요하게 원하지 않는 데이터도 함께 삽입된다.

2) 삭제 이상

일부 정보를 삭제함으로써 유지되어야 할 정보까지도 삭제되는 연대 삭제가 발생한다.

3) 갱신 이상

일부 속성 값을 갱신 함으로써 원하지 않는 정보의 이상 현상(무결성 파괴, 정보의 모순성)이 발생 한다.

▣ 정규화의 장점

1) 중복값이 줄어든다

궁극적으로 칼럼 간, 레코드 간, 테이블 간에 중복되는 데이터들을 최소화할 수 있다. 이것이 정규화의 최대 성과라고 할 수 있다. 사실은 모든 장점이 이것에서 시작된다.

2) NULL 값이 줄어든다

전체적으로 NULL 값의 사용이 줄어들게 된다.

3) 복잡한 코드로 데이터 모델을 보완할 필요가 없다

데이터에 중복된 값이 적고, 그 부모가 누구인지가 항상 명시되어 있는 상황에서 데이터의 무결성을 지키기 위한 복잡한 코드를 사용할 필요가 없어진다. 데이터베이스 코드를 작성하는데 복잡한 변환 과정과 조인 질의, NULL 값 처리들이 필요하다는 것은 그 데이터베이스 설계가 문제가 있다는 말이 기도 하다.

4) 새로운 요구 사항의 발견 과정을 돕는다

실제 정규화 과정에서 많은 엔터티 혹은 속성들이 태어나게 된다. 또한 많은 결정 과정에서 업무 담당자와의 협의를 통해서 현재뿐만 아니라 미래까지도 고려한 요구 사항의 발견 과정이기도 하다.

5) 업무 규칙의 정밀한 포착을 보증한다

정체화되고, 규칙의 Value화에도 도움을 주게 된다.

6) 데이터 구조의 안정성을 최대화한다

중복된 값이 최소화되고 모든 정보들이 자기가 있어야 할 자리에 존재하게 되기 때문에 향후 발생 하게 될 모델 변화에도 유연하게 대처할 수 있다.

▣ 정규화 단계

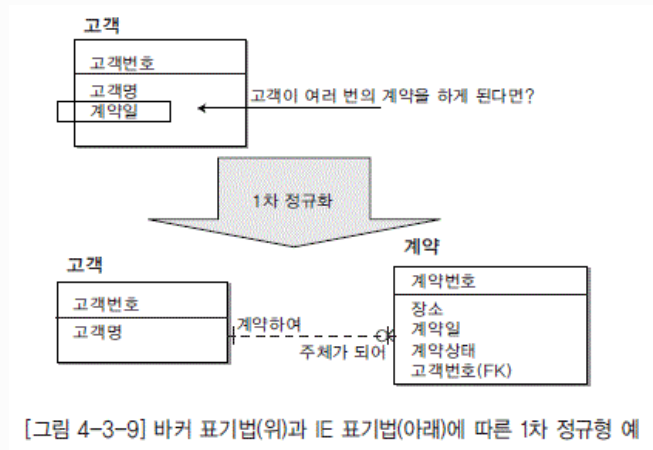
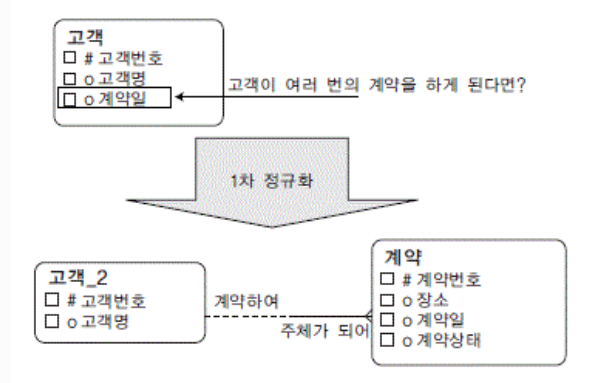
1) 1차 정규형 (1NF, First Normal Form)

가) 정의

- 모든 속성은 반드시 하나의 값을 가져야 한다. 즉, 반복 형태가 있어서는 안된다.
- 각 속성의 모든 값은 동일한 형식이어야 한다.
- 각 속성들은 유일한 이름을 가져야 한다.
- 레코드들은 서로 간에 식별 가능해야 한다.

나) 정규화 작업

- [그림 4-3-9]에서와 같이 고객 엔터티의 계약일 속성의 값이 여러 건이 된다면 1차 정규형을 위배하는 것이 된다.



[그림 4-3-9] 바커 표기법(위)과 IE 표기법(아래)에 따른 1차 정규형 예

- 어떤 속성이 다수의 값을 가지고 있다면 M:1 관계의 새로 엔터티를 추가한다.
- 관계형 모델에서는 관계(Relation) 정의상 한 속성이 하나의 값만을 가져야 한다. 그러므로 비정규형 관계는 엄밀히 말하면 관계로 간주할 수 없다.
- 비정규형 관계가 관계로서의 모습을 갖추기 위해선 여러 개의 복합적인 의미를 가지고 있는 속 성이 분해되어 하나의 의미만을 표현하는 속성들로 분해되어야 한다. 즉 속성수가 늘어나야 한다.
- 비정규형 관계가 관계로서의 모습을 갖추기 위해선 하나의 속성이 하나의 값을 가질 수 있어야 하며, 이 조건을 만족시키기 위해선 로우(Row)가 늘어나야 한다. 또는 다른 관계로 분리되어야 한다.
- 분석 또는 모델링 진행 과정에서 발생하며, 최종적인 모델링 완성 단계에서는 나올 수 없다. 그러나 분석(모델링) 초기 단계에는 상세히 분해된 속성보다는 위와 같은 레벨의 속성 추출이 복잡도를 줄일 수 있으므로 실전에서는 효율적이고 유리하게 이용될 수도 있다.

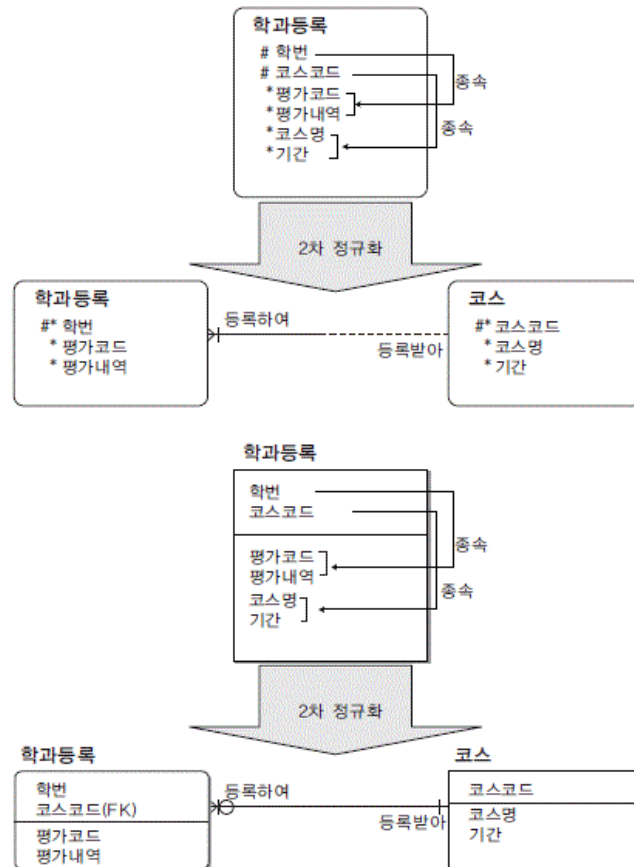
2) 2차 정규형 (2NF, Second Normal Form)

가) 정의

- 식별자가 아닌 모든 속성들은 식별자 전체 속성에 완전 종속되어야 한다.
- 이것을 물리 데이터 모델의 테이블로 말하면 기본키가 아닌 모든 칼럼들이 기본키에 종속적이어 야 2차 정규형을 만족할 수 있다는 것이다.

나) 정규화 작업

- [그림 4-3-10]에서와 같이 식별자가 학번 + 코스코드로 이루어진 학과등록 엔터티에서 학번 속성에 평가코드, 평가내역 속성들이 종속적이다. 그렇기 때문에 이것은 2차 정규형을 위반하고 있는 것이다.
- [그림 4-3-10]에서와 같이 식별자가 학번 + 코스코드로 이루어진 학과등록 엔터티에서 코스코드 속성에 코스명, 기간 속성들이 종속적이다. 그렇기 때문에 이 또한 2차 정규형을 위반하고 있는 것이다.
- 의미상의 주어 즉, 본질식별자를 알아야 식별자 부분 종속인지를 구분할 수 있다.



[그림 4-3-10] 바커 표기법(위)과 IE 표기법(아래)에 따른 2차 정규형 예

- 속성의 의미가 명확해야 종속성을 비교할 수 있다.
- 어떤 속성 식별자 전체에 종속되어 있지 않으면 잘못된 위치이며 새로운 엔터티 즉, 상위 부모 엔터티를 생성하고 UID BAR를 상속받게 된다.

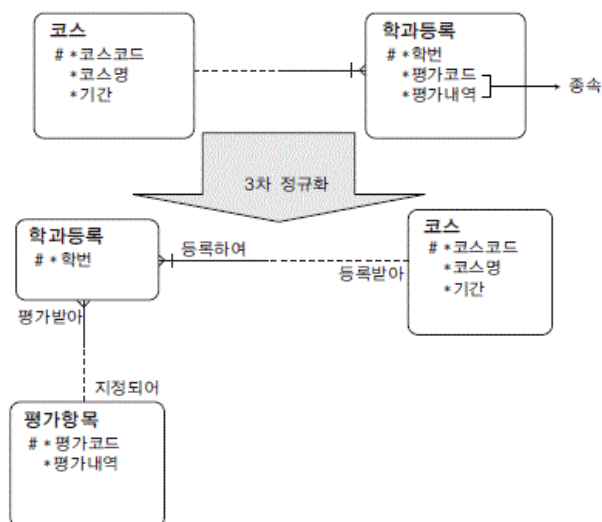
3) 3차 정규형(3NF, Third Normal Form)

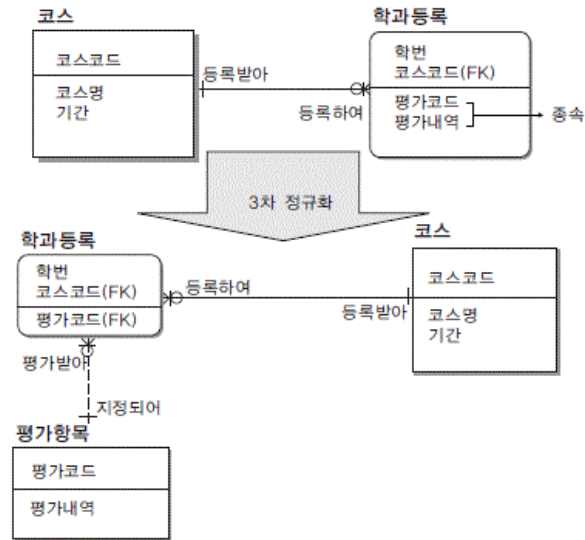
가) 정의

- 2차 정규형을 만족하고 식별자를 제외한 나머지 속성들 간의 종속이 존재하면 안된다. 이것이 3차 정규형을 만족하는 것이다.

나) 정규화 작업

- [그림 4-3-11]에서 학과등록 엔터티에서 평가코드, 평가내역 속성들이 서로 간에 종속적이다. 즉, 평가내역 속성은 평가코드 속성에 종속적이다. 그렇기 때문에 이것은 3차 정규형을 위반하고 있는 것이다.
- 3차 정규형을 위반하고 있을 시에는 [그림 4-3-11]에서의 평가항목 엔터티처럼 부모 엔터티가 생성되고 그 부모 엔터티로부터 UID Bar가 없는 관계를 상속받게 된다.





[그림 4-3-11] 바커 표기법(위)과 IE 표기법(아래)에 따른 3차 정규형 예

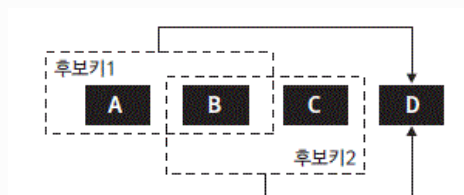
4) BCNF 정규형

가) 정의

- 모든 결정자가 키인 릴레이션이 BCNF이다. 반대로 어떠한 결정자 하나라도 키가 아닌 릴레이션이라면 BCNF가 될 수 없다.
- 기존의 2차 정규형과 3차 정규형을 보완하려는 목적으로 만들어졌다. 즉 부분 종속이나 이행 종속이 없는 3차 정규형도 변경 이상 현상이 나타날 수 있기 때문이고, 이것은 어떤 Non-Key 속성이 결정자로 동작하기 때문에 발생한다.

나) 3차 정규형의 문제점

- 한 릴레이션에 여러 개의 후보키(Candidate Key)가 있으며,
- 모든 후보키들이 적어도 둘 이상의 속성으로 이루어지는 복합(Composite) 키이며,
- 모든 후보키들이 적어도 하나 이상의 공통 속성이 포함되는 경우이다.



[그림 4-3-12] BCNF 정규형

다) 3차 정규형을 만족하고 BCNF가 아닌 경우

- 각 속성이 단 하나의 값으로 구성된 경우(1차 정규형)
- Non-Key 속성인 D는 후보키(Candidate Key)인 A+B 또는 B + C 에 완전 함수 종속이므로 2차 정규형에 속한다.
- 또한 Non-Key 속성이 D 하나 뿐이므로 Non-Key 속성 간의 종속 관계가 존재하지 않으므로 제 3차 정규형을 만족한다.

3. M:M 관계 해소

▣ M:M 관계의 의미

M:M 관계는 논리 데이터 모델링 과정에서 많이 나타난다. 이러한 관계는 데이터 모델이 아직 덜 완성된 모습이라고 할 수 있다. 그래서 M:M 관계는 최종적으로 완성된 데이터 모델에는 존재할 수 없는 형태라고 할 수 있다.

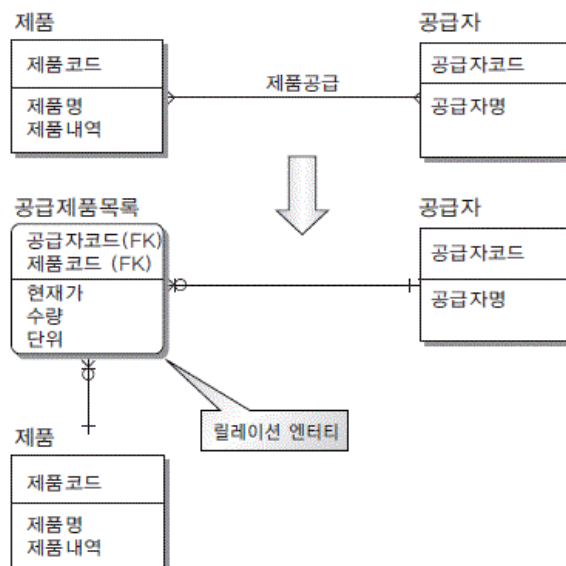
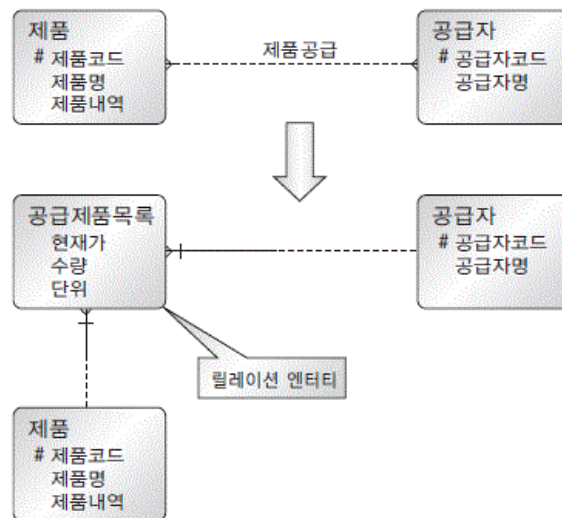
- 실제 업무 중 대부분은 M:M 관계이다. 즉, 기업이 관리하고 있는 많은 데이터 중에서 기업의 업무 내용에 해당하는 데이터가 이러한 M:M 관계로 표현되고 향후에 모델링이 더 진행됨에 따라 이것 이 해소된다.
- 키 엔터티와 키 엔터티 간에는 대부분 M:M 관계이다. 그래서 데이터 모델 상세화 단계에서 이러한 M:M 관계가 해소되면서 액션 엔터티가 새롭게 도출되기도 한다.
- 지속적으로 발생하는 대다수의 엔터티는 M:M 관계이다. 즉, 기업의 업무 내용을 관리하는 데이터는 대부분 이러한 관계에 의해서 생겨난 엔터티라고 볼 수 있다.

▣ M:M 관계 해소의 의미

M:M 관계는 불특정 관계로도 알려져 있으며, 데이터 구조에 있어서 어떠한 실제적 방법으로도 구현이 불가능하다. 이 것이 이 관계를 해결하는데 충분한 이유이지만, 데이터 모델 구축 작업 초기에 그렇게 하는 데에는 또 다른 동기가 있다.

- M:M 릴레이션십은 새로운 릴레이션 엔터티를 추가하여 M:1 관계로 변경한다.
- 연관 실체(associative, relational) 엔터티는 M:M 관계 미결 시 간과해 버렸을 추가 업무 규칙 또는 업무 논리를 내포할 수 있다. 특히 업무 규칙이 정밀하게 정의됨에 따라 하위 유형(subtype)계층 구조가 연관 실체로부터 나타나기 쉽다는 점, 분석 초기에 M:M 관계가 해결되지 않은 모델에서 이들이 간과된다는 점이다.

- M:M 관계는 데이터 종속성에 대한 결정을 어렵게 하여, 모델의 논리적 완성과 부분집합 식별 능력을 제한한다.
- M:M 관계 해결시까지 모델은 불안정 상태에 머물 것이다. 모델은 정규화되지 못하고, 모델에 대한 문서화 작업도 완비되지 못할 것이다.
- [그림 4-3-13]에서와 같이 제품 엔터티와 공급자 엔터티 간에는 M:M 관계가 존재한다. 이 관계는 제품 공급이라는 업무 내용에 의해서 두 엔터티 간에 생겨난 M:M 관계이다.
- 이 관계가 해소된 결과로 공급제품목록이라는 새로운 엔터티가 만들어지고 또한 각각의 두 엔터티 즉 제품, 공급자와 1:M의 관계를 부모로서 가지게 된다.



[그림 4-3-13] 바커 표기법(위)과 IE 표기법(아래)에 따른 M:M 관계 해소 예

실습 예제

[문] 다음 지문의 내용에 대한 엔터티를 정의하고 이들의 관계를 작성하시오.

고객은 각기 다른 배송지 정보를 설정하여 두고 주문 시에 선택할 수 있다. 단, 최초의 기본 배송지는 가입 시에 등록했 던 주소 지로 설정하며, 기본 배송지는 변경할 수 있다. 고객은 여러 상품을 하나의 주문으로 묶어 처리할 수 있으며, 하나의 주문은 신용카드, 포인트 등과 같은 여러 개의 결제 수단을 사용하여 결제할 수 있다.

참조무결성 규칙 정의

- 관계 테이블의 모든 외부 식별자 값은 관련 있는 관계 테이블의 모든 주 식별자 값이 존재해야 한다.
- 실체의 주 식별자(PK)와 마찬가지로 외부 식별자(FK)도 데이터 무결성에 관한 업무 규칙을 내포하고 있다.
- 데이터베이스 설계 관점에서 선택하지 말고, 사용자의 업무 규칙에 따라 적절한 규칙을 선택한다.

입력 규칙

자식 실체의 인스턴스(Instance)를 입력할 때 참조무결성 규칙의 종류는 다음과 같다.

1) Dependent

대응되는 부모 실체에 인스턴스가 있는 경우에만 자식 실체에 입력을 허용한다.

2) Automatic

자식 실체 인스턴스의 입력을 항상 허용하고, 대응되는 부모 건이 없는 경우 이를 자동 생성한다.

3) Nullify

자식 실체 인스턴스의 입력을 항상 허용하고, 대응되는 부모 건이 없는 경우 자식 실체의 참조키 (FK)를 Null 값으로 처리한다.

4) Default

자식 실체 인스턴스의 입력을 항상 허용하고, 대응되는 부모 건이 없는 경우 참조키(FK)를 지정된 기본 값으로 처리한다 .

5) Customized

특정한 검증 조건이 만족되는 경우에만 자식 실체 인스턴스의 입력을 허용한다.

6) No Effect

자식 실체 인스턴스의 입력을 조건 없이 허용한다.

▣ 삭제 규칙

부모 실체의 인스턴스를 삭제할 때(또는 그것의 주 식별자를 수정할 때) 사용되는 무결성 규칙은 다음과 같다.

1) Restrict

대응되는 자식 실체의 인스턴스가 없는 경우에만 부모 실체 인스턴스 삭제를 허용한다.

2) Cascade

부모 실체 인스턴스의 삭제를 항상 허용하고, 대응되는 자식 실체의 인스턴스를 자동 삭제한다.

3) Nullify

부모 실체 인스턴스의 삭제를 항상 허용하고, 대응되는 자식 실체의 인스턴스가 존재하면, 그것의 참조 키(FK)를 Null 값으로 수정한다.

4) Default

부모 실체 인스턴스의 삭제를 항상 허용하고, 대응되는 자식 실체의 인스턴스가 존재하면, 그것의 참조 키(FK)를 기본 값으로 수정한다.

5) Customized

특정한 검증 조건이 만족되는 경우에만 부모 실체 인스턴스의 삭제를 허용한다.

6) No Effect

부모 실체 인스턴스 삭제를 조건 없이 허용한다.

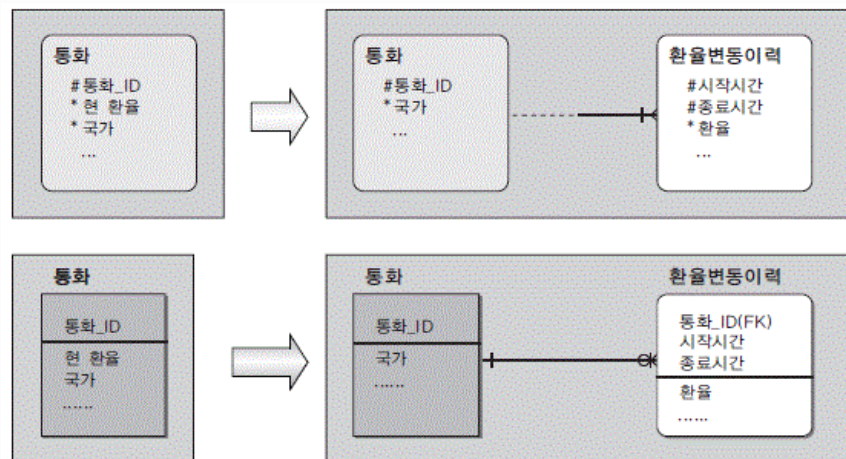
논리 데이터 모델링 이해 속성 정의 엔터티 상세화 **이력 관리 정의** 논리 데이터 모델 품질 검토

+ 이력 관리란 ?

데이터는 현재의 프로세스만 처리하고 버리는 것이 아니라 마치 후손에게 물려주어야 할 귀중한 문화유산처럼 오랜 기간의 데이터를 유지시켜 좀 더 가치있는 정보를 제공할 수 있는 밑거름이 되도록 해야 한다. 현재는 단지 하나의 점에 불과하지만 과거란 엄청난 개수의 점이 모여 있는 형상이다. 사실 현재의 데이터조차 제대로 다루기가 어려운 데 이미 지나가 버린 데이터에 연관한다는 것은 생 각처럼 그리 쉬운 일이 아닌 것은 분명하다. 굳이 위상수학(topology)을 동원하지 않더라도 수학 시 간에 배웠듯이 점으로 선분을 만들려면 무한대의 점이 필요하다. 이력은 선분이고 현재의 순간은 점 이므로 선분 -그것도 과거에 비해 현저하게 오랜 기간 - 을 관리해야 한다는 것은 결코 함부로 결정 해서는 안된다.

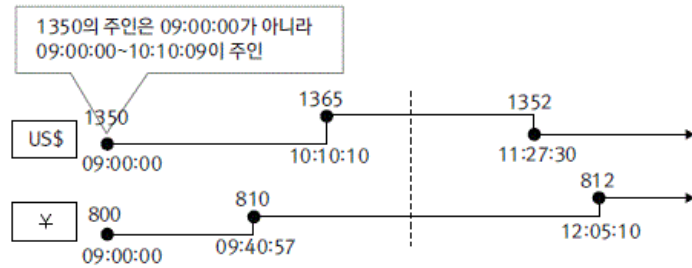
모든 업무는 언제 시작해서 언제 끝났는지에 관한 정보가 기록되어 있다. 가장 흔한 예로 주민등록 증의 뒷면을 보면 주소 변경란이 있다. 이사를 가서 주소지를 옮길 때마다 거기에는 주소 이력 정보가 기입된다. 언제 어느 동으로 이사를 갔으며, 변경된 주소는 무엇이나 하는 것 등이다.

예를 들면 [그림 4-3-14]에서와 같이 통화 데이터에서 관리되고 있던 환율 데이터에 대한 다음과 같은 업무 요구 사항이 발생하면 환율에 대한 이력을 관리하게 된다.



[그림 4-3-14] 바커 표기법(위)과 IE 표기법(아래)에 따른 통화 이력 예

- 어제의 환율은 얼마인가?
- 오늘 아침의 환율은 얼마인가?
- 환율의 변화에 대한 추이 분석을 하고자 한다.



[그림 4-3-15] 선분 이력 예

[그림 4-3-15]는 [그림 4-3-14]의 데이터 모델이 구현되어 실제 데이터가 들어가고 있는 모습이다. 즉, 환율별로 이력이 관리되고 있는 모습을 보여주고 있다.

+ 이력 관리 대상 선정

▣ 사용자 조사

데이터의 이력을 관리한다는 것은 관리하지 않을 때와 비교했을 때 많은 비용이 들어가는 일이다. 즉, 이력을 관리할 필요가 없는 데이터에 대해서 이력을 관리하는 것은 여러 가지 측면에서 낭비가 다. 그래서 다음과 같은 질문에 대한 사용자의 검증 과정을 거쳐야 한다.

- 변경 내역을 감시할 필요가 있는가?
- 시간의 경과에 따라 데이터가 변할 수 있는가?
- 시간의 경과에 따라 관계가 변할 수 있는가?
- 과거의 데이터를 조회할 필요가 있는가?
- 과거 버전을 보관할 필요가 있는가?

▣ 인력 데이터의 종류

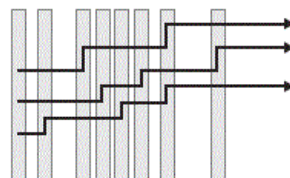
이력 데이터의 종류에는 크게 세 가지 정도의 종류가 있는데 여기에 따라서 이력을 관리하는 전략을 세워서 관리해야 한다.

1) 발생 이력(Occurrence History) 데이터

어떤 데이터가 발생할 때마다 이력 정보를 남겨야만 한다면 이력은 발생 이력이라고 볼 수 있다. 예를 들어 고객의 접속 기록을 남겨야만 하는 경우 고객이 웹사이트를 접속할 때마다 그 접속 데이터를 남긴다면 이것을 발생 이력이라고 할 수 있다. 이 방법은 전통적으로 이력을 관리하는 방법으로 사용되어 왔다.

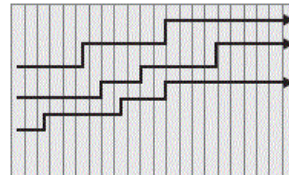
이러한 이벤트성 이력 데이터를 관리하는 방법으로는 [그림 4-3-16]의 그림처럼 이벤트가 발생할때에만 이력 데이터를 발생하는 방법이 있고, 아래 그림과 같이 이력이 발생하지 않더라도 날마다 데이터를 생성하는 경우도 있다.

■EVENT 발생 시마다 생성



Event가 발생할 때마다 사진을 찍어 두어야 이력을 관리할 수 있는가?

■DAILY마다 생성



아무 변화가 없는 경우도 데이터 생성 그러나 완벽한 이력 관리의 불가능

[그림 4-3-16] 발생 이력 예

2) 변경 이력 (Modification History) 데이터

데이터가 변경될 때마다 변경 전과 후의 차이를 확인해야 한다면 변경 이력을 남길 수 있다. 예를 들어, 고객이 주문을 하고서 주문 정보를 변경하였을 때 이전 주문과 변경된 새로운 주문 정보를 관리하기 위해서 변경된 새로운 주문 정보를 이력 정보로 남겨야 한다.

3) 진행 이력 (Progress History) 데이터

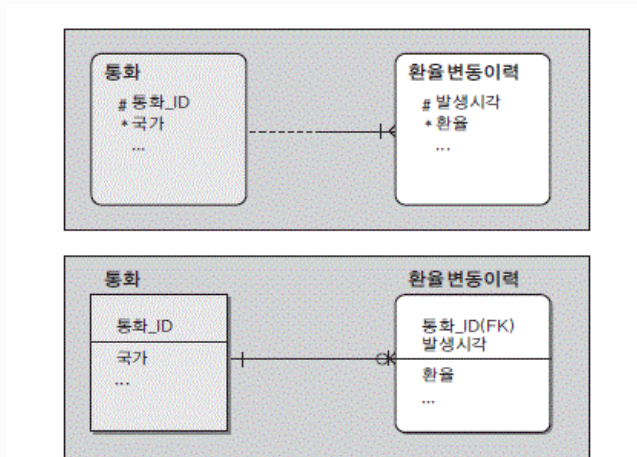
업무의 진행에 따라 이 데이터를 이력 정보로 남겨야만 하는 경우가 있다. 가장 대표적인 것이 바로 주문과 같은 업무 처리이다. 물론 이력이 관리되는 형태는 위의 변경 이력과 같은 형태로 관리 된다. 예를 들면, 주문의 업무 처리는 구매 신청 -> 입금 완료-> 배송 준비 중 -> 배송 중 -> 배송 완료 혹은 주문 취소 등과 같은 업무 진행 상황이 있다. 대부분의 주문 업무 처리의 경우 각 단계가 언제 누구에 의해서 처리되고, 현재 단계는 무엇인지에 관한 정보가 필요한 경우가 많다. 이러한 경우의 업무를 처리하기 위해서는 진행 이력이 중요하다.

■ 이력 관리 형태

- 시점 이력 : 데이터의 변경이 발생한 시각만을 관리
- 선분 이력 : 데이터 변경의 시작 시점부터 그 상태의 종료 시점까지 관리

1) 시점 이력

특정 통화의 환율이 변경되면 새로운 인스턴스가 생겨나고(New Record), 그 시점의 해당 통화의 환율과 발생 시각을 기록/보관함으로써 환율이 어느 시점에 얼마의 값으로 변경되었더라는 정보를 관리하는 것이다. 이러한 방식의 이력 관리 방법은 아래 예에서 특정한 시점의 데이터를 추출하고자 할 경우에 불필요한 작업을 수행하게 된다. 이러한 점을 주의하여야 한다.



[그림 4-3-17] 바커 표기법(위)과 1E 표기법(아래)에 따른 시점 이력 예

- 시점 이력 활용 예

```
SELECT 환율
FROM 환율 변동 이력
WHERE 발생 시각 = (SELECT MAX(발생시각)
FROM 환율 변동 이력
WHERE 발생 시각 <= 20020521 AND 통화_ID = 'USD')
AND 통화_ID = 'USD'
```

2) 선분 이력

각 통화의 특정 기간 동안 유효한 환율을 관리

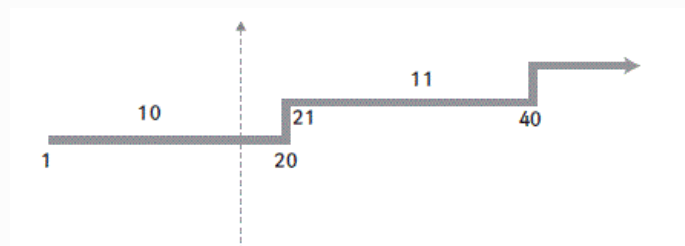
- 선분 이력 활용 예

```
SELECT 환율
FROM 환율 변동 이력
WHERE 발생 시각 between 시작 시각 and 종료 시각
AND 통화_ID = 'USD'
```

- 선분 이력 의미

[그림 4-3-19]를 예를 들어 선분 이력의 의미를 살펴 본다.

- 17을 가진 선분은 = 로 찾을 수는 없다.
- 그러나 17이 통과하는 선분을 찾아보자.
- 어떤 점이나 반드시 하나의 선분을 통과한다.
- 선분이 아무리 길어도 레코드는 하나이다.



[그림 4-3-19] 선분 이력의 의미

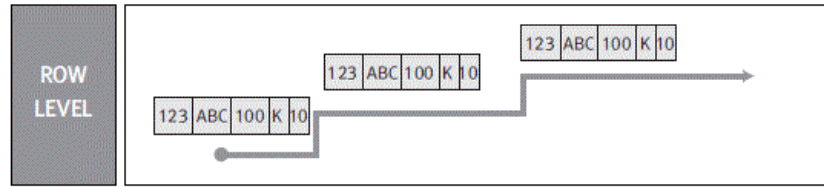
17의 지점을 [그림 4-3-19]에서와 같은 선분 이력 데이터에서 찾는 방법은 다음과 같다.

- 부등식 표현 : 시작점 <= 17 <= 종료점
- 연산자 표현 : 17 Between 시작점 and 종료점

■ 선분 이력 관리 유형

1) 인스턴스 레벨 이력 관리

하나의 인스턴스의 어떤 변경이라도 발생하면 전체 인스턴스를 새롭게 생성하는 방식의 이력 관리 유형이다.



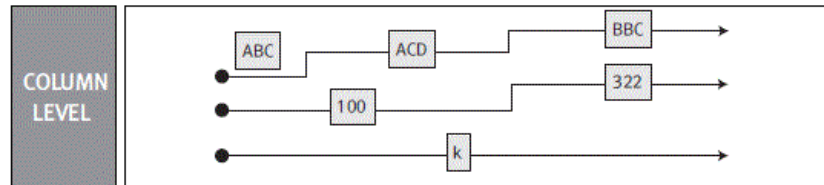
[그림 4-3-20] 인스턴스 이력 관리 예

이러한 방식의 이력 관리 유형의 특징은 다음과 같다.

- 한번의 액세스로 스냅샷을 참조하는 것이 가능하다. 즉, 한번의 액세스로 해당 시점의 모든 데이터를 참조하는 것이 가능하다.
- 로그성 데이터를 저장할 목적인 경우 적당한 방법이다.
- 다른 이력 관리 유형에 비해서 저장하기가 쉽다.
- 가장 큰 단점 중에 하나는 하나 이상의 칼럼에 변경이 발생했을 때 이벤트가 모호해진다는 점이다.
- 만약 이벤트가 지식 정보를 가지게 된다면 매우 치명적이다. 즉, 각각의 변경 이벤트가 하위의 데이터(엔터티)를 가지게 된다면 해당 이벤트를 찾기가 매우 어려워진다.
- 이력 관리의 다른 유형들에 비해서 저장 공간의 낭비를 초래할 수 있다.
- 실제 어떤 데이터가 변경된 것인지를 찾기 위해서는(즉, 이벤트를 찾기 위해서는) 과거의 데이터를 Merge해서 비교를 해야만 가능할 수 있다.
- 특정 순간의 스냅샷만 보는데 아니라면 처리가 복잡해지는 경향이 있다.
- 변화가 빈번하게 발생하는 상황이라면 고려해 볼 수 있는 유형이다.

2) 속성 레벨 이력 관리

[그림 4-3-21]에서와 같이 이력을 관리할 대상 속성에서 변화가 생길 때만 이력을 생성하는 방식이다



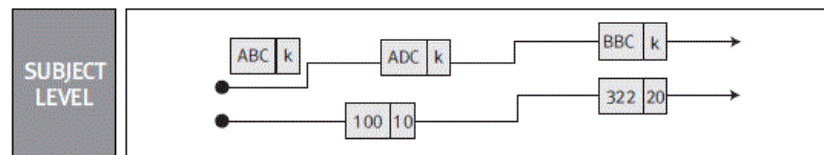
[그림 4-3-21] 속성 레벨 이력 관리 예

이러한 방식의 이력 관리 유형의 특징은 다음과 같다.

- 변경 이벤트가 매우 분명하게 관리된다. 즉, 실제 어떤 데이터가 변경되었는지가 분명하다.
- 하나의 이력 관리 엔터티에서 다른 엔터티와 통합 이력 관리가 가능하다.
- 변경된 것만 처리하기 때문에 독립적 처리가 가능하다.
- 변화가 발생할 가능성은 매우 낮으면서 이력 관리 대상 속성은 매우 많은 경우에 사용하는 것이 유리하다.
- 특정 속성들에 변화가 집중되는 경우에 해당 속성에 대해서만 이력을 관리할 수 있기 때문에 유리하다.
- 여러 속성에 대한 이력이 필요할 때 많은 Merge가 발생하게 된다.
- 이력 관리의 다른 유형들에 비해서 향후 사용될 데이터 액세스 쿼리에서 조건 검색이 조금 어렵다.
- 변화가 너무 많은 경우에는 적용이 곤란하다.

3) 주제 레벨 이력 관리

[그림 4-3-22]에서와 같이 내용이 유사하거나 연동될 확률이 높은 것별로 인스턴스 레벨 이력을 관리하는 방법이다.



[그림 4-3-22] 주제 레벨 이력 관리 예

이러한 방식의 이력 관리 유형의 특징은 다음과 같다.

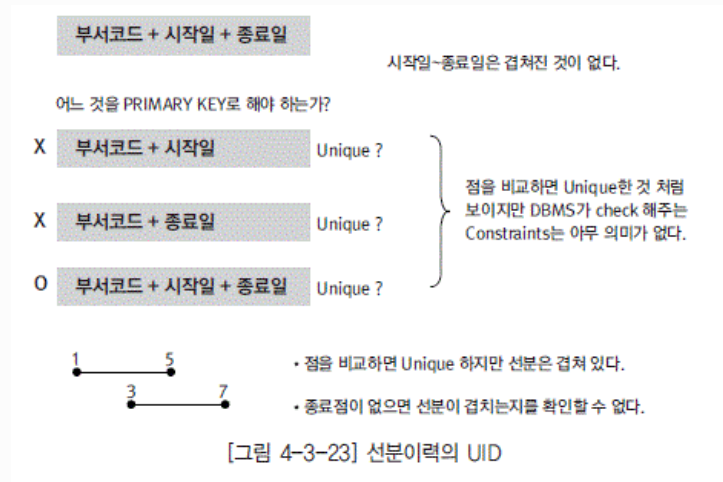
- 인스턴스 레벨과 속성 레벨의 장점을 모두 수용하는 형태의 이력 관리 형태이다.
- 목적이 분명한 엔터티를 생성함으로써 확장성을 확보할 수 있는 용도로 사용할 수 있다.
- 변경 부분만 처리가 가능하다.(독립적 처리 가능)
- 다른 엔터티와 통합 이력 관리가 가능하다.
- 속성 레벨의 단점을 해소할 수 있다.
- 전체를 참조할 때 인스턴스 레벨에 비해 Merge가 발생하는 문제점이 존재한다.
- 부문에 따라 변경 정도의 차이가 심한 경우 유리하다.

+ 선분 이력 관리용 식별자 확정

▣ 선분 이력에서 식별자 결정 시 고려사항

선분 이력을 관리하는 엔터티의 UID를 확정하는 것은 향후에 테이블로 만들어지고 사용될 때의 성능 측면도 고려되어야 한다.

[그림 4-3-23]에서와 같이 부서별로 이력이 관리되는 엔터티를 예를 들어 설명하면 의미적인 UID는 부서코드 + 이력시작일 + 이력종료일을 기본키로 하는 것이 유리하다. 하지만 이것은 [그림 4-3-23]에서와 같이 물리적인 측면, 즉 실제 데이터는 Unique하지만 의미적으로는 Unique하지 않는 일이 발생한다. 이러한 부분의 의미적인 Unique을 검증할 수 있는 조치를 병행해야 한다.



▣ 선분 이력에서 종료점 처리 시 주의사항

1) 종료점이 미정이므로 NULL

- 논리적으로는 타당하지만 비교가 불가능
- 인덱스를 사용하지 못하므로 수행 속도 저하

2) 수렴하므로 최대치 부여

- 아직 종료되지 않았으므로 무한히 계속되는 것으로 간주
- 최대치 부여 (예; 일자라면 9999/12/31)
- 가능한 TABLE creation 시 Default constraints 부여
- 수행 속도에 유리



논리 데이터 모델링

Home > DB 구축 가이드 > DA 가이드 > 데이터 모델링 > 논리 데이터 모델링

논리 데이터 모델링 이해 속성 정의 엔터티 상세화 이력 관리 정의 **논리 데이터 모델 품질 검토**

논리 데이터 모델 품질 검토 개요

데이터 모델 설계가 완료되면 모델러를 비롯한 이해관계자는 데이터 모델 리뷰 세션을 통해 작성된 데이터 모델의 품질을 검토한다. 데이터 모델 검토는 개념 데이터 모델링, 논리 데이터 모델링, 물리 데이터 모델링의 각 단계가 수행된 후 각 단계에서 작성된 개념 데이터 모델, 논리 데이터 모델, 물리 데이터 모델에 대해 이루어진다. 일반적으로 논리 데이터 모델과 물리 데이터 모델은 모든 이해 관계자들이 가장 관심을 갖고 검토하는 산출물로, 데이터 모델의 중요성을 생각하면 이 검토 과정이야말로 향후의 모든 공정에 대해 영향을 미칠 수 있는 매우 의미 있는 작업이라 할 수 있다. 데이터 모델을 검토하기 위해서는 모든 이해관계자가 동의하는 검토 기준이 필요하며, 통상 논리·물리 데이터 모델에 대한 검토 기준은 과목Ⅰ. 전사아키텍처 이해 부분에서 설명한 데이터아키텍처 정책 수립 시 DA원칙/표준에 포함되어야 할 중요한 사안이다.

데이터 모델의 품질 검토 기준은 주로 논리 데이터 모델과 물리 데이터 모델에 대해 적용하며, 조직에 따라서는 개념 데이터 모델에 대한 검토 기준을 추가하기도 한다. 기본적인 품질 검토 기준은 과목Ⅵ. 데이터 품질 관리 이해 부분에서 설명한 데이터 구조의 관리 기준을 준용할 수 있으며, 논리 데이터 모델에 대한 품질 기준을 좀 더 세분화해 보면 [표 4-3-1]과 같이 정의해 볼 수 있다. 논리 데이터 모델 품질 검토의 목적은 ‘완벽한 모델’보다 ‘(조직에) 적합한 모델’의 관점에서 생각해 볼 수 있으며, 이에 따라 논리 데이터 모델의 품질 기준도 조직에 따라 혹은 업무 상황이나 여건에 따라 가감하거나 변형하여 사용하기도 한다.

[표 4-3-1] 논리 데이터 모델의 품질 기준

기준 항목	설 명	검토 관점 사례
정 확성	데이터 모델이 표기법에 따라 정확하게 표현되었고, 업무영역 또는 요구사항이 정확하게 반영되었음을 의미함	- 사용된 표기법에 따라 데이터 모델이 정확하게 표현 되었는가 - 대상 업무영역의 업무 개념과 내용이 정확하게 표현 되었는가 - 요구사항의 내용이 정확하게 반영 되었는가 - 업무규칙이 정확하게 표현 되었는가
완전성	데이터 모델의 구성 요소를 정의하는데 있어서 누락을 최소화하고, 요구사항 및 업무영역 반영에 있어서 누락이 없음을 의미함	- 모델 표현의 충실성(완성도) - 필요한 항목(엔터티/속성 설명 등)들의 작성 상태 - 논리 데이터 모델링 단계에서 결정해야 할 항목들의 작성 상태(속성의 선택성(optional), 식별자, 정규화, 엔터티/속성의 중복배제, 이력관리 등) - 요구사항 반영 및 업무 영역 반영의 완전성: 목적하는 업무 영역을 기술(설계)하는데 있어서 논리 데이터 모델 구성요소(엔터티, 속성, 관계 등)들이 누락없이 정의 된 정도
준거성	제반 준수 요건들이 누락 없이 정확하게 준수 되었음을 의미함	- 데이터 표준, 표준화 규칙 등을 준수하였는가 - 법적 요건을 준수 하였는가
최신성	데이터 모델이 현행 시스템의 최신 상태를 반영하고 있고, 이슈사항들이 지체없이 반영되고 있음을 의미	- 업무상의 변경이나 결정사항 등이 시의 적절하게 반영되고 있는가 - 최근의 이슈사항이 반영 되었는가 - 현행 데이터 모델은 현행 시스템과 일치 하는가
일관성	여러 영역에서 공통 사용되는 데이터 요소가 전사 수준에서 한 번만 정의되고 이를 여러 다	- 여러 주제영역에서 공통적으로 사용되는 엔터티는 일관성 있게 사용되는가(전사 수준에서 한 번만 정의되고 이를 여러 다른 영역에서 참조·활용한다는 의미에서 통합성이라 하

	른 영역에서 참조·활용되면서, 모델 표현상의 일관성을 유지하고 있음을 의미함	기도 함) - 모델 표현상의 일관성을 유지하고 있는가
활용성	작성된 모델과 그 설명 내용이 이해관계자에게 의미를 충분히 전달할 수 있으면서, 업무 변화 시에 설계 변경이 최소화되도록 유연하게 설계되어 있음을 의미	- 작성된 설명 내용이나 모델 표기 등이 사용자나 모델을 보는 사람에게 충분히 이해가 될 수 있고, 모델의 작성 의도를 명확하게 이해할 수 있는가(의사소통의 충분성) - 데이터 모델은 유연성을 갖고 있는가(오류가 적고 업무변화에 유연하게 대응하여 데이터 구조의 변경이 최소화 될 수 있는 설계 결과)

➤ 논리 데이터 모델 품질 검토 체크리스트의 활용

논리 데이터 모델의 품질 검토 기준에 따라서 논리 데이터 모델에 정의된 엔터티, 관계, 속성 등 데이터 모델의 주요 구성요소와 논리 데이터 모델 전반에 대한 체크리스트를 구성할 수 있으며, 이를 통해 논리 데이터 모델의 품질 검토를 보다 용이하게 수행할 수 있다. [표 4-3-2]는 논리 데이터 모델의 주요 구성 요소별로 품질 검토 기준 항목을 적용하여 작성한 품질 검토 체크리스트의 사례이다.

[표 4-3-2] 논리 데이터 모델 품질 검토 체크리스트 사례

검토대상	검토항목	검토 내용
엔터티	엔터티명	- 사용된 표기법에 따라 데이터 모델이 정확하게 표현 되었는가 - 대상 업무영역의 업무 개념과 내용이 정확하확하게 반영 되었는가 - 업무규칙이 정확하게 표현 되었는가 - 데이터 집합의 개요나 성격, 관리 목적 등을 설명하였는가? - 데이터 집합 구성상의 특징이 설명되어 있는가? - 데이터 집합의 생명주기나 오너쉽 등을 비롯한 기타 특이사항에 대한 내용을 포함하고 있는가? - 설명된 내용은 모든 이해관계자가 이해하고 의사소통 하는 데에 어려움이 없도록 쉽고 상세하게 기술되었는가?
	엔터티 정의	- 도출된 엔터티는 요구사항을 충족하거나 업무 영역을 설명하기에 충분한가? - 우선순위에 따른 엔터티 분류 관점에서 중요한 키엔터티나 메인엔터티가 누락되지 않았는가? - 엔터티는 서브타입을 사용하여 구체적·입체적으로 정의되었는가? - 서브타입 정의 시 구분자 속성은 명확하게 정의하였는가? - 서브타입 구성은 엔터티의 성격을 설명하기에 충분한가? - 서브타입은 충분히 도출되었는가? (전체집합=Σ서브타입) - 향후의 업무 변화 가능성에 대비하여 모델 변경을 최소화할 수 있도록 유연성, 확장성이 고려되었는가?
	통합 수준	- 업무 행위의 주체가 될 수 있는 전사관계자와 같은 중요 기준데이터는 통합이 고려되었는가? - 엔터티간 동질성을 부여할 수 있는 유사 목적·구성의 엔터티에 대해 통합이 고려되었는가? (서브타입 사용) - 코드 엔터티는 통합이 고려되었는가? - 계층구조 엔터티에 대한 통합은 고려되었는가? - 배타관계 엔터티의 통합은 고려되었는가? - 다른 영역에서 동일 목적의 엔터티는 동일 명칭과 구조로 일관되게 사용되었는가? - 엔터티 분리는 명확하고 합리적인 근거와 목적에 의해 적절한 형태로 이루어졌는가?
	권한	- 메타데이터 권한을 정의 하였는가? (엔터티 생성/변경/삭제) - 데이터 오너쉽을 정의 하였는가? (데이터 생성/변경/삭제)
	발생 건수/빈도	- 현재의 데이터 저장 건수/빈도는 파악하였는가? - 향후 예상되는 데이터 저장 건수/빈도의 변화가능성은 파악하였는가?
	다른 엔터티와의 관계	- 다른 엔터티와 하나 이상의 관계를 가지고 있는가?
	법규 준수	- 관련 법규에서 요구하는 데이터를 보관하기 위한 엔터티를 정의하였는가? - 조직 특성에 비추어 보호가 요구되는 엔터티를 식별하였는가?
	요구사항 추적가능성	- 정의된 엔터티는 요구사항과 매핑이 되었는가?
	속성명	- 속성명은 명명규칙을 준수하였는가? - 제한요건에 따라 약어를 사용한 경우 약어사용 규칙을 준수하였는가? - 의미 전달이 명확한 명칭을 사용하였는가?
		- 엔터티명이나 엔터티 성격에 맞는 속성이 도출되었는가? - 엔터티의 성격이나 목적에 비추어 속성은 충분히 도출되었는가?

속성	속성 정의	- 속성의 선택성 결정은 적절한가? - 유일값 원칙에 위배되는 속성이 존재하는가? - 속성의 원자단위 구성은 적절한가? - 속성의 관리 목적상 상세화 여부 검토가 수행되었는가? - 인조 속성의 생성 단위나 생성 규칙이 정의되었는가?
	속성 설명	- 속성의 개요나 성격, 관리 목적 등을 설명하였는가? - 속성으로 관리하고자 하는 데이터의 형태적 혹은 구성상의 특징이 포함되어 있는가? - 데이터 집합으로서 속성의 생명주기나 오너쉽 등을 비롯한 기타 특이사항에 대한 내용을 포함하고 있는가? - 설명된 내용은 모든 이해관계자가 이해하고 의사소통 하는 데에 어려움이 없도록 쉽고 상세하게 기술되었는가?
	속성 유형	- 엔터티 성격에 맞게 식별자가 정의되었는가? - 관계를 통해 상속받은 관계속성은 명확한 역할명을 사용하고 있는가?
	식별자 정의	- 모든 본질식별자가 적절하게 파악되었는가? - 인조식별자 사용 시 대응하는 속성 구성이 파악되었는가? - 실질식별자로서 본질식별자나 인조식별자를 사용하는 기준을 준수하고 있는가?
	법규 준수	- 법규상 암호화 대상인 속성은 식별하였는가? - 법규상 필요한 속성은 정의되었는가? - 법규상 수집·보관에 따른 제약이 존재하는 속성은 처리 방안을 고려하고 적용하였는가?
	도메인 정의	- 표준 도메인을 정의하여 적용하였는가? - 속성의 도메인은 일관성 있게 정의되었는가?
	추출 속성의 정의	- 추출 속성은 명확하고 합리적인 이유를 토대로 정의하였는가? - 추출 속성의 원시 속성은 식별하였는가? - 추출 속성은 추출 방법 또는 산식이 명확하게 정의되었는가? - 추출 속성의 빈도(구성 수준)는 적절한가?
	요구사항 추적가능성	- 속성 수준에서 필요한 요구사항 매핑은 수행되었는가?
	오너쉽 정의	- 속성 수준에서 데이터 오너쉽 정의가 필요한 경우 데이터 오너쉽이 정의 되었는가?
관계	관계명	- 관계명이 누락된 관계가 존재하는가? - 관계명 부여 규칙 존재 시 이를 준수하였는가? - 두 엔터티 간의 업무적 관계를 자식(자식 엔터티)이 바라보는 부모(부모 엔터티)의 역할 관점으로 파악하여 관계명을 표현하였는가?
	관계 정의	- 업무 영역을 설명하거나 요구사항을 충족하는데 있어서 필요한 관계들이 충분히 도출·정의되었는가? - 관계 정의는 업무 영역의 내용이나 요구사항과 일치하는가? - 배타적 관계가 정의된 경우 업무 내용과 일치하는가? - 배타적 관계가 정의된 경우 현재의 업무 개선 관점이 고려되었는가? - 배타 관계 해소를 위한 검토가 수행되었는가?
	관계 설명	- 관계가 왜 존재해야 하는지의 관점에서 기술하고 있는가?(업무규칙, 정규화 등)
	관계 표현	- 표기법에 따라 정확하게 표현하였는가? - 관계에 표현된 기수성·선택성은 업무규칙을 정확하게 설명하는가?
	식별자 상속	- 자식에 상속된 관계 속성은 정확한 역할명으로 표현되었는가? - 모든 관계 속성들의 출처(또는 관계)가 명확하게 파악되었는가?
	요구사항 추적가능성	- 관계에 대해 필요한 요구사항 매핑은 수행되었는가?
	외부키 (외래키)*	- 외부키(외래키)가 부모 엔터티의 주식별자(실질식별자)와 일치하는가? - 외부키 항목이 기본키와 기본키가 아닌 속성에 펼쳐져 있는가? - 자식에 상속된 관계 속성의 선택성은 적절한가?
	참조무결성**	- 업무규칙에 근거하여 참조무결성을 정의하였는가?
모델 전반	주제영역 적절성	- 주제영역의 구성은 적절한가?
	논리 모델 상세화	- 데이터 모델상에 정규화가 미흡한 부분이 존재하는가? - 최종적인 논리 데이터 모델에서 다대다 관계는 모두 해소 하였는가?
	이력 관리	- 이력관리 대상 선정과 이력관리 방법은 적절한가?

※ *, ** 표기한 부분은 방법론이나 모델링 도구에 따라 물리 데이터 모델의 검토 항목으로 보기도 함.



물리 데이터 모델링 이해

물리 요소 조사 및 분석

논리-물리 모델 변환

반정규화

물리 데이터 모델 품질 검토

물리 데이터 모델 정의

물리 데이터 모델이란 논리적 모델을 특정 데이터베이스로 설계함으로써 생성된, 데이터를 저장할 수 있는 물리적인 스키마를 말한다. 데이터 모델의 엔터티와 서브타입은 논리적인 집합이며, 만약 관계형 데이터베이스로 설계한다면 이 단계에 와서 물리적인 테이블(Table)로 확정된다. 하나의 논리적 집합(엔터티 혹은 서브타입)은 하나 이상의 테이블이 될 수 있으며, 경우에 따라서는 속성의 일부만으로 생성될 수 있다.

물리 데이터 모델링은 논리 데이터 모델을 사용하고자 하는 각 DBMS의 특성을 고려하여 데이터 베이스 저장 구조(물리 데이터 모델)로 변환하는 것이다. 여기에서 물리 데이터 모델링과 데이터베이스 디자인과의 개념을 정리하자면 물리 데이터 모델링은 데이터의 구조에 관련된 것들을 물리적인 모습까지 설계하는 것이고, 반면에 데이터베이스 디자인은 이러한 물리적인 모델(설계도면)을 DBMS 관점의 오브젝트로 생성하는 최적의 설계(디자인)를 하는 것이다. 데이터베이스 디자인의 예로는 오브젝트별 저장공간의 효율적 사용 계획, 오브젝트 파티셔닝 설계, 최적의 인덱스 설계 등이 여기에 속한다고 할 수 있다. 물론, 이러한 기준에 대해서 이론이 있을 수 있을 것이다. 하지만, 이 수험서에서는 이러한 기준을 가지고 물리 데이터 모델링과 데이터베이스 디자인을 구분하였다.

물리 데이터 모델 의의

물리적 데이터 모델링은 관계 데이터 모델링(RDM, Relation Data Modeling)이라고도 한다. 사전적으로 작성된 논리적 데이터 모델을 각각의 관계형 데이터베이스 관리 시스템의 특성, 기능, 성능 등을 고려하여 데이터베이스의 물리적인 구조(Schema)를 작성해가는 과정이다. 많은 사람들이 물리 데이터 모델링을 단순히 설계된 논리 데이터 모델의 개체 명칭이나 속성 명칭, 데이터 형태, 길이, 영역값등을 변환하는 것 정도로 생각하고 있다. 그러나 물리적 데이터 모델링 단계는 논리 데이터 모델에서 도출된 내용 변환을 포함하여 데이터의 저장공간, 데이터의 분산, 데이터 저장 방법 등을 함께 고려하는 단계이다. 또한, 이 과정에서 결정되는 많은 부분이 데이터베이스 운용 성능(Performance)으로 나타나므로 소홀히 다루면 안된다.

논리 데이터 모델-물리 데이터 모델

하나의 논리적 데이터 모델을 가지고 서로 다른 형태의 물리적 데이터 모델을 설계하는 경우는 크게 네 가지로 나눌 수가 있다.

분산 데이터베이스 구축 시

분산 데이터베이스를 구축하고자 할 때 노드별로 자신이 원하는 형태의 물리적 모델을 생성하고자 할 때 적용하는 경우이다.

물리 데이터 모델 비교

각자 나름대로의 특징을 가지고 있는 여러 개의 물리적 모델을 생성하여 종합적인 비교 검토를 하기 위하여 적용하는 경우이다.

물리적 환경의 변화

논리적인 모델에는 변화가 발생하지 않지만 물리적인 환경에는 변경이 발생했을 때 기존의 물리적 모델을 새로운 목표 물리적 모델로 개선하고자 할 때 적용하는 경우이다.

물리적 모델의 형상관리

물리적 모델이 세월의 흐름에 따라 조금씩 변해갈 때 그 이력을 관리할 목적으로 여러 개의 버전을 보유하고자 할 때 사용하는 경우이다.



물리 데이터 모델링

Home > DB 구축 가이드 > DA 가이드 > 데이터 모델링 > 물리 데이터 모델링

물리 데이터 모델링 이해

물리 요소 조사 및 분석

논리-물리 모델 변환

반정규화

물리 데이터 모델 품질 검토

+ 시스템 구축 관련 명명 규칙

사내의 시스템 구축과 관련된 명명 규칙을 파악하여 물리 데이터 모델의 각 요소의 내용에 이를 적용해야 한다.

+ 하드웨어 자원 파악

▣ CPU

중앙처리 장치의 성능과 집중적인 부하가 발생하는 시간 등을 파악한다.

▣ MEMORY

전체 메모리의 규모 및 시스템이 사용하는 메모리 영역을 포함하여 사용 가능한 메모리 영역을 파악한다.

▣ DISK

전체 디스크의 크기, 분할된 형태, 현재 디스크 활용률 등을 파악하고 사용 가능한 공간을 확인 한다.

▣ I/O Controller

현재 입/출력 컨트롤러의 성능 및 적절하게 운용되고 있는가를 파악한다.

▣ Network

네트워크와 관련된 모든 내용을 파악한다. 여기에 관련된 내용으로는 다음과 같은 것들이 존재한다.

- 현재 처리 가능한 속도
- 집중적인 부하가 발생하는 시간대
- 동시 접속 최대 가용 사이트 수

+ 운영체제 및 DBMS 버전 파악

데이터베이스 운영 환경과 관련된 운영체제의 관련 요소를 파악하고 적절하게 관리되고 있는가를 파악한다. 특히 인스턴스 관리 기법 등에 대해서 파악하고 분석한다.

+ DBMS 파라미터 정보 파악

DBMS 환경 적용 단계에서 가장 중요하게 고려해야 하는 단계이다. 물론 DBMS 파라미터는 데이터베이스 관리 시스템 별로 많은 차이가 있으며 관리하는 방법도 서로 다르다. 따라서 자신들의 DBMS가 관리하는 파라미터의 종류와 관리 대상들을 정확하게 파악하고 정의해야 한다. 특히 데이터베이스 관리를 위한 데이터 저장 공간 관리 기법과 메모리 관리 기법 등과 관련된 파라미터들에 대해서는 세심한 주의를 기울여야 한다. 그리고 데이터 쿼리에서 활용하는 옵티마이저(Optimizer)의 운영 방법 등도 중요한 고려사항이 된다.

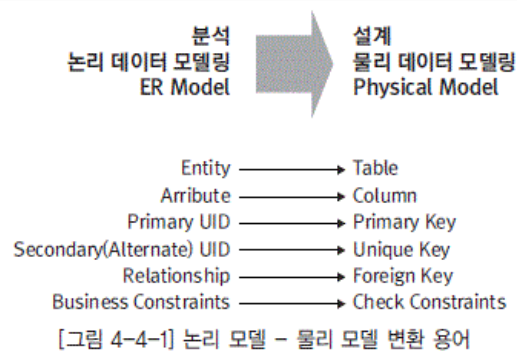
+ 데이터베이스 운영과 관련된 관리 요소 파악

- 사용자 관리 기법 및 정책
- 백업/복구 기법 및 정책
- 보안 관리 정책

물리 데이터 모델링 이해 물리 요소 조사 및 분석 **논리-물리 모델 변환** 반정규화 물리 데이터 모델 품질 검토

+ 논리 데이터 모델-물리 데이터 모델 변환(Transformation) 용어

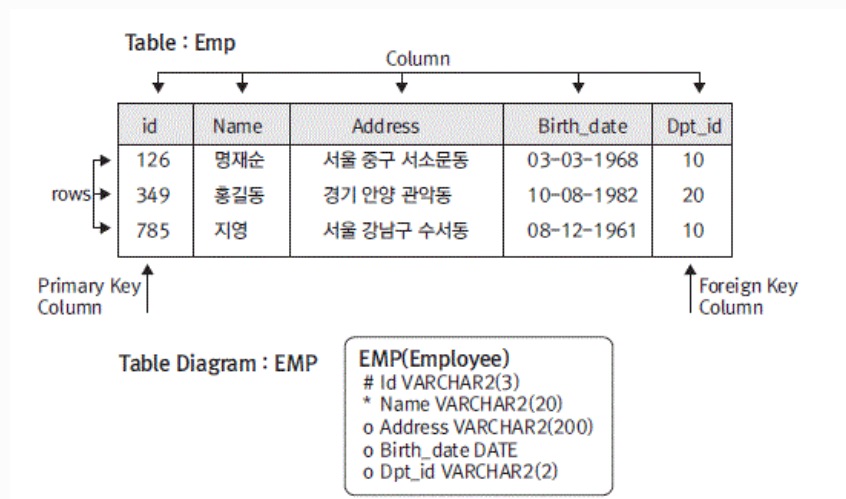
논리 영역과 물리 영역을 보는 시각은 여러 가지 관점에서 조금씩은 다르다. 특히 학자, 모델링 툴도 이러한 차이는 존재한다. 하지만, 하지만 이 책에서는 다음과 같은 기준으로 논리 데이터 모델의 영역과 물리 데이터 모델의 영역을 구분하여 접근하고 있다.



+ 엔터티-테이블 변환

▣ 테이블 설명

테이블은 데이터를 저장하기 위해서 생성된 데이터베이스에서의 가장 기본적인 오브젝트이다. 기본적인 모습은 아래와 같은 모양으로 만들어지게 된다.



EMP

id: VARCHAR2 (3) NOT NULL
Name: VARCHAR2 (20) NOT NULL
Address: VARCHAR2 (200) NULL
Birth_date: DATE NULL
Dpt_id: VARCHAR2 (2) NULL

[그림 4-4-2] 바커 표기법(위)과 IE 표기법(아래)에 따른 테이블 레이아웃

1)테이블(Table)

테이블은 기본적으로 칼럼(Column)과 로우(Row)를 가진다. 각각의 칼럼은 지정된 유형의 데이터 값을 저장하는 데 사용된다. [그림 4-4-2]에서 테이블 EMP는 직원 정보를 저장하기 위해서 생성된 구조이다.

2)로우(Rows)

테이블의 한 로우에 대응. 튜플, 인스턴스, 어커런스라고도 한다.

3)칼럼(Columns)

각 직원 개인의 관리 항목에 대한 Value를 저장한다.

4)기본키(Primary keys)

하나의 칼럼 혹은 몇 개의 칼럼 조합으로 어떤 경우라도 테이블 내에 동일한 값을 갖는 튜플이 존재하지 않도록 한다.

5)외래키(Foreign keys)

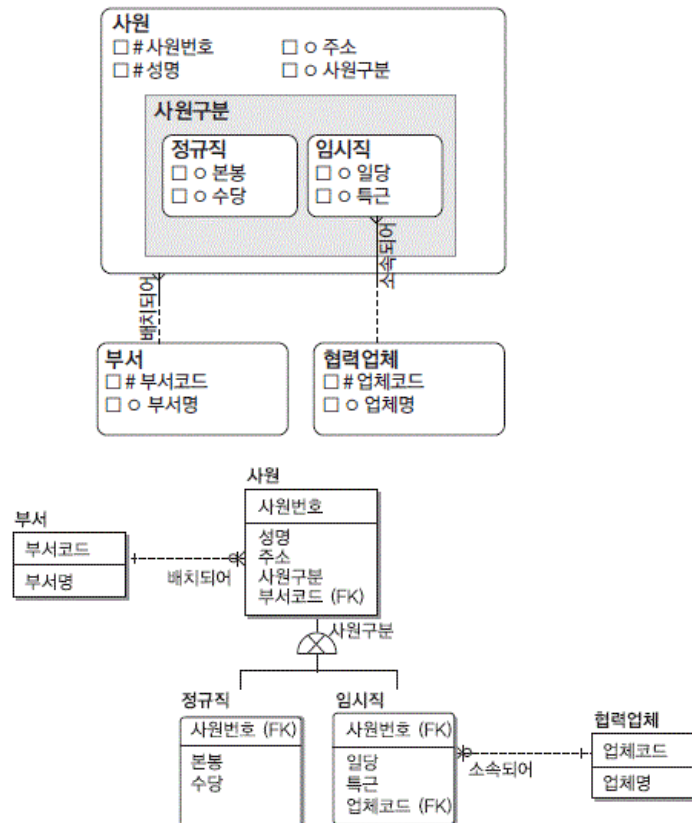
외부 데이터 집합과의 관계를 구현한 구조이다.

서브타입 변환(Transformation)

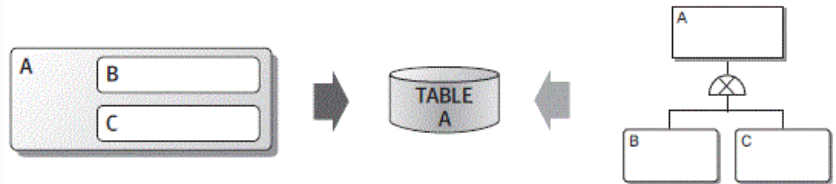
논리 데이터 모델에서는 비즈니스 또는 업무를 데이터 모델로 표현하기 위해서는 최대한 상세한 표현이 필수적이다. 그러한 목적을 달성하기 위해서 가능한 집합(엔터티)의 구성을 서브타입을 사용하여 구체적으로 표현하는 것이 통상적이다. 또한 각각의 서브타입들이 독립적인 속성(Attribute), 관계(Relationship)를 가지고 있는 경우에는 이러한 서브타입(Sub Type)을 사용한 집합의 표현은 필수적이다. 이렇게 논리 모델링에서 표현된 서브타입은 물리 데이터 모델에서는 테이블의 형태로 설계되어야 한다. 하지만 이러한 서브타입 모델은 단순 엔터티-테이블 변환과는 다른 몇 가지 방법을 통해서 테이블로 변환 작업을 하게 된다.

- 슈퍼타입 기준 테이블 변환
- 서브타입 기준 테이블 변환
- 개별타입 기준 테이블 변환

[그림 4-4-3]과 같은 구체적인 논리 데이터 모델을 통해서 위 방법들을 살펴본다.



[그림 4-4-3] 바커 표기법(위)과 IE 표기법(아래)에 따른 서브타입 예



[그림 4-4-4] 바커 표기법(좌)과 IE 표기법(우)에 따른 슈퍼타입 기준 테이블 변환 예

슈퍼타입 기준 테이블 변환

- 서브타입을 슈퍼타입에 통합하여 하나의 테이블로 만든다
- 이 통합된 테이블에는 모든 서브타입의 데이터를 포함해야 한다
- 주로 서브타입에 적은 양의 속성이나 관계를 가진 경우에 적용된다

가) 절차

- 슈퍼타입으로 테이블 명칭 부여
- 서브타입을 구분할 수 있도록 칼럼 추가
- 슈퍼타입의 속성을 칼럼명으로
- 서브타입의 속성을 칼럼명으로
- 슈퍼타입의 관계를 FK로
- 서브타입의 관계를 FK로

나) 테이블 사례

[그림 4-4-3]의 논리 모델에서의 서브타입을 하나의 테이블로 통합하는 경우 테이블의 모습은 [그림 4-4-5]의 사례 데이터 표와 같다.

테이블명 : EMPLOYEE

칼럼명	EMPNO	ENAME	TYPE	SAL	DEPT	HRATE	OTRATE	UNION
키형태	PK				FK1			FK2
Nulls/ Unique	NN,U	NN	NN		NN			
견본 데이터	7540	홍길동	1	890000	40			
	5579	성삼문	1	950000	35			
	6714	김야박	1	827000	20			
	9451	오재구	2	20000	25	60000	1000	201
	3040	유성원	2	35000	40	80000	1500	180

[그림 4-4-5] 테이블 사례 예

- TYPE
서브타입을 구분할 수 있는 칼럼이다. 즉 사원 구분에 해당하는 칼럼이다.
- DEPT
위의 구분이 정규직일 경우에 부서로부터의 관계로 인해서 생성된 칼럼이다.
- UNION
위의 구분이 임시직일 경우에 협력 업체로부터의 관계로 인해서 생성된 칼럼이다.

다) 하나의 테이블로의 통합이 유리한 경우

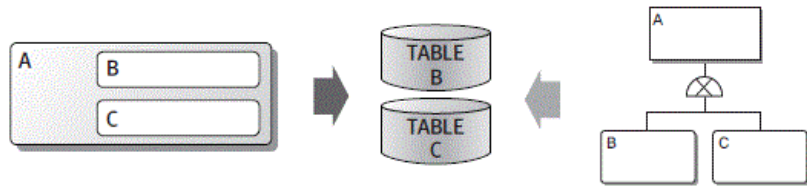
- 데이터 액세스가 좀더 간편
- 뷰를 활용하여 각 서브타입만을 액세스하거나 수정 가능
- 수행 속도가 좋아지는 경우가 많다
- 서브타입 구분없는 임의 집합의 가공 용이
- 다수의 서브타입을 통합한 경우 조인(JOIN) 감소 효과가 크다
- 복잡한 처리를 하나의 SQL로 통합하기가 용이

라) 하나의 테이블로의 통합이 불리한 경우

- 특정 서브타입의 NOT NULL 제한 불가
- 테이블의 칼럼 수가 증가
- 테이블의 블록 수가 증가
- 처리시마다 서브타입의 구분(TYPE)이 필요해지는 경우가 많다
- 인덱스(INDEX) 크기가 증가

서브타입 기준 테이블 변환

- 슈퍼 타입 속성들을 각 서브타입에 추가하여 각각의 서브타입마다 하나의 테이블로 만든다
- 분할된 테이블에는 해당 서브타입의 데이터만 포함돼야 한다
- 주로 서브타입에 많은 양의 속성이나 관계를 가진 경우에 적용된다



[그림 4-4-6] 바커 표기법(좌)과 IE 표기법(우)에 따른 서브타입 기준 테이블 변환 예

가) 절차

- 서브타입마다 테이블 명칭 부여
- 서브타입의 속성을 칼럼명으로
- 테이블마다 슈퍼타입의 속성을 칼럼으로
- 서브타입마다 해당되는 관계들을 FK로
- 테이블마다 슈퍼타입의 관계를 FK로

나) 테이블 사례

위의 [그림 4-4-3] 논리 모델에서의 서브타입을 여러 개의 테이블로 분할하는 경우 테이블의 모습은 [그림 4-4-7]과 [그림 4-4-8]의 데이터 사례 표와 같다. [그림 4-4-7]은 정규직 사원에 대한 표본 테이블의 모습과 인스턴스를 나타낸다. [그림 4-4-8]은 임시직 사원에 대한 표본 테이블의 모습과 인스턴스를 나타낸다.

테이블명 : REG_EMP

칼럼명	EMPNO	ENAME	SAL	DEPT
키형태	PK			FK
Nulls/Unique	NN,U	NN	NN	NN
건본 데이터	9840	김문기	920000	40
	2543	박종민	850000	35
	6587	정종수	927000	20
	9476	심주현	720000	35
	5476	장국선	935000	25

[그림 4-4-7] 테이블 예 : 정규직 사원

테이블명 : TEM_EMP

칼럼명	EMPNO	ENAME	HRATE	OTRATE	UNION	DEPT
키형태	PK				FK1	FK2
Nulls/Unique	NN,U	NN	NN	NN	NN	NN
건본 데이터	7540	홍길동	20000	6500	201	40
	5579	성삼문	50000	8900	180	35
	6714	김이박	27000	6500	201	20
	9451	오재구	20000	6000	201	35
	3040	유성원	35000	8000	180	25

[그림 4-4-8] 테이블 예 : 임시직 사원

다) 여러 개의 테이블로 분할한 경우가 유리한 경우

- 각 서브타입 속성들의 선택 사양 명확한 경우에 유리하다.
- 처리시마다 서브타입 유형 구분이 불필요하다.
- 전체 테이블 스캔시 유리하다.
- 단위 테이블의 크기가 감소한다.

라) 여러 개의 테이블로 분할한 경우가 불리한 경우

- 서브타입 구분 없이 데이터 처리하는 경우 UNION이 발생한다.
- 처리 속도가 감소하는 경우가 많다
- 트랜잭션 처리시 여러 테이블을 처리하는 경우가 증가한다.
- 복잡한 처리의 SQL 통합이 어려워진다.
- 부분 범위 처리가 불가능해 질 수 있다.
- 여러 테이블을 합친 뷰는 조회만 가능하다.
- UID 유지관리가 어렵다.

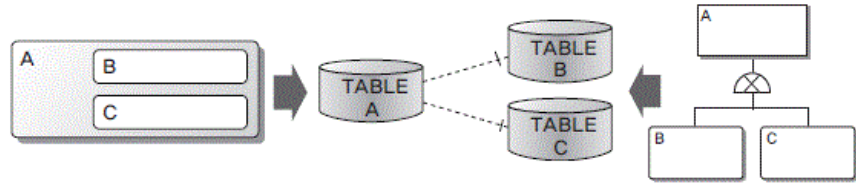
■ 개별타입 기준 테이블 변환

- 슈퍼타입과 서브타입을 각각 테이블로 변환한 경우이다.
- 슈퍼타입과 서브타입 테이블 간에는 1:1 관계가 생성된다.(한 쪽을 모두 합치면 전체와 같게 된다는 면에서 개념적으로 아크 관계와 유사)

다음의 여러 가지 경우를 만족할 때 사용

- 전체 데이터 처리가 빈번하게 발생할 경우에 적용한다.
- 서브타입의 처리는 주로 독립적으로 발생할 경우에 적용한다.
- 테이블을 통합했을 때 칼럼의 수가 너무 많아지는 경우에 적용한다.
- 서브타입의 칼럼 수가 많은 경우에 적용한다.

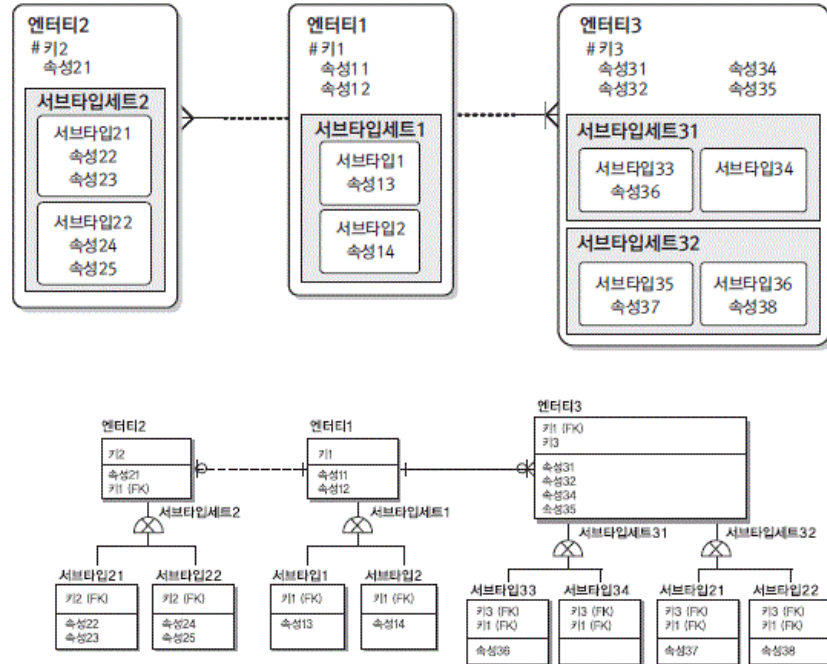
- 트랜잭션이 주로 공통 부분(슈퍼타입)에서 발생한다.
- 슈퍼타입의 처리 범위가 넓고 빈번하여 단일 테이블 클러스터링을 해야 할 때 적용한다.



[그림 4-4-9] 바커 표기법(좌)과 IE 표기법(우)에 따른 개별타입 기준 테이블 변환 예

▣ 서브타입 변환 예

[그림 4-4-10]과 같은 논리적 데이터 모델이 있다고 하자.



[그림 4-4-10] 바커 표기법(위)과 IE 표기법(아래)에 따른 서브타입 변환 대상 예

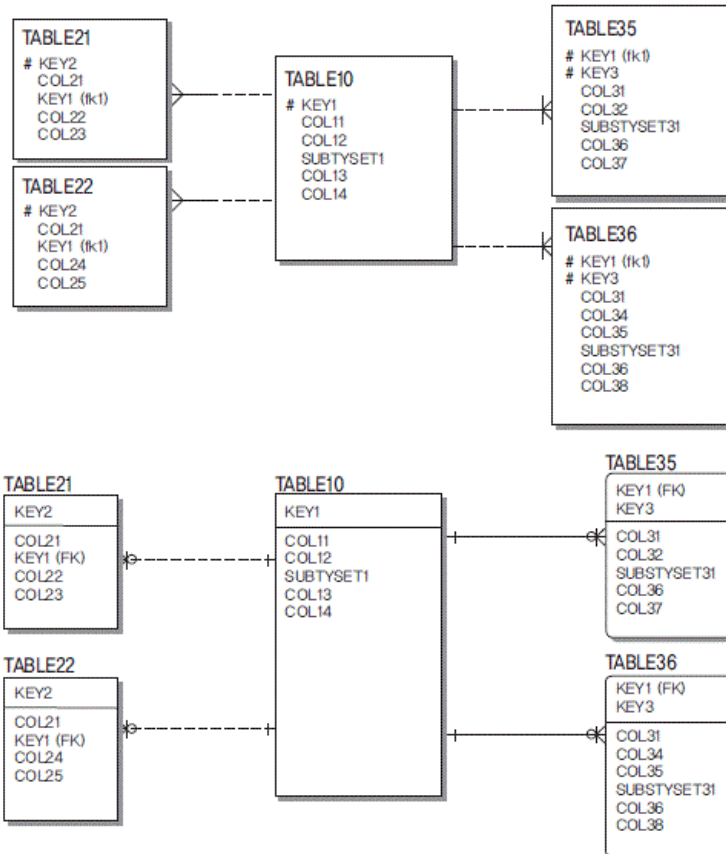
[그림 4-4-10]의 데이터 모델을 물리적 모델로 전환하는 방법은 다양하다. 하나의 데이터 모델을 이용하여 하나 이상의 노드(분산 데이터베이스를 구현하였을 때의 각각의 단위 데이터베이스)에 자 신만의 데이터베이스를 설계할 수 있을 것이다. 데이터 모델의 전부를 물리적 모델로 전환할 수도 있 지만 필요에 따라 원하는 것만 전환하고자 할 수도 있을 것이다.

뿐만 아니라 엔터티를 하나의 테이블로 할 수도 있겠지만 각각의 서브타입 혹은 몇 개를 묶어서 하 나의 테이블로 결정하는 경우도 있을 수 있다. 만약 [그림 4-4-10]의 '엔터티3'처럼 서브타입 세트를 이용하여 여러 차원에서 분류한 서브타입을 가지고 있다면 자신이 원하는 서브타입 세트의 일부서브타입만 테이블로 전환하는 것도 가능하다. 하나의 엔터티를 여러 개의 테이블로 설계하고 싶다면 최소한 논리적 데이터 모델에는 서브타입으로 반드시 정의되어 있어야 가능하다. 즉, 물리적 모델을 결정하는 단계에서 기존에 정의해 둔 서브타입이 아닌 다른 방법으로 테이블을 분할하고 싶다면 논리적 데이터 모델에 새로운 서브타입 세트를 추가로 정의해야 한다는 것을 의미한다.

속성을 물리적 모델로 전환하는 경우에도 전부나 일부만 선택하는 것은 얼마든지 가능하다. 이러 한 개념을 활용하면 하나의 논리적 데이터 모델을 이용하여 여러 개로 수직 분할된 물리 모델을 생성 할 수 있다.

1) Case 1 : 서브타입을 테이블로 분리

속성에 붙어 있는 숫자가 같은 칼럼이 서로 변환된 것으로 간주한다.(예: 속성31 --> COL31)



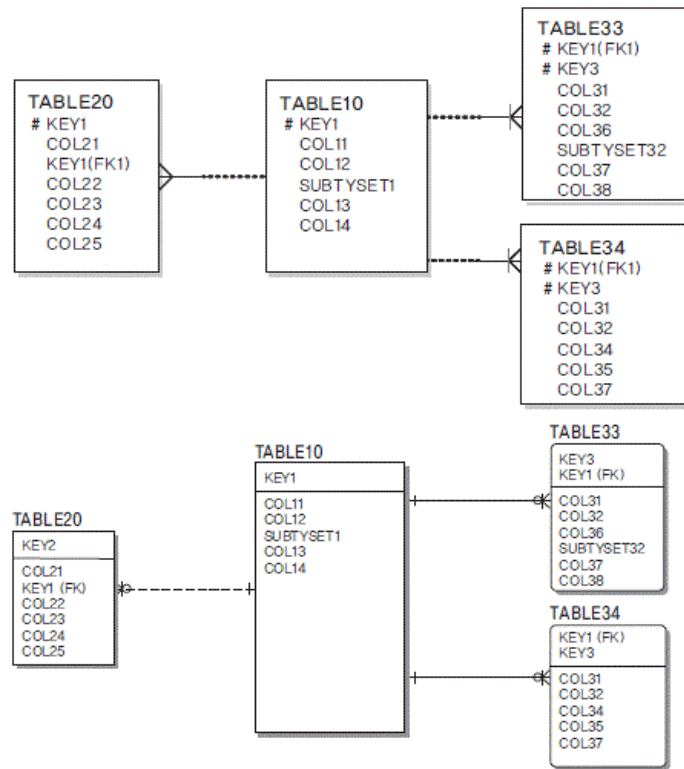
[그림 4-4-11] 바커 표기법(위)과 IE 표기법(아래)에 따른 물리 모델 예제 1

- 엔터티1은 그대로 TABLE10으로 전환되었다. 서브타입 세트란 일종의 구분코드라는 속성이라고 할 수 있으며, 서브타입세트1은 칼럼(SUBTYSET1)으로 전환되었다. 엔터티1에 있던 서브타입1,2는 서브타입세트1의 속성에 들어가는 값의 내용으로 칼럼으로 전환할 필요가 없다.
- 엔터티2는 서브타입별로 테이블을 분리한 모습이다. 서브타입21은 TABLE21로, 서브타입22는 TABLE22로 전환되었다. 공통 속성인 키2, 속성21은 양쪽 모두에 전환되었고, 엔터티
- 엔터티3은 두 종류의 서브타입 세트를 가지고 있다. 위의 물리 모델은 이 중에서 서브타입세트32에 있는 서브타입별로 분리한 모습이다. 자세히 살펴보면 슈퍼타입에 있던 공통 속성 중에서 자신이 필요로 하는 일부분만 전환되었음을 발견할 수 있다. 바커표기법으로 표현할 모델에서 관계선에 세로선(UID BAR)이 붙어 있다면 식별자의 일부가 되겠다는 뜻이므로 물리적 모델이 될 때는 당연히 기본키(PK, Primary Key)이면서 외래키(FK, Foreign Key)가 되어야 한다.

2) Case 2 : 서브타입을 통합 테이블로 생성

물리적 모델에서는 [그림 4-4-12] 같이 동일한 논리적 모델을 기반으로 했지만 앞서 예제와는 다른 모습의 모델을 만들 수도 있다.

[그림 4-4-12]의 물리 모델은 앞서 소개한 물리적 모델과는 테이블의 개수도 다르며, 경우에 따라 테이블 명칭은 동일하지만 내용은 다르게 정의할 수도 있다.



[그림 4-4-12] 바커 표기법(위)과 IE 표기법(아래)에 따른 물리 모델 예제 2

실습 예제

[문] 다음 지문의 내용을 보고 제시된 커뮤니티관리 논리 데이터 모델을 적합한 물리 데이터 모델로 변환하시오.

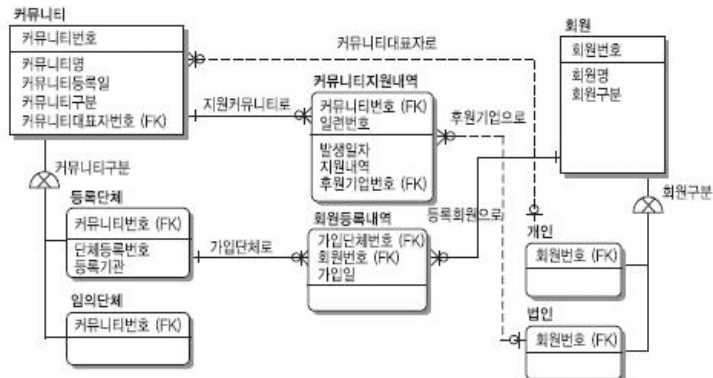
- 우리가 관리하는 커뮤니티에 대해 전체를 대상으로 하는 이벤트나 검색 경우가 빈번하다.
- 커뮤니티 대표자가 누구인지는 등록단체에 대해서만 관리할 것이다.
- 사실상 등록단체에 가입한 회원은 개인회원뿐이고, 앞으로도 그럴 것이다.
- 법인 회원과 개인 회원은 관리할 항목들이 많이 다르고, 같이 검색할 일은 별로 없다.

칼럼명 약어 사용 규칙 : 단어와 단어는 '_'로 연결

커뮤니티 → cmnt, 번호 → no, 명(이름) → nm, 등록일 → endt,
구분 → gb, 대표자 → ldr, 단체 → org, 등록번호 → regno,
등록기관 → regag, 일련 → ser, 발생일자 → actdt, 지원내역 → supdt,
회원 → mbr, 등록내역 → regdt, 가입단체 → jnorg, 가입일 → jndt,
후원기업 → spnco, 법인 → busi, 개인 → prsn,



〈바커 표기법에 따른 커뮤니티관리 논리 데이터 모델〉



〈IE 표기법에 따른 커뮤니티관리 논리 데이터 모델〉

테이블 목록 정의서

갑을 은행 계정계 시스템	갑을 은행 계정계 시스템									
	모델명	인버터 V3					단 계			
	문서번호		버 전				작성일자	2005/10/23		
테이블 목록										
테이블 ID	테이블 명	Type	분류	Table Space	Partition 여부	발생 주기	월간발생량	보존기(한월)	출력수	수정일
EMP	Employee	STANDARD								2005/01/19
inv_issue#st	미결제항리스트	STANDARD								2005/01/19
inv_issue	미결제사항	STANDARD								2005/01/19
inv_attribute	속성	STANDARD								2005/01/19
usr_menugrp	메뉴 구성정보	STANDARD								2005/01/19
usr_menu	단위 메뉴	STANDARD	Key							2005/01/19
da_model	주제영역(DA_MODEL)	STANDARD								2005/01/19
usr_alloc	사용 자할당	STANDARD	Main							2005/01/19
usr_user	사용자	STANDARD	Key							2005/01/19
USR_ROLE	역할	STANDARD	Key							2005/01/19
inv_index	INDEX	STANDARD								2005/01/19
inv_project	Project	STANDARD	Key							2005/01/19

〔그림 4-4-13〕 테이블 목록 정의서 예

엔티티를 테이블로 변환하는 과정이 완료되면 [그림 4-4-13]과 같은 테이블 목록 정의서를 작성한다. 테이블 목록 정의서는 줄여서 테이블 목록이라 부르기도 하며, 전체 테이블이 목록으로 요약 관리되어야 한다.

+ 속성-칼럼 변환

속성이나 관계를 물리 데이터 모델 객체로 변환하는데 있어서 사례 데이터 표를 작성해 보는 것은 실제 데이터가 어떤 형태로 저장되는지, 어떠한 예외사항이 존재할 수 있는지 등을 용이하게 파악할 수 있도록 도움을 주기 때문에 여기서는

사례 데이터 표를 활용하여 변환 과정을 설명한다.

□ 일반 속성 변환

- 엔터티에 있는 각 속성들에 대한 컬럼명을 사례 데이터 표의 컬럼명 란에 기록한다
- 컬럼의 명칭은 속성의 명칭과 반드시 일치할 필요는 없으나 프로그래머와 사용자의 혼동을 피하기 위해 가능한 표준화된 약어를 사용한다
- 표준화된 약어의 사용은 SQL 해독 시간을 감소시킨다.
- SSQL의 예약어(reserved word)의 사용을 피한다.
- 가급적 컬럼 명칭은 짧은 것이 좋으며, 짧은 명칭은 개발자의 생산성에 긍정적인 영향을 준다.
- 필수 입력 속성은 Nulls/Unique 란에 NN을 표시한다.
- 실제 테이블에 대한 설계를 검증하기 위한 목적으로 가능하다면 표본 데이터를 입력시킨다.

□ Primary UID 기본키(Primary Key) 변환

논리 데이터 모델에서의 Primary UID는 물리 데이터 모델에서는 기본키로 생성된다. 실제 DDL에서는 기본키 제약 조건의 형태로 오브젝트가 생성된다.

1) 변환 절차

- 사례 데이터 표의 키 형태 란에 엔터티의 Primary UID에 속하는 모든 속성에 PK를 표시
- PK로 표시된 모든 컬럼은 Nulls/Unique 란에 반드시 NN,U로 표시되어야 함
- 여러 개의 컬럼으로 UID가 구성되어 있는 경우는 각각의 컬럼에 NN,U1을 표시
- 또 다른 Unique Key(Secondary UID)가 있다면 U2로 표시

2) 변환 예

컬럼명	EMPNO	ENAME	JOB
키 형태	PK		
Nulls/Unique	NN,U	NN	
건본 데이터	1354	홍길동	부장
	4562	박경수	사원
	2763	김정구	
	2874	신동파	차장
	6538	정명훈	

[그림 4-4-14] 기본키 변환

□ Primary UID(관계의 UID Bar) 기본키(Primary Key) 변환

논리 데이터 모델에서 Primary UID에 속하는 속성 중에는 해당 엔터티 자체에서 생성된 것도 존 재하지만 다른 집합(엔터티)으로부터의 관계에 의해서 생성되는 UID 속성(관계 속성)도 존재 한다. 이러한 관계 속성 UID의 변환은 기본적인 속성 UID 변환과는 약간 다르다.

1) 변환 절차

- 테이블에 외래키 컬럼을 포함시킨다
- PK의 일부분으로 표시
 - Nulls/Unique 란에 각각 NN,U1을 표시
 - 키 형태란에 PK, FK를 표시
 - 여러 UID BAR가 있는 경우는 (PK, FK1), (PK, FK2), ...
 - 여러 컬럼으로 구성된 경우 PK,FK1을 각각 표시

- 추가된 FK 컬럼에 표본 데이터를 추가

2) 변환 예

컬럼명	CCODE	STID	SDATE	EDATE	GRADE
키 형태	PK, FK1	PK, FK2	PK		
Nulls/Unique	NN,U	NN,U	NN,U		
건본 데이터	344	4567	940525	940630	
	401	2341	940520		
	344	5672	940601	940630	B
	401	4132	940712	940731	D
	743	3564	940827	940920	A

[그림 4-4-15] 바커 표기법(좌)과 IE 표기법(우)에 따른 기본키 변환 예 : 외래키에 의한

▣ Secondary (Alternate) UID Unique Key 변환

논리 데이터 모델에서 정의한 Secondary UID 또는 Alternate Key 들은 해당 집합과 상태 집합과의 선택적인 관계를 가질 수 있도록 하는 데 중요한 역할을 수행한다. 이러한 Secondary UID들은 물리 데이터 모델에서는 Unique Key로 생성된다. 변환 절차는 기본적으로 Primary UID 변환 절차와 동일하다.

▣ 테이블 정의서

갑을 은행 계정계 시스템													
모델 명		인버터 V3							단 계				
문서번호					버 전				작성일자		2005/10/23		
테이블 정의서													
테이블 ID		EMP	테이블명	Employee	Table Space			시스템	인버터V3	종관수			
실제		System											
테이블정의													
파티션 정의													
파티션키													
데이터처리형태													
특이사항													
Note													
No	칼럼명	한글명	Type	Len	Null	UK	PK	FK	참조테이블	참조칼럼	유형	Validation Rule	비고
1	Id	시스템 ID	VARCHAR2	3			Y						
2	Name	시스템 Name	VARCHAR2	20									
3	Address	Description	VARCHAR2	200	Y								
4	Birth_date	일자	DATE		Y						System		
5	Dot_id	부서	VARCHAR2	2	Y						System		

[그림 4-4-16] 테이블 정의서 예

기본적인 테이블 변환, 칼럼 변환 작업이 완성되면 [그림 4-4-16]과 같은 테이블 정의서를 생성할 수 있다. 대부분의 시스템 구축 프로세스상에서 개발자들이 프로그램 개발을 수행하는 단계에서 가장 많이 참조하는 산출물 중 하나이다.

+ 관계 변환

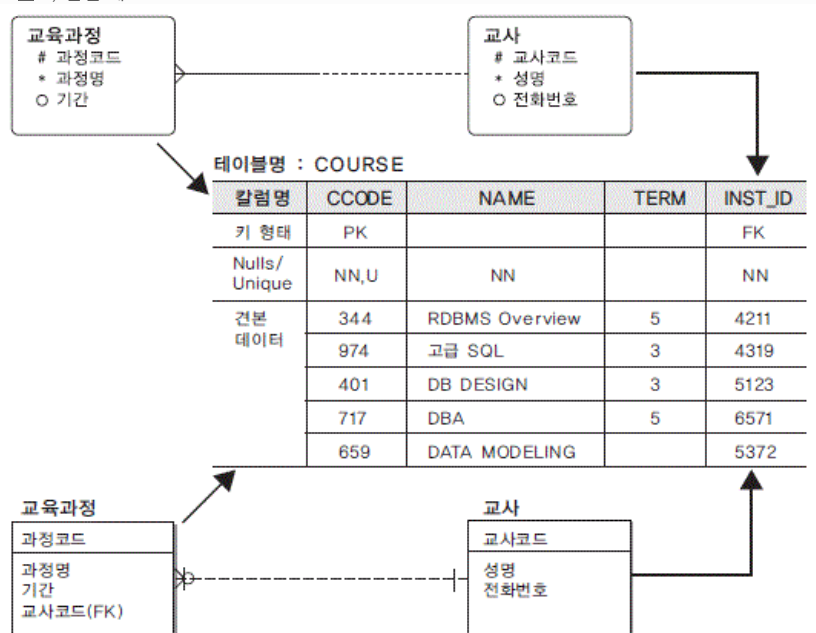
▣ 1:M 관계 변환

논리 데이터 모델에서 존재하는 관계 중에?? 따라 서 관계 칼럼의 선택사항이 결정되게 된다.

■ 1) 변환 절차

- 1(One) 에 있는 PK를 M(Many)의 FK로 변환
 - FK의 명칭 결정
 - 키 형태 란에 FK 표시
 - Nulls/Unique 란에 NN 표시(Must Be 관계시)
 - 필수 관계가 아닌 경우에는 NN를 체크하지 않는다.
- 표본 데이터 추가
- UID BAR 가 있는 경우는 전단계에서 실시

■ 2) 변환 예

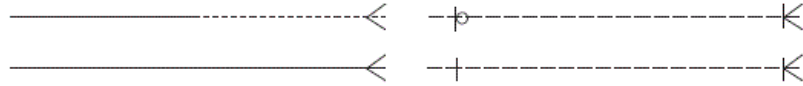


[그림 4-4-17] 바커 표기법(위)과 IE 표기법(아래)에 따른 1:M 관계 변환 예

■ 3) 1:M 관계에서 1 쪽이 Mandatory 관계일 때의 변환시 주의사항

- 자식 쪽의 레코드(Row)가 반드시 하나 이상은 되어야만 부모 쪽의 레코드(Row)를 생성할 수 있다.

- 자식 쪽의 레코드(Row)를 삭제할 경우에는 전체를 다 삭제할 수는 없고 반드시 하나 이상의 자식 레코드(Row)를 남겨두어야 한다. 또는 자식, 부모 레코드(Row)를 동시에 삭제해야 한다.



[그림 4-4-18] 바커 표기법(좌)과 IE 표기법(우)에 따른 1:M Mandatory 관계

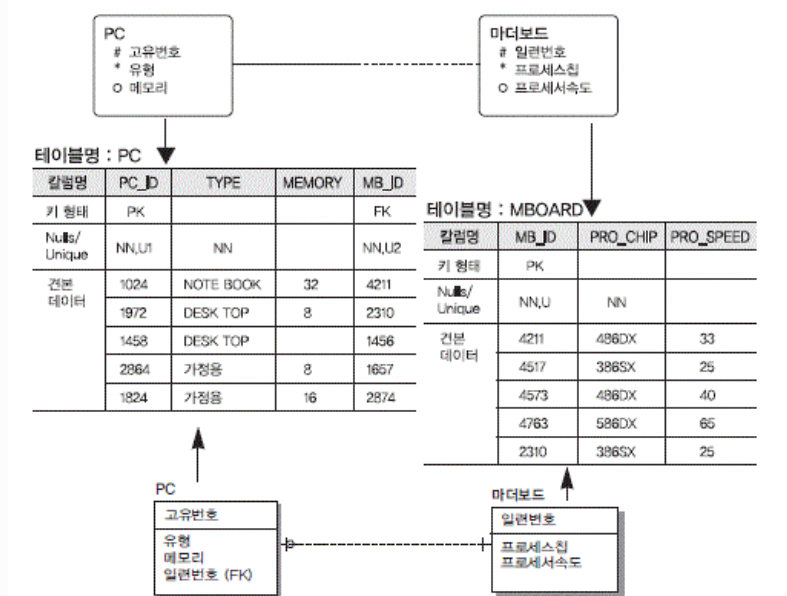
□ 1:1 관계 변환

1:1 관계는 논리 모델에서는 자주 발생하지는 않는 관계이다. 이러한 1:1 관계를 물리 모델로 변환하는 과정은 관계의 Optionality(기수성)에 따라서 다른 방법으로 적용된다. 양쪽 다 Optional인 경우에는 보다 빈번하게 사용되는 테이블이 외래키를 가지는 것이 유리하다.

■ 1) 변환절차

- Mandatory 반대쪽에 있는 테이블의 기본키를 Mandatory 쪽 테이블의 외래키로 변환한다.
- NN 표시를 한다..

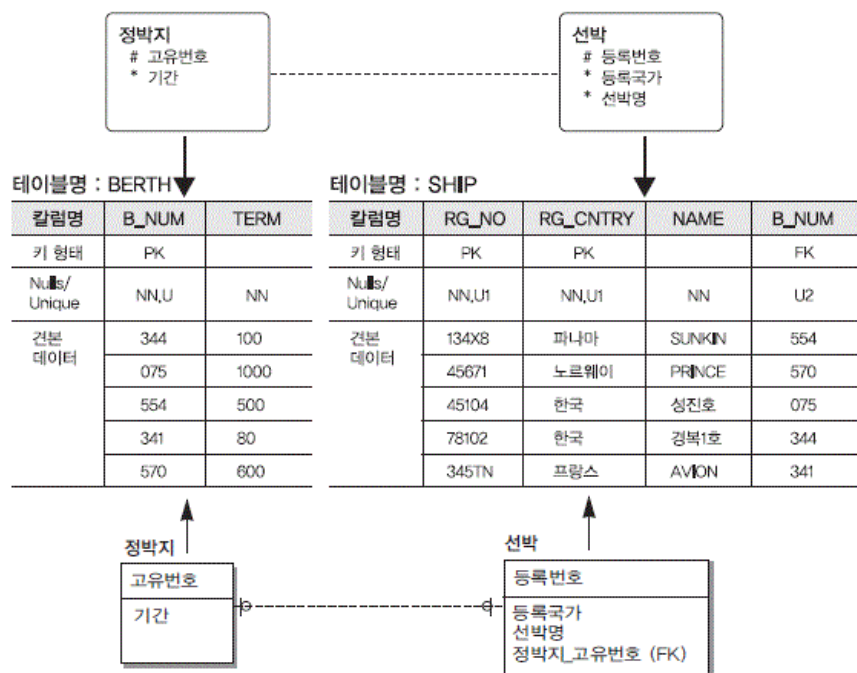
■ 2) 변환 예



[그림 4-4-19] 바커 표기법(위)과 IE 표기법(아래)에 따른 1:1 Mandatory 관계 변환 예

■ 3) 변환시 주의사항

- 1:1 관계에 의해서 생긴 모든 외래키 부분은 Unique Key가 필수적이다.
- 한쪽이 Optional이고 다른 한쪽이 Mandatory라면 Mandatory쪽의 테이블에 외래키가 생성 된다.
- 양쪽다 Mandatory라면 변환시에 어떤 테이블에 외래키를 생성할 것인지를 선택해야 한다.



[그림 4-4-20] 바커 표기법(위)과 IE 표기법(아래)에 따른 1:1 Optional 관계 변환 예

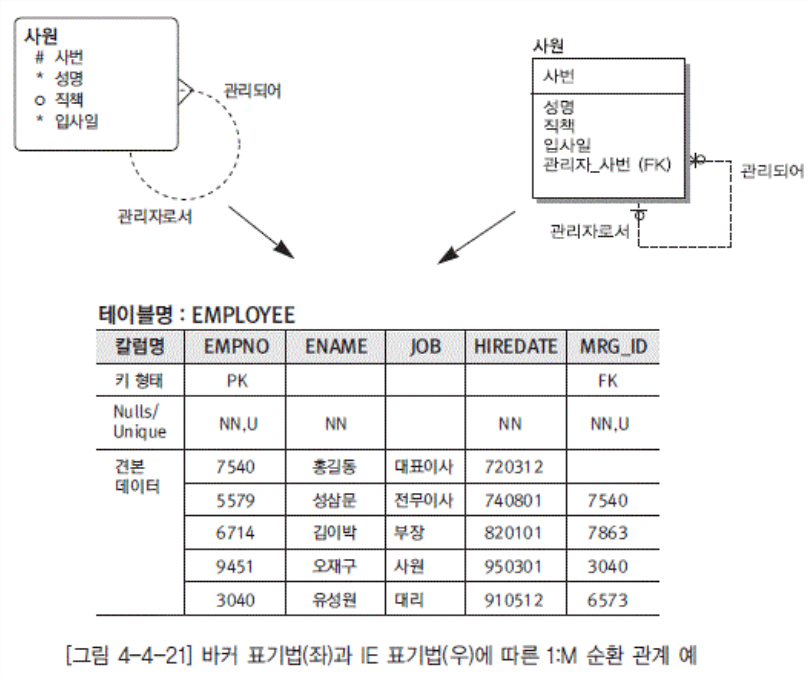
□ 1:M 순환 관계 변환

대부분의 경우는 데이터의 계층 구조를 표현하기 위해서 이러한 관계를 사용한다. 특성상 최상위 관계 속성은 항상 Optional인 형태의 관계이어야 한다. 하지만 경우에 따라서는 최상위의 관계 속성에 특정 값을 지정하는 경우도 존재한다.

■ 1) 변환 절차

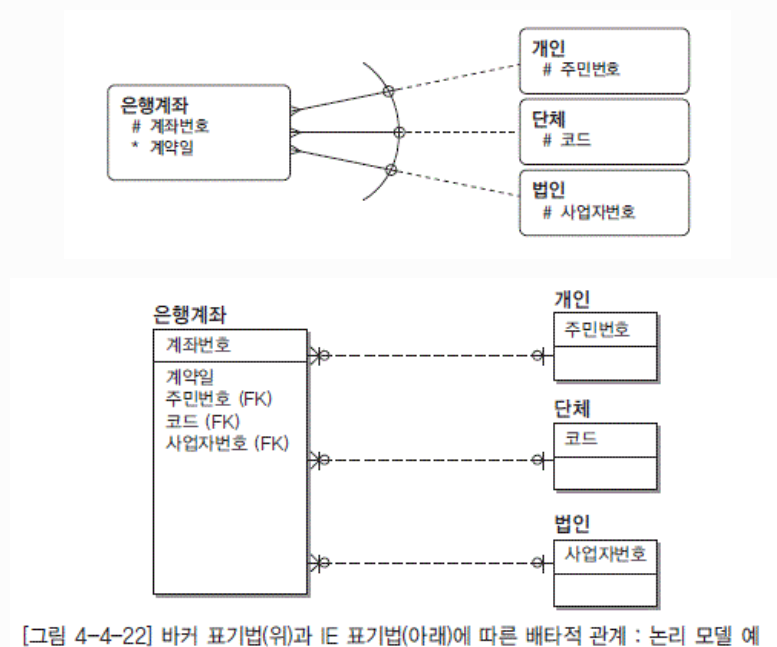
- 해당 테이블 내에 외래키 칼럼을 추가한다. 외래키는 같은 테이블 내의 다른 로우의 기본키 칼럼을 참조하게 된다.
- 외래키 칼럼 명칭은 가능한 한 관계 명칭을 반영한다. 외래키는 결코 NN(Not Null) 이 될 수 없다.

■ 2) 변환 예



□ 배타적 관계 변환

[그림 4-4-22]와 같은 논리 모델에서의 배타적 관계의 모델은 실제 데이터 환경에서는 자주 등장하게 된다. 하지만 이러한 관계를 물리 데이터 모델로 생성하는 방법은 일반적인 관계를 물리 데이터 모델로 변환하는 것과는 다르다. 여기에는 두 가지 정도의 대표적인 방법을 가지고 설명한다.



■ 1) 외래키 (Foreign Key) 분리 방법

각각의 관계를 관계 칼럼으로 생성하는 방법이다. 이 방법을 사용하면 실제 외래키 제약조건 (Foreign Key Constraints)을 생성할 수 있다는 장점이 있다. 하지만 각각의 키 칼럼들이 Optional이어야 한다. 또한 다음과 같은 체크 제약조건(Check Constraints)을 추가적으로 생성하여야 한다.



• 추가적인 체크 제약조건

CHECK (JUMIN_NO IS NOT NULL
AND PART_NO IS NULL
AND BUSINESS_ID IS NULL)

OR (JUMIN_NO IS NULL
AND PART_NO IS NOT NULL
AND BUSINESS_ID IS NULL)
OR (JUMIN_NO IS NULL
AND PART_NO IS NULL
AND BUSINESS_ID IS NOT NULL)

■ 2) 외래키 결합 방법

각각의 관계를 하나의 관계 칼럼으로 생성하는 방법이다. 이 방법을 사용하면 실제 외래키 제약조건을 생성할 수 없다는 단점이 있다. 또한 각각의 관계를 선택적으로 구분할 수 있는 추가적인 칼럼이 필요하게 된다.

칼럼명	ACCTNO	CDATE	JUMIN_NO	TYPE
키형태	PK		FK	
Nulls/ Unique	NN,U	NN	NN	NN
건본 데이터	7540	940614	581101-101211	J
	5579	931201	298-02-11	P
	6714	940516	711024-281071	J
	9451	930718	10-234-1955	B
	3040	921009	211-05-29	P

[그림 4-4-24] 배타적 관계 변환 : 외래키 결합 예

■ 구분 칼럼 추가

[그림 4-4-24]에서와 같이 구분 칼럼 TYPE이 추가되어 배타적 관계의 각 테이블들이 구분된다.

- -J : 개인
- -P : 단체
- -B : 법인

+ 관리상 필요한 칼럼 추가

□ 개념

논리 데이터 모델에는 존재하지 않지만 관리상의 이유로 혹은 데이터베이스를 이용하는 프로그래밍이 좀더 빠르게 수행되도록 하기 위해서 테이블이나 칼럼을 추가할 수 있다. 예를 들면 해당 데이터를 등록한 일자나 시스템 번호 등 같은 관리상의 이유로 필요한 것들이다.

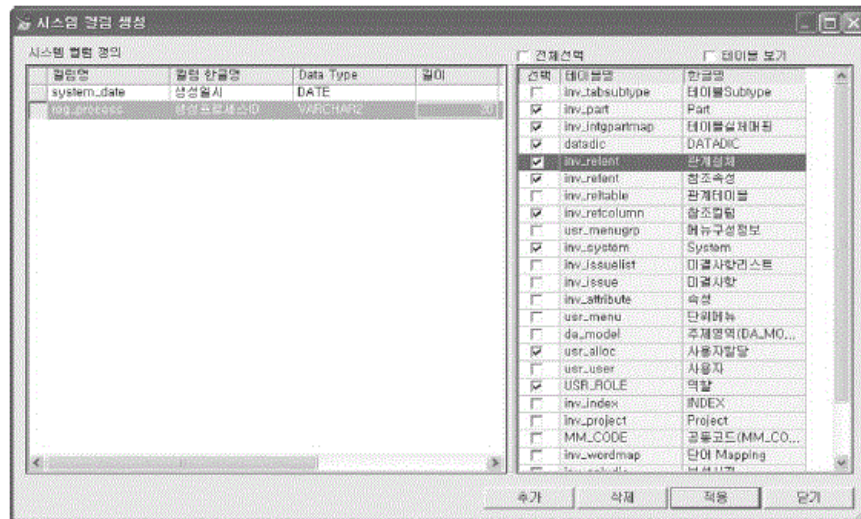
□ 시스템 칼럼 추가 예

[그림 4-4-25]는 생성일시, 생성 프로세스 ID라는 논리 데이터 모델에서는 존재하지 않는 속성인 데도 불구하고 물리 데이터 모델에서 추가하여 생성하고 있는 예를 보여주고 있다.

+ 데이터 타입 선택

□ 개념

물리 데이터 모델링 단계에서 일어나는 많은 문제 중의 하나가 칼럼의 데이터 형식을 잘못 설정하는 데에서 발생한다. 특정 칼럼의 데이터 형식을 선택하는 것은 논리 데이터 모델에서 정의된 논리적인 데이터 타입(정보 타입 : Information Type)을 물리적인 DBMS의 특성과 성능 등을 고려하여 최적의 데이터 타입을 선택하는 작업이다. 여기에서의 DBMS별 특성은 이 책을 통해 설명하는 것은 불가능하다. 따라서 여기에서는 대표적인 데이터 타입의 선택 기준에 대해서만 언급한다. 여기에서는 데이터 타입의 선택에 대한 예제를 들어 설명한다. 단, 모든 DBMS를 예로 들 수는 없기 때문에 MS SQL Server를 예로 들어 설명한다.



■ 문자 타입 (Character Data Types)

논리 데이터 모델에서의 형식이 문자였다면 세부적으로 많은 문자 형식들 중에서 칼럼의 값이 어 떤 범주를 만족하는지 를 판단해야 한다. 여기에는 현재 칼럼이 가지는 값의 특성도 고려되어야 하고, 미래에 가질 값의 특성들도 고려되어야 만 한다.

- 1) 세부 문자 타입 선택을 위한 기준

가) 영문만 사용되는가?

유니코드 형식은 모든 문자를 2바이트(Byte) 체계로 설계한 국제 표준 문자 형식이다. 한글을 저장하기 위해서 반드시 유니코드 형식을 사용해야 하는 것은 아니다. 하지만 일반 문자 형식들에 값을 지정한다면 영문은 1바이트, 한글은 2바이트라는 부조화를 일으키게 된다. 이러한 이유로 유니코드를 저장하는 데이터 타입에는 일반적으로 NCHAR, NVARCHAR등과 같이 앞에 N이 들어 가는 데이터 타입을 사용한다.

나) 4000자 혹은 8000자 이상의 문자열이 포함되는가?

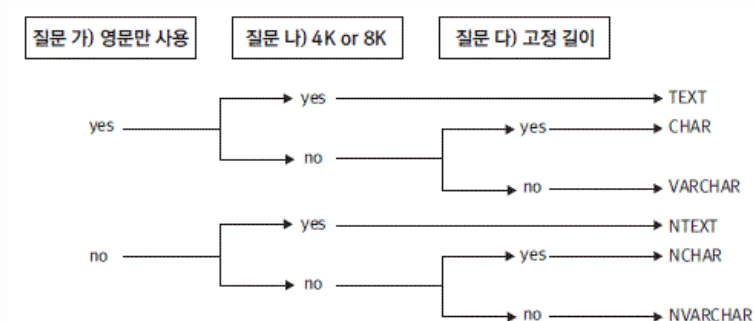
일반적인 문자들을 저장하는 데이터 타입은 4K 혹은 8K를 상한선으로 하고 있다. 물론 이 기준은 DBMS마다 조금씩 다르다. 이렇게 큰 데이터를 저장하기 위해서는 일반적인 문자열을 저장하는 데이터 타입이 아닌 다른 데이터 타입을 사용해야 한다. 예를 들면, LONG, TEXT 등과 같은 데이터 타입을 사용할 수 있다.

다) 입력되는 값의 길이가 일정한가?

값의 길이가 일정하다는 것은 입력되는 칼럼 값의 길이가 일정하다는 것을 의미한다. 반대로 가 변적이라는 것은 입력되는 값의 길이가 일정하지 않다는 것을 의미한다.

- ## 2) 문자 형식 데이터 타입 설정 예

[[그림 4-4-26]은 MS SQL Server에서의 데이터 타입을 기준으로 예를 든 것이다.



■ 숫자 타입 (Numeric Types)

숫자 타입의 데이터 타입도 DBMS마다 많은 형식이 존재한다. 많은 숫자 타입들 중에서 주어진 상황에 맞는 가장 적절한 데이터 타입을 설정해야 한다.

- 1) 세부 숫자 타입 선택을 위한 기준

가) 정말 숫자 데이터인지 판단한다.

많은 경우 숫자처럼 보이는 숫자가 아닌 값들을 관리하는 경우가 존재한다. 예를 들면 6810301633318 과 같은 주민등록번호

록번호는 숫자처럼 보이지만 숫자가 아니다. 즉, 우리가 이 주민 번호를 가지고 연산을 하거나 할 가능성이 있는지를 보면 이것은 숫자 타입이 아닌 문자 타입의 데 이터라는 것을 알 수 있다.

나) 세부 숫자 타입 결정

■ 불린(Boolean)

참(TRUE) 혹은 거짓(False)을 저장하는 경우에 선택한다.

■ 정수(Integer)

소수점 이하를 처리하지 않는 경우에 선택한다.

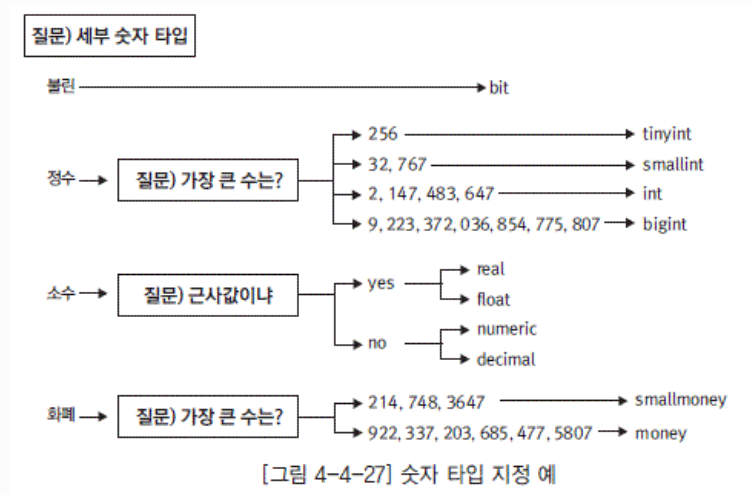
■ 소수(Decimal)

소수점 이하를 처리하는 경우에 선택한다.

■ 화폐(Money)

금액을 저장하기 위한 경우에 선택한다.

2) 숫자 형식 데이터 타입 지정 예



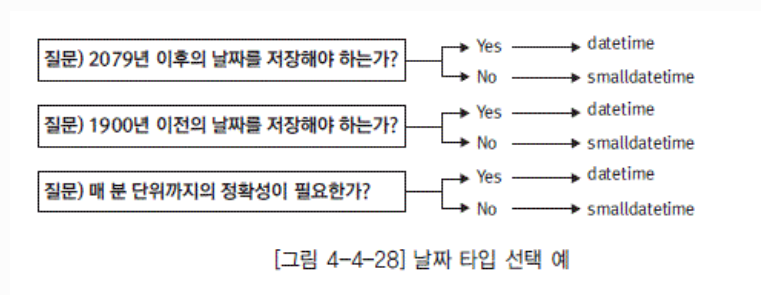
□ 날짜 타입 (Datetime Types)

특정 데이터 항목에 대해서 날짜 타입으로 할 것인지 아니면 문자 타입으로 할 것인지는 이미 논리 데이터 모델에서 결정된다. 그렇기 때문에 물리 데이터 모델링에서는 논리 데이터 모델링에서 날짜 타입으로 결정된 부분을 DBMS 특성에 맞게 여러 개의 날짜 타입들 중에서 어떤 날짜 타입을 선택 할 것인지를 결정하는 것이다.

1) 세부 날짜 타입 선택을 위한 기준

대부분의 DBMS에서는 날짜 타입에 일자뿐만 아니라 시분초의 정보도 같이 저장한다. 심지어는 0.001초 차이까지도 저장하기도 한다. 그래서 그냥 일반적인 시간까지를 저장할 것이냐 아니면 이러 한 정밀한 시간을 저장할 것이냐에 따라 날짜 타입을 결정한다.

2) 세부 날짜 타입 지정 예



+ 데이터 표준 적용

□ 개념

논리 데이터 모델링 과정에서 정의된 엔티티, 속성, 관계들을 여러가지 기준으로 물리 데이터 모델 로 변환한다. 이 과정에서 필수적으로 엔티티명에 해당하는 테이블명을 생성하고, 속성 또는 관계에 해당하는 칼럼명을 생성하게 된다. 이러한 이름을 변환하는 과정에서 전사적으로 미리 생성된 데이터 표준을 따르게 된다. 이러한 데이터 표준에는 대표적으로 표준 용어, 표준 도메인, 표준 명명 규칙 등이 존재한다.

□ 데이터 표준 적용 대상

물리 데이터 모델에서의 데이터 표준화는 다음과 같은 객체를 대상으로 수행하게 된다.

■

■ 1) 데이터베이스(Database)

테이블의 집합으로 통합 모델링 단계의 주제 영역이나 애플리케이션 모델링 단계의 업무 영역에 대응되는 오브젝트이다.

■ 2) 스토리지 그룹(Storage Group)

DASD(Direct Access Storage Device) 즉, 물리적인 디스크(Disk)를 묶어서 하나의 그룹으로 정의해 놓은 것이며 테이블스페이스, 인덱스 스페이스 생성시 스토리지 그룹명을 지정하여 물리적인 영역을 할당하도록 한다.

■ 3) 테이블스페이스(Tablespace)

테이블이 생성되는 물리적인 영역이며, 하나의 테이블 스페이스에 하나 또는 그 이상의 테이블을 저장할 수 있다.

■ 4) 테이블(Table)

논리 설계 단계의 엔터티에 대응하는 객체이다.

■ 5)칼럼(Column)

논리 설계 단계의 속성에 대응하는 객체이다.

■ 6) 인덱스(Index)

테이블에서 특정 조건의 데이터를 효율적으로 검색하기 위한 색인 데이터로 대표적인 인덱스 대상으로는 기본키(Primary Key), 외래키(Foreign Key)등이 있다.

■ 7) 뷰(View)

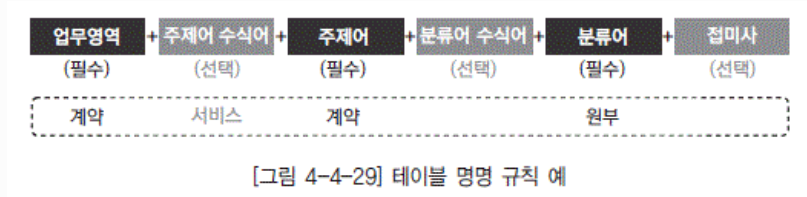
테이블에 대한 재정의로서 물리적으로 테이블의 특정 칼럼, 특정 로우를 뷰로 정의하여 특정 사용 자만 접근이 가능하도록 할 수 있다.

▣ 데이터 표준 적용 방법

■ 1) 명명 규칙에 의한 표준화 적용

테이블에 대한 명명 규칙과 적용 기준들의 예를 들면 다음과 같다.

- 논리 데이터 모델을 물리 데이터 모델로 전환시 테이블명은 엔터티 한글명과 동일하게 용어를 사용하면서 해당 용어를 영문명으로 전환한다.
- 영문명은 영문 약어를 사용하며, 표준 용어 사전에 등록된 표준 영문 약어를 참조한다.
- 테이블의 명명 순서는 업무 영역 + 주제어 수식어 + 주제어 + 분류어 수식어 + 분류어 + 접미사 순으로 한다.



- 테이블 영문명에서는 각 어소별 구분자로 Under Bar(_)를 사용한다.
예) 업무관계자_사원_정보 IVPT_EMP_INFO

■ 2) 표준 용어집에 의한 표준화 적용

사전에 사용될 모든 객체명과 해당 객체에 대한 데이터 타입, 길이 등의 표준을 정의해 놓고 이 표준 들을 적용하는 방식이다. 현재 대부분의 회사들은 위 두 가지의 방법을 병행하여 사용하고 있다고 볼 수 있다.

논리 데이터 모델링의 마지막에 진행되었던 정규화 작업이 완료되면 데이터 모델은 데이터의 중복을 최소화하고 데이터의 일관성과 정확성, 안정성을 보장하는 데이터 구조가 완성된다. 이러한 정규화된 데이터 모델은 시스템의 성능 향상, 개발 과정의 편의성, 운영의 단순화를 위해 정규화의 원칙들에 위배되는 행위를 의도적으로 수행하게 된다. 이러한 일련의 과정을 반정규화 과정이라 할 수 있다. 이러한 과정에는 크게 테이블 관점, 칼럼 관점에서의 반정규화 과정이 존재한다. 이러한 작업은 동전의 양면과도 같다. 즉, 반정규화된 데이터 구조는 성능과 관리효율을 증대시킬 수 있지만, 데이터의 일관성 및 정합성을 해칠 위험을 내포하고 있고, 또한 이를 유지하는데도 그만큼 비용이 발생하여 지나치면 오히려 성능에도 악영향을 미칠 수 있기 때문에, 데이터 모델의 각 구성 요소인 엔터티, 속성, 관계에 대해 데이터의 일관성과 무결성을 우선으로 할지 데이터베이스의 성능과 단순화에 우선순위를 둘 것인지를 적절하게 조정하는 것이 중요하고, 또한 다양한 경험을 필요로 하는 작업이다.

+ 테이블 분할

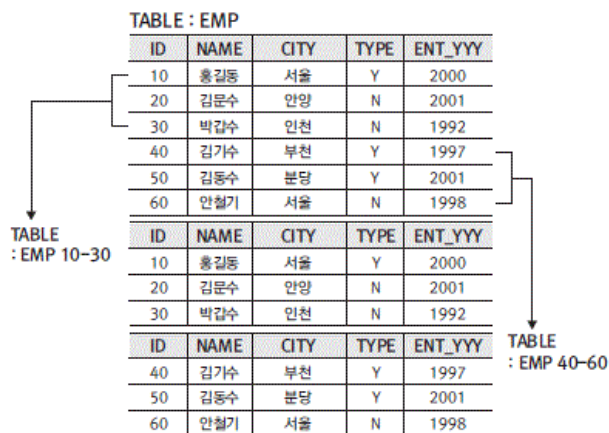
▣ 개념

하나의 테이블을 수직 혹은 수평 분할하는 것을 테이블 분할 또는 파티셔닝이라고 한다. 여기에서의 파티셔닝이라는 용어는 데이터베이스 디자인 단계에서의 데이터를 저장하는 방식의 파티셔닝과는 구분되는 개념이다.

▣ 수평 분할(Horizontal Partitioning)

1) 개념

레코드(Record)를 기준으로 테이블을 분할하는 것을 말한다. [그림 4-4-30]의 사례는 EMP 테이블에 대해 기본키인 ID 칼럼의 값이 10에서 30까지를 EMP 10-30이라는 테이블로 분할하고, 나머지 40에서 60까지를 EMP 40-60이라는 테이블로 분리했다.



[그림 4-4-30] 수평 분할의 개념 예

2) 사용 의미

- 하나의 테이블에 데이터가 너무 많이 있고, 레코드 중에서 특정한 덩어리의 범위만을 주로 액세스 하는 경우에 사용한다.
- 분할된 각 테이블은 서로 다른 디스크에 위치시켜 물리적인 디스크의 효율성을 극대화할 수 있다.
- 현재는 이러한 수평 테이블의 분할은 DBMS 차원에서 제공하고 있다. 특히 분할의 방법 다양하게 제공하고 있는 추세이다. 분할의 대표적인 방법으로는 범위 분할, 해쉬 분할, 복합 분할 등의 기법이 사용된다. 이러한 DBMS 차원의 분할은 데이터베이스 디

자원에서 자세하게 다루어질 것이다.

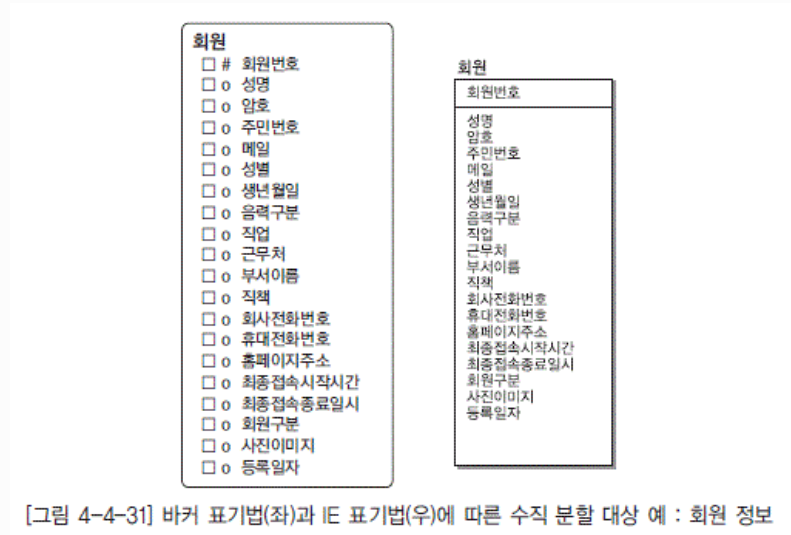
■ 수직 분할 (Vertical Partitioning)

1) 개념

하나의 테이블이 가지는 레코드의 개수가 많아서 수평 분할을 한다면 수직 분할은 하나의 테이블이 가지는 칼럼의 개수가 많아지기 때문에 일어난다. 이러한 수직 분할이 일어나는 이유는 다양하다.

- 조회 위주의 칼럼과 갱신 위주의 칼럼으로 나뉘는 경우
- 특별히 자주 조회되는 칼럼이 있는 경우
- 특정 칼럼 크기가 아주 큰 경우
- 특정 칼럼에 보안을 적용해야 하는 경우

그러면 각각의 내용에 대해서 간단하게 살펴본다.



[그림 4-4-31] 바커 표기법(좌)과 1E 표기법(우)에 따른 수직 분할 대상 예 : 회원 정보

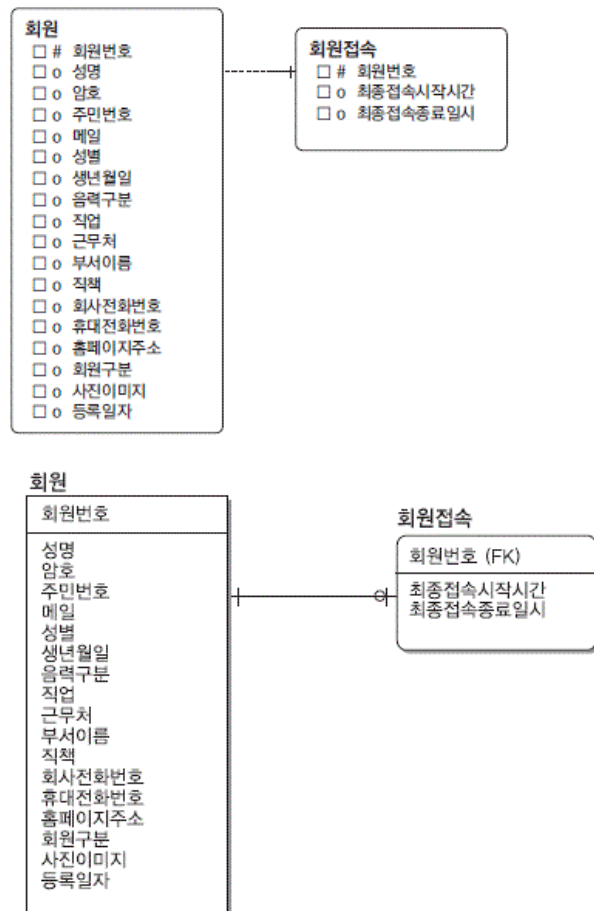
2) 갱신 위주의 칼럼 수직 분할

가) 개념

갱신 위주의 칼럼들을 분할하는 이유는 데이터를 갱신하는 작업이 일어날 때 업데이트하려는 레코드(Record), 즉 레코드에 잠금(Locking)을 수행하기 때문이다. 잠금은 데이터의 무결성을 지키기 위한 수단으로 하나의 프로세스가 특정 데이터 값을 변경하려고 할 때 변경 작업이 끝날 때까지 다른 프로세스가 이 데이터의 값을 변경하지 못하도록 금지하는 것이다.

나) DBMS 버전별 적용 검토

특정 DBMS의 경우에는 이러한 잠금이 레코드 전체에 걸리기 때문에 업데이트가 완료될 때까지 해당 레코드를 사용할 수 없게 하는 요인이 된다. 즉, 몇 개의 갱신 위주의 칼럼에 대한 작업이 나 머지 조회 위주의 칼럼 이용을 방해한다. 이러한 DBMS의 경우에는 이러한 갱신 위주의 칼럼들을 수직 분할하여 사용하는 것이 데이터 사용의 효율성을 증가시킨다.



[그림 4-4-32] 바커 표기법(위)과 1E 표기법(아래)에 따른 갱신 위주의 칼럼 수직 분할 예

3) 자주 조회되는 칼럼 분할

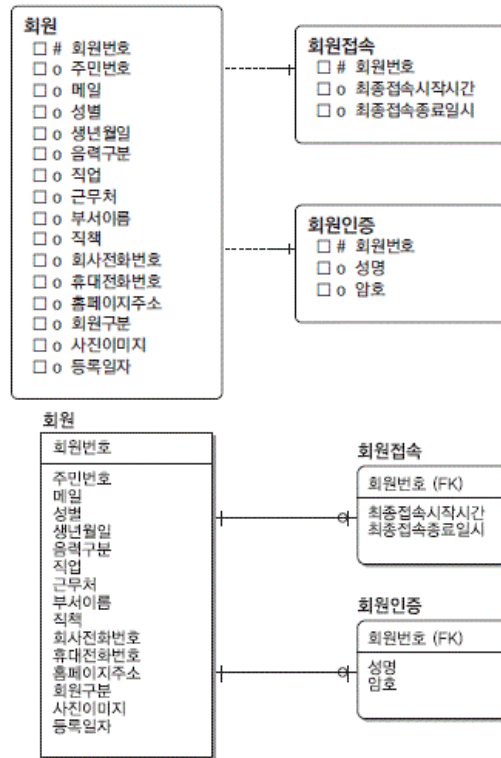
가) 개념

테이블의 특정한 칼럼들이 자주 조회된다면 이러한 칼럼들을 분리해서 별도의 테이블로 관리하면 조회되는 쿼리의 작업 성능을 향상시킬 수 있다. 즉 칼럼 수가 아주 많은 테이블에서 주로 사용되어지는 칼럼들이 극히 일부라고 가정한다면 이러한 일부 칼럼들로 이루어진 테이블을 생성하여 실제 물리적인 I/O의 양을 줄여서 데이터 액세스 성능을 향상시킬 수 있다.

나) 물리적인 DBMS 메커니즘 고려

DBMS는 액세스하고자 하는 모든 데이터를 초기에 물리적인 데이터 파일에서 메모리로 읽어들이게 된다. 또한 한번 읽어들이는 데이터는 읽고 나서 바로 지워지는 것이 아니라 일정기간 메모리에 저장되게 된다. 이러한 DBMS의 메커니즘상에서도 보듯이 읽어들이는 데이터의 양이 적다면 즉, 칼럼 수가 적은 테이블을 자주 읽는 작업을 많이 한다면 초기 데이터를 메모리로 적재하는 비용이 절약되고 또한 메모리상에 상대적으로 오래 머무를 수 있기 때문에 데이터의 재사용성을 높여주는 효과를 가져올 수 있다.

다) 회원 인증 테이블 사례



[그림 4-4-33] 바커 표기법(위)과 IE 표기법(아래)에 따른 자주 조회되는 칼럼의 분할 예

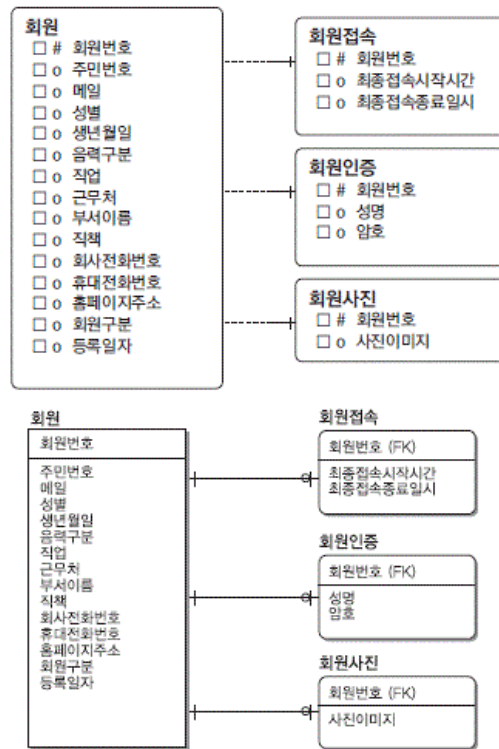
[그림 4-4-33]에서와 같이 회원 테이블에서 성명, 암호 칼럼을 분할하여 회원인증이라는 테이블로 만들었다. 애플리케이션마다 다르기는 하지만 이러한 두 가지 정보들은 회원 인증을 위해서 사이트에 접속할 때마다 반복해서 액세스되는 데이터이다.

4) 특정 칼럼의 크기가 아주 큰 경우 분할

가) 개념

특정 칼럼의 크기가 아주 큰 경우 분할이 일어나는 대개의 경우는 특정 칼럼의 크기가 크다는 것 보다는 특정한 데이터 형식에 기인하는 문형 문자열을 저장하기 위해 지원하는 데이터 타입은 크기는 2GB까지 저장할 수도 있다. 또는 이미지 데이터를 저장할 수도 있다. 이러한 텍스트 및 이미지와 같은 LOB(Large Objects) 데이터 형식을 지원하는 방법은 데이터베이스 시스템마다 약간의 차이는 있다. 하지만, 테이블의 칼럼에 이러한 텍스트 및 이미지 데이터가 포함될 때 성능이 저하될 가능성이 있다. 이것은 백업, 복원과 같은 관리나 프로그래밍과 같은 개발 부분에서 여러 가지 성능 저하 요인으로 작용할 수 있다는 것이다. 그래서 이러한 데이터 형식들을 분리할 수 있다.

나) 회원 사진 테이블 사례



[그림 4-4-34] 특정 칼럼이 아주 큰 경우 바커 표기법(위)과 1E 표기법(아래)에 따른 분할 예

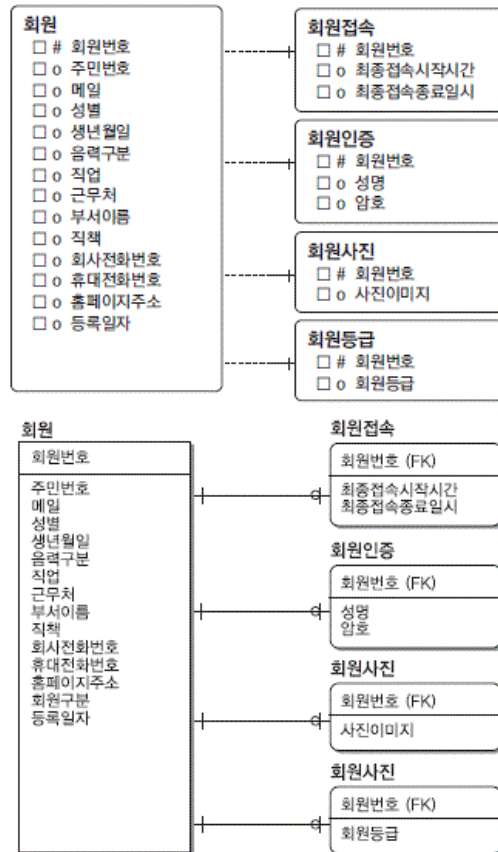
[그림 4-4-34]는 회원의 이미지 데이터를 저장하는 사진 이미지 칼럼을 수직 분할하여 별도의 회원사진 테이블을 생성한 사례이다.

5) 특정 칼럼에 보안을 적용해야 하는 경우의 분할

가) 개념

많은 데이터베이스 시스템이 테이블이나 뷰와 같은 객체들에 대해서는 SELECT, UPDATE, DELETE 등과 같은 권한을 제어할 수 있는 기능을 제공하고 있다. 하지만 테이블 내의 칼럼에 대해서는 이러한 권한(Permission) 제어 기능을 제공하지 않는다. 이러한 경우 해당 칼럼에 대해 권한을 제어하기 위해서는 보안을 적용하고자 하는 칼럼을 분리해 이를 별도의 테이블로 만들어 그 테이블에 대한 권한을 제어하기 위한 목적으로 수직 분할을 할 수 있다.

나) 회원 등급 테이블 사례



[그림 4-4-35] 특정 칼럼에 보안을 적용해야 하는 경우 바커 표기법(위)과 IE 표기법(아래)에 따른 분할 예

[그림 4-4-35]에서 회원 등급 칼럼은 서비스 기업의 특정한 사용자들에게만 부여하는 권한이다. 이 권한은 기업이 제공하는 서비스를 제어하고 설정할 수 있도록 하기 때문에 상당히 주의해서 관리되어야만 한다. 그래서 특정 칼럼에 보안을 적용하기 위해서 이를 회원 등급이라는 테이블로 분리하고 이 테이블에 대해서 데이터베이스가 제공하는 객체 권한(Object Privilege)을 조정해서 특정 사용자에게만 권한을 부여할 수 있도록 한다.

+ 중복 테이블 생성

개념

많은 양의 정보들을 자주 Group By, Sum 등과 같은 집계 함수를 이용해서 실시간으로 통계 정보들을 계산해 낼 수 있다. 하지만 대부분 이러한 계산의 유형은 매우 많은 양의 데이터가 대상이 되고, 하나의 테이블이 아닌 여러 개의 테이블에서 필요한 데이터를 추출하는 경우가 대부분이다. 이를 위해서 특정 통계 테이블을 두거나 중복 테이블을 추가할 수 있다.

중복 테이블 생성 판단 근거

- 정규화에 충실하면 종속성, 활용성은 향상되나 수행 속도 증가가 발생하는 경우 고려한다.
- 많은 범위를 자주 처리해야 하는 경우에 고려한다.
- 특정 범위의 데이터만 자주 처리되는 경우에 고려한다.
- 처리 범위를 줄이지 않고는 수행 속도를 개선할 수 없는 경우에 고려한다.
- 요약 자료만 주로 요구되는 경우에 고려한다.
- 추가된 테이블의 처리를 위한 오버헤드를 고려하여 결정한다.
- 인덱스의 조정이나 부분 범위 처리로 유도, 클러스터링을 이용하여 해결할 수 있는지를 철저히 검토한 후 결정한다.

이와 같은 상황이 존재한다고 판단된다면 논리 데이터 모델에는 존재하지 않지만 물리 데이터 모델에서 중복 테이블을 추가하여 생성할 수 있다.

중복 테이블 유형

중복 테이블에는 다양한 유형이 존재한다. 집계, 진행 테이블 추가를 검토할 수 있는 상황에 대한 예를 들면 다음과 같다.

1) 집계(통계) 테이블 추가

가) 집계 테이블 유형

- 단일 테이블의 GROUP BY
- 여러 테이블의 조인 GROUP BY

나) 집계 테이블 생성시 유의사항

- 로우 수와 활용도를 분석하고 시뮬레이션을 통해서 그 효율성에 대한 면밀한 검토가 선행되어야 한다.
- 집계 테이블에 단일 테이블 클러스터링을 한다면 집계 레벨을 좀더 낮춰 활용도를 높일 수 있는지를 검토해야 한다.
- 클러스터링, 결합 인덱스, 고단위 SQL 을 활용하면 굳이 집계 테이블 없이도 양호한 수행 속도를 낼 수 있다. 이러한 부분도 사전 검토되어야 한다.
- 집계 테이블을 다시 집계, 조인하면 추출할 수 있는지를 검토하여 지나친 집계 테이블을 만들지 않는 것이 좋다.
- 추가된 집계 테이블을 기존 응용 프로그램이 이용할 수 있는지를 찾아 보정시키는 노력이 필요하다.
- 데이터베이스 트리거의 오버헤드에 주의하고 데이터의 일관성 보장에 유의하여야 한다. 즉 집계 테이블과 원본 데이터는 일관성 유지가 매우 중요하다.

2) 진행 테이블 추가

가) 진행 테이블 추가 상황

- 여러 테이블의 조인이 빈번히 발생하며 처리 범위도 넓은 경우
- M:M 관계가 포함된 처리의 과정을 추적, 관리하는 경우
- 검색 조건이 여러 테이블에 걸쳐 다양하게 사용되며 복잡하고 처리량이 많은 경우

나) 진행 테이블 생성시 유의 사항

- 데이터량이 적절하고 활용도가 좋아지도록 기본키를 선정
- 필요에 따라 적절한 추출(Derived) 칼럼을 추가하여 집계 테이블의 역할도 하는 다목적 테이블을 구상
- 다중 테이블 클러스터링이나 조인 SQL을 적절히 이용하면 굳이 진행 테이블을 만들지 않아도 양호한 수행 속도를 낼 수 있는 경우가 많다.

+ 중복 칼럼 생성

▣ 개념

논리 데이터 모델링 과정에서 정규화를 통하여 중복 칼럼을 최대한 제거하는 작업을 수행한다. 이렇게 중복 데이터를 제거하는 이유는 여러 가지가 존재하지만 가장 중요한 이유 중에 하나는 데이터 의 적합성을 유지하기 위함이다. 그런데 물리 데이터 모델링 과정에서 이러한 정규화를 어기면서 다시 데이터의 중복(중복 칼럼 생성)을 수행하곤 한다.

▣ 중복 칼럼 생성 상황

- 빈번하게 조인을 일으키는 칼럼에 대해서 고려해 볼 수 있다.
- 조인의 범위가 다량인 경우를 온라인화해야 하는 경우처럼 속도가 중요한 칼럼에 대해서는 중복 칼럼을 고려할 수 있다.
- 액세스의 조건으로 자주 사용되는 칼럼에 대해서 고려해 볼 수 있다.
- 자주 사용되는 액세스 조건이 다른 테이블에 분산되어 있어 상세한 조건 부여에도 불구하고 액세스 범위를 줄이지 못하는 경우엔 자주 사용되는 조건들을 하나의 테이블로 모아서 조건의 변별성 을 극대화할 수 있다.
- 복사된 칼럼의 도메인은 원본 칼럼과 동일하게 해야 한다. 이것은 데이터의 일관성을 위해서 필수 적인 사항이다.
- 접근 경로의 단축을 위해서 부모 테이블의 칼럼을 자식 테이블에 중복시킬 수 있다.
- 상위 레벨의 테이블에 집계된 칼럼을 추가(M:1 관계)할 수 있다. 즉, 집계 칼럼을 추가한다.
- 하위 레벨의 테이블로 중복 칼럼을 복사(M:1 관계)할 수 있다.
- 연산된 결과를 주로 사용하는 경우에도 미리 연산을 하여 중복 칼럼을 생성할 수 있다.
- 여러 칼럼들의 수 밖에 없는 값이 검색의 조건으로 사용되는 경우에는 연산의 결과를 중복 칼럼으로 생성할 수 있다.
- 여러 개의 로우로 구성되는 값을 하나의 로우에 나열하는 경우이다. 즉 로우로 관리하던 데이터를 칼럼으로 관리하는 경우이다.
- 기본키의 칼럼이 길거나 여러 개의 칼럼으로 구성되어 있는 경우 인위적인 기본키를 추가할 수 있다.

▣ 중복 칼럼 생성시 유의사항

- 다중 테이블 클러스터링으로 해결할 수 있는지 검토한다.
- SQL GROUP 함수 이용하여 처리할 수 있는지 검토한다.
- 저장 공간의 지나친 낭비를 고려하여 적절한 대비책을 마련해야 한다.
- 반복 칼럼은 특별한 경우를 제외하고는 절대 사용할 필요가 없고, 있다면 sum(decode..) 용법과 같은 SQL 기법 등을 활용하여 이러한 부분을 피할 수 있도록 한다.
- 경우에 따라 상대 테이블의 ROWID를 복사하는 경우가 효과적일 때도 있다.
- 데이터의 일관성 보장에 유의해야 한다. 성능을 향상시키기 위해서 데이터의 일관성을 그르치는 일이 일어나서는 안된다.
- 칼럼의 중복이 지나치게 심하면 데이터 처리의 오버헤드가 발생하게 된다.
- 사용자나 프로그램은 반드시 원본 칼럼만 수정하는 것이 바람직하다.
- 일반적으로 수행 속도를 우려해서 지나치게 많은 중복 칼럼을 생성하고 있는 것이 현실이다. 가능하 면 중복 칼럼을 적게 가져가는 것이 바람직하다.
- 클러스터링, 결합 인덱스, 적절한 SQL을 이용하면 특별한 경우를 제외하고는 거의 해결 가능하기 때문에 이 부분을 먼저 적극적으로 고려해 보는 것이 바람직하다.
- 중복 칼럼을 이용하면 손쉽게 액세스 효율을 개선할 수 있으나 지나친 중복화는 반드시 데이터 일관성 오류 발생의 개연성 증가 및 데이터 처리 오버헤드 증가라는 반대급부가 있다는 것을 염두 에 두고 수행해야 한다.
- JOIN, SUB-QUERY 액세스 경로의 최적화 방안을 보다 철저히 강구해야 한다.



물리 데이터 모델링

Home > DB 구축 가이드 > DA 가이드 > 데이터 모델링 > 물리 데이터 모델링

물리 데이터 모델링 이해 물리 요소 조사 및 분석 논리-물리 모델 변환 반영규화 물리 데이터 모델 품질 검토

+ 물리 데이터 모델 품질 검토 개요

물리 데이터 모델 설계가 완료되면 이를 데이터베이스 객체로 생성하고 개발 단계로 넘어가기 전에 모델러를 비롯한 이해관계자들이 모여 물리 데이터 모델에 대한 리뷰 세션을 통해 작성된 데이터 모델의 품질을 검토하는 것이 바람직하다. 3장에서 언급한 바와 같이 데이터 모델 검토는 개념 데이터 모델링, 논리 데이터 모델링, 물리 데이터 모델링의 각 단계가 수행된 후 각 단계에서 작성된 개념 데이터 모델, 논리 데이터 모델, 물리 데이터 모델에 대해 이루어진다. 특히 물리 데이터 모델은 시스템 성능에 대해 직접적인 영향을 미치기 때문에 향후에 발생할 수 있는 성능 문제를 사전에 검토하여 최소화하는 노력이 절대적으로 필요하다. 논리 데이터 모델의 검토 시와 마찬가지로 물리 데이터 모델을 검토하기 위해서는 모든 이해관계자가 동의하는 검토 기준이 필요하며, 이 또한 과목 I. 전사 아키텍처 이해 부분에서 설명한 데이터아키텍처 정책 수립 시 DA원칙/표준에 포함되어야 할 중요한 사안이다.

물리 데이터 모델링의 범위에 대해서는 어느 정도의 이견이 있을 수 있기 때문에 여기서는 이 장에서 설명한 내용을 중심으로 물리 데이터 모델의 품질 검토 기준을 제시하였으며, 조직에 따라서는 과목 VI에서 설명하는 데이터베이스 설계 내용까지를 포함하여 품질 검토 범위에 포함하기도 하고, 데이터베이스 설계 내용에 해당하는 부분을 별도의 품질 검토 대상으로 보기도 하므로, 각자의 환경에 맞게 적용하는 것이 중요하다. 기본적인 품질 검토 기준은 과목 II. 데이터 품질 관리 이해 부분에서 설명한 데이터 구조의 관리 기준을 준용할 수 있으며, 데이터 모델에 대한 품질 기준을 좀 더 세분화해 보면 [표 4-4-1]과 같이 정의해 볼 수 있다. [표 4-4-1]은 3장의 논리 데이터 모델 품질 검토에서 제시한 [표 4-3-1]의 내용을 물리 데이터 모델 관점으로 재작성한 것이다. 물리 데이터 모델 품질 검토의 목적은 '성능'과 '오류 예방'의 관점에서 생각해 볼 수 있으며, 이에 따라 물리 데이터 모델의 품질 기준도 조직에 따라 혹은 업무 상황이나 여건에 따라 가감하거나 변형하여 사용하기도 한다.

[표 4-4-1] 물리 데이터 모델 품질 기준

기준 항목	설 명	검토 관점 사례
정확성	데이터 모델이 표기법에 따라 정확하게 표현되었고, 업무 영역 또는 요구사항이 정확하게 반영되었음을 의미함	- 사용된 표기법에 따라 물리 데이터 모델이 정확하게 표현되었는가 - 대상 업무영역의 업무 개념과 내용이 정확하게 표현되었는가 - 요구사항의 내용이 정확하게 반영되었는가 - 업무규칙이 정확하게 표현/적용되었는가
완전성	데이터 모델의 구성 요소를 정의하는데 있어서 누락을 최소화하고, 요구사항 및 업무 영역 반영에 있어서 누락이 없음을 의미함	- 물리 데이터 모델 작성 항목의 충실성(완성도) - 필요한 설명 항목(테이블/칼럼 설명 등)들의 작성 상태 - 물리 모델링 단계에서 결정해야 할 항목들의 작성 상태(칼럼의 데이터 타입 및 길이, Null 허용 여부, 서브타입 변환, 배타관계 변환, PK, FK, 데이터 무결성 관련 제약사항 등등. 필요에 따라서는 저장공간 지정, 테이블/인덱스 생성 관련 파라미터 결정 사항 등까지도 포함) - 요구사항 반영 및 업무 영역 반영의 완전성: 목적하는 업무 영역을 기술(설계)한 논리 데이터 모델의 구성요소(엔터티, 속성, 관계, 식별자 등)들이 누락없이 물리 데이터 모델로 변환되어 정의된 정도 (단, 특별한 목적에 의해 일부만 물리 데이터 모델로 변환될 수 있으며, 이 경우 목적이 명확해야 함)
준거성	제반 준수 요건들이 누락 없이 정확하게 준수되었음을 의미함	- 데이터 표준, 표준화 규칙 등을 준수하였는가 - 법적 요건을 준수하였는가 - 법적 요건을 준수하기에 충분하도록 도메인이 정의되었는가
최신성	데이터 모델이 현행 시스템의 최신 상태를 반영하고 있고, 이슈사항들이 지체없이 반영되고 있음을 의미	- 업무상의 변경이나 결정사항 등이 시의 적절하게 반영되고 있는가 - 최근의 이슈사항이 반영되었는가 - 현행 데이터 모델은 현행 시스템과 일치하는가

일관성	여러 영역에서 공통 사용되는 데이터 요소가 전사 수준에서 한 번만 정의되고 이를 여러 다른 영역에서 참조·활용되면서, 모델 표현상의 일관성을 유지하고 있음을 의미함	- 여러 주제영역에서 공통적으로 사용되는 엔터티는 일관성 있게 변환되었는가 (전사 수준에서 한 번만 정의되고 이를 여러 다른 영역에서 참조·활용한다는 의미에서 통합성이라 하기도 함) - 모델 표현상의 일관성을 유지하고 있는가 - 동일·유사 목적·용도의 칼럼들은 일관성 있게 정의되었는가 - 조인 대상 칼럼들은 일관성 있게 정의 되었는가
활용성	작성된 모델과 그 설명 내용이 이해관계자에게 의미를 충분히 전달할 수 있으면서, 업무 변화 시에 설계 변경이 최소화되도록 유연하게 설계되어 있음을 의미	- 작성된 설명 내용이나 모델 표기 등이 사용자나 모델을 보는 사람에게 충분히 이해가 될 수 있고, 모델의 작성 의도를 명확하게 이해할 수 있는가 (의사소통의 충분성) - PK, UK 등의 칼럼 구성은 데이터 무결성을 보장하면서 데이터 액세스를 효율화하기에 충분한가 - 논리 데이터 모델의 유연성이 물리 데이터 모델에도 반영되었는가 (오류가 적고 업무 변화에 유연하게 대응하여 데이터 구조의 변경이 최소화 될 수 있는 설계 결과) - 코드화 대상 칼럼에 대한 코드 정의는 업무 지원 및 적용에 충분한가

➤ 물리 데이터 모델 품질 검토 체크리스트의 활용

물리 데이터 모델의 품질 검토 기준에 따라서 물리 데이터 모델에 정의된 테이블, 칼럼, Key 구성, 무결성 제약조건 등 물리 데이터 모델의 주요 구성요소와 물리 데이터 모델 전반에 대한 체크리스트 를 구성할 수 있으며, 이를 통해 물리 데이터 모델의 품질 검토를 보다 용이하게 수행할 수 있다. [표 4-4-2]는 물리 데이터 모델의 주요 구성 요소별로 품질 검토 기준 항목을 적용하여 작성한 품질 검토 체크리스트의 사례이다.

[표 4-4-2] 물리 데이터 모델 품질 검토 체크리스트 사례 제4장 물리 데이터 모델링 데이터아키텍처

검토대상	검토항목	검토 내용
테이블	테이블명	- 명명 규칙을 준수하였는가? - 제한요건에 따라 약어를 사용한 경우 약어사용 규칙을 준수하였는가? - 의미 전달이 명확한 명칭을 사용하였는가? - 테이블 한글명은 엔터티 명칭과 일치하는가?
	테이블 설명	- 데이터 집합의 개요나 성격, 관리 목적 등을 설명하였는가? - 데이터 집합 구성상의 특징이 설명되어 있는가? - 데이터 집합의 생명주기나 오너쉽 등을 비롯한 기타 특이사항에 대한 내용 을 포함하고 있는가? - 설명된 내용은 모든 이해관계자가 이해하고 의사소통 하는 데에 어려움이 없도록 쉽고 상세하게 기술되었는가?
	테이블 정의	- 특별한 이유가 존재하지 않는 한 논리 데이터 모델의 엔터티가 누락없이 물 리 데이터 모델로 변환되었는가? - 테이블 형태는 성능을 고려하여 결정되었는가? (일반 힙 테이블, IOT, 클 러스터, 클러스터드 테이블, 파티션 등) - 서브타입의 변환 형태는 명확한 판단 기준에 의해 결정되었는가? - 테이블 생성 관련 파라미터들은 적절하고 충분히 정의되었는가? - 향후의 업무 변화 가능성에 대비하여 모델 변경을 최소화할 수 있도록 유연 성, 확장성이 고려되었는가?
	통합 수준	- 다른 영역에서 동일 목적으로 사용되는 테이블은 동일 명칭과 구조로 일관되게 사용되었는가?
	권한	- 메타데이터 권한을 정의 하였는가? (테이블 생성/변경/삭제) - 데이터 오너쉽을 정의 하였는가? (데이터 생성/변경/삭제) - 테이블에 대한 접근 권한을 정의하였는가? - 테이블에 대한 접근 권한은 적절하게 정의되었는가?
	발생 건수/ 빈도	- 현재의 데이터 저장 건수/빈도는 파악하였는가? - 향후 예상되는 데이터 저장 건수/빈도의 변화가능성은 파악하였는가?
	법규 준수	- 관련 법규에서 요구하는 데이터를 보관하기 위한 테이블을 정의하였는가? - 조직 특성에 비추어 보호가 요구되는 테이블을 식별하였는가?
	요구사항 추적가능성	- 정의된 테이블은 요구사항과 매핑이 되었는가?
	칼럼명	- 칼럼명은 명명규칙을 준수하였는가? - 제한요건에 따라 약어를 사용한 경우 약어사용 규칙을 준수하였는가? - 의미 전달이 명확한 명칭을 사용하였는가? - 칼럼의 한글명은 속성명과 일치하는가?
	칼럼 정의	- 시스템 내부적으로만 사용되는 칼럼(입력자, 입력일시, 변경자, 변경일시, 내부적으로만 사용되는 identity 칼럼 등) 외에 논리 모델 상에 정의된 속성과의 얼라인먼트가 유지되고 있는가? - 논리 모델의 인조식별자와 같이 일련번호를 저장하는 칼럼의 일련번호 생성 규칙과 방법이 정의되었는가? (Identity 칼럼, Sequence 객체 등)
		- 논리 모델에 정의된 속성의 개요나 성격, 관리 목적, 저장 데이터의 형태적 또는 구성상의 특징 등이 물리 모델에 적합한 설명으로 작성되었는가? - 데이터 집합으로서 속성의 생명주기나 오너쉽 등을 비롯한 기타 특이사항

칼럼	칼럼 설명	에 대한 내용을 포함하고 있는가? - 설명된 내용은 모든 이해관계자가 이해하고 의사소통 하는 데에 어려움이 없도록 쉽고 상세하게 기술되었는가?
	Primary Key	- PK 칼럼의 구성은 데이터의 유일성을 보장하기에 충분한가? - FK 칼럼에 대한 참조무결성 규칙은 정의되었는가?
	Unique Key	- PK 외에 본질식별자에 해당하는 모든 칼럼에 UK가 정의되었는가? - 대체식별자로 정의된 칼럼에 대해 UK가 정의되었는가?
	법규 준수	- 법규상 암호화 대상인 칼럼의 데이터타입과 길이는 암호화를 고려하여 정의되었는가?
	도메인 정의	- 표준 도메인을 정의하여 적용하였는가? - 칼럼의 도메인(데이터 타입, 길이, Not Null 제약, Default 제약, Check 제약 등)은 일관성 있게 정의되었는가?
	추출 칼럼의 정의	- 추출 칼럼에 대한 추출 방법 또는 산식이 명확하게 정의되었는가? - 추출 칼럼에 대한 추출 방법 또는 산식을 적용한 방법은 적절한가? (어플리케이션 로직, 트리거, 기타 등)
	요구사항 추적가능성	- 칼럼 수준에서 필요한 요구사항 매핑은 수행되었는가?
	오너쉽 정의	- 칼럼 수준에서 데이터 오너쉽 정의가 필요한 경우 데이터 오너쉽이 정의되었는가?
FK	참조무결성 규칙 정의	- 논리 모델 상의 관계들은 특별한 이유가 없는 한 누락없이 참조무결성 규칙이 정의되었는가? - 정의된 참조무결성 규칙은 업무 영역의 내용이나 요구사항과 일치하는가? - 배타적 관계의 FK 칼럼에 대한 참조무결성 규칙을 적용하기 위한 방법은 적절한가? (DBMS의 FK 제약조건, 어플리케이션 로직, DB트리거 등)
	요구사항 추적가능성	- 참조무결성 규칙에 대해 필요한 요구사항 매핑은 수행되었는가?
	FK 일관성	- FK 칼럼은 참조하는 PK 칼럼과 도메인 정의가 일치하는가?
모델전반	반정규화	- 반정규화 방법과 형태, 적용 범위는 적절한가? - 반정규화된 테이블이나 칼럼은 명확한 목적에 근거하고 있는가?
	인덱스 정의 주1)	- PK 인덱스 외에 추가적인 인덱스가 고려되었는가? - PK 인덱스를 포함하여 인덱스의 구성 칼럼들은 적절하게 선정되었는가? (엑세스 경로 분석 수행 여부) - 인덱스 생성에 관련된 파라미터들은 충분하고 적절하게 정의되었는가?
	스토리지 정의 주2)	- 스토리지 구성은 I/O 분산과 액세스 성능을 고려하여 정의했는가? - 스토리지 정의 파라미터들은 적절하게 정의되었는가? - 테이블과 인덱스는 적절한 테이블스페이스에 할당되었는가?

※ 주1)과 주2)는 방법론이나 모델링 도구에 따라 데이터베이스 설계의 검토 항목으로 보기도 함.